



Pamantasan ng Lungsod ng Maynila
University of the City of Manila
Intramuros, Manila, Philippines



SeaStack

A Compiler Proposal presented to the
Computer Science Department
College of Information Systems and Technology Management

In Partial Fulfillment of the Requirements for the Degree
Bachelor of Science in Computer Science (BSCS)

Submitted by:

Cfarers

Aliswag, K. V. J.

Alteza, J. R. D.

Cayacap, F. A. S.

Del Rosario, J. M. A.

Magaan, F. M. V.

Mulleno, J. M. A.

Submitted to:

PROF. LEISYL M. MAHUSAY

PROF. JONATHAN C. MORANO

May 2026

Table of Contents

Overview of the Programming Language.....	2
I. STRUCTURE OVERVIEW.....	3
II. GENERAL RULES.....	4
III. SPECIFIC RULES.....	11
DECLARATIONS.....	11
DATA TYPES AND LITERALS.....	12
IDENTIFIERS.....	18
VARIABLES.....	19
CONSTANTS.....	21
ARRAYS.....	22
STRUCTURES.....	25
OPERATORS.....	30
OPERATOR PRECEDENCE.....	34
EXPRESSIONS.....	35
STATEMENTS.....	40
FUNCTIONS.....	59
COMMENTS.....	66
IV. REGULAR DEFINITION.....	67
V. REGULAR EXPRESSION.....	69
VI. TRANSITION DIAGRAM.....	73
RESERVED WORDS.....	73
RESERVED SYMBOLS.....	76
IDENTIFIERS.....	80
COIN LITERALS.....	82
DIME LITERALS.....	84
PARCH AND SCROLL LITERALS.....	85
COMMENTS.....	86
VII. CONTEXT FREE GRAMMAR.....	87
VIII. FIRST SET.....	105
IX. FOLLOW SET.....	113
X. PREDICT SET.....	123
XI. TEST SCRIPTS.....	157

Overview of the Programming Language

SeaStack is a high-level programming language designed to combine structured programming principles with a thematic and engaging syntax inspired by the ocean voyager theme. It takes its foundation from the C language, adapting familiar constructs such as functions, loops, conditionals, and structures while introducing an oceanic twist to make coding both intuitive and enjoyable.

The integration of the Ocean Voyager theme enhances the language by aligning programming commands with navigational and seafaring terms. This thematic consistency creates an immersive environment that not only strengthens recall of syntax but also sparks creativity in the programming experience.

SeaStack supports a wide range of constructs, including global and local declarations, expressions, statements, arrays, structures, and user-defined functions that enable programmers to implement both simple and complex logic. These features make SeaStack suitable for learning core programming concepts, developing simple yet structured algorithms, and introducing problem-solving skills more interactively.

Overall, the SeaStack programming language makes programming accessible and enjoyable by combining solid foundations with a creative oceanic theme. It encourages learners to see coding as a voyage of discovery, emphasizing clarity, engagement, and a fun learning experience.

I. STRUCTURE OVERVIEW

```

<global_declaration/s> | λ

LOCKE <DATA_TYPE> <id> = <value>!!    ~ Global Constant | λ
<DATA_TYPE> <id>!!                    ~ Global Variable | λ
<DATA_TYPE> <id>{<size>}!!            ~ Global Array | λ

MAST <id> [                            ~ Structure Definition | λ
    <member_declaration/s>
]!!

<DATA_TYPE> <id>(<parameter/s>) [    ~ Returning Function | λ
    <local_declaration/s> | λ
    <statement/s> | λ
    BACK <return_value>!!
]

ABYSS <id>(<parameter/s>) [          ~ Non-returning Function | λ
    <local_declaration/s> | λ
    <statement/s>
    BACK!! | λ
]

~~ muti-line
comment ~~

AHOY0 [                                ~ Main Program
    <local_declaration/s> | λ

    <DATA_TYPE> <id>!!                ~ Local Variable | λ
    <DATA_TYPE> <id>{<size>}!!        ~ Local Array | λ

    MAST <id> <variable/s> !!        ~ Structure Declaration | λ

    <statement/s>
]
    
```

II. GENERAL RULES

1. SeaStack supports variables, constants, arrays, structures, expressions, statements, subfunctions, comments, and a single AHOY function. It also supports global and local declarations.
2. Global declarations include variables, constants, arrays, and structure and function definition. These are located at the start of the program and can be accessed or modified anywhere within the program.
3. Local declarations include variables, arrays, and structure variables, and it overrides global declarations of the same name and type. These are declared at the start of any function body and can only be accessed or modified within the function in which they are declared.
4. SeaStack uses two exclamation marks (!!) as a statement terminator.
5. The body of functions, structures, conditional statements and looping statements, as well as the initial values of an array and structure variable, must be enclosed within a pair of square brackets ([]).
6. SeaStack recognizes the following reserved words as data types: COIN, DIME, PARCH, SCROLL, and BOOL.
7. COIN is any whole number. It must contain at least one (1) digit and may contain up to sixteen (16) digits only. Leading zeros are not allowed.
8. DIME is any number with a decimal component. The integer part must contain at least one (1) digit and may contain up to sixteen (16) digits only. The decimal part may contain up to eight (8) digits. Leading zeroes, and trailing zeroes exceeding 8 digits, are not allowed.
9. PARCH is any single ASCII character, except newline and single quotation mark, or a single escape sequence, enclosed in a pair of single quotation marks (' ').
10. SCROLL is any sequence of ASCII characters, except newline and double quotation marks, enclosed in a pair of double quotation marks (" ").
11. BOOL can be either AYE (**true**) or NAY (**false**) only.
12. Identifiers are used to name variables, constants, arrays, structures, structure member or variable, and subfunctions. It must only start with a lowercase letter, and can be followed by a sequence of lowercase letters, digits, and underscores only. Identifiers should have a length of at least one (1) and at most twenty (20) valid characters.
13. Variables are values that can be accessed and modified in the program. It is declared with the data type, then the identifier, and the statement terminator.
14. Constants are fixed values that can be accessed but cannot be modified in the program. It must be initialized at the time of declaration, beginning with the reserved word LOCKE, then the data type, identifier, assignment operator (=), the assigned value, and the statement terminator.
15. Arrays are collections of elements that share the same data type. An array declaration begins with the data type, then the identifier, a fixed size enclosed in curly braces ({ }),

-
- and the statement terminator. Arrays in SeaStack can be either one-dimensional or two-dimensional only.
16. Structures are groups of several related variables of different data types. It is defined after the declaration of any global constant, variable, or array, and before any function. It starts with the reserved word **MAST**, followed by the identifier, then the member declaration enclosed in square brackets, and ends with the statement terminator.
 17. A structure variable can only be declared inside the **AHOY** function. It must start with the reserved word **MAST**, followed by the structure member identifier, then the variable identifier, and end with the statement terminator.
 18. SeaStack supports arithmetic, assignment, unary, relational, logical, and concatenation operators. Operators follow the given operator precedence rule.
 19. Operators must not appear consecutively without any operand in between.
 20. SeaStack supports arithmetic, relational, logical, unary, and **SCROLL** expressions.
 21. Arithmetic expressions perform mathematical calculations on operands of type **COIN** or **DIME** only with arithmetic operators (+, -, *, /, %, ^).
 22. Relational expressions compare exactly two operands of type **COIN** or **DIME** only with less than (<), greater than (>), less than or equal (<=), or greater than or equal (>=) operators. Relational expressions using the operators equal to (==) or not equal to (!=) can compare two operands of any compatible data type.
 23. Logical expressions compare operands of type **BOOL** only with logical operators (!, !#, ||, &&).
 24. Unary expressions perform increment or decrement on operands of type **COIN** only with unary operators (+#, -#) as a prefix.
 25. **SCROLL** expressions combine operands of type **SCROLL** with the concatenation operator (&).
 26. SeaStack supports input, output, assignment, conditional, looping, and jump statements.
 27. Input statements use the reserved word **ASK** to get user input, while output statements use the reserved word **ECHO** to display output.
 28. Assignment statements assign a value to the left-hand operand with the assignment operators (=, +=, -=, *=, /=, %=, ^=).
 29. Conditional statements execute when a certain condition is met. **LOOK**, **DROP**, **DROLOOK**, and **CHART** are reserved words used for conditional statements. The body of conditional statements must be enclosed within square brackets.
 30. Looping statements execute repeatedly based on a given condition. **HOIST**, **HEAVE**, and **HAUL-HEAVE** are reserved words used for looping statements. The body of looping statements must be enclosed within square brackets.
 31. Jump statements immediately execute a block of code in another part of the program. **LAND**, **SAIL**, and **BACK** are reserved words used for jump statements.
 32. The **AHOY** function serves as the main function, found at the end of the program. It must not have a return type, not include any parameter, and not have any **BACK** statement. It cannot be called by any function.

-
33. Subfunctions must be defined after the declaration of any global constant, variable, array, or structure definition, and before the AHOY function. It can either be a returning or nonreturning function. It must begin with a return type, followed by a unique identifier, then a pair of parentheses () with an optional parameter list inside, and the body enclosed within square brackets ([]).
 34. Returning functions must have a single BACK statement located at the end of the body. Statements, excluding the single BACK statement, are optional.
 35. Nonreturning functions and the AHOY function must have at least one statement, excluding jump statements.
 36. A returning function can only have a return type of COIN, DIME, PARCH, SCROLL, or BOOL and must end with a single BACK statement that returns a value matching the declared return type.
 37. A nonreturning function must only have a return type of ABYSS and must not return any value. It can end with a single BACK statement without a return value.
 38. Comments are ignored during execution. A single-line comment starts with a tilde symbol (~) followed by a sequence of characters, and ends with a newline.
 39. A multi-line comment starts with a tilde and open parenthesis (~(), and is enclosed with a closing parenthesis followed by a tilde ()~). A multi-line comment that is not enclosed is invalid.

RESERVED WORDS

Reserved Words	C Equivalent	Definition
ABYSS	void	Indicates that it will not return a value.
ADRIFT	default	Runs if no COURSE matches in a CHART statement.
AHOY	main	The starting or entry point of the program.
ASK	scanf	Reads input from the standard input stream.
AYE	true	Represents the value true.
BACK	return	Exits a function and optionally passes a value.
BOOL	bool	Used to store Boolean values, AYE (true) or NAY (false).
CHART	switch	Executes a block of code based on the value of an expression.
COIN	int	Represents whole numbers.
COURSE	case	Defines a branch inside a CHART statement.
DIME	double	Represents numbers with a decimal part.
DROP	else	Executes a block of code if the previous LOOK is false.
DROLOOK	else if	Tests multiple conditions after an initial LOOK.

Reserved Words	C Equivalent	Definition
ECHO	printf	Prints output to the standard output stream.

HAUL-HEAVE	do-while	Creates a loop that executes once and loops while the condition is true.
HEAVE	while	Creates a loop that runs as long as a condition is true.
HOIST	for	Creates a loop that runs a fixed number of times.
LAND	break	Used to exit from a loop or CHART statement.
LOCKE	const	Defines an initial value that cannot be modified.
LOOK	if	Executes a block of code if the condition is true.
MAST	struct	Defines a collection of variables of different data types.
NAY	false	Represents the value false.
PARCH	char	Represents a single character.
SAIL	continue	Skips the loop and jumps to the next iteration.
SCROLL	string	Represents a sequence of characters.

RESERVED SYMBOLS

Operators	
Arithmetic Operators	$+$, $-$, $*$, $/$, $\%$, $^$
Assignment Operators	$=$, $+=$, $-=$, $*=$, $/=$, $\%=$, $^=$
Unary Operators	$++$, $--$
Relational Operators	$<$, $>$, $<=$, $>=$, $==$, $!=$
Logical Operators	$!$, $!#$, $ $, $\&\&$
Concatenation Operator	$\&$

Other Symbols

RESERVED SYMBOLS	DEFINITION
@	Used as a prefix to reference a variable.
\$	Used to access the member of a structure variable.
,	Used to separate values.
:	Used to indicate the start of a COURSE's body.
{	Used to enclose the size or index of an array or the character of a SCROLL.
}	
(	Used to enclose parameters, arguments, grouped expressions, or conditions.
)	
[	Used to enclose a block of statements, initial values of an array, and member declaration or member initialization of a structure.
]	

Other Symbols

RESERVED SYMBOLS	DEFINITION
------------------	------------

ALISWAG, K. V. J.
ALTEZA, J. R. D.
CAYACAP, F. A. S.

DEL ROSARIO, J. M. A.
MAGAAN, F. M. V.
MULLENO, J. M. A.



'	Used to enclose a PARCH literal.
"	Used to enclose a SCROLL literal.
~	Used to start a single-line comment.
~(	Used to start a multi-line comment.
)~	Used to end a multi-line comment.
!!	Used as a statement terminator.

III. SPECIFIC RULES

DECLARATIONS

Establish the characteristics of variables, constants, arrays, or structures before they are used. SeaStack allows global and local declarations.

A. Global declarations - Located at the start of the program and can be accessed or modified anywhere within the program. Global declarations include the following:

1. Declaration or initialization of variables, arrays, or initialization of constants.
Constants can only be declared globally. Local constant declarations are invalid.
2. Definition of structures
3. Definition of subfunctions

B. Local declarations - Located at the start of any function body and can only be accessed or modified within the function they are declared in. If a local and global declaration share the same identifier, the local declaration will override the global declaration. Local declarations include the following:

1. Declaration or initialization of variables or arrays
2. Declaration or initialization of structure variables

DATA TYPES AND LITERALS

Data types specify which type of value a variable can hold and the type of operations that can be performed on it. The reserved words COIN, DIME, PARCH, SCROLL, and BOOL are used as data types, while ABYSS is used as a return type for functions that do not return a value.

A. COIN - A whole number without a decimal component within a specified range.

Rules for COIN

1. A COIN literal represents whole numbers. The inclusion of any character other than digits or the dash sign (-) is not permitted.
2. A COIN accepts a maximum of sixteen (**16**) digits. The accepted range is from -9999999999999999 to 9999999999999999.
3. Leading zeros are not allowed unless the value is zero, in which case one zero remains.
4. Positive COIN literals are unsigned and represented without a prefix.
5. Negative COIN literals must be denoted by a dash sign (-) placed immediately before the first digit. Any whitespace between the dash sign and the digit is prohibited.

Examples

VALID	INVALID
20	20. 5
1000000000000000	1,000,000,000,000,000
1234567890123456	1234567890123456 7
2000000000000001	00 2000000000000000
67	+ 67
-20	- 20
-34	3 a 4

B. DIME - A number with a decimal component within a specified range.

Rules for DIME

1. A DIME literal represents numbers with a decimal part. The inclusion of any character other than digits, the decimal point (.), or the dash sign (-) is not permitted.
2. A DIME accepts a maximum of sixteen (**16**) digits for the integer part and a maximum of eight (**8**) digits for the decimal part. The accepted range is from -9999999999999999.99999999 to 9999999999999999.99999999.
3. If the value has no integer part, a single leading zero is required before the decimal point. If no decimal part is provided, a decimal point followed by a single zero (**.0**) is automatically added to the value.
4. Leading zeros are not allowed unless they are the single leading zero required before the decimal point, or the value is zero. Any digit after the eighth digit in the decimal part is invalid.
5. Positive DIME literals are unsigned and represented without a prefix.
6. Negative DIME literals must be denoted by a dash sign (-) placed immediately before the first digit.

Examples

VALID	INVALID
0.12345678	000 .12345678
1.12345678	1.12345678 001
1234567890123456.12345678	1234567890123456 10 .12345678
12.5	12.5 a
0.123	. 123
20 → 20.0	20 .
-3.5	- 3.5
12.0	12. 0000

C. PARCH - A single character enclosed in single quotation marks (' ').

Rules for PARCH

1. PARCH represents a single ASCII character, except a newline, backslash, or single quotation mark, or a single escape sequence, enclosed in single quotation marks.
2. Using any symbol other than a pair of single quotation marks, or the lack thereof, is invalid to enclose a PARCH.
3. Any value enclosed in single quotation marks that exceeds or subceeds a single character or escape sequence is invalid.
4. PARCH recognizes only five escape sequences. These are valid only inside the pair of single quotation marks. Any instance of a backslash that is not followed by any valid escape sequence character is invalid.
 - a. `\s` : Single quote
 - b. `\n` : Newline
 - c. `\t` : Horizontal tab
 - d. `\0` : Null
 - e. `\\` : Backslash

Examples

VALID	INVALID
'A'	"A"
'+'	'++'
''	''
'\n'	'\m'
'\s'	'''
'\\'	'\\'

D. SCROLL - A sequence of characters enclosed in double quotation marks (" ").

Rules for SCROLL

1. SCROLL represents an ordered sequence of ASCII characters, except a newline, backslash, or double quotation mark, enclosed in double quotation marks.
2. Using any symbol other than a pair of double quotation marks, or the lack thereof, is invalid to enclose a SCROLL.
3. It must consist of at least one (1) valid ASCII character enclosed in double quotation marks.
4. Each character of a SCROLL is treated as a single-character SCROLL literal, and can be accessed using an index.
5. SCROLL recognizes only five escape sequences. These are valid only inside the pair of double quotation marks. Any instance of a backslash that is not followed by any valid escape sequence character is invalid.
 - a. `\d` : Double quote
 - b. `\n` : Newline
 - c. `\t` : Horizontal tab
 - d. `\0` : Null
 - e. `\\` : Backslash

Examples

VALID	INVALID
"Hello"	'Hello'
" "	""
"\d"	""
"CFarers\n"	"CFarers\ns"
"CFarers\n, AHOY!"	"CFarers\s, AHOY!"
"She said, \dHello World\d"	"She said, \dHello World"\d
"Line 1\nLine2"	"Line 1 Line2"

Rules for Accessing SCROLL

1. Individual characters in a SCROLL are accessed using an index operator. It must start with a literal or variable of type SCROLL, followed by the index enclosed in curly braces ({ }).
2. The index indicates the position of the character in the SCROLL and starts at zero (0). The index must be a literal or variable of type COIN only and must not be empty or exceed the length of the SCROLL.

Syntax

<SCROLL-lit>{<index>}
<id>{<index>}

Examples

VALID	INVALID
word{1}	word
"Hello"{0}	"Hello" }
message{i}	message{ i+1 }
"Hello">{1}	"Hello">{ 1.5 }
name{index}	name{ index() }
word{x}	word[2]

- E. BOOL** - BOOL represents either of two values: AYE (true) or NAY (false).

Rules for BOOL

1. BOOL is restricted to only two values: AYE or NAY; otherwise, it is invalid.

Examples

VALID	INVALID
AYE	TRUE, True
NAY	FALSE, False
AYE	1
NAY	0
AYE	aye

F. ABYSS - Represents the absence of a value.

Rules for ABYSS

1. ABYSS is used as a return type for a nonreturning function to signify that the function will not return any value to the calling function.
2. It cannot be used to declare or call variables, constants, arrays, or structures.
3. The keyword ABYSS, as well as any function defined with it, cannot be used as operands or values inside any expression or assignment statement.

Examples

VALID	INVALID
ABYSS function() [ECHO("Hello")!!]	ABYSS function ()!!
ABYSS run() [ECHO("Go")!!]	ABYSS count = 0!!
	LOCKE ABYSS val = 0!!
	ABYSS arr {5}!!
	MAST item [ABYSS id!!]!!

IDENTIFIERS

Identifiers are designated names given to identify a variable, constant, array, structure, structure member or variable, or subfunction.

Rules in Naming Identifiers

1. An identifier must begin with a lowercase letter only. Subsequent characters may consist of a combination of lowercase letters, digits, and underscores only. The use of uppercase letters and any other special character is strictly prohibited.
2. It must consist of at least one (1) lowercase letter and at most twenty (20) valid characters.
3. An identifier must be unique within its defined scope. Variables, arrays, constants, structures, structure members, structure variables, and subfunctions

within the same scope, must each have a unique identifier. Declaring or defining with an identical identifier is invalid.

4. SeaStack allows identifier shadowing. A local identifier may share the same name as a global identifier. In such cases, the local declaration will take precedence and override within its local scope.
5. Reserved words are strictly not allowed to be used as identifiers.

Examples

VALID	INVALID
number	Number
age	_age
person1	pERSON1
middle_name	middle_name\$
this_is_a_twentychar	this_is_a_twentyfourchar
coin	COIN
COIN age!! DIME gwa!!	COIN age!! DIME age!!
COIN x!! AHOY() [DIME x!!]	AHOY() [COIN x!! COIN x!!]

VARIABLES

Variables are user-defined containers for storing values.

Rules in Declaring a Variable

1. Declaring a variable must begin with the data type, followed by the identifier, and end with the statement terminator (!!).
2. Multiple declarations of variables with the same data type are allowed and are separated with a comma (,).
3. Multiple variables with different data types cannot be declared at once.
4. Declared variables are undefined and cannot be used in the program until it is assigned a value.

Syntax

<dtype> <id>!!

<dtype> <id1>, <id2>, ..., <idN>!!

Examples

VALID	INVALID
COIN age!!	COIN age
DIME pi!!	pi DIME!!
SCROLL greet, welcome!!	SCROLL greet welcome !!
PARCH letter, digit!!	PARCH letter, COIN digit!!
BOOL flag!!	BOOL flag, AYE !!
BOOL power, level, flag!!	BOOL power && flag!!

Rules in Initializing a Variable

1. Initializing a variable must begin with the data type, followed by the identifier, the assignment operator (=), then the assigned value, and end with the statement terminator (!!).
2. The initial value assigned to a variable must be of the same data type as the variable. It can be a literal, variable, constant, array element, structure member, function return value, or expression.
3. Initialization and declaration of multiple variables with the same data type is allowed and is separated with a comma (,). Multiple variables with different data types cannot be initialized at once.

Syntax

<code><dtype> <id> = <value>!!</code>
<code><dtype> <id1> = <value>, ..., <idN> = <value>!!</code>
<code><dtype> <id1> = <value>, <id2>, ..., <idN> = <value>!!</code>

Examples

VALID	INVALID
COIN age = 9!!	COIN age = '9' !!
PARCH letter = x!!	PARCH letter = x
DIME sec = add(x,y), min = (2.2*3.3)!!	COIN sec = 1, DIME min = 2.2!!
SCROLL name = students{3}!!	SCROLL name = 1*2+3!!
BOOL flag = AYE, on = menu\$first!!	BOOL flag = AYE, on = 9!!
BOOL flag = AYE, choice, on = NAY!!	BOOL flag = AYE, choice, on = !!

CONSTANTS

Constants are fixed values. It must be given an initial value at the time of declaration, and this value must not be reassigned or modified after initialization.

Rules in Declaring and Initializing Constants

1. A constant must be initialized at the time of declaration; otherwise, it is invalid. It must begin with the reserved word LOCKE, followed by the data type, the identifier, the assignment operator (=), the assigned value, and end with the statement terminator (!!).
2. The value to be assigned to a constant must only be a literal matching the declared data type.
3. Any attempt to reassign a new value to a constant is invalid.
4. Initialization of multiple constants with the same data type is allowed and is separated with a comma (,). Multiple constants with different data types cannot be initialized at once.

Syntax

LOCKE <dtype> <id> = <value>!!
LOCKE <dtype> <id1> = <value>, ..., <idN> = <value>!!

Examples

VALID	INVALID
LOCKE DIME pi = 3.14!!	LOCKE DIME pi!!
LOCKE SCROLL name = "Juan"!!	LOCKE SCROLL name = 123 !!
LOCKE BOOL flag = AYE!!	LOCKE BOOL flag = 10 > x !!
LOCKE SCROLL language = "English", programming_language = "SeaStack"!!	LOCKE SCROLL name = "Jose", nickname = names() !!
LOCKE COIN max_score = 100!!	LOCKE COIN max_score = 100!! max_score = 150!!

ARRAYS

A fixed-size collection of elements of the same data type.

Rules for Arrays

1. The size of an array must be specified upon declaration with a COIN literal only. The size must be at least one (1).
2. The elements of an array can be a literal, variable, constant, array element, structure member, function return value, or expression, which evaluates to the declared data type of the array.
3. The number of elements assigned to an array must not exceed the declared size. If fewer elements are provided than the declared size, the remaining indices without a value will be assigned a value of null.

A. One-dimensional Array - A one-dimensional array is a collection of elements of the same data type in a single row or list of values.

Rules for One-dimensional Array

1. A one-dimensional array declaration begins with the data type, followed by the identifier, the size enclosed in curly braces ({}), and the statement terminator (!!). Only one array can be declared or initialized at once.
2. It can be initialized with the assignment operator (=), followed by square brackets ([]) enclosing the array elements which are separated by commas (,), and end with the statement terminator (!!).

Syntax

<code><dtype> <id>{<size>}!!</code>
<code><dtype> <id>{<size>} = [<value1>, ..., <valueN>]!!</code>

Examples

VALID	INVALID
COIN numbers{2}!!	COIN numbers{}!!
DIME average{3}!!	DIME arr1 {5}, arr2{5}!!
PARCH letters{3} = ['A', 'B', 'C']!!	PARCH letters{3} = ['A', 'B', 'C', ' D' ']!!
BOOL flag{2} = [3<5, status(a, c)]!!	BOOL flag{2} = [AYE, 1]!!

B. Two-dimensional Array - A two-dimensional array is an array where each element is another array.

Rules for Two-dimensional Array

1. A two-dimensional array declaration begins with the data type, followed by the identifier, then the size of the row and the column, each enclosed in curly braces

{ { } }, and ends with the statement terminator (!!). Only one array can be declared or initialized at once.

2. It can be initialized using the assignment operator (=), with square brackets enclosing the elements ([]), and each element being another set of array elements, with each set enclosed in square brackets ([]).

Syntax

<code><dtype> <id>{<rows>} {<columns>}!!</code>
<code><dtype> <id>{<rows>} {<columns>} = [[<r1c1>, ..., <r1cN>], ..., [<rNc1>, ..., <rNcN>]]!!</code>

Examples

VALID	INVALID
DIME grades{2}{2}!!	DIME grades{2}{ } !!
PARCH letters{1}{2}!!	PARCH letters{1}{ x }!!
COIN table{1}{2} = [[1, add(x,y)]]!!	COIN table{1}{2} = [[1, 3>5]]!!
COIN table{2}{3} = [[1, 1, 1],[2, 2, 2]]!!	COIN table{2}{1} = [[1], [2], [3]]!!
COIN table{1}{3} = [[1, 2+3, 3]]!!	COIN table{ 1 , 3 } = [1, 2, 3]!!

Rules for Accessing Array Elements

1. Array elements are stored consecutively in an index starting from zero (0).
2. Accessing an array element starts with the array identifier, followed by the array index enclosed in curly braces. The index must be a literal or variable of type COIN only and must not be empty or less than zero.
3. Accessing array elements can be used to modify or assign a value to the specified index.

Syntax

<code><id>{<index>}</code>
<code><id>{<index>}{<index>}</code>

Examples

VALID	INVALID
<code>grade{3}</code>	<code>grade{}</code>
<code>nums{1}{2}</code>	<code>nums[1][2]</code>
<code>numbers{2}</code>	<code>numbers{2.5}</code>
<code>nums{x}</code>	<code>nums{get_num()}</code>
<code>table{i}{j}</code>	<code>table{i, j}</code>
<code>average{i}</code>	<code>average{COIN}</code>

STRUCTURES

A collection of several related variables of possibly different types. The items in a structure are called members, and they can be of any valid data type.

Rules for Defining Structures

1. Defining a structure begins with the reserved word **MAST**, followed by a unique identifier, then the member declarations enclosed within the square brackets ([]), and ends with the statement terminator (!!). It is defined after the declaration of any global constant, variable, or array, and before any function.
2. Declaring a member starts with the data type, followed by the identifier, and ends with the statement terminator (!!). There must be at least one member declaration in a structure definition.
3. Multiple members of the same data type can be declared at once by separating each member with a comma (,). Multiple members of different data types cannot be declared at once.

Syntax

<pre>MAST <id> [<dtype> <id>!!]!!</pre>
<pre>MAST <id> [<dtype> <id1>, <id2>, ..., <idN>!!]!!</pre>
<pre>MAST <id> [<dtype1> <id1>!! ... <dtypeN> <idN>!!]!!</pre>

Examples

VALID	INVALID
MAST person [SCROLL name!!]!!	person [SCROLL name!!]!!
MAST person [COIN age!!]!!	MAST person []!!
MAST person [COIN age, year!!]!!	MAST person [COIN age, DIME year!!]!!
MAST item [DIME price!!]!!	MAST item [DIME price]!!
MAST student [COIN age!! DIME gwa!!]!!	MAST student [COIN age!! DIME gwa!! I

Rules for Declaring Structure Variables

1. Declaring a structure variable must begin with the reserved word MAST, followed by the structure member identifier, then the structure variable identifier, and the statement terminator (!!). It can only be declared inside the AHOY function.
2. Multiple structure variables can be declared at once by separating each variable with a comma (,).

Syntax

MAST <struct_id> <var_id>!!
MAST <struct_id> <var_id1>, <var_id2>, ..., <var_idN>!!

Example

VALID	INVALID
-------	---------

MAST book book1!!	MAST book book1, MAST employee !!
MAST employee e1, e2, e3!!	MAST employee e (1+3) !!
MAST student s1, s2, s3!!	MAST S tudent s1!!
MAST book novel!!	MAST SCROLL book novell!!
MAST car c1, c2!!	MAST car c1 c2 !!

Rules for Initializing Structure Variables

1. Initializing a structure variable must begin with the reserved word MAST, followed by the structure identifier, the structure variable identifier, the assignment operator (=), then the assigned values enclosed in square brackets ([]), and end with the statement terminator (!!).
2. Only one structure variable can be initialized at once.
3. The value assigned to the members of a structure variable can be a literal, variable, constant, array element, structure member, function return value, or expression, which evaluates to the declared data type of the member.
4. Values are assigned sequentially to the members in the order of their declaration in the structure definition. This sequence is maintained until the initialization block ends or a member operator is encountered.
5. To initialize a specific member, start with the member operator (\$), followed by the member identifier, then the assignment operator, and the assigned value. The succeeding value will be assigned to the next member, unless specified otherwise with another member operator.
6. A member can only be left uninitialized if all succeeding members are also uninitialized, unless a member operator is used to skip the order. Uninitialized members will be assigned a value of null.
7. Providing a number of initial values that exceeds the number of defined members is invalid.

Syntax

MAST <struct_id> <var_id> = [<value>]!!
MAST <struct_id> <var_id> = [<value1>, ..., <valueN>]!!
MAST <struct_id> <var_id> = [\$<mem_id> = <value>]!!
MAST <struct_id> <var_id> = [\$<mem_id1> = <value1>, ..., \$<mem_idN> = <valueN>]!!

Examples

VALID	INVALID
MAST vehicle truck = ["Foton", "12-wheeler"]!!	vehicle truck = ["Foton", "12-wheeler"]!!
MAST blk3 s1 = ["Liza", 19, 1.00]!!	MAST blk3 s1 = ["Liza"], s2 = ["Ann"] !!
MAST movies movie1 = [\$title = "Fantastic Mr. Fox"]!!	MAST movies movie1 = [title = "Fantastic Mr. Fox"]!!
MAST books book1 = [\$title = "Gone Girl", "Mystery", \$price = 499.99]!!	MAST books book1 = [\$title = "Gone Girl", "Mystery", COIN x!!]!!
MAST animal insect = ["spider", "8 legs"]!!	MAST animal reptile, mammal , insect = ["spider", "8 legs"]!!
	MAST animal insect = {"spider", "8 legs"}!!

Rules for Accessing Members

1. To access a member, it must start with the structure variable identifier, followed by the member operator (\$), and then the member identifier.
2. Accessing members can be used to modify or assign a value to the specific member.

Syntax

<code><var_id>\$<mem_id></code>

Example

VALID	INVALID
student1\$name	student1.name
employee1\$salary	employee1\$
student1\$age	student1\$MAST
employee1\$role	employee1()\$role
student1\$age = 20!!	MAST\$age = 20!!

OPERATORS

Operators are symbols that provide instructions on how to perform arithmetic, relational, and logical calculations, concatenate SCROLL values, or perform other special operations. SeaStack supports arithmetic, assignment, unary, relational, logical, and other operators.

Rules for Operators

1. Seastack operators follow a standard operator precedence table.
2. Operators must not be used consecutively without any operand in between, unless the second operator is the unary minus (-), logical not (!), or logical double not (!#).
3. A dash sign (-) that is immediately followed by a digit, without any whitespace in between, is always considered as a negative digit. Otherwise, it is treated as a subtraction or unary minus operator.
4. Arithmetic operators follow the PEMDAS order, where operations inside parentheses are evaluated first, followed by exponents, then multiplication, division, or modulo, and finally, addition or subtraction.
5. Logical operators are evaluated using short-circuit rules, meaning that in logical AND, if the first operand evaluates to false, the second operand is not evaluated, and in logical OR, if the first operand evaluates to true, the second operand is not evaluated.
6. The modulo operation with negative values preserves the sign of the dividend or left operand.

A. Arithmetic Operators - Used to perform basic mathematical operations.

OPERATOR	DEFINITION
+	Adds operands.
-	Subtracts operands.
*	Multiplies operands.
/	Divides operands.
%	Gets the remainder from dividing operands.
^	Exponentiates an operand.

B. Assignment Operators - Used to assign values or update existing values.

OPERATOR	DEFINITION
=	Assignment operator. Assigns the value of the

	right-operand to the left-operand.
<code>+=</code>	Add and Assign operator. It adds the right operand to the left operand and assigns the result to the left operand.
<code>-=</code>	Subtract and Assign operator. It subtracts the right operand from the left operand and assigns the result to the left operand.
<code>*=</code>	Multiply and Assign operator. It multiplies the left operand by the right operand and assigns the result to the left operand.
<code>/=</code>	Divide and Assign operator. It divides the left operand by the right operand and assigns the result to the left operand.
<code>%=</code>	Modulo and Assign operator. It divides the left operand by the right operand and assigns the remainder to the left operand.
<code>^=</code>	Exponent and Assign operator. Raises the left operand to the power of the right operand and assigns the result to the left operand.

C. Unary Operators - Used to perform an operation on a single operand.

OPERATOR	DEFINITION
<code>+#</code>	Used to increment an operand.
<code>-#</code>	Used to decrement an operand.
<code>-</code>	Used to negate an operand.

D. Relational Operators - Used to compare two values and returns a BOOL value.

OPERATOR	DEFINITION
<	Returns AYE if the left operand is less than the right operand. Otherwise, returns NAY.
>	Returns AYE if the left operand is greater than the right operand. Otherwise, returns NAY.
>=	Returns AYE if the left operand is greater than or equal to the right. Otherwise, returns NAY.
<=	Returns AYE if the left operand is less than or equal to the right. Otherwise, returns NAY.
==	Returns AYE if the left operand is equal to the right operand. Otherwise, it returns NAY.
!=	Returns AYE if the left operand is not equal to the right operand. Otherwise, returns NAY.

E. Logical Operators - Used to combine or negate bool expressions.

Logical Operators	
RESERVED SYMBOLS	DEFINITION
&&	Logical AND. Returns AYE if both operands evaluate to true. Otherwise, it returns NAY.
	Logical OR. Returns AYE if at least one operand evaluates to true. Otherwise, it returns NAY.
!	Logical NOT. Returns AYE if the operand evaluates to false, and NAY if it evaluates to true.
!#	Logical DOUBLE NOT. Returns AYE if the operand evaluates to true, and NAY if it evaluates to false.

F. Other Operators - Used to perform a specific function.

OPERATOR	DEFINITION
&	Concatenation operator. Used to concatenate values of type SCROLL.
@	Address operator. Used to access the address of an operand.
\$	Member operator. Used to access a member of a structure.
{}	Index operator. Used to access an element of an array or a character of a SCROLL literal.
()	Call operator. Used as a placeholder of arguments to be passed to a function call.

OPERATOR PRECEDENCE

Precedence	Operator	Operation	Associativity
1	()	Function Call, Grouping	Left to Right
	{ }	Index Accessing	
	\$	Member Accessing	
2	+#, -#	Increment, Decrement	Right to Left
	!, !#	Logical Not, Logical Double Not	
	-	Unary Minus	
3	^	Exponent	Right to Left
4	*, /, %	Multiply, Divide, Modulo	Left to Right
5	+, -, &	Add, Subtract, Concatenation	
6	<, >	Less than, Greater than	
	<=, >=	Less than or Equal, Greater than or Equal	
7	==, !=	Equality, Inequality	
8	&&	Logical And	
9		Logical Or	
10	=	Assignment	
	+=, -=, *=, /=, %=, ^=	Compound Assignment	

EXPRESSIONS

Expressions combine operands with operators to produce a single value.

A. Arithmetic Expressions - It performs mathematical calculations.

Rules for Arithmetic Expressions

1. An arithmetic expression must begin with an operand, followed by an arithmetic operator, and end with an operand.
2. An arithmetic expression must contain at least one arithmetic operator and two operands of type COIN or DIME.
3. Operands may be literals, variables, constants, array elements, structure members, function return values, or grouped expressions that evaluate to a COIN or DIME only.
4. A dash sign (-) preceding a digit, without any whitespace in between, is always considered as a negative digit. If the dash sign is not preceding a digit without any intervening whitespace, and is preceded by an operand, it is treated as a subtraction operator. Otherwise, it is treated as a unary minus operator.
5. A unary minus operator is used to negate a variable, constant, array element, structure member, function return value, or grouped arithmetic expression.

Syntax

$\langle \text{operand} \rangle \langle \text{arith_op} \rangle \langle \text{operand} \rangle$
$\langle \text{operand1} \rangle \langle \text{arith_op} \rangle \langle \text{operand2} \rangle \dots \langle \text{arith_op} \rangle \langle \text{operandN} \rangle$

Examples

VALID	INVALID
$2 * (x + 3)$	$5 + * 2$
$5 - 3$	$5 - 3$
$\text{items}\{5\} * 10$	$\text{items}\{5\} + \text{"USD"}$
$(x * y + z) / (x - y)$	$(x * y + z) / ()$
$- x + 5.0$	$* x + 5.0$

B. Relational Expressions - It performs comparison of two operands.

Rules for Relational Expressions

1. A relational expression must begin with an operand, followed by a relational operator, and end with an operand.
2. A relational expression must contain exactly one relational operator and two operands.

3. For less than (<), greater than (>), less than or equal (<=), or greater than or equal (>=) operators, the operands must only be literals, variables, constants, array elements, structure members, function return values, arithmetic expressions, or grouped expressions that evaluate to a COIN or DIME only.
4. For equal to (==) and not equal to (!=) operators, the operands must only be literals, variables, constants, array elements, structure members, function return values, or grouped expressions. The operands must have the same data type, except for COIN and DIME values; otherwise, it is invalid. COIN and DIME values can be compared with each other using these operators.

Syntax

<code><operand> <rel_op> <operand></code>

Examples

VALID	INVALID
<code>(a + b) >= (a * b)</code>	<code>>= (a + b) (a * b)</code>
<code>5 < 20</code>	<code>5 < 10 < 20</code>
<code>4 <= (val * 2)</code>	<code>username > "admin"</code>
<code>1 == 1.5</code>	<code>"Hello" == AYE</code>
<code>val >= flag</code>	<code>val >> flag</code>

C. Logical Expressions - It performs comparison of BOOL operands.

Rules for Logical Expressions

1. For the logical not (!) and logical double not (!#) operators, it must start with exactly one operator followed by a single operand.
2. For the logical or (||) and logical and (&&) operators, it must start with an operand, followed by the logical operator, and end with an operand.
3. A logical expression must contain at least one logical operator, and its operands must only be literals, variables, constants, array elements, structure members, function return values, relational expressions, or grouped expressions that evaluate to a BOOL.

Syntax

<code><operand> <log_op> <operand></code>
<code><operand1> <log_op> <operand2> ... <log_op> <operandN></code>
<code>!<operand></code>
<code>!#<operand></code>

Examples

VALID	INVALID
<code>num1 > 0 && num2 < 10</code>	<code>&& (num1 > 0)</code>
<code>!NAY</code>	<code>AYE!NAY</code>
<code>count > 5 count < 100</code>	<code>(count > 5) (count < 100)</code>
<code>3 + 5 > 6 && 4 == 3</code>	<code>3 + 5 > 6 && 4</code>
<code>x > 5 && y < 10 !! 3 == 6</code>	<code>(x > 5) &&&</code>
<code>a && b c && d</code>	<code>a && b c && d</code>

D. Unary Expressions - It performs increment or decrement.

Rules for Unary Expressions

1. A unary expression must start with the unary operator followed by a single operand. Any whitespace between the operator and the operand is invalid.
2. The operand must be a variable, array element, or structure member of type COIN only.
3. A unary expression may appear either as part of a condition in a HOIST statement or as a standalone statement ending with the statement terminator (!!).

Syntax

<code><unary_op><operand></code>
--

Examples

VALID	INVALID
<code>-#x</code>	<code>-# x</code>
<code>+#count</code>	<code>++count</code>
<code>-#value</code>	<code>value-#</code>
<code>+#age{3}</code>	<code>+#(5+3)</code>
<code>-#flag\$count</code>	<code>+#9</code>

- E. SCROLL Expressions** - A combination of SCROLL values combined with the concatenation operator.

Rules for SCROLL Expressions

1. A SCROLL expression must begin with an operand, followed by the concatenation operator (&), and end with an operand.
2. Operands can be literals, variables, constants, array elements, structure members, function return values, grouped SCROLL expressions, or SCROLL expressions of type SCROLL.
3. SCROLL expressions are evaluated from left to right and return a single SCROLL value formed by appending the operands in order without altering any character.

Syntax

<operand> & <operand>
<operand1> & <operand2> ... & <operandN>

Examples

VALID	INVALID
"Hi" & "There"	"Hi" "There"
"Name: "&name1&" "&name2	"Name: " & name1 name2
("Good" & "Morning") & "Boss"	"Good & "Morning" & "Boss"
("Sea" & "Stack") & " " & "Year" & " " & "2026"	("Sea" & "Stack") &&& "Year" & ""& "2026"
"Hello" & "There"	"Hello" + "There"
greeting & " Boss"	greeting & !!

STATEMENTS

Statements are instructions that the computer executes. SeaStack supports input, output, assignment, conditional, looping, and jump statements.

A. Input Statements - Used to obtain user input.

Rules for Input Statements

1. It must start with the reserved word ASK, followed by the argument list enclosed in parentheses, and end with the statement terminator (!!).
2. The first argument consists of the format specifier/s only enclosed in double quotation marks, followed by a comma, then the second argument which is the addressing of target variable, array element, or structure member.
3. Format specifiers must begin with a percentage symbol (%), followed by the first character of the data type (%C: COIN, %D: DIME, %P: PARCH, %S: SCROLL, %B: BOOL). It must strictly match the data type, quantity, and order of the targets in the second argument, and must not be separated by any comma.
4. Addressing the target variable, array element, or structure member starts with the address symbol (@) followed by the identifier. Multiple target variables must be separated with a comma.

Syntax

ASK("<format_specifier>", @<id>)!!
ASK("<format_specifier1>...<format_specifierN>", @<id1>, ..., @<idN>)!!

Examples

VALID	INVALID
ASK("%C", @score)!!	ASK("%C", score)!!
ASK("%D%C", @gwa, @score)!!	ASK("%C%D", @cost)!!
ASK("%C%D%D", @age, @weight, @height)!!	ASK(" %s ", @name)!!
ASK("%C%C", @score, @total)!!	ASK(" %C, %C ", @score, @total)!!
ASK("%D", @gwa)!!	ASK("%D" @gwa)!!
ASK("%C", @player\$hp)!!	ASK("%C", @player.hp)!!

B. Output Statements - Used to display an output.

Rules for Output Statements

1. It must start with the reserved word ECHO, followed by the argument list enclosed in parentheses, and end with the statement terminator (!!).
2. The first argument is the text to be displayed and must be a SCROLL literal. It can be combined with format specifier/s which must be placed within the SCROLL literal.
3. A format specifier is used as a placeholder of a value to be displayed. If the first argument does not have any format specifier, it must not have a second argument.
4. The value/s in the second argument can be a literal, variable, constant, array element, structure member, function return value, or expression. The value/s must strictly match the data type, quantity, and order of the format specifier/s in the SCROLL literal. Multiple values must be separated with commas.

Syntax

ECHO("<SCROLL-lit>")!!
ECHO("<SCROLL-lit>", <value>)!!

Examples

VALID	INVALID
ECHO("Enter your age: ")!!	ECHO(Enter your age)!!
ECHO("Your score: %C", score)!!	ECHO("Your score: %C ")!!
ECHO("Hello\nWorld")!!	ECHO("Hello" & greet)!!
ECHO("Your name: %S", name)!!	ECHO("Your name: name", %S)!!
ECHO("%C+%C = %C", a, b, sum)!!	ECHO("%C+%C = %C", a, b, sum)

C. Assignment Statements - Used to assign a value to the operand.

Rules for Assignment Statements

1. It must start with an operand, then the assignment operator, the value to be assigned, and end with the statement terminator (!!). The operand must only be a declared variable, array element, or structure member.
2. For the standard assignment operator (=), the operand can be any data type, and the value to be assigned must be a literal, variable, constant, array element, structure member, function return value, expression, or grouped expression of the same data type as the operand.
3. For the compound assignment operators (+=, -=, *=, /=, %=, ^=), the operand must be of type COIN or DIME. The value to be assigned must be a literal, variable, constant, array element, structure member, function return value, or arithmetic expression that evaluate to a COIN or DIME.

Syntax

`<operand> <assignment_op> <value>!!`

Examples

VALID	INVALID
num1 = 10*2!!	num1 = 10
flag = AYE && 3 > (5+4)!!	9 = num2!!
username = "captain_01"!!	username() = "hi"!!
total *= 9!!	total * 9 = 5!!
height *= 3+5/(4-6)!!	height *= "tall" !!

D. Conditional Statements - Used to execute different blocks of code based on a given condition. The reserved words LOOK, DROP, DROPLOOK, and CHART are used for conditional statements.

a. LOOK - Executes the following block of statements if the given condition is true; otherwise, the block of statements is ignored.

Rules in LOOK Statements

1. It must start with the reserved word LOOK, then a condition enclosed in parentheses (()), then the body enclosed in square brackets ([]).

- The condition must be a literal, variable, constant, array element, structure member, function return value, relational expression, or grouped expression that evaluates to a BOOL data type. It must not be empty.
- The body supports all statements, except BACK statements, and can be empty. LAND and SAIL statements are optional and the body can only have either one located at the end.

Syntax

LOOK (<condition>) [<statement/s>]
LOOK (<condition>) []

Examples

VALID	INVALID
LOOK (b > 5) [ECHO("b equals 5")!!]	LOOK (COIN b = 5) [ECHO("Invalid condition")!!]
LOOK (AYE) [ECHO("Always true")!! SAIL!!]	LOOK (AYE) [SAIL!! ECHO("Always true")!!]
LOOK (3+5>6 4 < 5) []	LOOK (x = 10) []
LOOK (is_valid == AYE) [ECHO("Granted")!!]	LOOK (0) [ECHO("Empty")!!]
LOOK (score >= 90) [ECHO("Excellent")!!]	LOOK [ECHO("True")!!]

b. DROP - Executes the block of statements under DROP only if the preceding LOOK or DROLOOK conditions does not evaluate to true.

Rules in DROP Statements

- A DROP statement must have an initial LOOK statement and can be preceded by a DROLOOK statement.

2. The reserved word DROP must be followed by the body enclosed in square brackets ([]).
3. The body supports all statements, except BACK statements, and can be empty. LAND and SAIL statements are optional and the body can only have either one located at the end.

Syntax

<pre>LOOK (<condition>) [<statement/s>] DROP [<statement/s>]</pre>
<pre>LOOK (<condition>) [] DROP []</pre>

Examples

VALID	INVALID
<pre>LOOK (is_rainy) [] DROP []</pre>	<pre>LOOK (x > 5) [ECHO("x is greater")!!] DROP (x<5) []</pre>
<pre>LOOK (age >= 18) [ECHO("You are old.\n")!!] DROP [ECHO("You are not old.\n")!!]</pre>	<pre>LOOK (age >= 18) [] DROP [SAIL!! LAND!!]</pre>
<pre>LOOK (flag) [] DROP [ECHO("ON.")!!]</pre>	<pre>LOOK (is_safe) [] DROP [BACK !!]</pre>
<pre>LOOK (flag) [DROP [ECHO("ON.")!!]]</pre>	<pre>DROP [ECHO("ON.")!!]</pre>

<pre>LOOK (is_rainy) [] DROP []</pre>	<pre>LOOK (is_rainy) [] DROP ECHO("No Brackets")!!</pre>
---	--

c. DROPLOOK - Evaluates the conditions from top to bottom. If the condition does not evaluate to true, it proceeds to check the next condition. Once a condition is evaluated to be true, it executes the corresponding block of statements and skips the rest of the conditions.

Rules in DROPLOOK Statements

1. A DROPLOOK statement must have an initial LOOK statement and can be preceded by another DROPLOOK statement. It can be followed by another DROPLOOK statement or an optional final DROP statement.
2. The reserved word DROPLOOK must be followed by a condition enclosed in parentheses (()), then the body enclosed in square brackets ([]).
3. The condition must be a literal, variable, constant, array element, structure member, function return value, relational expression, or grouped expression that evaluates to a BOOL data type. It must not be empty.
4. The body supports all statements, except BACK statements, and can be empty. LAND and SAIL statements are optional and the body can only have either one located at the end.

Syntax

<pre>LOOK (<condition1>) [<statement/s>] DROPLOOK (<condition2>) [<statement/s>] ... DROPLOOK (<conditionN>) [<statement/s>]</pre>
<pre>LOOK (<condition1>) [<statement/s>] ... DROPLOOK (<conditionN>) [<statement/s>] DROP [<statement/s>]</pre>
<pre>LOOK (<condition1>) [] DROPLOOK (<condition2>) [] DROP [</pre>

ALISWAG, K. V. J.

ALTEZA, J. R. D.

CAYACAP, F. A. S.

DEL ROSARIO, J. M. A.

MAGAAN, F. M. V.

MULLENO, J. M. A.



SeaStack

]

Examples

VALID	INVALID
<pre>LOOK (score >= 90) [ECHO("Excellent.")!!] DROPLOOK (score >= 75) [ECHO("Passed.")!!] DROP [ECHO("Failed.")!!]</pre>	<pre>DROPLOOK (score >= 75) [ECHO("Missing LOOK")!!] DROP [ECHO("Failed.")!!]</pre>
<pre>LOOK (temp > 37.5) [ECHO("Very hot")!!] DROPLOOK (temp > 30.5) [ECHO("Hot")!!] DROPLOOK (temp > 20.5) [ECHO("Warm")!!]</pre>	<pre>LOOK () [ECHO("No Condition")!!] DROPLOOK () [ECHO("No Condition")!!] DROPLOOK () [ECHO("No Condition")!!]</pre>
<pre>LOOK (grade == 'A') [ECHO("Outstanding")!!] DROPLOOK (grade == 'B') [ECHO("Good")!! LAND!!]</pre>	<pre>LOOK (grade == 'A') [ECHO("Outstanding")!!] DROPLOOK (grade == 'B') [ECHO("Good")!! BACK!!]</pre>
<pre>LOOK (condition{0}) [ECHO("True")!!] DROPLOOK (condition) [ECHO("False")!!] DROP []</pre>	<pre>LOOK (3+6) [ECHO("Not Boolean")!!] DROPLOOK () [ECHO("No Condition")!!] DROP []</pre>
<pre>LOOK (temp > 100) [ECHO("Boiling")!!] DROPLOOK (temp > 0 && temp <= 100) [ECHO("Liquid")!!]</pre>	<pre>LOOK (temp > 100) [ECHO("Boiling")!!] DROPLOOK (COIN x) [ECHO("Wrong Cndtn")!!]</pre>

d. Chart Statements - Executes different parts of the code based on the value of a single variable.

Rules in CHART Statements

1. It must start with the keyword CHART, followed by a condition enclosed in parentheses (()), and end with the body enclosed in square brackets ([]).
2. The condition of a CHART statement must be a literal or variable of type COIN, PARCH, or SCROLL only and must not be empty.
3. The body of a CHART statement must consist of at least one COURSE body and can end with a single optional ADRIFT body.
4. Each COURSE body must start with the keyword COURSE, followed by a unique label, then a colon symbol (:), followed by the statements to be executed. The label must be a COIN, PARCH, or SCROLL literal. Accessing a SCROLL character is allowed given that the index is a COIN literal that is within the length of the SCROLL.
5. The body supports all statements, except BACK statements, and can be empty. LAND and SAIL statements are optional and the body can only have either one located at the end.
6. The optional ADRIFT body must start with the keyword ADRIFT, followed by a colon (:), then the statements to be executed. It supports all statements, except BACK and SAIL statements, and must have a LAND statement at the end.

Syntax

<pre> CHART (<condition>) [COURSE label: <statement/s>]</pre>
<pre> CHART (<condition>) [COURSE label1: <statement/s> ... COURSE labelN: <statement/s> ADRIFT: <statement/s> LAND!!]</pre>
<pre> CHART (<condition>) [COURSE label:]</pre>

Examples

VALID	INVALID
<pre> CHART (day) [COURSE 1: ECHO("Monday\n")!!</pre>	<pre> CHART (night, day) [COURSE 1: ECHO("Invalid Condition")!!</pre>

LAND!!]	LAND!!]
CHART (display) [COURSE 1: ECHO("Hello")!!]	CHART (letter) [COURSE a : ECHO("Variable as label")!! LAND!!
CHART (menu) [COURSE 1: ECHO("File Menu")!! SAIL!!]	CHART (menu) [ADRIFT : ECHO("No COURSE body")!! LAND!!]
CHART (letter) [COURSE 'A': ECHO("Excellent")!! LAND!! ADRIFT: ECHO("Below Average")!! LAND!!]	CHART (digit) [COURSE 3: ECHO("Variable as label")!! LAND!! ADRIFT: ECHO("Variable as label")!! SAIL!!]
CHART (grade) [COURSE 'A': ECHO("Pass")!! LAND!! ADRIFT: LAND!!]	CHART (grade) [COURSE 'A': ECHO("Pass")!! LAND!! ADRIFT: BACK!]

E. Looping Statements - A looping statement executes a block of statements repeatedly until a given condition is satisfied. The reserved words HOIST, HEAVE, and HAUL-HEAVE are used for looping statements.

a. HOIST - Iterates over a specific range of numbers.

Rules in HOIST Statement

1. It must always begin with a reserved word HOIST, followed by the condition enclosed in parentheses (()), and end with the body enclosed in square brackets ([]). The HOIST condition has three arguments: initialization, condition, and update.
2. The initialization argument defines the starting state of the loop. The value assigned in the initialization must be a literal, variable, constant, array element,

structure member, function return value, or arithmetic expression evaluating to type COIN only. It can be any of the three ways:

- a. Initialization of a COIN variable starting with the reserved word COIN, followed by the identifier, the assignment operator (=), the value, and the statement terminator (!!). Multiple initialization is allowed and must be separated with commas.
 - b. Assignment of a COIN value starting with the operand (variable, array element, or structure member), followed by the assignment operator, the value, and the statement terminator. Multiple assignments are allowed and must be separated with commas.
 - c. A lone statement terminator.
3. The condition is checked before each iteration. If the condition evaluates to NAY (false), the loop terminates; otherwise, the loop continues. The condition must be a relational expression wherein the operands can only be literals, variables, constants, array elements, structure members, function return values, or arithmetic expressions of type COIN or DIME, and it must not be empty. Multiple relational expressions are allowed and must strictly be separated with logical operators.
 4. The update argument runs after each iteration before re-evaluating the condition. It supports increment or decrement through a unary expression or assignment of values using a compound assignment operator. Multiple updates are allowed and must be separated with commas.
 5. The body supports all statements, except BACK statements, and can be empty. LAND and SAIL statements are optional and the body can only have either one located at the end.

Syntax

HOIST (<initialization>!! <condition>!! <update>) [<statement/s>]
HOIST (<initialization>!! <condition>!! <update>) []
Initialization
<dtype> <id> = <value>
<id> = <value>
λ
Condition

<operand><rel_op><operand>
<rel_exp><log_op><rel_exp>
Update
<unary_op><id>
<id> <compound_assign_op> <value>

Examples

VALID	INVALID
HOIST (COIN i = 0!! i < 5!! +#i) []	HOIST (COIN i = 0 i < 5 i++) []
HOIST (i{4} = 0!! i{4}<5!! +#i{4}) [ECHO("i = %C, j = %C\n", i, j)!!]	HOIST (COIN i = 0!! i--) [ECHO("No condition")!!]
HOIST (!! x > 0!! x+=1) [ECHO("Countdown: %C\n", x)!!]	HOIST() [ECHO("empty condition")!!]
HOIST (COIN i = 0, j = 10!! i < j!! +#i, -#j) []	HOIST (COIN i = 0, x = "start"!! i < j!! +#i, -#j) []
HOIST (COIN i = 0!! i < 10!! +#i) []	HOIST (COIN i = 0!! i < 10!! +#i) [BACK!!]
HOIST (COIN i = 0, j = 10!! i < 10!! +#i, +#j) [ECHO("i is %C, j is %C\n", i, j)!!]	HOIST (COIN i = 0, j = 10!! i < 10!! +#i+#j) [ECHO("i is %C, j is %C\n", i, j)!!]
HOIST (COIN i = 0 !! i < 10 && signal == 1 !! +#i) [ECHO("The signal is %C", signal)!!]	HOIST (COIN i = 0!! i < 10!! +#i) [BACK!!]
HOIST (COIN i = 1 !! i + 1 < 10 && flag == 1 !! +#i) [ECHO("Iteration: %C\n", i)!!]	HOIST (COIN i = 0, i < 10, +#i) []

b. HEAVE - Executes the block of statements repeatedly while the given condition evaluates to true.

Rules in HEAVE Statements

1. It must always begin with the reserved word HEAVE, followed by the condition enclosed in parentheses (()), then the body enclosed in square brackets ([]).
2. The condition must be a literal, variable, constant, array element, structure member, function return value, relational expression, or grouped expression that evaluates to a BOOL data type. It must not be empty.
3. The body supports all statements, except BACK statements, and can be empty. LAND and SAIL statements are optional and the body can only have either one located at the end.

Syntax

<pre>HEAVE (<condition>) [<statement/s>]</pre>
<pre>HEAVE (<condition>) []</pre>

Examples

VALID	INVALID
<pre>HEAVE (i < 5) [ECHO("i = %C\n", i)!! +#i!!]</pre>	<pre>HEAVE (i < 5)!! [i++!!]</pre>
<pre>HEAVE (choice != 3) [ECHO("Enter 3 to exit: ")!! ASK("%C", @choice)!!]</pre>	<pre>HEAVE choice !=3 [ECHO("Enter 3 to exit: ")!! ASK("%C", @choice)]</pre>
<pre>HEAVE (count > 0) [ECHO("Count: %C\n", count)!! -#count!!]</pre>	<pre>HEAVE 0 [ECHO("Empty condition")!!]</pre>
<pre>HEAVE (player\$hp > 0) []</pre>	<pre>HEAVE (100) []</pre>
<pre>HEAVE (fuel > 0) [ECHO("Flying...")!! fuel -= 10 !! SAIL !!]</pre>	<pre>HEAVE (fuel > 0) [ECHO("Flying...")!! fuel -= 10 !! BACK!!]</pre>

c. HAUL-HEAVE - Executes the block of statements at least once, then executes it repeatedly as long as the condition is AYE (**true**), and terminates once the condition evaluates to NAY (**false**).

Rules in HAUL-HEAVE Statements

1. The HAUL-HEAVE statement begins with the HAUL keyword, followed by the HAUL body enclosed in square brackets ([]).
2. The HAUL body must be followed by the HEAVE keyword, a condition enclosed in parentheses (()), and end with the statement terminator (!!).
3. The condition must be a literal, variable, constant, array element, structure member, function return value, relational expression, or grouped expression that evaluates to a BOOL data type. It must not be empty.
4. The body supports all statements, except BACK statements, and can be empty. LAND and SAIL statements are optional and the body can only have either one located at the end.

Syntax

HAUL [<statement/s>] HEAVE (<condition>)!!
HAUL [] HEAVE (<condition>)!!

Examples

VALID	INVALID
HAUL [ECHO("i = %C\n", i)!! +#i!!] HEAVE (i < 5)!!	HAUL [ECHO("No condition")!!] HEAVE!!
HAUL [+#x!!] HEAVE (x < 10)!!	HAUL [+#x!!] HEAVE (x + 10)!!
HAUL [ECHO("Processing...")!! SAIL !!] HEAVE (AYE)!!	HAUL [BACK !!] HEAVE (AYE)!!
HAUL [ECHO("Processing...")!!] HEAVE (tries < 4)!!	HAUL [ECHO("Processing...")!!] HEAVE [tries < 4]!!
HAUL [ECHO("Processing...")!!] HEAVE (is_waiting)!!	HAUL [ECHO("Processing...")!!] HEAVE (is_waiting)

F. Jump Statements - A jump statement immediately jumps to execute a block of code in another part of the program.

a. LAND Statement - It uses the reserved word LAND to immediately exit or terminate a conditional or looping statement.

Rules for LAND Statements

1. The LAND statement must begin with the reserved word LAND and end with the statement terminator (!!).
2. It is required to end an ADRIFT body. For other conditional and looping statements, it is optional and can only be placed at the end of the body. Otherwise, it is considered invalid.

Syntax

LAND!!

Examples

VALID	INVALID
LAND!!	LAND(1<3)!!
	Land!!
AHOY()[LOOK(3>5) [LAND!!]]	AHOY() [LAND!!]
AHOY() [CHART (day) [COURSE 1: ECHO("Monday")!! LAND!! ADRIFT: ECHO("Noramal Day")!! LAND!!]]	AHOY()[LOOK(3>5) [] LAND!!]
	AHOY() [LOOK (3 > 5) [LAND!! ECHO ("Land must be last")!!]]

b. SAIL Statement - It uses the reserved word SAIL to skip the rest of the current iteration and continue to the next iteration.

Rules for SAIL Statements

1. The SAIL statement must begin with the reserved word SAIL and end with the statement terminator (!!).
2. It must not be used in an ADRIFT body. For other conditional and looping statements, it is optional and can only be placed at the end of the body. Otherwise, it is considered invalid.

Syntax

SAIL!!

Examples

VALID	INVALID
SAIL!!	SAIL id !!
	Sail !!
AHOY() LOOK(3>5) [SAIL!!]]	AHOY() LOOK(3>5) [] SAIL!!]]
LOOK(x > y) [ECHO("Correct placement")!! SAIL!!]	AHOY() [LOOK(3 > 5) [SAIL!! ECHO ("Sail must be last")!!]]
AHOY() [CHART(anchor) [COURSE 1: ADRIFT: ECHO("This is valid.")!! LAND!!]]	AHOY() [CHART(anchor) [COURSE 1: ADRIFT: SAIL!! LAND!!]]

c. BACK Statement - It uses the reserved word BACK to terminate the execution of a function. It may be used to return a value to the calling function.

Rules for BACK Statements

1. A BACK statement must begin with the reserved word BACK, followed by an optional return value, and end with the statement terminator (!!).
2. For returning functions:

-
- a. A BACK statement is mandatory at the end of the body.
 - b. The keyword BACK must be followed by the return value. It must be a literal, variable, constant, array element, structure member, function return value, or expression of type COIN, DIME, BOOL, PARCH, or SCROLL only, and end with a statement terminator (!!).
 - c. The data type of the returned value must match the function's declared return type.
 3. For non-returning functions:
 - a. A BACK statement is optional.
 - b. The keyword BACK must be followed by the statement terminator (!!).
Indicating a return value is invalid.
 4. For AHOY function:
 - a. There must be no BACK statement.

Syntax

BACK <value>!!
BACK!!

Examples

VALID	INVALID
BACK a!!	BACK ECHO("Invalid")!!
BACK!!	Back!!
BACK!!	AHOY()[BACK!!]
COIN add() [BACK a + b!!]	COIN add() [BACK ECHO("No")!!]
ABYSS display() [ECHO("Hi")!! BACK!!]	ABYSS display() [ECHO("Hi")!! BACK 10!!]

FUNCTIONS

A function is a self-contained block of code designed to perform a specific task or related set of tasks. Functions may or may not return values depending on their intended purpose. The types of functions in SeaStack are the main function, returning functions, and non-returning functions, each serving distinct roles in program execution.

- A. AHOY Function** - The main function uses the reserved word AHOY and is required in every program. This function coordinates the flow of instructions by calling other functions and managing the program's control structure.

Rules for AHOY Function

1. A valid SeaStack program must have exactly one AHOY function. It must be located at the bottom of the program.
2. It starts with the reserved word AHOY followed by a pair of parentheses (**()**), and ends with the body enclosed in square brackets (**[]**).
3. The AHOY function cannot accept arguments, hence the parentheses must be left empty.
4. The body must have at least one statement, excluding jump statements, and must not have any BACK statement. Local declarations are optional and located at the top of the body, before any statement.

Syntax

<pre>AHOY() [<statement/s>]</pre>
<pre>AHOY() [<local_declaration/s> <statement/s>]</pre>

Examples

VALID	INVALID
<pre>AHOY() [COIN age = 18!! ECHO("Age: %C", age)!!]</pre>	<pre>COIN AHOY() [COIN x = 10!! BACK x!!]</pre>
<pre>AHOY() [COIN x = 10!! ECHO("X: %C\n", x)!!]</pre>	<pre>AHOY(x) [COIN x = 10!! ECHO("X: %C\n", x)!!]</pre>
<pre>AHOY() [COIN x = 5, y = 10, z = x + y!! ECHO("Sum: %C", z)!!]</pre>	<pre>AHOY { COIN x = 5, y = 10, z = x + y!! ECHO("Sum: %C", z)!! }</pre>
<pre>AHOY() [COIN x = 10!! COIN y = 20!! COIN sum = x + y!! ECHO("Sum: %C", sum)!!]</pre>	<pre>AHOY(1, 3) [ECHO("Hello, World")!!]</pre>
	<pre>AHOY() [ECHO("Hello, World")!!] COIN add(COIN x, COIN y) [<statement/s>!!]</pre>
	<pre>AHOY() [ECHO("Hello, World")!! BACK 0!!]</pre>

B. Returning Function - This performs a specific computation and returns a value.

Rules for Returning Function

1. A returning function must be defined before the AHOY function and after all the global declarations. Any function definition inside a returning function is invalid.

2. It must start with the return type, followed by a unique identifier, then the parameter list enclosed in parentheses (()), and end with the body enclosed in square brackets ([]).
3. The return type can only be COIN, DIME, PARCH, SCROLL, or BOOL.
4. The parameter list consists of parameter/s. A parameter starts with the data type followed by the identifier. Multiple parameters are allowed and must be separated with commas. The parameter list is optional and can be left empty if not needed.
5. The body must contain a mandatory BACK statement as its final statement to return a value. All other statements are optional. Local declarations are optional and located at the top of the body, before any statement.

Syntax

<pre><dtype> <id> (<parameter/s>) [BACK <value>!!]</pre>
<pre><dtype> <id> (<parameter/s>) [<statement/s> BACK <value>!!]</pre>
<pre><dtype> <id> (<parameter/s>) [<local declaration/s> <statement/s> BACK <value>!!]</pre>

Examples

VALID	INVALID
COIN add(COIN a, COIN b) [BACK a+b!!]	ABYSS addtwo(COIN a, COIN b) [BACK a + b!!]
COIN get_ten() [COIN value = 10!! BACK value!!]	PARCH get_char() [BACK 123 !!]
COIN get_max() [COIN max = 100!! BACK 100!!]	COIN sum() COIN x, COIN y [BACK x + y!!]
	COIN sum(COIN x COIN y) [BACK x + y!!]
PARCH get_char(SCROLL text) [PARCH first = 'X'!! BACK first!!]	BOOL get_text() [SCROLL text = "test"!!]
SCROLL get_name() [SCROLL name = "SeaStack"!! BACK name!!]	SCROLL get_number() [COIN num = 42!! BACK 42 !!]
SCROLL concat_strings(SCROLL first, SCROLL last) [SCROLL full = first & " " & last!! BACK full!!]	PARCH get_word() [SCROLL word = "hello"!! BACK 3+5 !!]

C. Non-returning Function - This operates but does not return a value to the caller.

Rules for Non-returning Function

1. A non-returning function must be defined before the AHOY function and after all the global declarations. Any function definition inside a non-returning function is invalid.
2. It must start with the reserved word ABYSS as its return type, indicating it does not return a value. It is followed by a unique identifier, then the parameter list enclosed in parentheses (()), and ends with the body enclosed in square brackets ([]).
3. The parameter list consists of parameter/s. A parameter starts with the data type followed by the identifier. Multiple parameters are allowed and must be separated with commas. The parameter list is optional and can be left empty if not needed.
4. The body must have at least one statement, excluding jump statements. It can end with an optional BACK statement without a return value. Local declarations are optional and located at the top of the body, before any statement.

Syntax

ABYSS <id> (<parameter/s>) [<statement/s>]
ABYSS <id> (<parameter/s>) [<statement/s> BACK!!]
ABYSS <id> (<parameter/s>) [<local declaration/s> <statement/s> BACK!!]

Examples

VALID	INVALID
<pre>ABYSS greet() [SCROLL msg = "Hello World"!! ECHO("%S", msg)!!]</pre>	<pre>ABYSS display() [ECHO("Hello")!! BACK 10!!]</pre>
<pre>ABYSS display() [ECHO("SeaStack")!!]</pre>	<pre>ABYSS display [ECHO("SeaStack")!!]</pre>
<pre>ABYSS display_sum(COIN a, COIN b) [COIN sum = a + b!! ECHO("Sum: %C", sum)!!]</pre>	<pre>ABYSS add_numbers(COIN a COIN b) [COIN sum = a + b!! ECHO("The sum is: %C, sum)!!]</pre>
<pre>ABYSS add_three(COIN x, COIN y, COIN z) [COIN sum = x + y + z!! ECHO("The sum is: %C", sum)!! BACK!!]</pre>	<pre>ABYSS result() []</pre>
<pre>ABYSS display_diff(COIN x, COIN y) [COIN diff!! diff = x - y!! ECHO("The difference is: %C", diff)!! BACK!!]</pre>	<pre>ABYSS display_diff(COIN x, COIN y) [COIN diff!! diff = x - y ECHO("The difference is: %C", diff)!! BACK diff!!]</pre>

Rules for Function Calling

1. A function is called by indicating its identifier followed by a pair of parentheses enclosing the arguments, separated by commas, and ending with the statement terminator (!!).
2. The number and types of arguments in a function call must exactly match the function's parameter list and any exceeding or subceeding argument is invalid. The argument can be literals, variables, constants, array elements, structure members, function return values, or expressions.
3. A subfunction can be called inside the AHOY function or another subfunction and must be defined before calling. The AHOY function cannot be called by any function.

Syntax

<code><id>()!!</code>
<code><id>(<argument1>,<argument2>, ..., <argumentN>)!!</code>

Examples

VALID	INVALID
<code>hello()!!</code>	<code>hello!!</code>
<code>add_numbers(5, 10)!!</code>	<code>add_numbers(5 10)!!</code>
<code>greet_name('A', 20)!!</code>	<code>greet_name{'A', 20}</code>
<code>increment(num)!!</code>	<code>increment(num!!</code>
<code>multiply(5, 3)!!</code>	<code>AHOY()!!</code>

COMMENTS

A comment is an annotation in the source code that helps with understanding the code but is ignored by the system during execution.

Rules for Comments

1. A single-line comment must start with a single tilde symbol (~) and terminates with a newline.
2. A multi-line comment must start with a tilde followed by an open parenthesis ~(. Any following character will be read as a comment until enclosed with a close parenthesis and tilde ~). Unclosed multi-line comments are invalid.
3. Any character following a tilde symbol ~ that is not immediately succeeded by an open parenthesis (is treated as a single-line comment.
4. Comments placed inside a SCROLL literal are considered as a part of the literal and not a comment. Comments in between COIN, DIME, PARCH, or BOOL literals are invalid.

Syntax

~<single-line-comment>
~(<multi-line comment/s>)~

Examples

VALID	INVALID
~This is a single line	~
~(This is a multi-line comment It spans multiple lines)~	/* Invalid Symbol */
COIN a = 10!! ~This comment is valid	COIN b = 15 ~Statement not properly terminated
COIN sum = 5 + 10!! ~ valid comment	COIN sum = 5 ~ comment + 10!!

IV. REGULAR DEFINITION

Name	Definition
lowlet	{a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z}
uplet	{A, B, C, D, E, H, L, M, N, P, S}
digit	{0, 1, 2, 3, 4, 5, 6, 7, 8, 9}
nonzero	{1, 2, 3, 4, 5, 6, 7, 8, 9}
alphanumeric	{lowlet, digit}
ASCII	{all printable characters}
whitespace	{ , \t, \n }
arith_op	{+, -, *, /, %, ^}
gen_op	{arith_op, <, >, =, !, &, }
comment	{~}
id_delim	{whitespace, gen_op, (,),], {, }, \$, , }
gen_op_delim	{whitespace, alphanumeric, -, (}
log_op_delim	{gen_op_delim, A, N, ', "}
minus_delim	{whitespace, lowlet, (}
not_delim	{lowlet, A, N, (}
concat_delim	{whitespace, lowlet, (, "}
assign_delim	{log_op_delim, [, !}
back_delim	{whitespace, (, !}
digit_delim	{whitespace, gen_op,),], }, :, , }
bool_delim	{whitespace,),], &, , !, =, , }
parch_delim	{bool_delim, :}
scr_delim	{parch_delim, { }
comma_delim	{assign_delim, @}

colon_delim	{ whitespace, lowlet, A, C, E, H, L, +, - }
terminator_delim	{ colon_delim, B, D, M, P, S,] }
closecb_delim	{ whitespace, gen_op,),], {, &, :, , }
closep_delim	{ whitespace, gen_op,), [,], , }
openp_delim	{ log_op_delim, B, C, D, P, S,), ! }
closesb_delim	{ whitespace, lowlet, uplet,], !, +, -, , }
opensb_delim	{ closesb_delim, digit, [, ' , " }
c_parch	{ ASCII } - { ' \n \ }
c_scr	{ ASCII } - { " \n \ }
es_parch	{ s, n, t, 0, \ }
es_scr	{ d, n, t, 0, \ }
multi_comment	{ ASCII } - { }

V. REGULAR EXPRESSION

RESERVED WORDS		
Reserved Word	Regular Expression	Token
ABYSS	(A)(B)(Y)(S)(S)	ABYSS
ADRIFT	(A)(D)(R)(I)(F)(T)	ADRIFT

AHOY	(A)(H)(O)(Y)	AHOY
ASK	(A)(S)(K)	ASK
AYE	(A)(Y)(E)	AYE
BACK	(B)(A)(C)(K)	BACK
BOOL	(B)(O)(O)(L)	BOOL
CHART	(C)(H)(A)(R)(T)	CHART
COIN	(C)(O)(I)(N)	COIN
COURSE	(C)(O)(U)(R)(S)(E)	COURSE
DIME	(D)(I)(M)(E)	DIME
DROP	(D)(R)(O)(P)	DROP
DROPLOOK	(D)(R)(O)(P)(L)(O)(O)(K)	DROPLOOK
ECHO	(E)(C)(H)(O)	ECHO
HAUL	(H)(A)(U)(L)	HAUL
HEAVE	(H)(E)(A)(V)(E)	HEAVE
HOIST	(H)(O)(I)(S)(T)	HOIST
LAND	(L)(A)(N)(D)	LAND
LOCKE	(L)(O)(C)(K)(E)	LOCKE
Reserved Word	Regular Expression	Token
LOOK	(L)(O)(O)(K)	LOOK
MAST	(M)(A)(S)(T)	MAST
NAY	(N)(A)(Y)	NAY
PARCH	(P)(A)(R)(C)(H)	PARCH
SAIL	(S)(A)(I)(L)	SAIL

SCROLL	(S)(C)(R)(O)(L)(L)	SCROLL
--------	--------------------	--------

RESERVED SYMBOLS		
Reserved Symbol	Regular Expression	Token
+	(+)	+
+#	(+)(#)	+#
+=	(+)(=)	+=
-	(-)	-
-#	(-)(#)	-#
-=	(-)(=)	-=
*	(*)	*
=	()(=)	*=
/	(/)	/
/=	(/)(=)	/=
%	(%)	%
%=	(%)(=)	%=
Reserved Symbol	Regular Expression	Token
^	(^)	^
^=	(^)(=)	^=
=	(=)	=
==	(=)(=)	==
!	(!)	!
!!	(!)(!)	!!
!#	(!)(#)	!#
!=	(!)(=)	!=

<	(<)	<
<=	(<)(=)	<=
>	(>)	>
>=	(>)(=)	>=
&	(&)	&
&&	(&)(&)	&&
	()()	
:	(:)	:
@	(@)	@
\$	(\$)	\$
,	(,)	,
{	({})	{
}	(})	}
(	((	(
Reserved Symbol	Regular Expression	Token
)	())	)
[	([	[
]	(])	]

LITERALS		
Literal	Regular Expression	Token
Identifier	(lowlet)(alphanumeric + _ + λ) ¹⁹	id
COIN	(- + λ)(digit)(digit + λ) ¹⁵	COIN-lit
DIME	(- + λ)(digit)(digit + λ) ¹⁵ (.)(digit)(digit + λ) ⁷	DIME-lit
PARCH	(')((c_parch) (\)(es_parch))(')	PARCH-lit

ALISWAG, K. V. J.
ALTEZA, J. R. D.
CAYACAP, F. A. S.

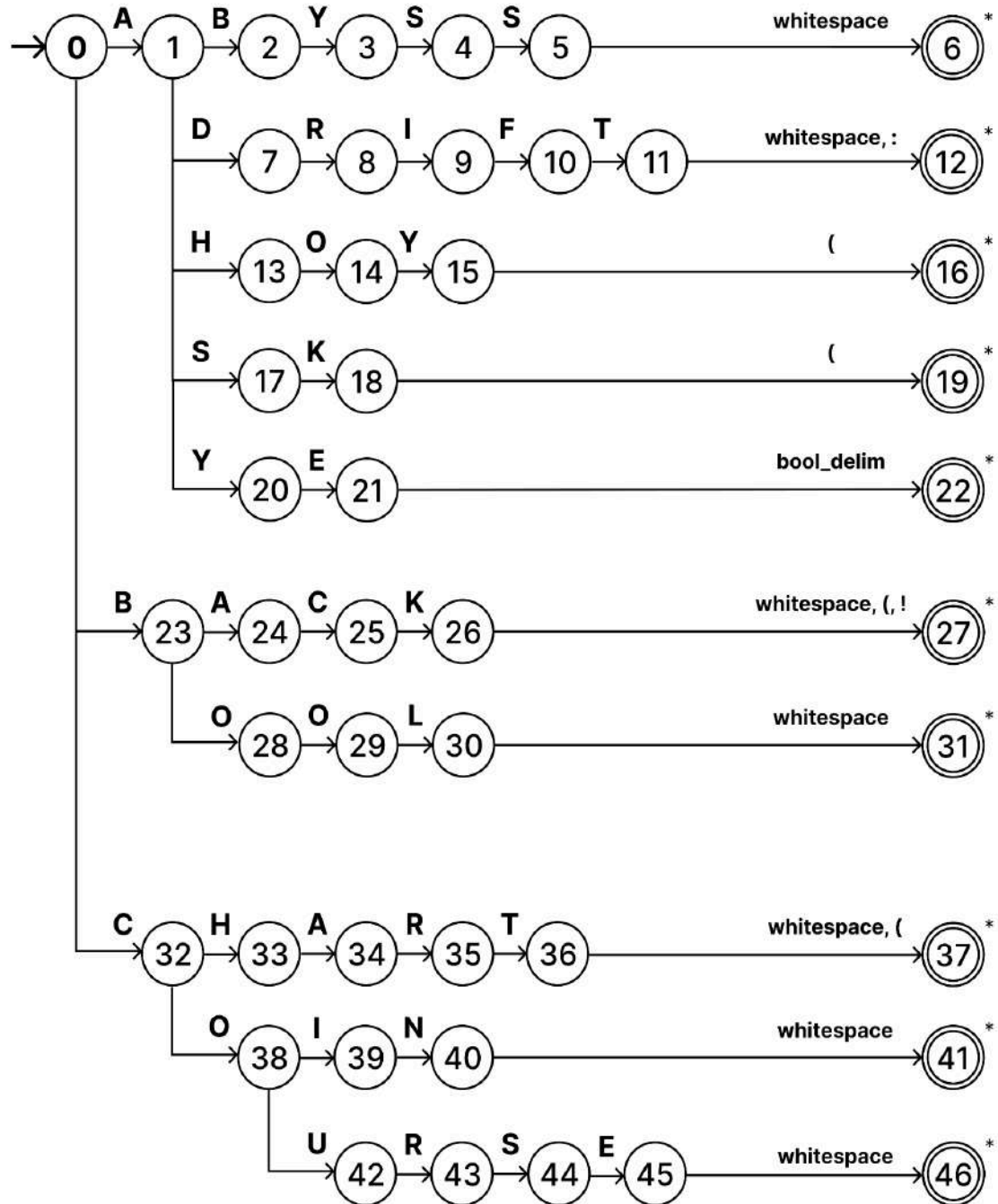
DEL ROSARIO, J. M. A.
MAGAAN, F. M. V.
MULLENO, J. M. A.

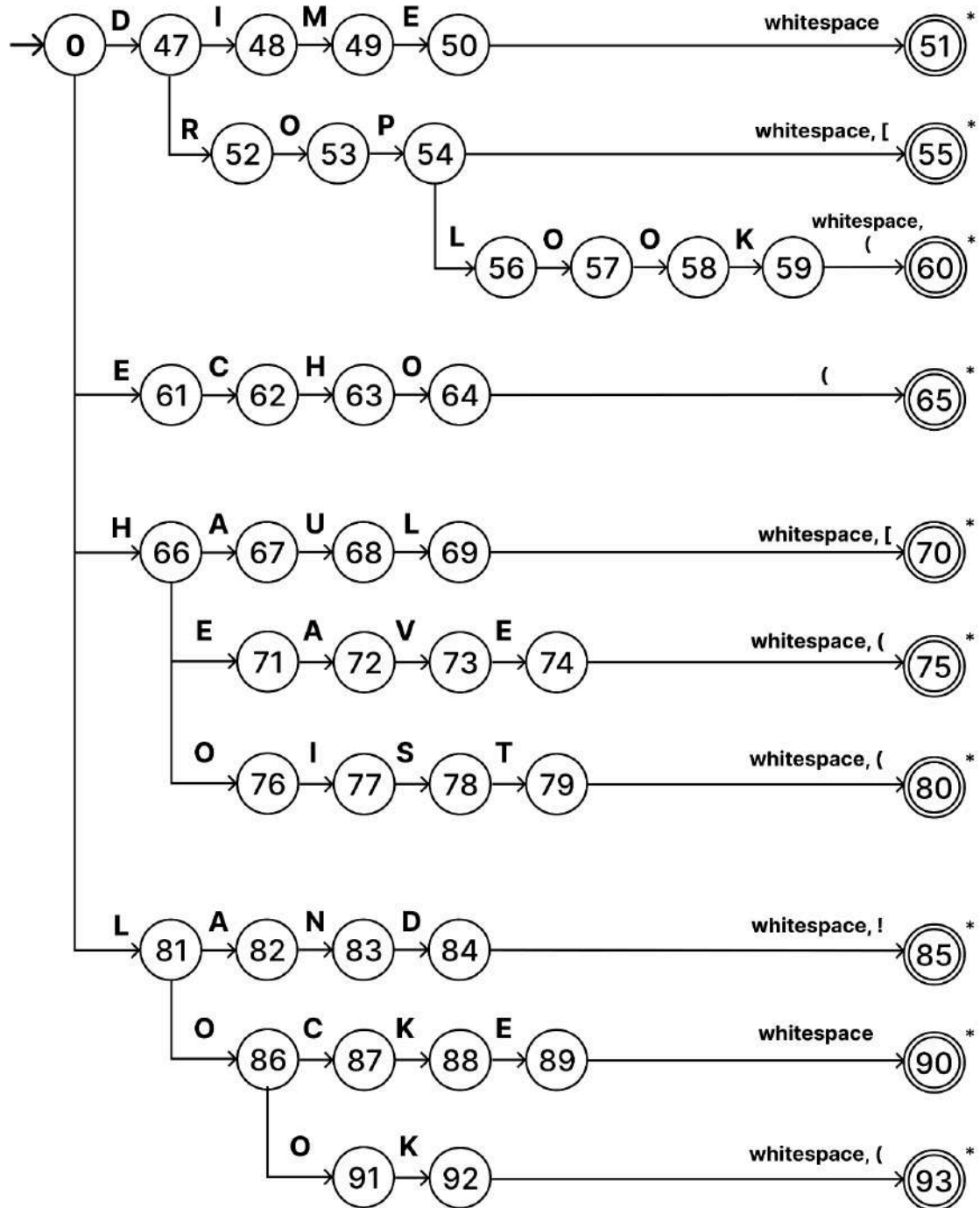


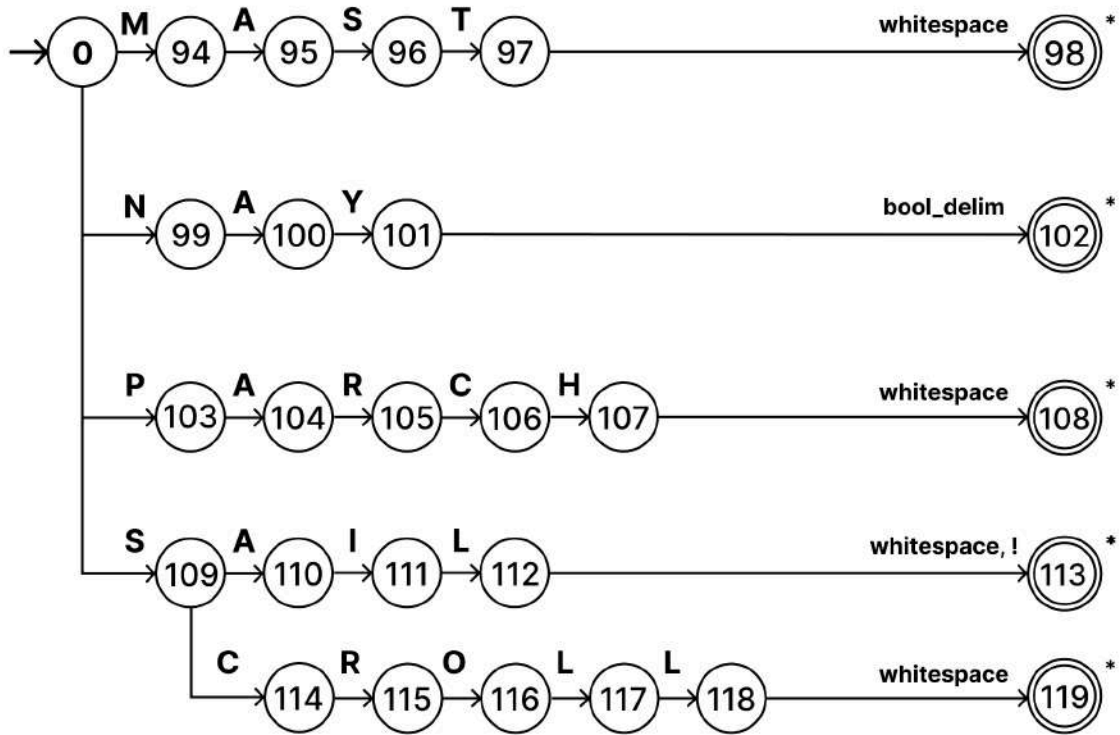
SCROLL	<code>((c_scr) (\)(es_scr))+</code>	SCROLL-lit
Single-line Comment	<code>(~)(ASCII)*(\n)</code>	single-comment
Multi-line Comment	<code>(~)(multi_comment)*(~)</code>	multi-comment

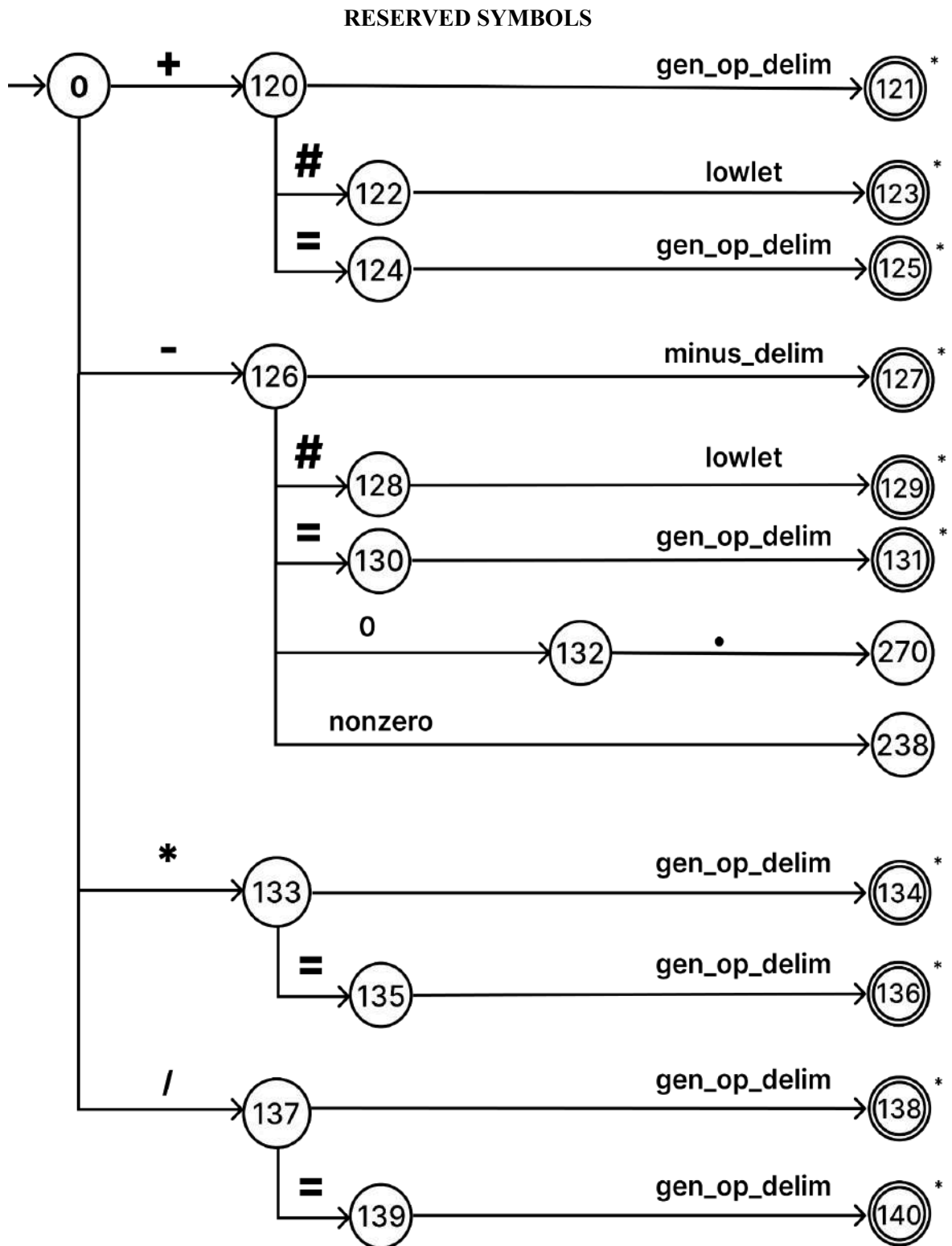
VI. TRANSITION DIAGRAM

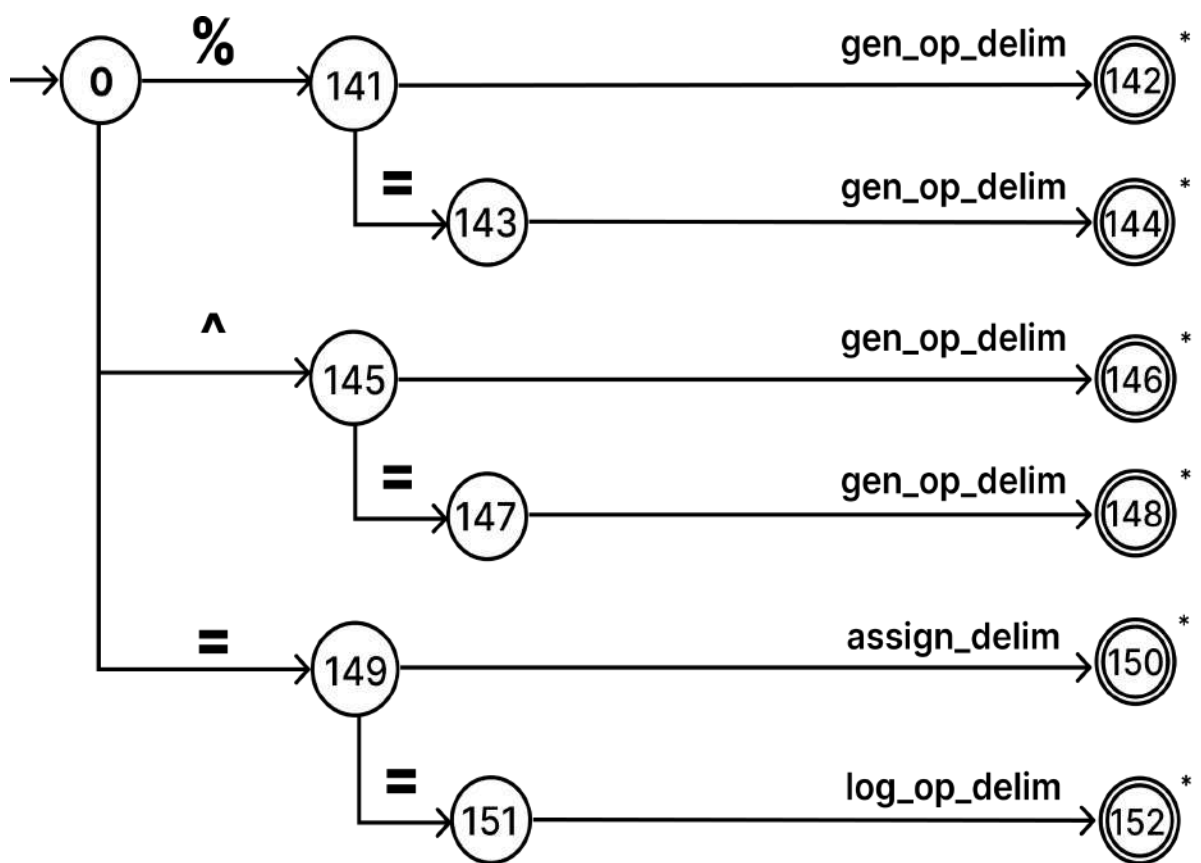
RESERVED WORDS

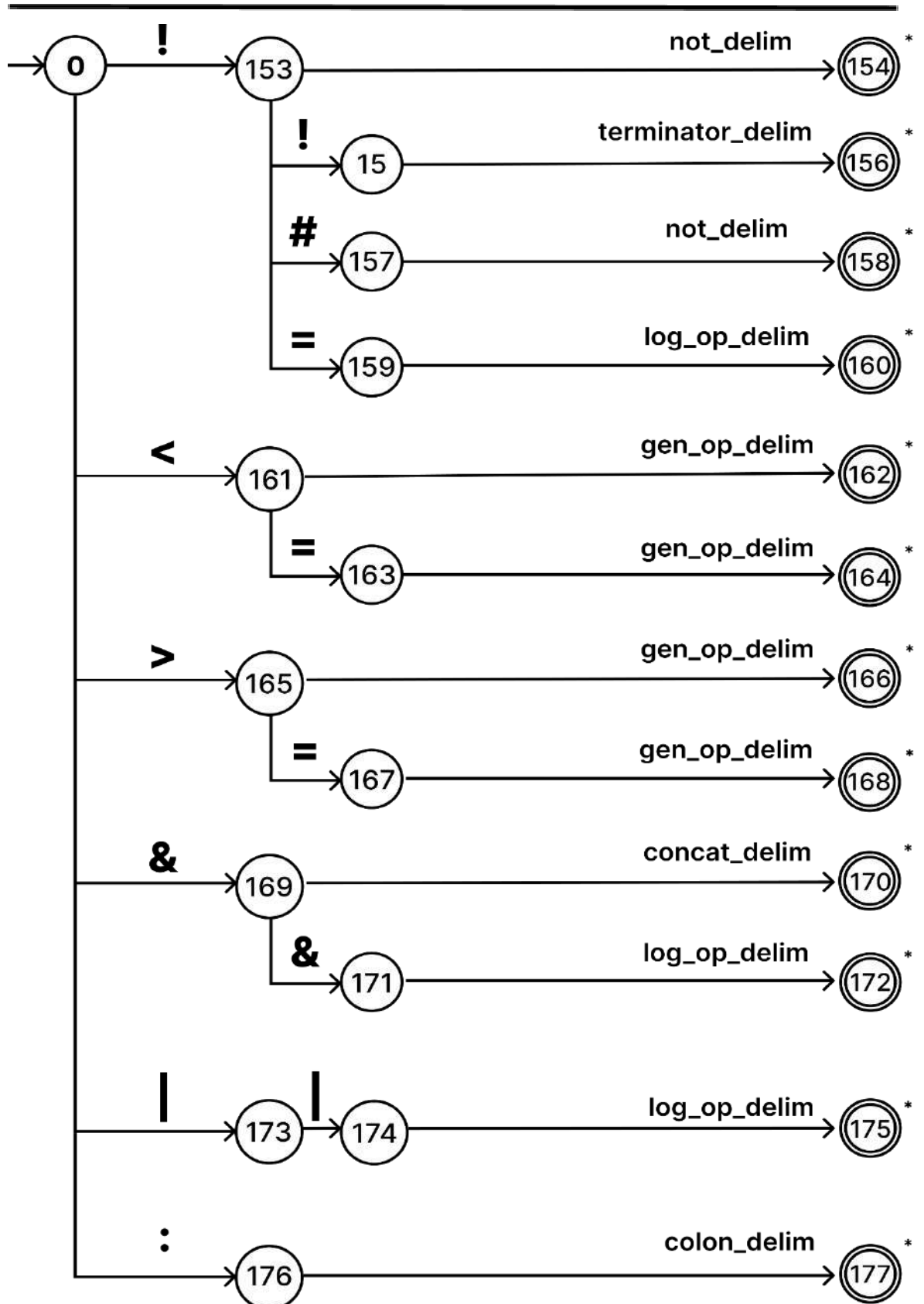


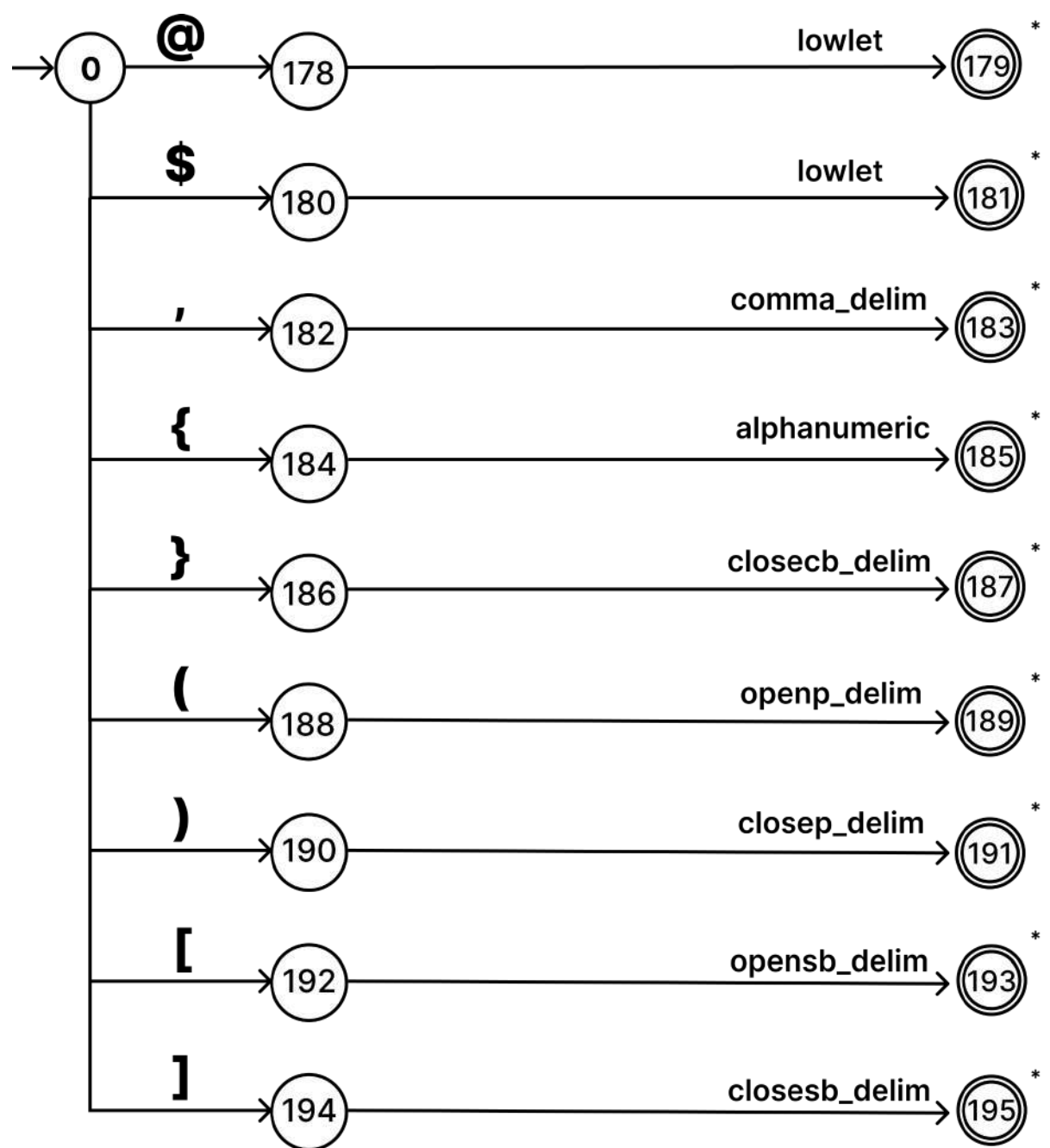




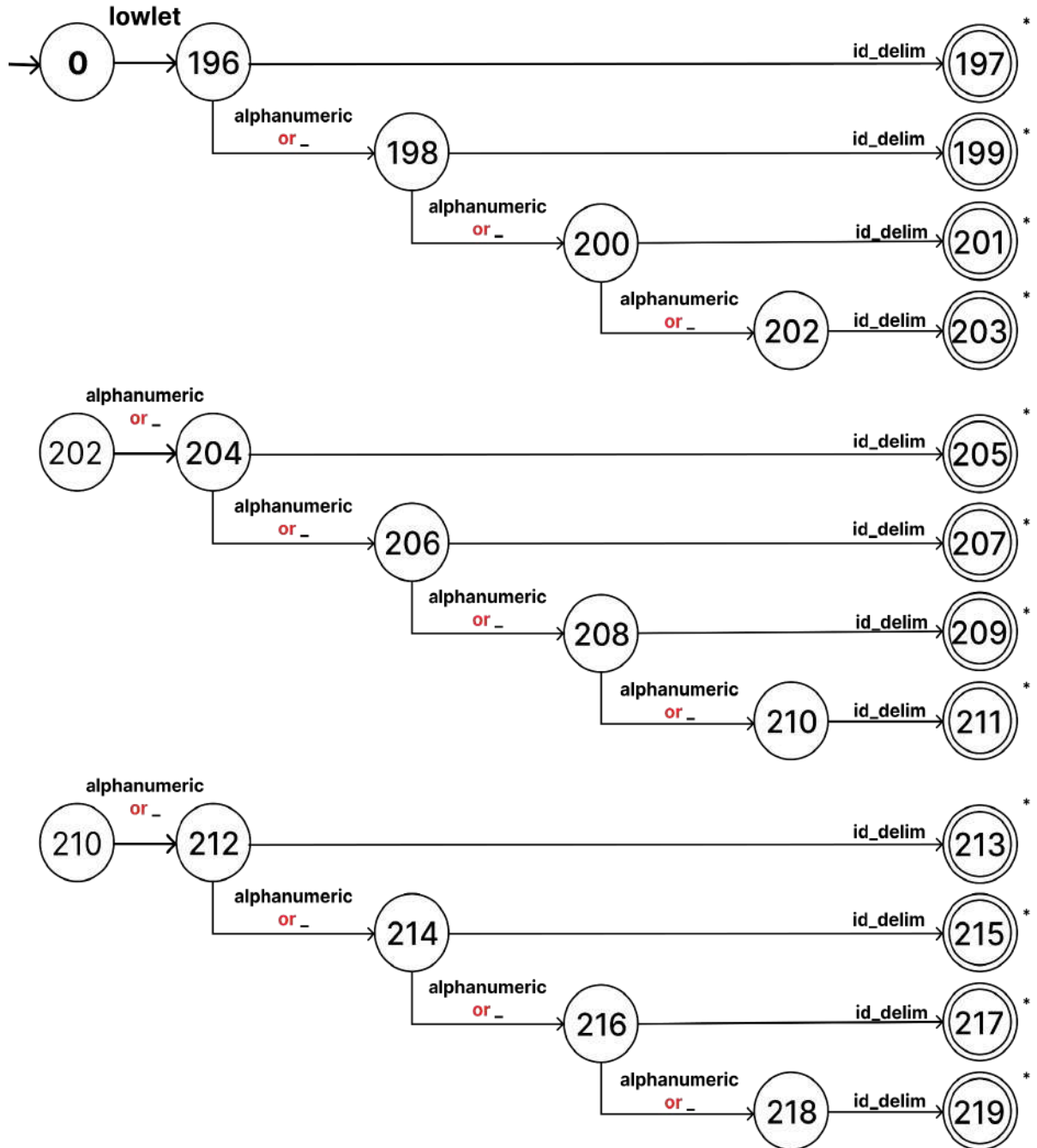


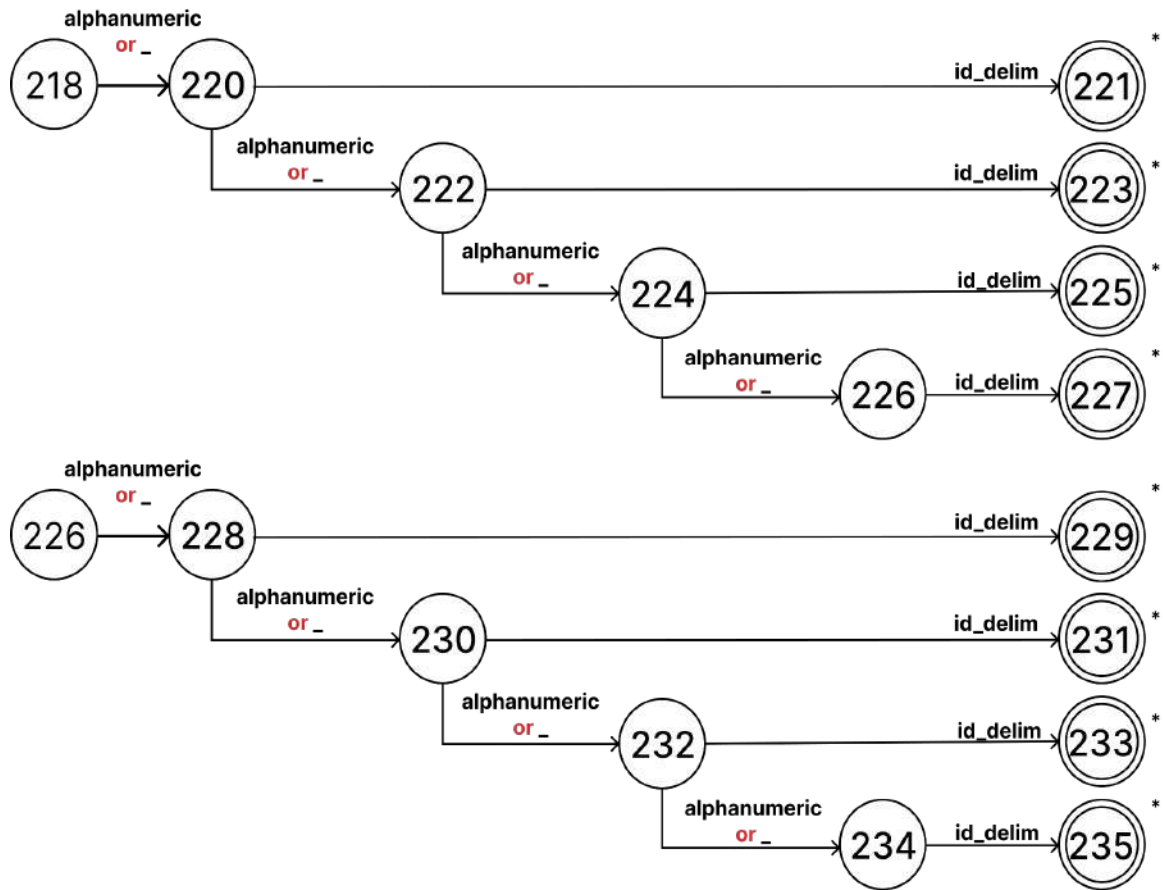




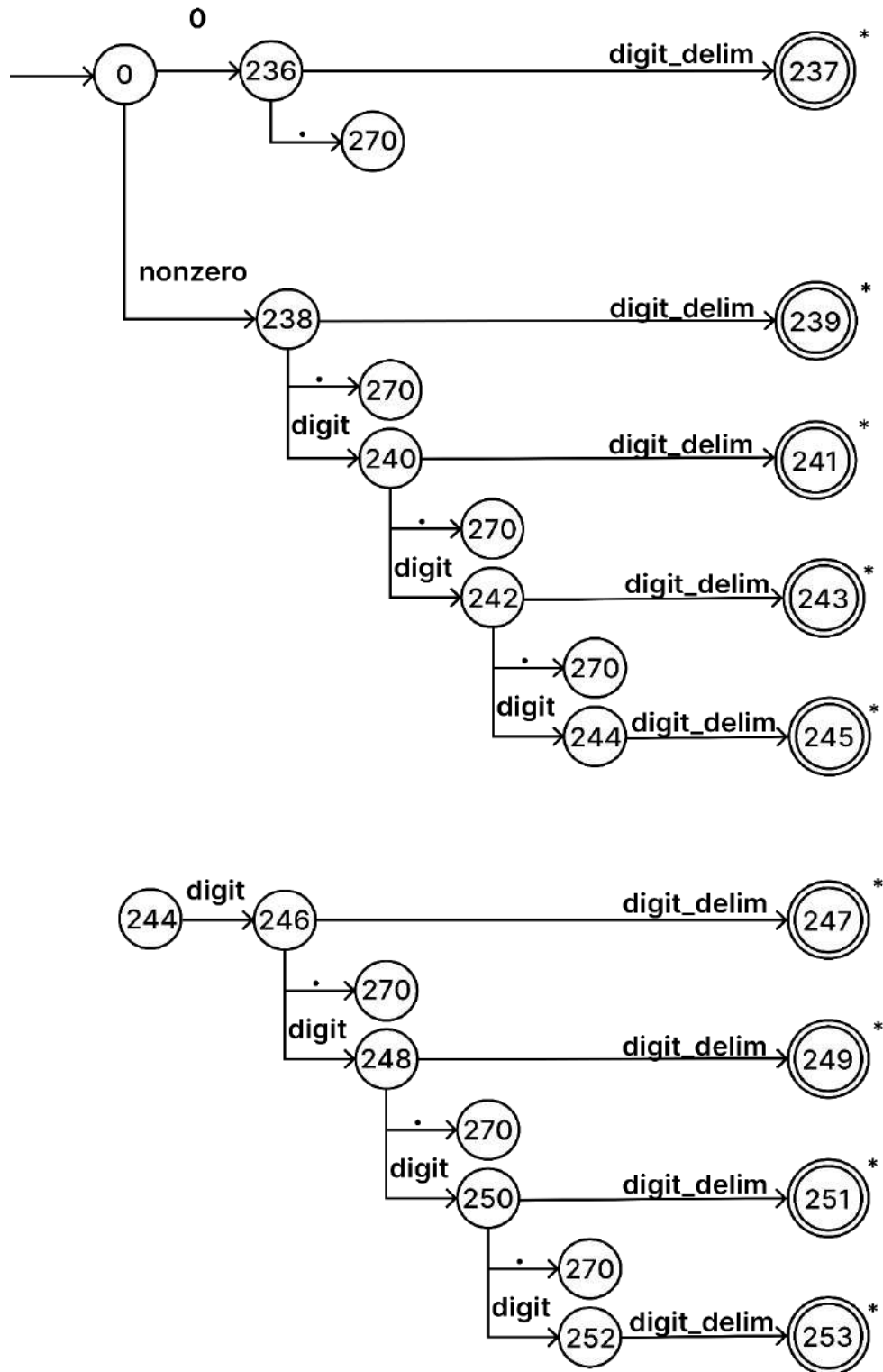


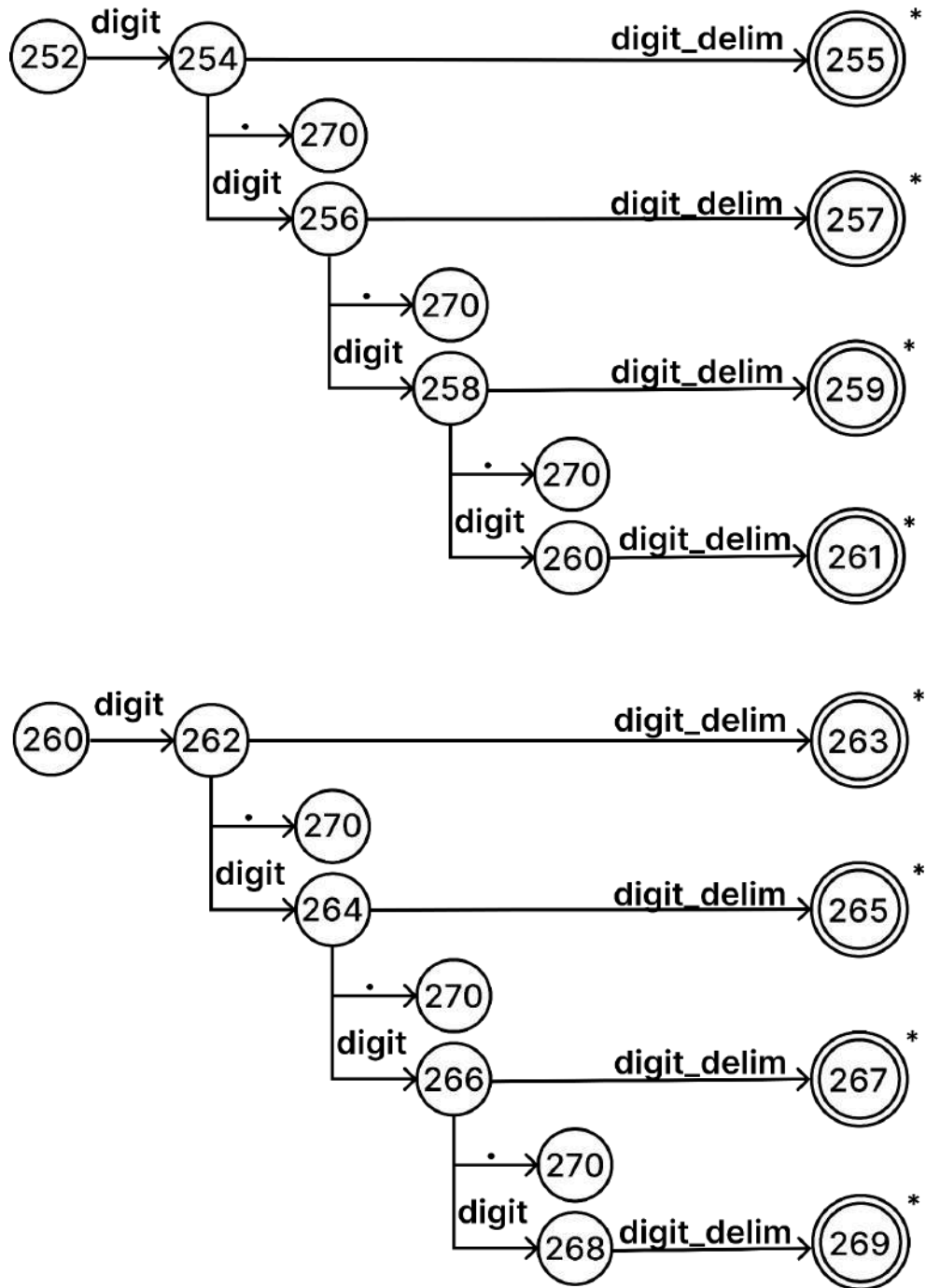
IDENTIFIERS



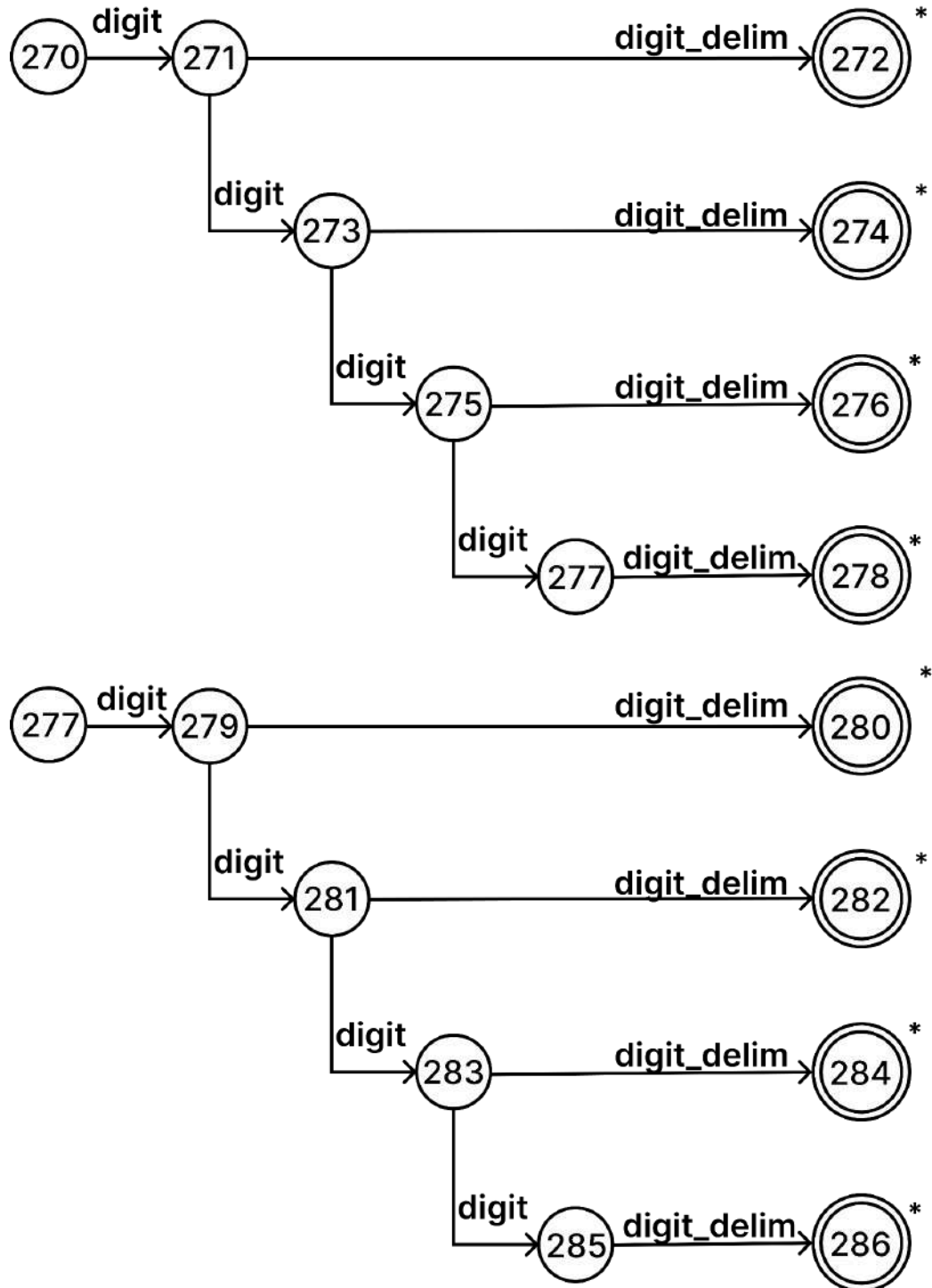


COIN LITERALS

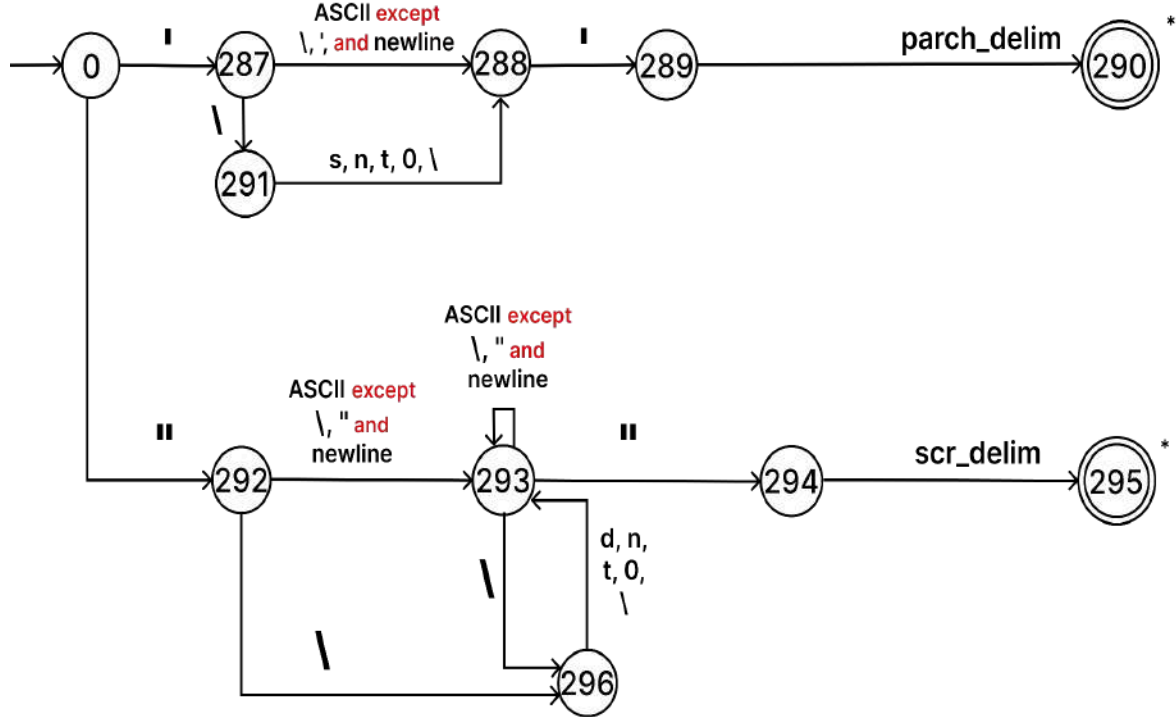




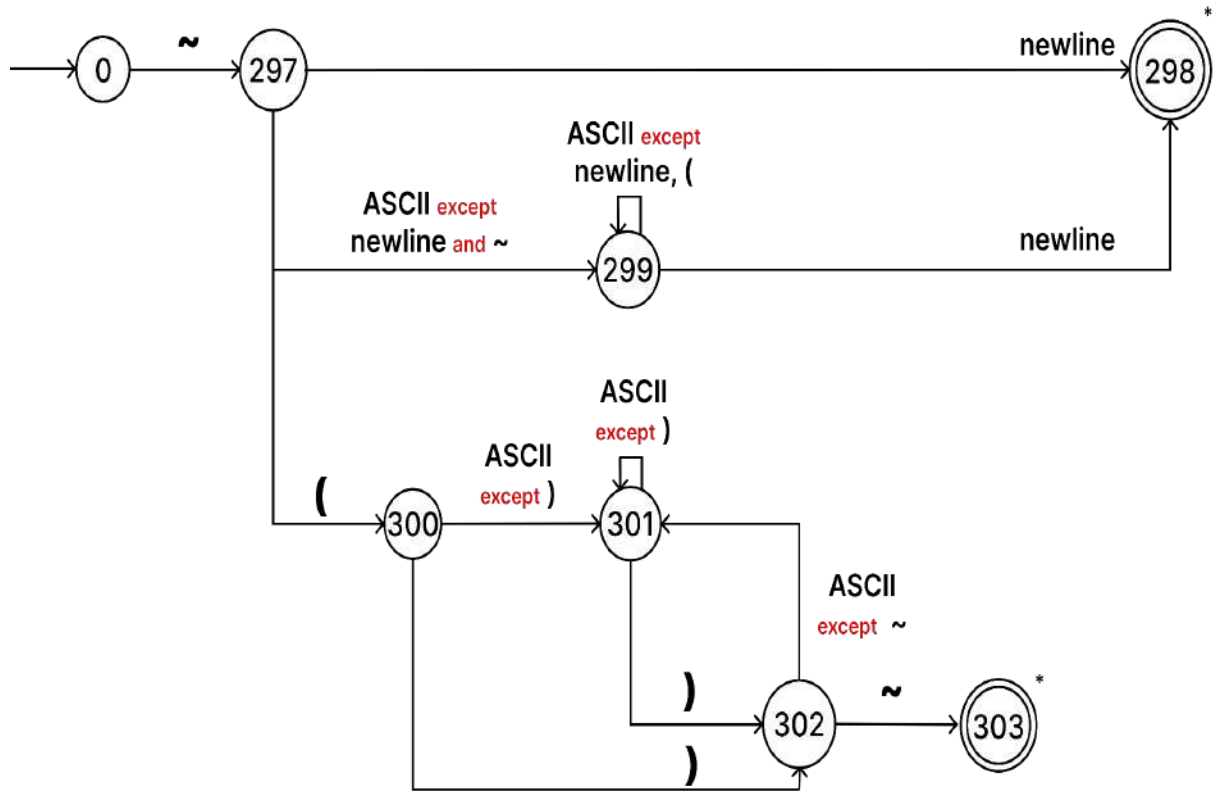
DIME LITERALS



PARCH AND SCROLL LITERALS



COMMENTS





VII. CONTEXT FREE GRAMMAR

NO.	PRODUCTION	→	PRODUCTION SET
1	<program>	→	<global-dec> AHoy() [<ahoy-local-dec> <ahoy-stmnts>]
2	<global-dec>	→	<var-arr-func>
3	<global-dec>	→	<const><global-dec>
4	<global-dec>	→	<struct><sub-func>
5	<global-dec>	→	<nonreturn-func>
6	<global-dec>	→	λ
7	<var-arr-func>	→	COIN id <coin-var-arr-func>
8	<var-arr-func>	→	DIME id <dime-var-arr-func>
9	<var-arr-func>	→	PARCH id <parch-var-arr-func>
10	<var-arr-func>	→	SCROLL id <scroll-var-arr-func>
11	<var-arr-func>	→	BOOL id <bool-var-arr-func>
12	<coin-var-arr-func>	→	<coin-var-arr>!! <global-dec>
13	<coin-var-arr-func>	→	<coin-func>
14	<coin-var-arr>	→	<coin-var>
15	<coin-var-arr>	→	<coin-arr>
16	<coin-var>	→	<coin-init><coin-init-mult>
17	<coin-init>	→	=<coin-val><coin-exp>
18	<coin-init>	→	λ
19	<coin-init-mult>	→	, id <coin-init><coin-init-mult>
20	<coin-init-mult>	→	λ
21	<coin-val>	→	COIN-lit
22	<coin-val>	→	<neg><neg-coin-val>
23	<neg>	→	-
24	<neg>	→	λ
25	<neg-coin-val>	→	id<id-tail>
26	<neg-coin-val>	→	(<coin-val-grp>)
27	<coin-exp>	→	<coin-arith><coin-exp>
28	<coin-exp>	→	λ
29	<coin-arith>	→	<arith-op><coin-val>
30	<coin-val-grp>	→	<coin-val><coin-grp-exp>
31	<coin-grp-exp>	→	<coin-arith><coin-grp-exp>
32	<coin-grp-exp>	→	λ
33	<arith-op>	→	+
34	<arith-op>	→	-
35	<arith-op>	→	*
36	<arith-op>	→	/
37	<arith-op>	→	%
38	<arith-op>	→	^



39	<coin-arr>	→	{COIN-lit} <coin-arr-tail>
40	<coin-arr-tail>	→	=[<coin-arr1>]
41	<coin-arr-tail>	→	{COIN-lit}<coin-arr2-tail>
42	<coin-arr-tail>	→	λ
43	<coin-arr1>	→	<coin-arr-val><cav-tail>
44	<coin-arr-val>	→	<coin-val><coin-arr-exp>
45	<coin-arr-exp>	→	<coin-arith><coin-arr-exp>
46	<coin-arr-exp>	→	λ
47	<cav-tail>	→	, <coin-arr1>
48	<cav-tail>	→	λ
49	<coin-arr2-tail>	→	=[<coin-arr2>]
50	<coin-arr2-tail>	→	λ
51	<coin-arr2>	→	[<coin-arr1>]<cav2-tail>
52	<cav2-tail>	→	, <coin-arr2>
53	<cav2-tail>	→	λ
54	<coin-func>	→	(<params>) [<local-dec> <ret-stmnts> BACK <coin-ret-val>!!] <sub-func>
55	<coin-ret-val>	→	<coin-val><coin-ret-exp>
56	<coin-ret-exp>	→	<coin-arith><coin-ret-exp>
57	<coin-ret-exp>	→	λ
58	<dime-var-arr-func>	→	<dime-var-arr>!! <global-dec>
59	<dime-var-arr-func>	→	<dime-func>
60	<dime-var-arr>	→	<dime-var>
61	<dime-var-arr>	→	<dime-arr>
62	<dime-var>	→	<dime-init><dime-init-mult>
63	<dime-init>	→	=<dime-init-val>
64	<dime-init>	→	λ
65	<dime-init-mult>	→	, id <dime-init><dime-init-mult>
66	<dime-init-mult>	→	λ
67	<dime-init-val>	→	<dime-val><dime-exp>
68	<dime-val>	→	DIME-lit
69	<dime-val>	→	COIN-lit
70	<dime-val>	→	<neg><neg-dime-val>
71	<neg-dime-val>	→	id<id-tail>
72	<neg-dime-val>	→	(<dime-grp-val>)
73	<dime-exp>	→	<dime-arith><dime-exp>
74	<dime-exp>	→	λ
75	<dime-arith>	→	<arith-op><dime-val>
76	<dime-grp-val>	→	<dime-val><dime-grp-exp>
77	<dime-grp-exp>	→	<dime-arith><dime-grp-exp>



78	<dime-grp-exp>	→	λ
79	<dime-arr>	→	{COIN-lit}<dime-arr-tail>
80	<dime-arr-tail>	→	=[<dime-arr1>]
81	<dime-arr-tail>	→	{COIN-lit}<dime-arr2-tail>
82	<dime-arr-tail>	→	λ
83	<dime-arr1>	→	<dime-arr-val><dav-tail>
84	<dime-arr-val>	→	<dime-val><dime-arr-exp>
85	<dime-arr-exp>	→	<dime-arith><dime-arr-exp>
86	<dime-arr-exp>	→	λ
87	<dav-tail>	→	, <dime-arr1>
88	<dav-tail>	→	λ
89	<dime-arr2-tail>	→	=[<dime-arr2>]
90	<dime-arr2-tail>	→	λ
91	<dime-arr2>	→	[<dime-arr1>]<dav2-tail>
92	<dav2-tail>	→	, <dime-arr2>
93	<dav2-tail>	→	λ
94	<dime-func>	→	(<params>) [<local-dec> <ret-stmnts> BACK <dime-ret-val>!!] <sub-func>
95	<dime-ret-val>	→	<dime-val><dime-ret-exp>
96	<dime-ret-exp>	→	<dime-arith><dime-ret-exp>
97	<dime-ret-exp>	→	λ
98	<parch-var-arr-func>	→	<parch-var-arr>!! <global-dec>
99	<parch-var-arr-func>	→	<parch-func>
100	<parch-var-arr>	→	<parch-var>
101	<parch-var-arr>	→	<parch-arr>
102	<parch-var>	→	<parch-init><parch-init-mult>
103	<parch-init>	→	=<parch-val>
104	<parch-init>	→	λ
105	<parch-init-mult>	→	, id <parch-init><parch-init-mult>
106	<parch-init-mult>	→	λ
107	<parch-val>	→	PARCH-lit
108	<parch-val>	→	id<id-tail>
109	<parch-arr>	→	{COIN-lit}<parch-arr-tail>
110	<parch-arr-tail>	→	=[<parch-arr1>]
111	<parch-arr-tail>	→	{COIN-lit}<parch-arr2-tail>
112	<parch-arr-tail>	→	λ
113	<parch-arr1>	→	<parch-val><pav-tail>
114	<pav-tail>	→	, <parch-arr1>
115	<pav-tail>	→	λ
116	<parch-arr2-tail>	→	=[<parch-arr2>]



117	<parch-arr2-tail>	→	λ
118	<parch-arr2>	→	[<parch-arr1><pav2-tail>
119	<pav2-tail>	→	, <parch-arr2>
120	<pav2-tail>	→	λ
121	<parch-func>	→	(<params>) [<local-dec> <ret-stmnts> BACK <parch-val>!!] <sub-func>
122	<scroll-var-arr-func>	→	<scroll-var-arr>!! <global-dec>
123	<scroll-var-arr-func>	→	<scroll-func><sub-func>
124	<scroll-var-arr>	→	<scroll-var>
125	<scroll-var-arr>	→	<scroll-arr>
126	<scroll-var>	→	<scroll-init><scroll-init-mult>
127	<scroll-init>	→	=<scroll-init-val>
128	<scroll-init>	→	λ
129	<scroll-init-mult>	→	, id <scroll-init><scroll-init-mult>
130	<scroll-init-mult>	→	λ
131	<scroll-init-val>	→	<scroll-val><scroll-exp>
132	<scroll-val>	→	SCROLL-lit<scr-char>
133	<scroll-val>	→	id<id-tail>
134	<scroll-val>	→	(<scroll-grp-val>)
135	<scr-char>	→	{<index>}
136	<scr-char>	→	λ
137	<index>	→	id
138	<index>	→	COIN-lit
139	<scroll-exp>	→	<scroll-concat><scroll-exp>
140	<scroll-exp>	→	λ
141	<scroll-concat>	→	& <scroll-val>
142	<scroll-grp-val>	→	<scroll-val><scroll-grp-exp>
143	<scroll-grp-exp>	→	<scroll-concat><scroll-grp-exp>
144	<scroll-grp-exp>	→	λ
145	<scroll-arr>	→	{COIN-lit}<scroll-arr-tail>
146	<scroll-arr-tail>	→	= [<scroll-arr1>]
147	<scroll-arr-tail>	→	{COIN-lit}<scroll-arr2-tail>
148	<scroll-arr-tail>	→	λ
149	<scroll-arr1>	→	<scroll-arr-val><sav-tail>
150	<scroll-arr-val>	→	<scroll-val><scroll-arr-exp>
151	<scroll-arr-exp>	→	<scroll-concat><scroll-arr-exp>
152	<scroll-arr-exp>	→	λ
153	<sav-tail>	→	, <scroll-arr1>
154	<sav-tail>	→	λ
155	<scroll-arr2-tail>	→	= [<scroll-arr2>]



156	<scroll-arr2-tail>	→	λ
157	<scroll-arr2>	→	[<scroll-arr1><sav2-tail>
158	<sav2-tail>	→	, <scroll-arr2>
159	<sav2-tail>	→	λ
160	<scroll-func>	→	(<params>) [<local-dec> <ret-stmnts> BACK <scroll-ret-val>!!] <sub-func>
161	<scroll-ret-val>	→	<scroll-val><scroll-ret-exp>
162	<scroll-ret-exp>	→	<scroll-concat><scroll-ret-exp>
163	<scroll-ret-exp>	→	λ
164	<bool-var-arr-func>	→	<bool-var-arr>!! <global-dec>
165	<bool-var-arr-func>	→	<bool-func>
166	<bool-var-arr>	→	<bool-var>
167	<bool-var-arr>	→	<bool-arr>
168	<bool-var>	→	<bool-init><bool-init-mult>
169	<bool-init>	→	=<bool-init-val>
170	<bool-init>	→	λ
171	<bool-init-mult>	→	, id <bool-init><bool-init-mult>
172	<bool-init-mult>	→	λ
173	<bool-init-val>	→	<bool-val><bool-exp>
174	<bool-val>	→	id<id-tail><bool-exp2>
175	<bool-val>	→	(<value-grp>)<bool-exp2>
176	<bool-val>	→	<bool><bool-eq>
177	<bool-val>	→	<digit><bool-arith><rel-eq>
178	<bool-val>	→	PARCH-lit<eq-op><parch-val>
179	<bool-val>	→	SCROLL-lit<scr-char><eq-op><scroll-val>
180	<bool>	→	<bool-lit>
181	<bool>	→	<not-op><not-ope>
182	<bool-lit>	→	AYE
183	<bool-lit>	→	NAY
184	<not-op>	→	!
185	<not-op>	→	!#
186	<not-ope>	→	id<id-tail>
187	<not-ope>	→	(<bool-grp-val>)
188	<not-ope>	→	<bool-lit>
189	<bool-eq>	→	<eq-op><bool-val>
190	<bool-eq>	→	λ
191	<digit>	→	COIN-lit
192	<digit>	→	DIME-lit
193	<digit>	→	-<neg-digit>
194	<neg-digit>	→	id<id-tail>



195	<neg-digit>	→	(<dime-grp-val>)
196	<bool-arith>	→	<dime-arith><bool-arith>
197	<bool-arith>	→	λ
198	<rel-eq>	→	<rel><bool-arith2><bool-eq>
199	<rel-eq>	→	<eq-digit><bool-arith3>
200	<rel>	→	<rel-op><dime-val>
201	<rel-op>	→	<
202	<rel-op>	→	>
203	<rel-op>	→	<=
204	<rel-op>	→	>=
205	<bool-arith2>	→	<dime-arith><bool-arith2>
206	<bool-arith2>	→	λ
207	<eq-digit>	→	<eq-op><dime-val>
208	<eq-op>	→	==
209	<eq-op>	→	!=
210	<bool-arith3>	→	<dime-arith><bool-arith3>
211	<bool-arith3>	→	λ
212	<bool-exp2>	→	<dime-arith><bool-arith><rel-eq>
213	<bool-exp2>	→	<rel><bool-arith2><bool-eq>
214	<bool-exp2>	→	<eq-op><eq-ope>
215	<bool-exp2>	→	λ
216	<eq-ope>	→	id<id-tail><bool-arith4><bool-rel>
217	<eq-ope>	→	(<value-grp>)<bool-arith4><bool-rel>
218	<eq-ope>	→	<digit><bool-arith4><bool-rel>
219	<eq-ope>	→	<eq-ope1>
220	<eq-ope1>	→	PARCH-lit
221	<eq-ope1>	→	SCROLL-lit<scr-char>
222	<eq-ope1>	→	<bool>
223	<bool-arith4>	→	<dime-arith><bool-arith4>
224	<bool-arith4>	→	λ
225	<bool-rel>	→	<rel><bool-arith3>
226	<bool-rel>	→	λ
227	<bool-exp>	→	<log-op><bool-val><bool-exp>
228	<bool-exp>	→	λ
229	<log-op>	→	
230	<log-op>	→	&&
231	<bool-grp-val>	→	<bool-ope-grp><bool-exp-grp>
232	<bool-ope-grp>	→	id<id-tail><bool-exp2-grp>
233	<bool-ope-grp>	→	(<value-grp>)<bool-exp2-grp>
234	<bool-ope-grp>	→	<bool><bool-eq-grp>



235	<bool-ope-grp>	→	<digit><bool-arith><releq-grp>
236	<bool-ope-grp>	→	PARCH-lit<eq-op><parch-val>
237	<bool-ope-grp>	→	SCROLL-lit<scr-char><eq-op><scroll-val>
238	<bool-eq-grp>	→	<eq-op><bool-ope-grp>
239	<bool-eq-grp>	→	λ
240	<releq-grp>	→	<rel><bool-arith2-grp><bool-eq-grp>
241	<releq-grp>	→	<eq-digit><bool-arith3-grp>
242	<bool-arith2-grp>	→	<dime-arith><bool-arith2-grp>
243	<bool-arith2-grp>	→	λ
244	<bool-arith3-grp>	→	<dime-arith><bool-arith3-grp>
245	<bool-arith3-grp>	→	λ
246	<bool-exp2-grp>	→	<dime-arith><bool-arith><releq-grp>
247	<bool-exp2-grp>	→	<rel><bool-arith2-grp><bool-eq-grp>
248	<bool-exp2-grp>	→	<eq-op><eq-ope-grp>
249	<bool-exp2-grp>	→	λ
250	<eq-ope-grp>	→	id<id-tail><bool-arith4-grp><bool-rel-grp>
251	<eq-ope-grp>	→	(<value-grp>)<bool-arith4-grp><bool-rel-grp>
252	<eq-ope-grp>	→	<digit><bool-arith4-grp><bool-rel-grp>
253	<eq-ope-grp>	→	<eq-ope1>
254	<bool-arith4-grp>	→	<dime-arith><bool-arith4-grp>
255	<bool-arith4-grp>	→	λ
256	<bool-rel-grp>	→	<rel><bool-arith3-grp>
257	<bool-rel-grp>	→	λ
258	<bool-exp-grp>	→	<log-op><bool-ope-grp><bool-exp-grp>
259	<bool-exp-grp>	→	λ
260	<bool-arr>	→	{COIN-lit}<bool-arr-tail>
261	<bool-arr-tail>	→	=[<bool-arr1>]
262	<bool-arr-tail>	→	{COIN-lit}<bool-arr2-tail>
263	<bool-arr-tail>	→	λ
264	<bool-arr1>	→	<bool-arr-val><bav-tail>
265	<bav-tail>	→	, <bool-arr1>
266	<bav-tail>	→	λ
267	<bool-arr-val>	→	<bool-ope-arr><bool-exp-arr>
268	<bool-ope-arr>	→	id<id-tail><bool-exp2-arr>
269	<bool-ope-arr>	→	(<value-grp>)<bool-exp2-arr>
270	<bool-ope-arr>	→	<bool><bool-eq-arr>
271	<bool-ope-arr>	→	<digit><bool-arith><releq-arr>
272	<bool-ope-arr>	→	PARCH-lit<eq-op><parch-val>
273	<bool-ope-arr>	→	SCROLL-lit<scr-char><eq-op><scroll-val>
274	<bool-eq-arr>	→	<eq-op><bool-ope-arr>



275	<bool-eq-arr>	→	λ
276	<releq-arr>	→	<rel><bool-arith2-arr><bool-eq-arr>
277	<releq-arr>	→	<eq-digit><bool-arith3-arr>
278	<bool-arith2-arr>	→	<dime-arith><bool-arith2-arr>
279	<bool-arith2-arr>	→	λ
280	<bool-arith3-arr>	→	<dime-arith><bool-arith3-arr>
281	<bool-arith3-arr>	→	λ
282	<bool-exp2-arr>	→	<dime-arith><bool-arith><releq-arr>
283	<bool-exp2-arr>	→	<rel><bool-arith2-arr><bool-eq-arr>
284	<bool-exp2-arr>	→	<eq-op><eq-ope-arr>
285	<bool-exp2-arr>	→	λ
286	<eq-ope-arr>	→	id<id-tail><bool-arith4-arr><bool-rel-arr>
287	<eq-ope-arr>	→	(<value-grp>)<bool-arith4-arr><bool-rel-arr>
288	<eq-ope-arr>	→	<digit><bool-arith4-arr><bool-rel-arr>
289	<eq-ope-arr>	→	<eq-ope1>
290	<bool-arith4-arr>	→	<dime-arith><bool-arith4-arr>
291	<bool-arith4-arr>	→	λ
292	<bool-rel-arr>	→	<rel><bool-arith3-arr>
293	<bool-rel-arr>	→	λ
294	<bool-exp-arr>	→	<log-op><bool-ope-arr><bool-exp-arr>
295	<bool-exp-arr>	→	λ
296	<bool-arr2-tail>	→	= [<bool-arr2>]
297	<bool-arr2-tail>	→	λ
298	<bool-arr2>	→	[<bool-arr1>]<bav2-tail>
299	<bav2-tail>	→	, <bool-arr2>
300	<bav2-tail>	→	λ
301	<bool-func>	→	(<params>) [<local-dec> <ret-stmnts> BACK <bool-ret-val>!!] <sub-func>
302	<bool-ret-val>	→	<bool-ope-ret><bool-exp-ret>
303	<bool-ope-ret>	→	id<id-tail><bool-exp2-ret>
304	<bool-ope-ret>	→	(<value-grp>)<bool-exp2-ret>
305	<bool-ope-ret>	→	<bool><bool-eq-ret>
306	<bool-ope-ret>	→	<digit><bool-arith><releq-ret>
307	<bool-ope-ret>	→	PARCH-lit<eq-op><parch-val>
308	<bool-ope-ret>	→	SCROLL-lit<scr-char><eq-op><scroll-val>
309	<bool-eq-ret>	→	<eq-op><bool-ope-ret>
310	<bool-eq-ret>	→	λ
311	<releq-ret>	→	<rel><bool-arith2-ret><bool-eq-ret>
312	<releq-ret>	→	<eq-digit><bool-arith3-ret>
313	<bool-arith2-ret>	→	<dime-arith><bool-arith2-ret>



314	<bool-arith2-ret>	→	λ
315	<bool-arith3-ret>	→	<dime-arith><bool-arith3-ret>
316	<bool-arith3-ret>	→	λ
317	<bool-exp2-ret>	→	<dime-arith><bool-arith><releq-ret>
318	<bool-exp2-ret>	→	<rel><bool-arith2-ret><bool-eq-ret>
319	<bool-exp2-ret>	→	<eq-op><eq-ope-ret>
320	<bool-exp2-ret>	→	λ
321	<eq-ope-ret>	→	id<id-tail><bool-arith4-ret><bool-rel-ret>
322	<eq-ope-ret>	→	(<value-grp>)<bool-arith4-ret><bool-rel-ret>
323	<eq-ope-ret>	→	<digit><bool-arith4-ret><bool-rel-ret>
324	<eq-ope-ret>	→	<eq-ope1>
325	<bool-arith4-ret>	→	<dime-arith><bool-arith4-ret>
326	<bool-arith4-ret>	→	λ
327	<bool-rel-ret>	→	<rel><bool-arith3-ret>
328	<bool-rel-ret>	→	λ
329	<bool-exp-ret>	→	<log-op><bool-ope-ret><bool-exp-ret>
330	<bool-exp-ret>	→	λ
331	<params>	→	<d-type> id <param-mult>
332	<params>	→	λ
333	<param-mult>	→	, <d-type> id <param-mult>
334	<param-mult>	→	λ
335	<d-type>	→	COIN
336	<d-type>	→	DIME
337	<d-type>	→	PARCH
338	<d-type>	→	SCROLL
339	<d-type>	→	BOOL
340	<ret-stmnts>	→	<statements><ret-stmnts>
341	<ret-stmnts>	→	λ
342	<id-tail>	→	{<index>}<elmt2>
343	<id-tail>	→	\$id
344	<id-tail>	→	(<args>)
345	<id-tail>	→	λ
346	<elmt2>	→	{<index>}
347	<elmt2>	→	λ
348	<args>	→	<value><args-mult>
349	<args>	→	λ
350	<args-mult>	→	, <value><args-mult>
351	<args-mult>	→	λ
352	<value>	→	id<id-tail><exp>
353	<value>	→	(<value-grp>)<exp>

354	<value>	→	<digit><digit-tail>
355	<value>	→	PARCH-lit<parch-tail>
356	<value>	→	SCROLL-lit<scr-char><scroll-tail>
357	<value>	→	<bool><logeq-var>
358	<digit-tail>	→	<arith-var><releq-var>
359	<digit-tail>	→	λ
360	<arith-var>	→	<dime-arith><arith-var>
361	<arith-var>	→	λ
362	<releq-var>	→	<rel><arith-var2><logeq-var>
363	<releq-var>	→	<eq-digit><arith-var3><log-var>
364	<releq-var>	→	λ
365	<arith-var2>	→	<dime-arith><arith-var2>
366	<arith-var2>	→	λ
367	<arith-var3>	→	<dime-arith><arith-var3>
368	<arith-var3>	→	λ
369	<logeq-var>	→	<logeq-op><log-ope><log-var>
370	<logeq-var>	→	λ
371	<logeq-op>	→	<log-op>
372	<logeq-op>	→	<eq-op>
373	<log-var>	→	<log-op><log-ope><log-var>
374	<log-var>	→	λ
375	<parch-tail>	→	<eq-op>PARCH-lit<log-var>
376	<parch-tail>	→	λ
377	<scroll-tail>	→	<concat-var>
378	<scroll-tail>	→	<eq-op><scroll-val><log-var>
379	<scroll-tail>	→	λ
380	<concat-var>	→	& <scroll-val> <concat-var>
381	<concat-var>	→	λ
382	<exp>	→	<dime-arith><arith-var><releq-var>
383	<exp>	→	<rel><arith-var2><logeq-var>
384	<exp>	→	<log-op><log-ope><log-var>
385	<exp>	→	<eq-op><eq-var><log-var>
386	<exp>	→	& <scroll-val> <concat-var>
387	<exp>	→	λ
388	<log-ope>	→	id<id-tail><log-exp>
389	<log-ope>	→	(<value-grp>)<log-exp>
390	<log-ope>	→	<bool><log-booleq>
391	<log-ope>	→	<digit><bool-arith><log-releq>
392	<log-ope>	→	PARCH-lit<eq-op><parch-val>
393	<log-ope>	→	SCROLL-lit<scr-char><eq-op><scroll-val>



394	<log-booleq>	→	<eq-op><log-ope>
395	<log-booleq>	→	λ
396	<log-releq>	→	<rel><arith-var2><log-booleq>
397	<log-releq>	→	<eq-digit><arith-var3>
398	<log-exp>	→	<dime-arith><arith-var><log-releq>
399	<log-exp>	→	<rel><arith-var2><log-booleq>
400	<log-exp>	→	<eq-op><eq-var>
401	<log-exp>	→	λ
402	<eq-var>	→	id<id-tail><arith-var4><rel-var>
403	<eq-var>	→	(<value-grp>)<arith-var4><rel-var>
404	<eq-var>	→	<digit><arith-var4><rel-var>
405	<eq-var>	→	<eq-ope1>
406	<arith-var4>	→	<dime-arith><arith-var4>
407	<arith-var4>	→	λ
408	<rel-var>	→	<rel><arith-var3>
409	<rel-var>	→	λ
410	<value-grp>	→	id<id-tail><exp-grp>
411	<value-grp>	→	(<value-grp>)<exp-grp>
412	<value-grp>	→	<digit><digit-tail-grp>
413	<value-grp>	→	PARCH-lit<parch-tail-grp>
414	<value-grp>	→	SCROLL-lit<scr-char><scroll-tail-grp>
415	<value-grp>	→	<bool><logeq-grp>
416	<digit-tail-grp>	→	<arith-grp><releqvar-grp>
417	<digit-tail-grp>	→	λ
418	<arith-grp>	→	<dime-arith><arith-grp>
419	<arith-grp>	→	λ
420	<releqvar-grp>	→	<rel><arith-grp2><logeq-grp>
421	<releqvar-grp>	→	<eq-digit><arith-grp3><bool-exp-grp>
422	<releqvar-grp>	→	λ
423	<arith-grp2>	→	<dime-arith><arith-grp2>
424	<arith-grp2>	→	λ
425	<arith-grp3>	→	<dime-arith><arith-grp3>
426	<arith-grp3>	→	λ
427	<logeq-grp>	→	<logeq-op><bool-ope-grp><bool-exp-grp>
428	<logeq-grp>	→	λ
429	<parch-tail-grp>	→	<eq-op>PARCH-lit<bool-exp-grp>
430	<parch-tail-grp>	→	λ
431	<scroll-tail-grp>	→	<concat-str>
432	<scroll-tail-grp>	→	<eq-op><scroll-val><bool-exp-grp>
433	<scroll-tail-grp>	→	λ
434	<concat-grp>	→	& <scroll-val> <concat-grp>



435	<concat-grp>	→	λ
436	<exp-grp>	→	<dime-arith><arith-grp><releqvar-grp>
437	<exp-grp>	→	<rel><arith-grp2><logeq-grp>
438	<exp-grp>	→	<log-op><bool-ope-grp><bool-exp-grp>
439	<exp-grp>	→	<eq-op><eq-ope-grp><bool-exp-grp>
440	<exp-grp>	→	& <scroll-val><concat-grp>
441	<exp-grp>	→	λ
442	<const>	→	LOCKE <const-init>!!
443	<const-init>	→	COIN <coin-locke><coin-locke-mult>
444	<const-init>	→	DIME <dime-locke><dime-locke-mult>
445	<const-init>	→	PARCH <parch-locke><parch-locke-mult>
446	<const-init>	→	SCROLL <scroll-locke><scroll-locke-mult>
447	<const-init>	→	BOOL <bool-locke><bool-locke-mult>
448	<coin-locke>	→	id = COIN-lit
449	<coin-locke-mult>	→	, <coin-locke><coin-locke-mult>
450	<coin-locke-mult>	→	λ
451	<dime-locke>	→	id = <locke-digit>
452	<locke-digit>	→	COIN-lit
453	<locke-digit>	→	DIME-lit
454	<dime-locke-mult>	→	, <dime-locke><dime-locke-mult>
455	<dime-locke-mult>	→	λ
456	<parch-locke>	→	id = PARCH-lit
457	<parch-locke-mult>	→	, <parch-locke><parch-locke-mult>
458	<parch-locke-mult>	→	λ
459	<scroll-locke>	→	id = SCROLL-lit<scr-id>
460	<scr-id>	→	{COIN-lit}
461	<scr-id>	→	λ
462	<scroll-locke-mult>	→	, <scroll-locke> <scroll-locke-mult>
463	<scroll-locke-mult>	→	λ
464	<bool-locke>	→	id = <locke-bool>
465	<locke-bool>	→	AYE
466	<locke-bool>	→	NAY
467	<bool-locke-mult>	→	, <bool-locke> <bool-locke-mult>
468	<bool-locke-mult>	→	λ
469	<struct>	→	MAST id [<mem-dec> <mem-dec-tail>] !! <struct>
470	<struct>	→	λ
471	<mem-dec>	→	<d-type> id <mem-mult>!!
472	<mem-mult>	→	, id <mem-mult>
473	<mem-mult>	→	λ
474	<mem-dec-tail>	→	<mem-dec> <mem-dec-tail>



475	<mem-dec-tail>	→	λ
476	<sub-func>	→	<return-func>
477	<sub-func>	→	<nonreturn-func>
478	<sub-func>	→	λ
479	<return-func>	→	COIN id<coin-func>
480	<return-func>	→	DIME id<dime-func>
481	<return-func>	→	PARCH id<parch-func>
482	<return-func>	→	SCROLL id<scroll-func>
483	<return-func>	→	BOOL id<bool-func>
484	<nonreturn-func>	→	ABYSS id (<params>) [<ahoy-local-dec> <nonret-stmnts> <nonret-back>] <sub-func>
485	<nonret-stmnts>	→	<nonret-stmnt><nonret-tail>
486	<nonret-tail>	→	<nonret-stmnts>
487	<nonret-tail>	→	λ
488	<nonret-back>	→	BACK!!
489	<nonret-back>	→	λ
490	<nonret-stmnt>	→	id <assign-tail> !!
491	<nonret-stmnt>	→	ASK (SCROLL-lit, <addr>) !!
492	<nonret-stmnt>	→	ECHO (SCROLL-lit <args-mult>) !!
493	<nonret-stmnt>	→	LOOK (<condition>) [<look-body> <jump-stmnt>] <nonret-look-tail>
494	<nonret-stmnt>	→	CHART (<chart-cond>) [<courses> <course-tail> <adrift-case>]
495	<nonret-stmnt>	→	HOIST (<hoist-init>!! <hoist-cond>!! <hoist-upd>) [<look-body> <jump-stmnt>]
496	<nonret-stmnt>	→	HEAVE (<condition>) [<look-body> <jump-stmnt>]
497	<nonret-stmnt>	→	HAUL [<look-body> <jump-stmnt>] HEAVE (<condition>)!!
498	<nonret-stmnt>	→	<unary-exp>!!
499	<nonret-look-tail>	→	DROPLOOK (<condition>) [<look-body> <jump-stmnt>] <nonret-look-tail>
500	<nonret-look-tail>	→	DROP [<look-body> <jump-stmnt>]
501	<nonret-look-tail>	→	λ
502	<local-dec>	→	<var-arr><local-dec>
503	<local-dec>	→	<struct-dec>
504	<local-dec>	→	λ
505	<var-arr>	→	COIN id <coin-var-arr>!!
506	<var-arr>	→	DIME id <dime-var-arr>!!
507	<var-arr>	→	PARCH id <parch-var-arr>!!
508	<var-arr>	→	SCROLL id <scroll-var-arr>!!
509	<var-arr>	→	BOOL id <bool-var-arr>!!
510	<struct-dec>	→	MAST id id <str-dec-init>!! <struct-dec>
511	<struct-dec>	→	λ



512	<str-dec-init>	→	, id <str-dec-tail>
513	<str-dec-init>	→	=[<str-val><str-val-tail>]
514	<str-dec-init>	→	λ
515	<str-dec-tail>	→	, id <str-dec-tail>
516	<str-dec-tail>	→	λ
517	<str-val>	→	<value-str>
518	<str-val>	→	\$id = <value-str>
519	<str-val-tail>	→	, <str-val> <str-val-tail>
520	<str-val-tail>	→	λ
521	<value-str>	→	id<id-tail><exp-str>
522	<value-str>	→	(<value-grp>)<exp-str>
523	<value-str>	→	<digit><digit-tail-str>
524	<value-str>	→	PARCH-lit<parch-tail-str>
525	<value-str>	→	SCROLL-lit<scr-char><scroll-tail-str>
526	<value-str>	→	<bool><logeq-str>
527	<digit-tail-str>	→	<arith-str><releq-str>
528	<digit-tail-str>	→	λ
529	<arith-str>	→	<dime-arith><arith-str>
530	<arith-str>	→	λ
531	<releq-str>	→	<rel><arith-str2><logeq-str>
532	<releq-str>	→	<eq-digit><arith-str3><bool-exp-arr>
533	<releq-str>	→	λ
534	<arith-str2>	→	<dime-arith><arith-str2>
535	<arith-str2>	→	λ
536	<arith-str3>	→	<dime-arith><arith-str3>
537	<arith-str3>	→	λ
538	<logeq-str>	→	<logeq-op><log-ope-str><bool-exp-arr>
539	<logeq-str>	→	λ
540	<parch-tail-str>	→	<eq-op>PARCH-lit<bool-exp-arr>
541	<parch-tail-str>	→	λ
542	<scroll-tail-str>	→	<concat-str>
543	<scroll-tail-str>	→	<eq-op><scroll-val><bool-exp-arr>
544	<scroll-tail-str>	→	λ
545	<concat-str>	→	& <scroll-val> <concat-str>
546	<concat-str>	→	λ
547	<exp-str>	→	<dime-arith><arith-str><releq-str>
548	<exp-str>	→	<rel><arith-str2><logeq-str>
549	<exp-str>	→	<log-op><bool-ope-arr><bool-exp-arr>
550	<exp-str>	→	<eq-op><eq-ope-arr><bool-exp-arr>
551	<exp-str>	→	& <scroll-val><concat-str>

552	<exp-str>	→	λ
553	<statements>	→	<assign-stmnt>
554	<statements>	→	<ask-stmnt>
555	<statements>	→	<echo-stmnt>
556	<statements>	→	<look-stmnt>
557	<statements>	→	<chart-stmnt>
558	<statements>	→	<hoist-stmnt>
559	<statements>	→	<heave-stmnt>
560	<statements>	→	<haul-stmnt>
561	<statements>	→	<unary-exp>!!
562	<assign-stmnt>	→	id<assign-tail>!!
563	<assign-tail>	→	<arr-str><assign-body>
564	<assign-tail>	→	(<args>)
565	<arr-str>	→	{<index>}<arr-str-tail>
566	<arr-str>	→	\$id
567	<arr-str>	→	λ
568	<arr-str-tail>	→	{<index>}
569	<arr-str-tail>	→	λ
570	<assign-body>	→	= <value-as>
571	<assign-body>	→	<arith-assign-op><dime-ret-val>
572	<value-as>	→	id<id-tail><exp-as>
573	<value-as>	→	(<value-grp>)<exp-as>
574	<value-as>	→	<digit><digit-tail-as>
575	<value-as>	→	PARCH-lit<parch-tail-as>
576	<value-as>	→	SCROLL-lit<scr-char><scroll-tail-as>
577	<value-as>	→	<bool><logeq-as>
578	<digit-tail-as>	→	<arith-as><releq-as>
579	<digit-tail-as>	→	λ
580	<arith-as>	→	<dime-arith><arith-as>
581	<arith-as>	→	λ
582	<releq-as>	→	<rel><arith-as2><logeq-as>
583	<releq-as>	→	<eq-digit><arith-as3><bool-exp-ret>
584	<releq-as>	→	λ
585	<arith-as2>	→	<dime-arith><arith-as2>
586	<arith-as2>	→	λ
587	<arith-as3>	→	<dime-arith><arith-as3>
588	<arith-as3>	→	λ
589	<logeq-as>	→	<logeq-op><bool-ope-ret><bool-exp-ret>
590	<logeq-as>	→	λ
591	<parch-tail-as>	→	<eq-op>PARCH-lit<log-as>
592	<parch-tail-as>	→	λ



593	<scroll-tail-as>	→	<concat-as>
594	<scroll-tail-as>	→	<eq-op><scroll-val><bool-exp-ret>
595	<scroll-tail-as>	→	λ
596	<concat-as>	→	& <scroll-val> <concat-as>
597	<concat-as>	→	λ
598	<exp-as>	→	<dime-arith><arith-as><releq-as>
599	<exp-as>	→	<rel><arith-as2><logeq-as>
600	<exp-as>	→	<log-op><bool-ope-ret><bool-exp-ret>
601	<exp-as>	→	<eq-op><eq-ope-ret><bool-exp-ret>
602	<exp-as>	→	& <scroll-val><concat-as>
603	<exp-as>	→	λ
604	<arith-assign-op>	→	+=
605	<arith-assign-op>	→	-=
606	<arith-assign-op>	→	*=
607	<arith-assign-op>	→	/=
608	<arith-assign-op>	→	%=
609	<arith-assign-op>	→	^=
610	<ask-stmnt>	→	ASK (SCROLL-lit, <addr>) !!
611	<addr>	→	@id<arr-str><addr-tail>
612	<addr-tail>	→	, <addr>
613	<addr-tail>	→	λ
614	<echo-stmnt>	→	ECHO (SCROLL-lit <args-mult>) !!
615	<look-stmnt>	→	LOOK (<condition>) [<look-body> <jump-stmnt>] <look-tail>
616	<condition>	→	<bool-grp-val>
617	<look-body>	→	<statements><look-body>
618	<look-body>	→	λ
619	<jump-stmnt>	→	SAIL!!
620	<jump-stmnt>	→	LAND!!
621	<jump-stmnt>	→	λ
622	<look-tail>	→	DROPLOOK (<condition>) [<look-body> <jump-stmnt>] <look-tail>
623	<look-tail>	→	DROP [<look-body> <jump-stmnt>]
624	<look-tail>	→	λ
625	<chart-stmnt>	→	CHART (<chart-cond>) [<courses> <course-tail> <adrift-case>]
626	<chart-cond>	→	id<id-tail>
627	<chart-cond>	→	<chart-const>
628	<chart-const>	→	COIN-lit
629	<chart-const>	→	PARCH-lit
630	<chart-const>	→	SCROLL-lit<scr-id>
631	<courses>	→	COURSE <chart-const> : <course-body> <course-jmp>
632	<course-body>	→	<statements><course-body>



633	<course-body>	→	λ
634	<course-jmp>	→	SAIL!!
635	<course-jmp>	→	LAND!!
636	<course-jmp>	→	λ
637	<course-tail>	→	<courses><course-tail>
638	<course-tail>	→	λ
639	<adrift-case>	→	ADRIFT : <adrift-body> LAND!!
640	<adrift-case>	→	λ
641	<adrift-body>	→	<statements><adrift-body>
642	<adrift-body>	→	λ
643	<hoist-stmnt>	→	HOIST (<hoist-init>!! <hoist-cond>!! <hoist-upd>) [<look-body> <jump-stmnt>]
644	<hoist-init>	→	COIN id = <coin-val> <init1-mult>
645	<hoist-init>	→	id<arr-str> = <coin-val> <init2-mult>
646	<hoist-init>	→	λ
647	<init1-mult>	→	, id = <coin-val> <init1-mult>
648	<init1-mult>	→	λ
649	<init2-mult>	→	, id<arr-str> = <coin-val> <init2-mult>
650	<init2-mult>	→	λ
651	<hoist-cond>	→	<dime-val><bool-arith><releq-op><dime-val><bool-arith3-grp> <hoist-log>
652	<releq-op>	→	<rel-op>
653	<releq-op>	→	<eq-op>
654	<hoist-log>	→	<log-op><hoist-cond>
655	<hoist-log>	→	λ
656	<hoist-upd>	→	<upd><upd-mult>
657	<upd>	→	<unary-op>id<arr-str>
658	<upd>	→	id<arr-str><arith-assign-op><dime-grp-val>
659	<upd-mult>	→	, <upd><upd-mult>
660	<upd-mult>	→	λ
661	<heave-stmnt>	→	HEAVE (<condition>) [<look-body> <jump-stmnt>]
662	<haul-stmnt>	→	HAUL [<look-body> <jump-stmnt>] HEAVE (<condition>)!!
663	<unary-exp>	→	<unary-op>id<arr-str>
664	<unary-op>	→	+#
665	<unary-op>	→	-#
666	<ahoy-local-dec>	→	<var-arr><ahoy-local-dec>
667	<ahoy-local-dec>	→	<ahoy-struct-dec>
668	<ahoy-local-dec>	→	λ
669	<ahoy-struct-dec>	→	MAST id id <str-dec-init>!! <ahoy-struct-dec>
670	<ahoy-struct-dec>	→	λ
671	<ahoy-stmnts>	→	<ahoy-stmnt><ahoy-tail>



672	<ahoy-tail>	→	<ahoy-stmnts>
673	<ahoy-tail>	→	λ
674	<ahoy-stmnt>	→	id<assign-tail>!!
675	<ahoy-stmnt>	→	ASK (SCROLL-lit, <addr>) !!
676	<ahoy-stmnt>	→	ECHO (SCROLL-lit <args-mult>) !!
677	<ahoy-stmnt>	→	LOOK (<condition>) [<look-body> <jump-stmnt>]
678	<ahoy-stmnt>	→	CHART (<chart-cond>) [<courses> <course-tail> <adrift-case>]
679	<ahoy-stmnt>	→	HOIST (<hoist-init>!! <hoist-cond>!! <hoist-upd>) [<look-body>
680	<ahoy-stmnt>	→	HEAVE (<condition>) [<look-body> <jump-stmnt>]
681	<ahoy-stmnt>	→	HAUL [<look-body> <jump-stmnt>] HEAVE (<condition>)!!
682	<ahoy-stmnt>	→	<unary-exp>!!
683	<ahoy-look-tail>	→	DROLOOK (<condition>) [<look-body><jump-stmnt>] <ahoy-look-tail>
684	<ahoy-look-tail>	→	DROP [<look-body><jump-stmnt>]
685	<ahoy-look-tail>	→	λ



VIII. FIRST SET

NO.	PRODUCTION	→	FIRST SET
1	First (<program>)	→	COIN, DIME, PARCH, SCROLL, BOOL, LOCKE, MAST, ABYSS, AHOY
2	First (<global-dec>)	→	COIN, DIME, PARCH, SCROLL, BOOL, LOCKE, MAST, ABYSS, λ
3	First (<var-arr-func>)	→	COIN, DIME, PARCH, SCROLL, BOOL
4	First (<coin-var-arr-func>)	→	=, ,, {, {, !!
5	First (<coin-var-arr>)	→	,, =, {, λ
6	First (<coin-var>)	→	=, ,, λ
7	First (<coin-init>)	→	=, λ
8	First (<coin-init-mult>)	→	,, λ
9	First (<coin-val>)	→	{, -, COIN-lit, id
10	First (<neg>)	→	-, λ
11	First (<neg-coin-val>)	→	{, id
12	First (<coin-exp>)	→	%, *, +, -, /, ^, λ
13	First (<coin-arith>)	→	%, *, +, -, /, ^
14	First (<coin-val-grp>)	→	{, -, COIN-lit, id
15	First (<coin-grp-exp>)	→	%, *, +, -, /, ^, λ
16	First (<arith-op>)	→	%, *, +, -, /, ^
17	First (<coin-arr>)	→	{
18	First (<coin-arr-tail>)	→	=, {, λ
19	First (<coin-arr1>)	→	{, -, COIN-lit, id
20	First (<coin-arr-val>)	→	{, -, COIN-lit, id
21	First (<coin-arr-exp>)	→	%, *, +, -, /, ^, λ
22	First (<cav-tail>)	→	,, λ
23	First (<coin-arr2-tail>)	→	=, λ
24	First (<coin-arr2>)	→	[
25	First (<cav2-tail>)	→	,, λ
26	First (<coin-func>)	→	{
27	First (<coin-ret-val>)	→	{, -, COIN-lit, id
28	First (<coin-ret-exp>)	→	%, *, +, -, /, ^, λ
29	First (<dime-var-arr-func>)	→	{, ,, =, {, !!
30	First (<dime-var-arr>)	→	,, =, {, λ
31	First (<dime-var>)	→	,, =, λ
32	First (<dime-init>)	→	=, λ
33	First (<dime-init-mult>)	→	,, λ
34	First (<dime-init-val>)	→	{, -, COIN-lit, DIME-lit, id
35	First (<dime-val>)	→	{, -, COIN-lit, DIME-lit, id
36	First (<neg-dime-val>)	→	{, id



37	First (<dime-exp>)	→	%, *, +, -, /, ^, λ
38	First (<dime-arith>)	→	%, *, +, -, /, ^
39	First (<dime-grp-val>)	→	(, -, COIN-lit, DIME-lit, id
40	First (<dime-grp-exp>)	→	%, *, +, -, /, ^, λ
41	First (<dime-arr>)	→	{
42	First (<dime-arr-tail>)	→	=, {, λ
43	First (<dime-arr1>)	→	(, -, COIN-lit, DIME-lit, id
44	First (<dime-arr-val>)	→	(, -, COIN-lit, DIME-lit, id
45	First (<dime-arr-exp>)	→	%, *, +, -, /, ^, λ
46	First (<dav-tail>)	→	„ λ
47	First (<dime-arr2-tail>)	→	=, λ
48	First (<dime-arr2>)	→	[
49	First (<dav2-tail>)	→	„ λ
50	First (<dime-func>)	→	(
51	First (<dime-ret-val>)	→	(, -, COIN-lit, DIME-lit, id
52	First (<dime-ret-exp>)	→	%, *, +, -, /, ^, λ
53	First (<parch-var-arr-func>)	→	(, „ =, {, !!
54	First (<parch-var-arr>)	→	„ =, {, λ
55	First (<parch-var>)	→	„ =, λ
56	First (<parch-init>)	→	=, λ
57	First (<parch-init-mult>)	→	„ λ
58	First (<parch-val>)	→	PARCH-lit, id
59	First (<parch-arr>)	→	{
60	First (<parch-arr-tail>)	→	=, {, λ
61	First (<parch-arr1>)	→	PARCH-lit, id
62	First (<pav-tail>)	→	„ λ
63	First (<parch-arr2-tail>)	→	=, λ
64	First (<parch-arr2>)	→	[
65	First (<pav2-tail>)	→	„ λ
66	First (<parch-func>)	→	(
67	First (<scroll-var-arr-func>)	→	(, „ =, {, !!
68	First (<scroll-var-arr>)	→	„ =, {, λ
69	First (<scroll-var>)	→	„ =, λ
70	First (<scroll-init>)	→	=, λ
71	First (<scroll-init-mult>)	→	„ λ
72	First (<scroll-init-val>)	→	(, SCROLL-lit, id
73	First (<scroll-val>)	→	(, SCROLL-lit, id
74	First (<scr-char>)	→	{, λ



75	First (<index>)	→	COIN-lit, id
76	First (<scroll-exp>)	→	&, λ
77	First (<scroll-concat>)	→	&
78	First (<scroll-grp-val>)	→	(, SCROLL-lit, id
79	First (<scroll-grp-exp>)	→	&, λ
80	First (<scroll-arr>)	→	{
81	First (<scroll-arr-tail>)	→	=, {, λ
82	First (<scroll-arr1>)	→	(, SCROLL-lit, id
83	First (<scroll-arr-val>)	→	(, SCROLL-lit, id
84	First (<scroll-arr-exp>)	→	&, λ
85	First (<sav-tail>)	→	„ λ
86	First (<scroll-arr2-tail>)	→	=, λ
87	First (<scroll-arr2>)	→	[
88	First (<sav2-tail>)	→	„ λ
89	First (<scroll-func>)	→	(
90	First (<scroll-ret-val>)	→	(, SCROLL-lit, id
91	First (<scroll-ret-exp>)	→	&, λ
92	First (<bool-var-arr-func>)	→	„ =, {, (
93	First (<bool-var-arr>)	→	„ =, {, λ
94	First (<bool-var>)	→	„ =, λ
95	First (<bool-init>)	→	=, λ
96	First (<bool-init-mult>)	→	„ λ
97	First (<bool-init-val>)	→	-, id, (, !, !#, AYE, NAY, COIN-lit, DIME-lit, PARCH-lit, SCROLL-lit
98	First (<bool-val>)	→	-, id, (, !, !#, AYE, NAY, COIN-lit, DIME-lit, PARCH-lit, SCROLL-lit
99	First (<bool>)	→	AYE, NAY, !, !#
100	First (<bool-lit>)	→	AYE, NAY
101	First (<not-op>)	→	!, !#
102	First (<not-ope>)	→	id, (, AYE, NAY
103	First (<bool-eq>)	→	==, !=, λ
104	First (<digit>)	→	-, COIN-lit, DIME-lit
105	First (<neg-digit>)	→	(, id
106	First (<bool-arith>)	→	%, *, +, -, /, ^, λ
107	First (<rel-eq>)	→	<, >, <=, >=, ==, !=
108	First (<rel>)	→	<, >, <=, >=
109	First (<rel-op>)	→	<, >, <=, >=
110	First (<bool-arith2>)	→	%, *, +, -, /, ^, λ
111	First (<eq-digit>)	→	==, !=
112	First (<eq-op>)	→	==, !=



113	First (<bool-arith3>)	→	%, *, +, -, /, ^, λ
114	First (<bool-exp2>)	→	%, *, +, -, /, ^, <, >, <=, >=, ==, !=, λ
115	First (<eq-ope>)	→	-, id, (, !, !#, AYE, NAY, COIN-lit, DIME-lit, PARCH-lit, SCROLL-lit
116	First (<eq-ope1>)	→	PARCH-lit, SCROLL-lit, AYE, NAY, !, !#
117	First (<bool-arith4>)	→	%, *, +, -, /, ^, λ
118	First (<bool-rel>)	→	<, >, <=, >=, λ
119	First (<bool-exp>)	→	&&, , λ
120	First (<log-op>)	→	&&,
121	First (<bool-grp-val>)	→	-, id, (, !, !#, AYE, NAY, COIN-lit, DIME-lit, PARCH-lit, SCROLL-lit
122	First (<bool-ope-grp>)	→	-, id, (, !, !#, AYE, NAY, COIN-lit, DIME-lit, PARCH-lit, SCROLL-lit
123	First (<bool-eq-grp>)	→	==, !=, λ
124	First (<bool-eq-arr>)	→	==, !=, λ
125	First (<releq-grp>)	→	<, >, <=, >=, ==, !=
126	First (<bool-arith2-grp>)	→	%, *, +, -, /, ^, λ
127	First (<bool-arith3-grp>)	→	%, *, +, -, /, ^, λ
128	First (<bool-exp2-grp>)	→	%, *, +, -, /, ^, <, >, <=, >=, ==, !=, λ
129	First (<eq-ope-grp>)	→	-, id, (, !, !#, AYE, NAY, COIN-lit, DIME-lit, PARCH-lit, SCROLL-lit
130	First (<bool-arith4-grp>)	→	%, *, +, -, /, ^, λ
131	First (<bool-rel-grp>)	→	<, >, <=, >=, λ
132	First (<bool-exp-grp>)	→	&&, , λ
133	First (<bool-arr>)	→	{
134	First (<bool-arr-tail>)	→	=, {, λ
135	First (<bool-arr1>)	→	-, id, (, !, !#, AYE, NAY, COIN-lit, DIME-lit, PARCH-lit, SCROLL-lit
136	First (<bav-tail>)	→	„ λ
137	First (<bool-arr-val>)	→	-, id, (, !, !#, AYE, NAY, COIN-lit, DIME-lit, PARCH-lit, SCROLL-lit
138	First (<bool-ope-arr>)	→	-, id, (, !, !#, AYE, NAY, COIN-lit, DIME-lit, PARCH-lit, SCROLL-lit
139	First (<releq-arr>)	→	<, >, <=, >=, ==, !=
140	First (<bool-arith2-arr>)	→	%, *, +, -, /, ^, λ
141	First (<bool-arith3-arr>)	→	%, *, +, -, /, ^, λ
142	First (<bool-exp2-arr>)	→	%, *, +, -, /, ^, <, >, <=, >=, ==, !=, λ
143	First (<eq-ope-arr>)	→	-, id, (, !, !#, AYE, NAY, COIN-lit, DIME-lit, PARCH-lit, SCROLL-lit
144	First (<bool-arith4-arr>)	→	%, *, +, -, /, ^, λ
145	First (<bool-rel-arr>)	→	<, >, <=, >=, λ
146	First (<bool-exp-arr>)	→	&&, , λ
147	First (<bool-arr2-tail>)	→	=, λ
148	First (<bool-arr2>)	→	[
149	First (<bav2-tail>)	→	„ λ
150	First (<bool-func>)	→	(

151	First (<bool-ret-val>)	→ -, id, (, !, !#, AYE, NAY, COIN-lit, DIME-lit, PARCH-lit, SCROLL-lit
152	First (<bool-ope-ret>)	→ -, id, (, !, !#, AYE, NAY, COIN-lit, DIME-lit, PARCH-lit, SCROLL-lit
153	First (<bool-eq-ret>)	→ ==, !=, λ
154	First (<releq-ret>)	→ <, >, <=, >=, ==, !=
155	First (<bool-arith2-ret>)	→ %, *, +, -, /, ^, λ
156	First (<bool-arith3-ret>)	→ %, *, +, -, /, ^, λ
157	First (<bool-exp2-ret>)	→ %, *, +, -, /, ^, <, >, <=, >=, ==, !=, λ
158	First (<eq-ope-ret>)	→ -, id, (, !, !#, AYE, NAY, COIN-lit, DIME-lit, PARCH-lit, SCROLL-lit
159	First (<bool-arith4-ret>)	→ %, *, +, -, /, ^, λ
160	First (<bool-rel-ret>)	→ <, >, <=, >=, λ
161	First (<bool-exp-ret>)	→ &&, , λ
162	First (<params>)	→ COIN, DIME, PARCH, SCROLL, BOOL, λ
163	First (<param-mult>)	→ ,, λ
164	First (<d-type>)	→ COIN, DIME, PARCH, SCROLL, BOOL
165	First (<ret-stmnts>)	→ id, ASK, ECHO, LOOK, CHART, HOIST, HEAVE, HAUL, +#, -#, λ
166	First (<id-tail>)	→ {, \$, (, λ
167	First (<elmt2>)	→ {, λ
168	First (<args>)	→ -, id, (, !, !#, AYE, NAY, COIN-lit, DIME-lit, PARCH-lit, SCROLL-lit, λ
169	First (<args-mult>)	→ ,, λ
170	First (<value>)	→ -, id, (, !, !#, AYE, NAY, COIN-lit, DIME-lit, PARCH-lit, SCROLL-lit
171	First (<digit-tail>)	→ %, *, +, -, /, ^, <, >, <=, >=, ==, !=, λ
172	First (<arith-var>)	→ %, *, +, -, /, ^, λ
173	First (<releq-var>)	→ <, >, <=, >=, ==, !=, λ
174	First (<arith-var2>)	→ %, *, +, -, /, ^, λ
175	First (<arith-var3>)	→ %, *, +, -, /, ^, λ
176	First (<logeq-var>)	→ &&, , !=, ==, λ
177	First (<logeq-op>)	→ &&, , !=, ==
178	First (<log-var>)	→ &&, , λ
179	First (<parch-tail>)	→ ==, !=, λ
180	First (<scroll-tail>)	→ &, ==, !=, λ
181	First (<concat-var>)	→ &, λ
182	First (<exp>)	→ %, *, +, -, /, ^, <, >, <=, >=, &&, , !=, ==, &, λ
183	First (<log-ope>)	→ -, id, (, !, !#, AYE, NAY, COIN-lit, DIME-lit, PARCH-lit, SCROLL-lit
184	First (<log-booleq>)	→ ==, !=, λ
185	First (<log-releq>)	→ <, >, <=, >=, ==, !=
186	First (<log-exp>)	→ %, *, +, -, /, ^, <, >, <=, >=, ==, !=, λ
187	First (<eq-var>)	→ -, id, (, !, !#, AYE, NAY, COIN-lit, DIME-lit, PARCH-lit, SCROLL-lit
188	First (<arith-var4>)	→ %, *, +, -, /, ^, λ



189	First (<rel-var>)	→	<, >, <=, >=, λ
190	First (<value-grp>)	→	-, id, (, !, !#, AYE, NAY, COIN-lit, DIME-lit, PARCH-lit, SCROLL-lit
191	First (<digit-tail-grp>)	→	%, *, +, -, /, ^, <, >, <=, >=, ==, !=, λ
192	First (<arith-grp>)	→	%, *, +, -, /, ^, λ
193	First (<releqvar-grp>)	→	<, >, <=, >=, ==, !=, λ
194	First (<arith-grp2>)	→	%, *, +, -, /, ^, λ
195	First (<arith-grp3>)	→	%, *, +, -, /, ^, λ
196	First (<logeq-grp>)	→	&&, , !=, ==, λ
197	First (<parch-tail-grp>)	→	==, !=, λ
198	First (<scroll-tail-grp>)	→	&, ==, !=, λ
199	First (<concat-grp>)	→	&, λ
200	First (<exp-grp>)	→	%, *, +, -, /, ^, <, >, <=, >=, ==, !=, &&, , &, λ
201	First (<const>)	→	LOCKE
202	First (<const-init>)	→	COIN, DIME, PARCH, SCROLL, BOOL
203	First (<coin-locke>)	→	id
204	First (<coin-locke-mult>)	→	„ λ
205	First (<dime-locke>)	→	id
206	First (<locke-digit>)	→	COIN-lit, DIME-lit
207	First (<dime-locke-mult>)	→	„ λ
208	First (<parch-locke>)	→	id
209	First (<parch-locke-mult>)	→	„ λ
210	First (<scroll-locke>)	→	id
211	First (<scr-id>)	→	{, λ
212	First (<scroll-locke-mult>)	→	„ λ
213	First (<bool-locke>)	→	id
214	First (<locke-bool>)	→	AYE, NAY
215	First (<bool-locke-mult>)	→	„ λ
216	First (<struct>)	→	MAST, λ
217	First (<mem-dec>)	→	COIN, DIME, PARCH, SCROLL, BOOL
218	First (<mem-mult>)	→	„ λ
219	First (<mem-dec-tail>)	→	COIN, DIME, PARCH, SCROLL, BOOL, λ
220	First (<sub-func>)	→	COIN, DIME, PARCH, SCROLL, BOOL, ABYSS, λ
221	First (<return-func>)	→	COIN, DIME, PARCH, SCROLL, BOOL
222	First (<nonreturn-func>)	→	ABYSS
223	First (<nonret-stmnts>)	→	id, ASK, ECHO, LOOK, CHART, HOIST, HEAVE, HAUL, +#, -#
224	First (<nonret-tail>)	→	id, ASK, ECHO, LOOK, CHART, HOIST, HEAVE, HAUL, +#, -#, λ
225	First (<nonret-back>)	→	BACK, λ
226	First (<nonret-stmnt>)	→	id, ASK, ECHO, LOOK, CHART, HOIST, HEAVE, HAUL, +#, -#



227	First (<nonret-look-tail>)	→	DROPLOOK, DROP, λ
228	First (<local-dec>)	→	COIN, DIME, PARCH, SCROLL, BOOL, MAST, λ
229	First (<var-arr>)	→	COIN, DIME, PARCH, SCROLL, BOOL
230	First (<struct-dec>)	→	MAST, λ
231	First (<str-dec-init>)	→	„ =, λ
232	First (<str-dec-tail>)	→	„ λ
233	First (<str-val>)	→	-, id, (, !, !#, AYE, NAY, COIN-lit, DIME-lit, PARCH-lit, SCROLL-lit, \$
234	First (<str-val-tail>)	→	„ λ
235	First (<value-str>)	→	-, id, (, !, !#, AYE, NAY, COIN-lit, DIME-lit, PARCH-lit, SCROLL-lit
236	First (<digit-tail-str>)	→	%, *, +, -, /, ^, <, >, <=, >=, ==, !=, λ
237	First (<arith-str>)	→	%, *, +, -, /, ^, λ
238	First (<releq-str>)	→	<, >, <=, >=, ==, !=, λ
239	First (<arith-str2>)	→	%, *, +, -, /, ^, λ
240	First (<arith-str3>)	→	%, *, +, -, /, ^, λ
241	First (<logeq-str>)	→	&&, , !=, ==, λ
242	First (<parch-tail-str>)	→	!=, ==, λ
243	First (<scroll-tail-str>)	→	&, !=, ==, λ
244	First (<concat-str>)	→	&, λ
245	First (<exp-str>)	→	%, *, +, -, /, ^, <, >, <=, >=, &&, , ==, !=, &, λ
246	First (<statements>)	→	id, ASK, ECHO, LOOK, CHART, HOIST, HEAVE, HAUL, +#, -#
247	First (<assign-stmnt>)	→	id
248	First (<assign-tail>)	→	\$, {, =, +=, -=, *=, /=, %=, ^=, (
249	First (<arr-str>)	→	\$, {, λ
250	First (<arr-str-tail>)	→	{, λ
251	First (<assign-body>)	→	=, +=, -=, *=, /=, %=, ^=
252	First (<value-as>)	→	-, id, (, !, !#, AYE, NAY, COIN-lit, DIME-lit, PARCH-lit, SCROLL-lit
253	First (<digit-tail-as>)	→	%, *, +, -, /, ^, <, >, <=, >=, ==, !=, λ
254	First (<arith-as>)	→	%, *, +, -, /, ^, λ
255	First (<releq-as>)	→	<, >, <=, >=, ==, !=, λ
256	First (<arith-as2>)	→	%, *, +, -, /, ^, λ
257	First (<arith-as3>)	→	%, *, +, -, /, ^, λ
258	First (<logeq-as>)	→	&&, , !=, ==, λ
259	First (<parch-tail-as>)	→	!=, ==, λ
260	First (<scroll-tail-as>)	→	&, !=, ==, λ
261	First (<concat-as>)	→	&, λ
262	First (<exp-as>)	→	%, *, +, -, /, ^, <, >, <=, >=, &&, , ==, !=, &, λ
263	First (<arith-assign-op>)	→	+=, -=, *=, /=, %=, ^=
264	First (<ask-stmnt>)	→	ASK



265	First (<addr>)	→	@
266	First (<addr-tail>)	→	„ λ
267	First (<echo-stmnt>)	→	ECHO
268	First (<look-stmnt>)	→	LOOK
269	First (<condition>)	→	-, id, (, !, !#, AYE, NAY, COIN-lit, DIME-lit, PARCH-lit, SCROLL-lit
270	First (<look-body>)	→	id, ASK, ECHO, LOOK, CHART, HOIST, HEAVE, HAUL, +#, -#, λ
271	First (<jump-stmnt>)	→	LAND, SAIL, λ
272	First (<look-tail>)	→	DROPLOOK, DROP, λ
273	First (<chart-stmnt>)	→	CHART
274	First (<chart-cond>)	→	COIN-lit, PARCH-lit, SCROLL-lit, id
275	First (<chart-const>)	→	COIN-lit, PARCH-lit, SCROLL-lit
276	First (<courses>)	→	COURSE
277	First (<course-body>)	→	id, ASK, ECHO, LOOK, CHART, HOIST, HEAVE, HAUL, +#, -#, λ
278	First (<course-jmp>)	→	LAND, SAIL, λ
279	First (<course-tail>)	→	COURSE, λ
280	First (<adrift-case>)	→	ADRIFT, λ
281	First (<adrift-body>)	→	id, ASK, ECHO, LOOK, CHART, HOIST, HEAVE, HAUL, +#, -#, λ
282	First (<hoist-stmnt>)	→	HOIST
283	First (<hoist-init>)	→	COIN, id, λ
284	First (<init1-mult>)	→	„ λ
285	First (<init2-mult>)	→	„ λ
286	First (<hoist-cond>)	→	(, -, COIN-lit, DIME-lit, id
287	First (<releq-op>)	→	<, >, <=, >=, ==, !=
288	First (<hoist-log>)	→	&&, , λ
289	First (<hoist-upd>)	→	+#, -#, id
290	First (<upd>)	→	+#, -#, id
291	First (<upd-mult>)	→	„ λ
292	First (<heave-stmnt>)	→	HEAVE
293	First (<haul-stmnt>)	→	HAUL
294	First (<unary-exp>)	→	+#, -#
295	First (<unary-op>)	→	+#, -#
296	First (<ahoy-local-dec>)	→	COIN, DIME, PARCH, SCROLL, BOOL, MAST, λ
297	First (<ahoy-struct-dec>)	→	MAST, λ
298	First (<ahoy-stmnts>)	→	id, ASK, ECHO, LOOK, CHART, HOIST, HEAVE, HAUL, +#, -#
299	First (<ahoy-tail>)	→	id, ASK, ECHO, LOOK, CHART, HOIST, HEAVE, HAUL, +#, -#, λ
300	First (<ahoy-stmnt>)	→	id, ASK, ECHO, LOOK, CHART, HOIST, HEAVE, HAUL, +#, -#
301	First (<ahoy-look-tail>)	→	DROPLOOK, DROP, λ

IX. FOLLOW SET

NO.	PRODUCTION	→	FOLLOW SET
1	Follow (<program>)	→	\$
2	Follow (<global-dec>)	→	AHOY
3	Follow (<var-arr-func>)	→	!!
4	Follow (<coin-var-arr-func>)	→	!!
5	Follow (<coin-var-arr>)	→	!!, -
6	Follow (<coin-var>)	→	!!
7	Follow (<coin-init>)	→	!!, (, ,, -, =, COIN-lit, [, id, {
8	Follow (<coin-init-mult>)	→	%, *, +, -, /, ^
9	Follow (<coin-val>)	→	!!, (, ,, -, =, COIN-lit, [, id, {
10	Follow (<neg>)	→	!!, -
11	Follow (<neg-coin-val>)	→	!!, (, ,, -, =, COIN-lit, [, id, {
12	Follow (<coin-exp>)	→	)
13	Follow (<coin-arith>)	→	!, !!, !#, !=, %, &, &&, (,), *, +, ,, -, /, <, <=, =, ==, >, >=, AYE, BOOL, COIN, COIN-lit, DIME, DIME-lit, HAUL, NAY, PARCH, PARCH-lit, SCROLL, SCROLL-lit, [,], ^, id, {,
14	Follow (<coin-val-grp>)	→	!, !!, !#, !=, %, &, &&, (,), *, +, ,, -, /, <, <=, =, ==, >, >=, AYE, BOOL, COIN, COIN-lit, DIME, DIME-lit, HAUL, NAY, PARCH, PARCH-lit, SCROLL, SCROLL-lit, [,], ^, id, {,
15	Follow (<coin-grp-exp>)	→	!!
16	Follow (<arith-op>)	→	!!
17	Follow (<coin-arr>)	→	]
18	Follow (<coin-arr-tail>)	→	[
19	Follow (<coin-arr1>)	→	[
20	Follow (<coin-arr-val>)	→	]
21	Follow (<coin-arr-exp>)	→	!!
22	Follow (<cav-tail>)	→	]
23	Follow (<coin-arr2-tail>)	→	]
24	Follow (<coin-arr2>)	→	ABYSS
25	Follow (<cav2-tail>)	→	!!
26	Follow (<coin-func>)	→	!!
27	Follow (<coin-ret-val>)	→	!!
28	Follow (<coin-ret-exp>)	→	!!
29	Follow (<dime-var-arr-func>)	→	!!
30	Follow (<dime-var-arr>)	→	(, -, COIN-lit, DIME-lit, id
31	Follow (<dime-var>)	→	!!
32	Follow (<dime-init>)	→	(, -, COIN-lit, DIME-lit, id



33	Follow (<dime-init-mult>)	→	!, !!, !#, !=, %, &, &&, (,), *, +, ,, -, /, <, <=, =, ==, >, >=, AYE, BOOL, COIN, COIN-lit, DIME, DIME-lit, HAUL, NAY, PARCH, PARCH-lit, SCROLL, SCROLL-lit, [,], ^, id, {,
34	Follow (<dime-init-val>)	→	!, !!, !#, !=, %, &, &&, (,), *, +, ,, -, /, <, <=, =, ==, >, >=, AYE, BOOL, COIN, COIN-lit, DIME, DIME-lit, HAUL, NAY, PARCH, PARCH-lit, SCROLL, SCROLL-lit, [,], ^, id, {,
35	Follow (<dime-val>)	→	(, -, COIN-lit, DIME-lit, id
36	Follow (<neg-dime-val>)	→	!, !!, !#, !=, %, &, &&, (,), *, +, ,, -, /, <, <=, =, ==, >, >=, AYE, BOOL, COIN, COIN-lit, DIME, DIME-lit, HAUL, NAY, PARCH, PARCH-lit, SCROLL, SCROLL-lit, [,], ^, id, {,
37	Follow (<dime-exp>)	→	)
38	Follow (<dime-arith>)	→	)
39	Follow (<dime-grp-val>)	→	!!
40	Follow (<dime-grp-exp>)	→	!!
41	Follow (<dime-arr>)	→	]
42	Follow (<dime-arr-tail>)	→	[
43	Follow (<dime-arr1>)	→	[
44	Follow (<dime-arr-val>)	→	]
45	Follow (<dime-arr-exp>)	→	!!
46	Follow (<dav-tail>)	→	]
47	Follow (<dime-arr2-tail>)	→	]
48	Follow (<dime-arr2>)	→	ABYSS
49	Follow (<dav2-tail>)	→	!!
50	Follow (<dime-func>)	→	!!
51	Follow (<dime-ret-val>)	→	!!
52	Follow (<dime-ret-exp>)	→	!!
53	Follow (<parch-var-arr-func>)	→	!!
54	Follow (<parch-var-arr>)	→	!!
55	Follow (<parch-var>)	→	!!
56	Follow (<parch-init>)	→	!, !!, !#, !=, %, &, (,), *, +, ,, -, /, <, <=, =, ==, >, >=, AYE, COIN-lit, DIME-lit, NAY, PARCH-lit, SCROLL-lit, [, ^, id, {
57	Follow (<parch-init-mult>)	→	!!
58	Follow (<parch-val>)	→	!!
59	Follow (<parch-arr>)	→	]
60	Follow (<parch-arr-tail>)	→	]
61	Follow (<parch-arr1>)	→	!!
62	Follow (<pav-tail>)	→	]
63	Follow (<parch-arr2-tail>)	→	]
64	Follow (<parch-arr2>)	→	]
65	Follow (<pav2-tail>)	→	]



66	Follow (<parch-func>)	→	!!
67	Follow (<scroll-var-arr-func>)	→	!!
68	Follow (<scroll-var-arr>)	→	(, SCROLL-lit, id
69	Follow (<scroll-var>)	→	!!
70	Follow (<scroll-init>)	→	(, SCROLL-lit, id
71	Follow (<scroll-init-mult>)	→	!, !!, !#, !=, %, &, (,), *, +, +#, ,, -, -#, /, <, <=, =, ==, >, >=, ASK, AYE, CHART, COIN-lit, DIME-lit, ECHO, HAUL, HEAVE, HOIST, LOCKE, LOOK, NAY, PARCH-lit, SCROLL-lit, [, ^, id, {
72	Follow (<scroll-init-val>)	→	!, !!, !#, !=, %, &, &&, (,), *, +, +#, ,, -, -#, /, <, <=, =, ==, >, >=, ASK, AYE, CHART, COIN-lit, DIME-lit, ECHO, HAUL, HEAVE, HOIST, LOCKE, LOOK, NAY, PARCH-lit, SCROLL-lit, [,], ^, id, {,
73	Follow (<scroll-val>)	→	}
74	Follow (<scr-char>)	→	(, SCROLL-lit, id
75	Follow (<index>)	→	!!, (,), ,, =, SCROLL-lit, [, id, {
76	Follow (<scroll-exp>)	→	)
77	Follow (<scroll-concat>)	→	)
78	Follow (<scroll-grp-val>)	→	!!
79	Follow (<scroll-grp-exp>)	→	!!
80	Follow (<scroll-arr>)	→	]
81	Follow (<scroll-arr-tail>)	→	[
82	Follow (<scroll-arr1>)	→	[
83	Follow (<scroll-arr-val>)	→	]
84	Follow (<scroll-arr-exp>)	→	!!
85	Follow (<sav-tail>)	→	]
86	Follow (<scroll-arr2-tail>)	→	]
87	Follow (<scroll-arr2>)	→	ABYSS
88	Follow (<sav2-tail>)	→	!!
89	Follow (<scroll-func>)	→	!!
90	Follow (<scroll-ret-val>)	→	!!
91	Follow (<scroll-ret-exp>)	→	!!
92	Follow (<bool-var-arr-func>)	→	!!
93	Follow (<bool-var-arr>)	→	!, !#, (, -, AYE, COIN-lit, DIME-lit, NAY, PARCH-lit, SCROLL-lit, id
94	Follow (<bool-var>)	→	!!
95	Follow (<bool-init>)	→	!, !#, (, -, AYE, COIN-lit, DIME-lit, NAY, PARCH-lit, SCROLL-lit, id
96	Follow (<bool-init-mult>)	→	!, !#, (, -, AYE, COIN-lit, DIME-lit, NAY, PARCH-lit, SCROLL-lit, id

97	Follow (<bool-init-val>)	→	!, !!, !#, !=, %, &, &&, (,), *, +, ,, -, /, <, <=, =, ==, >, >=, AYE, COIN-lit, DIME-lit, NAY, PARCH-lit, SCROLL-lit, [,], ^, id, {,
98	Follow (<bool-val>)	→	!, !!, !#, !=, %, &, &&, (,), *, +, ,, -, /, <, <=, =, ==, >, >=, AYE, COIN-lit, DIME-lit, NAY, PARCH-lit, SCROLL-lit, [,], ^, id, {,
99	Follow (<bool>)	→	-, COIN-lit, DIME-lit
100	Follow (<bool-lit>)	→	!, !!, !#, !=, %, &, &&, (,), *, +, ,, -, /, <, <=, =, ==, >, >=, AYE, COIN-lit, DIME-lit, NAY, PARCH-lit, SCROLL-lit, [,], ^, id, {,
101	Follow (<not-op>)	→	!, !#, (, -, AYE, COIN-lit, DIME-lit, NAY, PARCH-lit, SCROLL-lit, id
102	Follow (<not-ope>)	→	!, !!, !#, !=, %, &, &&, (,), *, +, ,, -, /, <, <=, =, ==, >, >=, AYE, BOOL, COIN, COIN-lit, DIME, DIME-lit, NAY, PARCH, PARCH-lit, SCROLL, SCROLL-lit, [,], ^, id, {,
103	Follow (<bool-eq>)	→	!, !!, !#, !=, %, &, &&, (,), *, +, ,, -, /, <, <=, =, ==, >, >=, AYE, BOOL, COIN, COIN-lit, DIME, DIME-lit, NAY, PARCH, PARCH-lit, SCROLL, SCROLL-lit, [,], ^, id, {,
104	Follow (<digit>)	→	!, !!, !#, %, (,), *, +, +#, ,, -, -#, /, <, <=, =, >, >=, AYE, COIN-lit, DIME-lit, NAY, PARCH-lit, SCROLL-lit, [, ^, id, {
105	Follow (<neg-digit>)	→	!, !#, (, -, AYE, COIN-lit, DIME-lit, NAY, PARCH-lit, SCROLL-lit, id
106	Follow (<bool-arith>)	→	!, !!, !#, !=, %, &, &&, (,), *, +, ,, -, /, <, <=, =, ==, >, >=, AYE, COIN-lit, DIME-lit, NAY, PARCH-lit, SCROLL-lit, [,], ^, id, {,
107	Follow (<rel-eq>)	→	!, !!, !#, !=, %, &, &&, (,), *, +, ,, -, /, <, <=, =, ==, >, >=, AYE, COIN-lit, DIME-lit, NAY, PARCH-lit, SCROLL-lit, [,], ^, id, {,
108	Follow (<rel>)	→	(, id
109	Follow (<rel-op>)	→	!, !!, !#, !=, &&, (,), ,, -, =, ==, AYE, COIN-lit, DIME-lit, NAY, PARCH-lit, SCROLL-lit, [, id, {,
110	Follow (<bool-arith2>)	→	!, !!, !#, !=, %, &, &&, (,), *, +, ,, -, /, <, <=, =, ==, >, >=, AYE, COIN-lit, DIME-lit, NAY, PARCH-lit, SCROLL-lit, [, ^, id, {,
111	Follow (<eq-digit>)	→	!, !#, (, -, AYE, COIN-lit, DIME-lit, NAY, PARCH-lit, SCROLL-lit, id
112	Follow (<eq-op>)	→	!, !#, (, -, AYE, COIN-lit, DIME-lit, NAY, PARCH-lit, SCROLL-lit, id
113	Follow (<bool-arith3>)	→	!, !#, (, -, AYE, COIN-lit, DIME-lit, NAY, PARCH-lit, SCROLL-lit, id
114	Follow (<bool-exp2>)	→	!, !!, !#, !=, %, &, (,), *, +, ,, -, /, <, <=, =, ==, >, >=, AYE, COIN-lit, DIME-lit, NAY, PARCH-lit, SCROLL-lit, [, ^, id, {
115	Follow (<eq-ope>)	→	&&,
116	Follow (<eq-ope1>)	→	!, !#, (, -, AYE, COIN-lit, DIME-lit, NAY, PARCH-lit, SCROLL-lit, id
117	Follow (<bool-arith4>)	→	!, !#, (, -, AYE, COIN-lit, DIME-lit, NAY, PARCH-lit, SCROLL-lit, id
118	Follow (<bool-rel>)	→	!!, !=, %, &&,), *, +, -, /, <, <=, =, ==, >, >=, AYE, NAY, [, ^, {,
119	Follow (<bool-exp>)	→	)
120	Follow (<log-op>)	→	), =, {
121	Follow (<bool-grp-val>)	→	), =, {
122	Follow (<bool-ope-grp>)	→	[
123	Follow (<bool-eq-grp>)	→	), =, {
124	Follow (<bool-eq-arr>)	→	!=,), <, <=, =, ==, >, >=, {
125	Follow (<releq-grp>)	→	), +#, -#, =, id, {



126	Follow (<bool-arith2-grp>)	→	), =, {
127	Follow (<bool-arith3-grp>)	→	), =, {
128	Follow (<bool-exp2-grp>)	→	{
129	Follow (<eq-ope-grp>)	→	), =, {
130	Follow (<bool-arith4-grp>)	→	), =, {
131	Follow (<bool-rel-grp>)	→	!!
132	Follow (<bool-exp-grp>)	→	!!
133	Follow (<bool-arr>)	→	]
134	Follow (<bool-arr-tail>)	→	]
135	Follow (<bool-arr1>)	→	!, !#, (, -, AYE, COIN-lit, DIME-lit, NAY, PARCH-lit, SCROLL-lit, id
136	Follow (<bav-tail>)	→	[
137	Follow (<bool-arr-val>)	→	[
138	Follow (<bool-ope-arr>)	→	%, *, +, -, /, [, ^
139	Follow (<releq-arr>)	→	[
140	Follow (<bool-arith2-arr>)	→	[
141	Follow (<bool-arith3-arr>)	→	[
142	Follow (<bool-exp2-arr>)	→	=, [
143	Follow (<eq-ope-arr>)	→	[
144	Follow (<bool-arith4-arr>)	→	!, !#, !=, %, (, *, +, -, /, <, <=, ==, >, >=, AYE, COIN-lit, DIME-lit, NAY, PARCH-lit, SCROLL-lit,], ^, id
145	Follow (<bool-rel-arr>)	→	!!
146	Follow (<bool-exp-arr>)	→	]
147	Follow (<bool-arr2-tail>)	→	]
148	Follow (<bool-arr2>)	→	ABYSS
149	Follow (<bav2-tail>)	→	!!
150	Follow (<bool-func>)	→	!!,,
151	Follow (<bool-ret-val>)	→	!!,,
152	Follow (<bool-ope-ret>)	→	!!,,
153	Follow (<bool-eq-ret>)	→	!!,%,*+, -, /, ^
154	Follow (<releq-ret>)	→	!!,,
155	Follow (<bool-arith2-ret>)	→	!!,,
156	Follow (<bool-arith3-ret>)	→	!!,,

157	Follow (<bool-exp2-ret>)	→	!!, ,, BOOL, COIN, DIME, PARCH, SCROLL
158	Follow (<eq-ope-ret>)	→	!!, ,
159	Follow (<bool-arith4-ret>)	→	!!
160	Follow (<bool-rel-ret>)	→	)
161	Follow (<bool-exp-ret>)	→	)
162	Follow (<params>)	→	id
163	Follow (<param-mult>)	→	BACK
164	Follow (<d-type>)	→	!, !!, !#, !=, %, &, &&, (,), *, +, +#, ,, -, -#, /, <, <=, =, ==, >, >=, ASK, AYE, BOOL, CHART, COIN, COIN-lit, DIME, DIME-lit, ECHO, HAUL, HEAVE, HOIST, LOCKE, LOOK, NAY, PARCH, PARCH-lit, SCROLL, SCROLL-lit, [,], ^, id, {,
165	Follow (<ret-stmnts>)	→	!, !!, !#, !=, %, &, &&, (,), *, +, +#, ,, -, -#, /, <, <=, =, ==, >, >=, ASK, AYE, BOOL, CHART, COIN, COIN-lit, DIME, DIME-lit, ECHO, HAUL, HEAVE, HOIST, LOCKE, LOOK, NAY, PARCH, PARCH-lit, SCROLL, SCROLL-lit, [,], ^, id, {,
166	Follow (<id-tail>)	→	)
167	Follow (<elmt2>)	→	)
168	Follow (<args>)	→	!=, %,), *, +, -, /, <, <=, ==, >, >=, ^
169	Follow (<args-mult>)	→	!=, %,), *, +, -, /, <, <=, ==, >, >=, ^
170	Follow (<value>)	→	!, !#, !=, %, (,), *, +, -, /, <, <=, ==, >, >=, AYE, COIN-lit, DIME-lit, NAY, PARCH-lit, SCROLL-lit, ^, id
171	Follow (<digit-tail>)	→	!=, %,), *, +, -, /, <, <=, ==, >, >=, ^
172	Follow (<arith-var>)	→	!=, %, &, &&,), *, +, -, /, <, <=, ==, >, >=, ^,
173	Follow (<releq-var>)	→	!=, %, &,), *, +, -, /, <, <=, ==, >, >=, ^
174	Follow (<arith-var2>)	→	!=, %,), *, +, -, /, <, <=, ==, >, >=, ^
175	Follow (<arith-var3>)	→	!=,), <, <=, =, ==, >, >=, {
176	Follow (<logeq-var>)	→	!=, %,), *, +, -, /, <, <=, ==, >, >=, ^
177	Follow (<logeq-op>)	→	!=, %,), *, +, -, /, <, <=, ==, >, >=, ^
178	Follow (<log-var>)	→	!=, %,), *, +, -, /, <, <=, ==, >, >=, ^
179	Follow (<parch-tail>)	→	!=, %,), *, +, -, /, <, <=, ==, >, >=, ^
180	Follow (<scroll-tail>)	→	!=, %,), *, +, -, /, <, <=, ==, >, >=, ^
181	Follow (<concat-var>)	→	!=, %, &,), *, +, -, /, <, <=, ==, >, >=, ^
182	Follow (<exp>)	→	!=, %, &,), *, +, -, /, <, <=, ==, >, >=, ^
183	Follow (<log-ope>)	→	!=, %, &,), *, +, -, /, <, <=, ==, >, >=, ^
184	Follow (<log-booleq>)	→	!=, %, &,), *, +, -, /, <, <=, ==, >, >=, ^
185	Follow (<log-releq>)	→	!=, %, &,), *, +, -, /, <, <=, ==, >, >=, ^
186	Follow (<log-exp>)	→	!=, %, &,), *, +, -, /, <, <=, ==, >, >=, ^
187	Follow (<eq-var>)	→	!=, %, &,), *, +, -, /, <, <=, ==, >, >=, ^
188	Follow (<arith-var4>)	→	)



189	Follow (<rel-var>)	→	)
190	Follow (<value-grp>)	→	%,), *, +, -, /, ^
191	Follow (<digit-tail-grp>)	→	!=, &,), =, ==, {
192	Follow (<arith-grp>)	→	), =, {
193	Follow (<releqvar-grp>)	→	!=, &,), =, ==, {, !!
194	Follow (<arith-grp2>)	→	), =, {
195	Follow (<arith-grp3>)	→	)
196	Follow (<logeq-grp>)	→	)
197	Follow (<parch-tail-grp>)	→	)
198	Follow (<scroll-tail-grp>)	→	)
199	Follow (<concat-grp>)	→	!!
200	Follow (<exp-grp>)	→	!!
201	Follow (<const>)	→	COIN-lit, DIME-lit
202	Follow (<const-init>)	→	!!
203	Follow (<coin-locke>)	→	!!,,
204	Follow (<coin-locke-mult>)	→	!!,,
205	Follow (<dime-locke>)	→	!!
206	Follow (<locke-digit>)	→	!!,{
207	Follow (<dime-locke-mult>)	→	!!
208	Follow (<parch-locke>)	→	AYE, NAY
209	Follow (<parch-locke-mult>)	→	), :, AYE, NAY
210	Follow (<scroll-locke>)	→	!!
211	Follow (<scr-id>)	→	BOOL, COIN, DIME, PARCH, SCROLL
212	Follow (<scroll-locke-mult>)	→	BOOL, COIN, DIME, PARCH, SCROLL
213	Follow (<bool-locke>)	→	!!
214	Follow (<locke-bool>)	→	ABYSS
215	Follow (<bool-locke-mult>)	→	BOOL, COIN, DIME, PARCH, SCROLL
216	Follow (<struct>)	→	!!
217	Follow (<mem-dec>)	→	]
218	Follow (<mem-mult>)	→	ABYSS
219	Follow (<mem-dec-tail>)	→	ABYSS
220	Follow (<sub-func>)	→	ABYSS
221	Follow (<return-func>)	→	DROP, DROPLOOK,]



222	Follow (<nonreturn-func>)	→	DROP, DROLOOK,]
223	Follow (<nonret-stmnts>)	→	]
224	Follow (<nonret-tail>)	→	+#, -#, ASK, CHART, ECHO, HAUL, HEAVE, HOIST, LOOK, id
225	Follow (<nonret-back>)	→	+#, -#, ASK, CHART, ECHO, HAUL, HEAVE, HOIST, LOOK, id
226	Follow (<nonret-stmnt>)	→	BACK, {
227	Follow (<nonret-look-tail>)	→	+#, -#, ASK, BACK, CHART, ECHO, HAUL, HEAVE, HOIST, LOOK, MAST, id, {
228	Follow (<local-dec>)	→	BACK, {
229	Follow (<var-arr>)	→	!!
230	Follow (<struct-dec>)	→	!!
231	Follow (<str-dec-init>)	→	!=, %, *, +, -, /, <, <=, ==, >, >=,], ^
232	Follow (<str-dec-tail>)	→	]
233	Follow (<str-val>)	→	!=, %, *, +, -, /, <, <=, ==, >, >=,], ^
234	Follow (<str-val-tail>)	→	!=, %, *, +, -, /, <, <=, ==, >, >=,], ^
235	Follow (<value-str>)	→	!=, %, *, +, -, /, <, <=, ==, >, >=,], ^
236	Follow (<digit-tail-str>)	→	!=, %, *, +, -, /, <, <=, ==, >, >=,], ^
237	Follow (<arith-str>)	→	!=, %, &, *, +, -, /, <, <=, ==, >, >=,], ^
238	Follow (<releq-str>)	→	[
239	Follow (<arith-str2>)	→	!=, %, *, +, -, /, <, <=, ==, >, >=,], ^
240	Follow (<arith-str3>)	→	!=, %, *, +, -, /, <, <=, ==, >, >=,], ^
241	Follow (<logeq-str>)	→	!=, %, *, +, -, /, <, <=, ==, >, >=,], ^
242	Follow (<parch-tail-str>)	→	!=, %,), *, +, -, /, <, <=, ==, >, >=,], ^
243	Follow (<scroll-tail-str>)	→	!=, %, *, +, -, /, <, <=, ==, >, >=,], ^
244	Follow (<concat-str>)	→	+#, -#, ADRIFT, ASK, BACK, CHART, COIN, COURSE, DROP, DROLOOK, ECHO, HAUL, HEAVE, HOIST, LAND, LOOK, id, {
245	Follow (<exp-str>)	→	+#, -#, ADRIFT, ASK, BACK, CHART, COIN, COURSE, DROP, DROLOOK, ECHO, HAUL, HEAVE, HOIST, LAND, LOOK, id, {
246	Follow (<statements>)	→	!!
247	Follow (<assign-stmnt>)	→	!!, !=, %, *, +, -, /, <, <=, =, ==, >, >=, @, HAUL, LOOK, ^
248	Follow (<assign-tail>)	→	!!, !=, %, *, +, -, /, <, <=, =, ==, >, >=, @, HAUL, LOOK, ^
249	Follow (<arr-str>)	→	!!
250	Follow (<arr-str-tail>)	→	!!
251	Follow (<assign-body>)	→	!!
252	Follow (<value-as>)	→	!!, %, *, +, -, /, ^
253	Follow (<digit-tail-as>)	→	!!
254	Follow (<arith-as>)	→	!!, !=, &, ==
255	Follow (<releq-as>)	→	!!,,
256	Follow (<arith-as2>)	→	!!
257	Follow (<arith-as3>)	→	!!



258	Follow (<logeq-as>)	→	!!
259	Follow (<parch-tail-as>)	→	!!
260	Follow (<scroll-tail-as>)	→	!!
261	Follow (<concat-as>)	→	!!, (, ,, =, {
262	Follow (<exp-as>)	→	+#, -#, ADRIFT, ASK, BACK, CHART, COIN, COURSE, DROP, DROLOOK, ECHO, HAUL, HEAVE, HOIST, LAND, LOOK, id, {
263	Follow (<arith-assign-op>)	→	)
264	Follow (<ask-stmnt>)	→	)
265	Follow (<addr>)	→	+#, -#, ADRIFT, ASK, BACK, CHART, COIN, COURSE, DROP, DROLOOK, ECHO, HAUL, HEAVE, HOIST, LAND, LOOK, id, {
266	Follow (<addr-tail>)	→	+#, -#, ADRIFT, ASK, BACK, CHART, COIN, COURSE, DROP, DROLOOK, ECHO, HAUL, HEAVE, HOIST, LAND, LOOK, id, {
267	Follow (<echo-stmnt>)	→	)
268	Follow (<look-stmnt>)	→	CHART
269	Follow (<condition>)	→	]
270	Follow (<look-body>)	→	+#, -#, ADRIFT, ASK, BACK, CHART, COIN, COURSE, DROP, DROLOOK, ECHO, HAUL, HEAVE, HOIST, LAND, LOOK, id, {
271	Follow (<jump-stmnt>)	→	+#, -#, ADRIFT, ASK, BACK, CHART, COIN, COURSE, DROP, DROLOOK, ECHO, HAUL, HEAVE, HOIST, LAND, LOOK, id, {
272	Follow (<look-tail>)	→	)
273	Follow (<chart-stmnt>)	→	), :
274	Follow (<chart-cond>)	→	+#, -#, ASK, CHART, ECHO, HAUL, HEAVE, HOIST, LOOK, id
275	Follow (<chart-const>)	→	+#, -#, ADRIFT, ASK, CHART, ECHO, HAUL, HEAVE, HOIST, LOOK, id
276	Follow (<courses>)	→	+#, -#, ASK, CHART, ECHO, HAUL, HEAVE, HOIST, LOOK, id
277	Follow (<course-body>)	→	HOIST
278	Follow (<course-jmp>)	→	]
279	Follow (<course-tail>)	→	LAND
280	Follow (<adrift-case>)	→	+#, -#, ADRIFT, ASK, BACK, CHART, COIN, COURSE, DROP, DROLOOK, ECHO, HAUL, HEAVE, HOIST, LAND, LOOK, id, {
281	Follow (<adrift-body>)	→	!!
282	Follow (<hoist-stmnt>)	→	!!
283	Follow (<hoist-init>)	→	!!
284	Follow (<init1-mult>)	→	!!
285	Follow (<init2-mult>)	→	!, !#, %, (, *, +, -, /, AYE, COIN-lit, DIME-lit, NAY, PARCH-lit, SCROLL-lit, ^, id
286	Follow (<hoist-cond>)	→	!!
287	Follow (<releq-op>)	→	)
288	Follow (<hoist-log>)	→	HAUL
289	Follow (<hoist-upd>)	→	)



290	Follow (<upd>)	→	+#, -#, ADRIFT, ASK, BACK, CHART, COIN, COURSE, DROP, DROPLOOK, ECHO, HAUL, HEAVE, HOIST, LAND, LOOK, id, {
291	Follow (<upd-mult>)	→	+#, -#, ADRIFT, ASK, BACK, CHART, COIN, COURSE, DROP, DROPLOOK, ECHO, HAUL, HEAVE, HOIST, LAND, LOOK, id, {
292	Follow (<heave-stmnt>)	→	!!
293	Follow (<haul-stmnt>)	→	id
294	Follow (<unary-exp>)	→	BACK, DROP, DROPLOOK,]
295	Follow (<unary-op>)	→	BACK, DROP, DROPLOOK,]
296	Follow (<ahoy-local-dec>)	→	id, ASK, ECHO, LOOK, CHART, HOIST, HEAVE, HAUL, +#, -#
297	Follow (<ahoy-struct-dec>)	→	id, ASK, ECHO, LOOK, CHART, HOIST, HEAVE, HAUL, +#, -#
298	Follow (<ahoy-stmnts>)	→	DROP, DROPLOOK
299	Follow (<ahoy-tail>)	→	DROP, DROPLOOK
300	Follow (<ahoy-stmnt>)	→	]
301	Follow (<ahoy-look-tail>)	→	id, ASK, ECHO, LOOK, CHART, HOIST, HEAVE, HAUL, +#, -#,]

X. PREDICT SET

	First = (Production Set)	PREDICT SET	
1	First (<global-dec> AHOY() [<ahoy-local-dec> <ahoy-stmnts>]) =	First (<global-dec>) =	{ COIN,DIME,PARCH,SCROLL, BOOL,LOCKE,MAST,ABYSS }
2	First (<var-arr-func>) =	First (<var-arr-func>) =	{ COIN,DIME,PARCH,SCROLL,BOOL }
3	First (<const><global-dec>) =	First (<const>) =	{ LOCKE }
4	First (<struct><sub-func>) =	First (<struct>) =	{ MAST }
5	First (<nonreturn-func>) =	First (<nonreturn-func>) =	{ ABYSS }
6	First (λ) =	Follow (<global-dec>) =	{ AHOY }
7	First (COIN id <coin-var-arr-func>) =		{ COIN }
8	First (DIME id <dime-var-arr-func>) =		{ DIME }
9	First (PARCH id <parch-var-arr-func>) =		{ PARCH }
10	First (SCROLL id <scroll-var-arr-func>) =		{ SCROLL }
11	First (BOOL id <bool-var-arr-func>) =		{ BOOL }
12	First (<coin-var-arr>!! <global-dec>) =	First (<coin-var-arr>) =	{ ,,=,{ }
13	First (<coin-func>) =	First (<coin-func>) =	{ (}
14	First (<coin-var>) =	First (<coin-var>) =	{ =, , }
15	First (<coin-arr>) =	First (<coin-arr>) =	{ { }
16	First (<coin-init><coin-init- mult>) =	First (<coin-init>) =	{ = }
17	First (=<coin-val><coin-exp >) =		{ = }
18	First (λ) =	Follow (<coin-init>) =	{ !!,(,,,-,=,COIN-lit,[,id,{ }
19	First (, id <coin-init><coin-init-m ult>) =		{ , }
20	First (λ) =	Follow (<coin-init-mult>) =	{ %,*,+,-,/ , ^ }

21	First (COIN-lit) =		{ COIN-lit }
22	First ($\langle \text{neg} \rangle \langle \text{neg-coin-val} \rangle$) =	First ($\langle \text{neg} \rangle$) =	{ - }
23	First (-) =		{ - }
24	First (λ) =	Follow ($\langle \text{neg} \rangle$) =	{ !!, - }
25	First (id $\langle \text{id-tail} \rangle$) =		{ id }
26	First ($\langle \text{coin-val-grp} \rangle$) =		{ (}
27	First ($\langle \text{coin-arith} \rangle \langle \text{coin-exp} \rangle$) =	First ($\langle \text{coin-arith} \rangle$) =	{ %, *, +, -, /, ^ }
28	First (λ) =	Follow ($\langle \text{coin-exp} \rangle$) =	{) }
29	First ($\langle \text{arith-op} \rangle \langle \text{coin-val} \rangle$) =	First ($\langle \text{arith-op} \rangle$) =	{ %, *, +, -, /, ^ }
30	First ($\langle \text{coin-val} \rangle \langle \text{coin-exp-grp} \rangle$) =	First ($\langle \text{coin-val} \rangle$) =	{ (, -, COIN-lit, id }
31	First ($\langle \text{coin-arith} \rangle \langle \text{coin-grp-exp} \rangle$) =	First ($\langle \text{coin-arith} \rangle$) =	{ %, *, +, -, /, ^ }
32	First (λ) =	Follow ($\langle \text{coin-grp-exp} \rangle$) =	{ !! }
33	First (+) =		{ + }
34	First (-) =		{ - }
35	First (*) =		{ * }
36	First (/) =		{ / }
37	First (%) =		{ % }
38	First (^) =		{ ^ }
39	First ({COIN-lit} $\langle \text{coin-arr-tail} \rangle$) =		{ { }
40	First ($=[\langle \text{coin-arr1} \rangle]$) =		{ = }
41	First ($\langle \text{COIN-lit} \rangle \langle \text{coin-arr2-tail} \rangle$) =		{ { }
42	First (λ) =	Follow ($\langle \text{coin-arr-tail} \rangle$) =	{ [}
43	First ($\langle \text{coin-arr-val} \rangle \langle \text{cav-tail} \rangle$) =	First ($\langle \text{coin-arr-val} \rangle$) =	{ (, -, COIN-lit, id }
44	First ($\langle \text{coin-val} \rangle \langle \text{coin-arr-exp} \rangle$) =	First ($\langle \text{coin-val} \rangle$) =	{ (, -, COIN-lit, id }
45	First ($\langle \text{coin-arith} \rangle \langle \text{coin-arr-exp} \rangle$) =	First ($\langle \text{coin-arith} \rangle$) =	{ %, *, +, -, /, ^ }

46	First (λ) =	Follow (<coin-arr-exp>) =	{ !! }
47	First (, <coin-arr1>) =		{ , }
48	First (λ) =	Follow (<cav-tail>) =	{ } }
49	First (=[<coin-arr2>]) =		{ = }
50	First (λ) =	Follow (<coin-arr2-tail>) =	{ } }
51	First ([<coin-arr1>]<cav2-tail>) =		{ [}
52	First (, <coin-arr2>) =		{ , }
53	First (λ) =	Follow (<cav2-tail>) =	{ !! }
54	First ((<params>) [<local-dec> <ret-stmnts> BACK <coin-ret-val>!!] <sub-func>) =		{ (}
55	First (<coin-val><coin-ret-exp>) =	First (<coin-val>) =	{ (, -, COIN-lit, id }
56	First (<coin-arith><coin-ret-exp>) =	First (<coin-arith>) =	{ %, *, +, -, /, ^ }
57	First (λ) =	Follow (<coin-ret-exp>) =	{ !! }
58	First (<dime-var-arr>!! <global-dec>) =	First (<dime-var-arr>) =	{ ,, =, {, }
59	First (<dime-func>) =	First (<dime-func>) =	{ (}
60	First (<dime-var>) =	First (<dime-var>) =	{ ,, =, }
61	First (<dime-arr>) =	First (<dime-arr>) =	{ { }
62	First (<dime-init><dime-init-mult>) =	First (<dime-init>) =	{ = }
63	First (<dime-init-val>) =		{ = }
64	First (λ) =	Follow (<dime-init>) =	{ (, -, COIN-lit, DIME-lit, id }
65	First (, id <dime-init><dime-init-mult>) =		{ , }
66	First (λ) =	Follow (<dime-init-mult>) =	{ !, !!, !#, !=, %, &, &&, (,), *, +, ,, -, /, <, <=, =, ==, >, >=, AYE, BOOL, COIN, COIN-lit, DIME, DIME-lit, H AUL, NAY, PARCH, PARCH-lit, SCROLL, SCR OLL-lit, [,], ^, id, {, }
67	First (<dime-val><dime-exp>) =	First (<dime-val>) =	{ (, -, COIN-lit, DIME-lit, id }
68	First (DIME-lit) =		{ DIME-lit }

69	First (COIN-lit) =		{ COIN-lit }
70	First ($\langle \text{neg} \rangle \langle \text{neg-dime-val} \rangle$) =	First ($\langle \text{neg} \rangle$) =	{ -, }
71	First (id $\langle \text{id-tail} \rangle$) =		{ id }
72	First ($\langle \langle \text{dime-grp-val} \rangle \rangle$) =		{ (}
73	First ($\langle \text{dime-arith} \rangle \langle \text{dime-exp} \rangle$) =	First ($\langle \text{dime-arith} \rangle$) =	{ %, *, +, -, /, ^ }
74	First (λ) =	Follow ($\langle \text{dime-exp} \rangle$) =	{) }
75	First ($\langle \text{arith-op} \rangle \langle \text{dime-val} \rangle$) =	First ($\langle \text{arith-op} \rangle$) =	{ %, *, +, -, /, ^ }
76	First ($\langle \text{dime-val} \rangle \langle \text{dime-grp-exp} \rangle$) =	First ($\langle \text{dime-val} \rangle$) =	{ (, -, COIN-lit, DIME-lit, id }
77	First ($\langle \text{dime-arith} \rangle \langle \text{dime-grp-exp} \rangle$) =	First ($\langle \text{dime-arith} \rangle$) =	{ %, *, +, -, /, ^ }
78	First (λ) =	Follow ($\langle \text{dime-grp-exp} \rangle$) =	{ !! }
79	First ($\langle \{ \text{COIN-lit} \} \langle \text{dime-arr-tail} \rangle$) =		{ { }
80	First ($\langle \text{dime-arr1} \rangle$) =		{ = }
81	First ($\langle \{ \text{COIN-lit} \} \langle \text{dime-arr2-tail} \rangle$) =		{ { }
82	First (λ) =	Follow ($\langle \text{dime-arr-tail} \rangle$) =	{ [}
83	First ($\langle \text{dime-arr-val} \rangle \langle \text{dav-tail} \rangle$) =	First ($\langle \text{dime-arr-val} \rangle$) =	{ (, -, COIN-lit, DIME-lit, id }
84	First ($\langle \text{dime-val} \rangle \langle \text{dime-arr-exp} \rangle$) =	First ($\langle \text{dime-val} \rangle$) =	{ (, -, COIN-lit, DIME-lit, id }
85	First ($\langle \text{dime-arith} \rangle \langle \text{dime-arr-exp} \rangle$) =	First ($\langle \text{dime-arith} \rangle$) =	{ %, *, +, -, /, ^ }
86	First (λ) =	Follow ($\langle \text{dime-arr-exp} \rangle$) =	{ !! }
87	First (, $\langle \text{dime-arr1} \rangle$) =		{ , }
88	First (λ) =	Follow ($\langle \text{dav-tail} \rangle$) =	{] }
89	First ($\langle \text{dime-arr2} \rangle$) =		{ = }
90	First (λ) =	Follow ($\langle \text{dime-arr2-tail} \rangle$) =	{] }

91	First ([<dime-arr1>]<dav2-tail>) =		{ [}
92	First (, <dime-arr2>) =		{ , }
93	First (λ) =	Follow (<dav2-tail>) =	{ !! }
94	First ((<params>) [<local-dec> <ret-stmnts> BACK <dime-ret-val>!!] <sub-func>) =		{ (}
95	First (<dime-val><dime-ret-exp>) =	First (<dime-val>) =	{ (,-,COIN-lit,DIME-lit,id }
96	First (<dime-arith><dime-ret-exp>) =	First (<dime-arith>) =	{ %,*,+,-,/,^ }
97	First (λ) =	Follow (<dime-ret-exp>) =	{ !! }
98	First (<parch-var-arr>!! <global-dec>) =	First (<parch-var-arr>) =	{ ,,=,{, }
99	First (<parch-func>) =	First (<parch-func>) =	{ (}
100	First (<parch-var>) =	First (<parch-var>) =	{ ,,= }
101	First (<parch-arr>) =	First (<parch-arr>) =	{ { }
102	First (<parch-init><parch-init-mult>) =	First (<parch-init>) =	{ = }
103	First (= <parch-val>) =		{ = }
104	First (λ) =	Follow (<parch-init>) =	{ !,!,!#,!=,%,&,(,),*,+,,,,-,/,<,<=,=,>,>=,AYE, COIN-lit,DIME-lit,NAY,PARCH-lit,SCROLL-lit,[,^,id,{ }
105	First (, id <parch-init><parch-init-mult>) =		{ , }
106	First (λ) =	Follow (<parch-init-mult>) =	{ !! }
107	First (PARCH-lit) =		{ PARCH-lit }
108	First (id<id-tail>) =		{ id }
109	First ({COIN-lit}<parch-arr-tail>) =		{ { }
110	First (= [<parch-arr1>]) =		{ = }
111	First ({COIN-lit}<parch-arr2-tail>) =		{ { }
112	First (λ) =	Follow (<parch-arr-tail>) =	{] }

113	First (<parch-val><pav-tail>) =	First (<parch-val>) =	{ PARCH-lit,id }
114	First (, <parch-arr1>) =		{ , }
115	First (λ) =	Follow (<pav-tail>) =	{ }
116	First (=<parch-arr2>) =		{ = }
117	First (λ) =	Follow (<parch-arr2-tail>) =	{ }
118	First ([<parch-arr1><pav2-tail>) =		{ [}
119	First (, <parch-arr2>) =		{ , }
120	First (λ) =	Follow (<pav2-tail>) =	{ }
121	First (((<params> [<local-dec> <ret-stmnts> BACK <parch-val>!!] <sub-func>)) =		{ (}
122	First (<scroll-var-arr>!! <global-dec>) =	First (<scroll-var-arr>) =	{ ,,=,{ }
123	First (<scroll-func><sub-func>) =	First (<scroll-func>) =	{ (}
124	First (<scroll-var>) =	First (<scroll-var>) =	{ ,,=}
125	First (<scroll-arr>) =	First (<scroll-arr>) =	{ { }
126	First (<scroll-init><scroll-init-mult>) =	First (<scroll-init>) =	{ = }
127	First (=<scroll-init-val>) =		{ = }
128	First (λ) =	Follow (<scroll-init>) =	{ (,SCROLL-lit,id }
129	First (, id <scroll-init><scroll-init-mult>) =		{ , }
130	First (λ) =	Follow (<scroll-init-mult>) =	{ !,!!,!#,!=,%,&,(,),*,+,+##,-,-#/,<,<=,=,>,> =,ASK,AYE,CHART,COIN-lit,DIME-lit,ECH O,HAUL,HEAVE,HOIST,LOCKE,LOOK,NA Y,PARCH-lit,SCROLL-lit,[,^,id,{ }
131	First (<scroll-val><scroll-exp>) =	First (<scroll-val>) =	{ (,SCROLL-lit,id }
132	First (SCROLL-lit<scr-char>) =		{ SCROLL-lit }
133	First (id<id-tail>) =		{ id }

134	First (((<scroll-grp-val>)) =		{ (}
135	First ({<index>}) =		{ { }
136	First (λ) =	Follow (<scr-char> =	{ (,SCROLL-lit,id }
137	First (id) =		{ id }
138	First (COIN-lit) =		{ COIN-lit }
139	First (<scroll-concat><scrol l-exp>) =	First (<scroll-concat> =	{ & }
140	First (λ) =	Follow (<scroll-exp> =	{) }
141	First (& <scroll-val>) =		{ & }
142	First (<scroll-val><scroll-gr p-exp>) =	First (<scroll-val>) =	{ (,SCROLL-lit,id }
143	First (<scroll-concat><scrol l-grp-exp>) =	First (<scroll-concat>) =	{ & }
144	First (λ) =	Follow (<scroll-grp-exp>) =	{ !! }
145	First ({COIN-lit}<scroll-arr-t ail>) =		{ { }
146	First ([<scroll-arr1>]) =		{ = }
147	First ({COIN-lit}<scroll-arr2- tail>) =		{ { }
148	First (λ) =	Follow (<scroll-arr-tail>) =	{ [}
149	First (<scroll-arr-val><sav-t ail>) =	First (<scroll-arr-val>) =	{ (,SCROLL-lit,id }
150	First (<scroll-val><scroll-arr -exp>) =	First (<scroll-val>) =	{ (,SCROLL-lit,id }
151	First (<scroll-concat><scrol l-arr-exp>) =	First (<scroll-concat>) =	{ & }
152	First (λ) =	Follow (<scroll-arr-exp>) =	{ !! }
153	First (, <scroll-arr1>) =		{ , }
154	First (λ) =	Follow (<sav-tail>) =	{ [}
155	First ([<scroll-arr2>]) =		{ = }
156	First (λ) =	Follow (<scroll-arr2-tail>) =	{ [}
157	First ([<scroll-arr1>]<sav2-t ail>) =		{ [}
158	First (, <scroll-arr2>) =		{ , }
159	First (λ) =	Follow (<sav2-tail>) =	{ !! }

160	First ((<params> [<local-dec> <ret-stmnts> BACK <scroll-ret-val>!!] <sub-func>) =		{ (}
161	First (<scroll-val><scroll-ret-exp>) =	First (<scroll-val>) =	{ (,SCROLL-lit,id }
162	First (<scroll-concat><scroll-ret-exp>) =	First (<scroll-concat>) =	{ & }
163	First (λ) =	Follow (<scroll-ret-exp>) =	{ !! }
164	First (<bool-var-arr>!! <global-dec>) =	First (<bool-var-arr>) =	{ ,,=,{, }
165	First (<bool-func>) =	First (<bool-func>) =	{ (}
166	First (<bool-var>) =	First (<bool-var>) =	{ ,,=, }
167	First (<bool-arr>) =	First (<bool-arr>) =	{ { }
168	First (<bool-init><bool-init-mult>) =	First (<bool-init>) =	{ = }
169	First (= <bool-init-val>) =		{ = }
170	First (λ) =	Follow (<bool-init>) =	{ !,!#,(-,AYE,COIN-lit,DIME-lit,NAY,PARCH-lit,SCROLL-lit,id }
171	First (, id <bool-init><bool-init-mult>) =		{ , }
172	First (λ) =	Follow (<bool-init-mult>) =	{ !,!#,(-,AYE,COIN-lit,DIME-lit,NAY,PARCH-lit,SCROLL-lit,id }
173	First (<bool-val><bool-exp>) =	First (<bool-val>) =	{ -,id,(,!,!#,AYE,NAY,COIN-lit,DIME-lit,PARC H-lit,SCROLL-lit }
174	First (id<id-tail><bool-exp2>) =		{ id }
175	First ((<value-grp><bool-exp2>) =		{ (}
176	First (<bool><bool-eq>) =	First (<bool>) =	{ AYE,NAY,!,!# }
177	First (<digit><bool-arith><rel-eq>) =	First (<digit>) =	{ -,COIN-lit,DIME-lit }

178	First (PARCH-lit<eq-op><p arch-val>) =		{ PARCH-lit }
179	First (SCROLL-lit<scr-char ><eq-op><scroll-val>) =		{ SCROLL-lit }
180	First (<bool-lit>) =	First (<bool-lit>) =	{ AYE,NAY }
181	First (<not-op><not-ope>) =	First (<not-op>) =	{ !,!# }
182	First (AYE) =		{ AYE }
183	First (NAY) =		{ NAY }
184	First (!) =		{ ! }
185	First (!#) =		{ !# }
186	First (id<id-tail>) =		{ id }
187	First ((<bool-grp-val>)) =		{ (}
188	First (<bool-lit>) =	First (<bool-lit>) =	{ AYE,NAY }
189	First (<eq-op><bool-val>) =	First (<eq-op>) =	{ ==,!= }
190	First (λ) =	Follow (<bool-eq>) =	{ !,!!,!#,!=,%,&&,(,),*,+,,,,-,/,<,<=,==,>,>=,AY E,BOOL,COIN,COIN-lit,DIME,DIME-lit,NAY ,PARCH,PARCH-lit,SCROLL,SCROLL-lit,[,] ,^,id,{, }
191	First (COIN-lit) =		{ COIN-lit }
192	First (DIME-lit) =		{ DIME-lit }
193	First (<neg-digit>) =		{ - }
194	First (id<id-tail>) =		{ id }
195	First ((<dime-grp-val>)) =		{ (}
196	First (<dime-arith><bool-ari th>) =	First (<dime-arith>) =	{ %,*,+,-,/,^ }
197	First (λ) =	Follow (<bool-arith>) =	{ !,!!,!#,!=,%,&&,(,),*,+,,,,-,/,<,<=,==,>,>=, AYE,COIN-lit,DIME-lit,NAY,PARCH-lit,SCR OLL-lit,[,],^,id,{, }
198	First (<rel><bool-arith2><b ool-eq>) =	First (<rel>) =	{ <,>,<=,>= }
199	First (<eq-digit><bool-arith 3>) =	First (<eq-digit>) =	{ ==,!= }

200	First ($\langle \text{rel-op} \rangle \langle \text{dime-val} \rangle$) =	First ($\langle \text{rel-op} \rangle$) =	{ $\langle , \rangle, \langle =, \rangle, \langle = \rangle$ }
201	First ($\langle \rangle$) =		{ $\langle \rangle$ }
202	First ($\langle \rangle$) =		{ $\langle \rangle$ }
203	First ($\langle = \rangle$) =		{ $\langle = \rangle$ }
204	First ($\langle = \rangle$) =		{ $\langle = \rangle$ }
205	First ($\langle \text{dime-arith} \rangle \langle \text{bool-arith2} \rangle$) =	First ($\langle \text{dime-arith} \rangle$) =	{ $\%, *, +, -, /, ^$ }
206	First (λ) =	Follow ($\langle \text{bool-arith2} \rangle$) =	{ !, !!, !#, !=, %, &, &, (,), *, +, ,, -, /, <, <=, =, ==, >, >=, AYE, COIN-lit, DIME-lit, NAY, PARCH-lit, SCROLL-lit, [^, id, {, }
207	First ($\langle \text{eq-op} \rangle \langle \text{dime-val} \rangle$) =	First ($\langle \text{eq-op} \rangle$) =	{ ==, != }
208	First ($\langle == \rangle$) =		{ = }
209	First ($\langle != \rangle$) =		{ != }
210	First ($\langle \text{dime-arith} \rangle \langle \text{bool-arith3} \rangle$) =	First ($\langle \text{dime-arith} \rangle$) =	{ $\%, *, +, -, /, ^$ }
211	First (λ) =	Follow ($\langle \text{bool-arith3} \rangle$) =	{ !, !!, !#, (, -, AYE, COIN-lit, DIME-lit, NAY, PARCH-lit, SCROLL-lit, id }
212	First ($\langle \text{dime-arith} \rangle \langle \text{bool-arith} \rangle \langle \text{rel-eq} \rangle$) =	First ($\langle \text{dime-arith} \rangle$) =	{ $\%, *, +, -, /, ^$ }
213	First ($\langle \text{rel} \rangle \langle \text{bool-arith2} \rangle \langle \text{bool-eq} \rangle$) =	First ($\langle \text{rel} \rangle$) =	{ $\langle , \rangle, \langle =, \rangle, \langle = \rangle$ }
214	First ($\langle \text{eq-op} \rangle \langle \text{eq-ope} \rangle$) =	First ($\langle \text{eq-op} \rangle$) =	{ ==, != }
215	First (λ) =	Follow ($\langle \text{bool-exp2} \rangle$) =	{ !, !!, !#, !=, %, &, (,), *, +, ,, -, /, <, <=, =, ==, >, >=, AYE, COIN-lit, DIME-lit, NAY, PARCH-lit, SCROLL-lit, [^, id, { }
216	First ($\langle \text{id} \rangle \langle \text{id-tail} \rangle \langle \text{bool-arith4} \rangle \langle \text{bool-rel} \rangle$) =		{ id }
217	First ($\langle \langle \text{value-grp} \rangle \rangle \langle \text{bool-arith4} \rangle \langle \text{bool-rel} \rangle$) =		{ (}
218	First ($\langle \text{digit} \rangle \langle \text{bool-arith4} \rangle \langle \text{bool-rel} \rangle$) =	First ($\langle \text{digit} \rangle$) =	{ -, COIN-lit, DIME-lit }
219	First ($\langle \text{eq-ope1} \rangle$) =	First ($\langle \text{eq-ope1} \rangle$) =	{ PARCH-lit, SCROLL-lit, AYE, NAY, !, !# }

220	First (PARCH-lit) =		{ PARCH-lit }
221	First (SCROLL-lit<scr-char>) =		{ SCROLL-lit }
222	First (<bool>) =	First (<bool>) =	{ AYE,NAY,!,# }
223	First (<dime-arith><bool-arith4>) =	First (<dime-arith>) =	{ %,*,+,-,/,<^ }
224	First (λ) =	Follow (<bool-arith4>) =	{ !,!#,(,-,AYE,COIN-lit,DIME-lit,NAY,PARCH-lit,SCROLL-lit,id }
225	First (<rel><bool-arith3>) =	First (<rel>) =	{ <,>,<=,>= }
226	First (λ) =	Follow (<bool-rel>) =	{ !,!=,%,&&,*,+,-,<,<=,=,=>,>=,AYE,NAY,[,^,{, }
227	First (<log-op><bool-val><bool-exp>) =	First (<log-op>) =	{ &&, }
228	First (λ) =	Follow (<bool-exp>) =	{) }
229	First () =		{ }
230	First (&&) =		{ && }
231	First (<bool-ope-grp><bool-exp-grp>) =	First (<bool-ope-grp>) =	{ -,id,(,!,#,AYE,NAY,COIN-lit,DIME-lit,PARCH-lit,SCROLL-lit }
232	First (id<id-tail><bool-exp2-grp>) =		{ id }
233	First ((<value-grp><bool-exp2-grp>) =		{ (}
234	First (<bool><bool-eq-grp>) =	First (<bool>) =	{ AYE,NAY,!,# }
235	First (<digit><bool-arith><releq-grp>) =	First (<digit>) =	{ -,COIN-lit,DIME-lit }
236	First (PARCH-lit<eq-op><parch-val>) =		{ PARCH-lit }
237	First (SCROLL-lit<scr-char><eq-op><scroll-val>) =		{ SCROLL-lit }
238	First (<eq-op><bool-ope-grp>) =	First (<eq-op>) =	{ ==,!= }

239	First (λ) =	Follow (<bool-eq-arr>) =	{ !=, <, <=, =, >, >=, { }
240	First (<rel><bool-arith2-grp> ><bool-eq-grp>) =	First (<rel>) =	{ <, >, <=, >= }
241	First (<eq-digit><bool-arith 3-grp>) =	First (<eq-digit>) =	{ ==, != }
242	First (<dime-arith><bool-arith2-grp>) =	First (<dime-arith>) =	{ %, *, +, -, /, ^ }
243	First (λ) =	Follow (<bool-arith2-grp>) =	{ }, =, { }
244	First (<dime-arith><bool-arith3-grp>) =	First (<dime-arith>) =	{ %, *, +, -, /, ^ }
245	First (λ) =	Follow (<bool-arith3-grp>) =	{ }, =, { }
246	First (<dime-arith><bool-arith> ><releq-grp>) =	First (<dime-arith>) =	{ %, *, +, -, /, ^ }
247	First (<rel><bool-arith2-grp> ><bool-eq-grp>) =	First (<rel>) =	{ <, >, <=, >= }
248	First (<eq-op><eq-ope-grp>) =	First (<eq-op>) =	{ ==, != }
249	First (λ) =	Follow (<bool-exp2-grp>) =	{ { }
250	First (id<id-tail><bool-arith 4-grp><bool-rel-grp>) =		{ id }
251	First ((<value-grp><bool-arith4-grp><bool-rel-grp>) =		{ (}
252	First (<digit><bool-arith4-grp> ><bool-rel-grp>) =	First (<digit>) =	{ -, COIN-lit, DIME-lit }
253	First (<eq-ope1>) =	First (<eq-ope1>) =	{ PARCH-lit, SCROLL-lit, AYE, NAY, !, !# }
254	First (<dime-arith><bool-arith4-grp>) =	First (<dime-arith>) =	{ %, *, +, -, /, ^ }
255	First (λ) =	Follow (<bool-arith4-grp>) =	{ }, =, { }
256	First (<rel><bool-arith3-grp>) >) =	First (<rel>) =	{ <, >, <=, >= }
257	First (λ) =	Follow (<bool-rel-grp>) =	{ !! }

258	First ($\langle \text{log-op} \rangle \langle \text{bool-ope-grp} \rangle \langle \text{bool-exp-grp} \rangle$) =	First ($\langle \text{log-op} \rangle$) =	{ &&, }
259	First (λ) =	Follow ($\langle \text{bool-exp-grp} \rangle$) =	{ !! }
260	First ($\langle \text{COIN-lit} \rangle \langle \text{bool-arr-tail} \rangle$) =		{ {} }
261	First ($\langle \text{bool-arr1} \rangle$) =		{ = }
262	First ($\langle \text{COIN-lit} \rangle \langle \text{bool-arr2-tail} \rangle$) =		{ {{ } }
263	First (λ) =	Follow ($\langle \text{bool-arr-tail} \rangle$) =	{ [] }
264	First ($\langle \text{bool-arr-val} \rangle \langle \text{bav-tail} \rangle$) =	First ($\langle \text{bool-arr-val} \rangle$) =	{ -,id,(,!,!#,AYE,NAY,COIN-lit,DIME-lit,PARC H-lit,SCROLL-lit }
265	First ($\langle \text{bool-arr1} \rangle$) =		{ , }
266	First (λ) =	Follow ($\langle \text{bav-tail} \rangle$) =	{ [] }
267	First ($\langle \text{bool-ope-arr} \rangle \langle \text{bool-exp-arr} \rangle$) =	First ($\langle \text{bool-ope-arr} \rangle$) =	{ -,id,(,!,!#,AYE,NAY,COIN-lit,DIME-lit,PARC H-lit,SCROLL-lit }
268	First ($\langle \text{id-id-tail} \rangle \langle \text{bool-exp2-arr} \rangle$) =		{ id }
269	First ($\langle \langle \text{value-grp} \rangle \langle \text{bool-exp2-arr} \rangle$) =		{ () }
270	First ($\langle \text{bool} \rangle \langle \text{bool-eq-arr} \rangle$) =	First ($\langle \text{bool} \rangle$) =	{ AYE,NAY,!,!# }
271	First ($\langle \text{digit} \rangle \langle \text{bool-arith} \rangle \langle \text{releq-arr} \rangle$) =	First ($\langle \text{digit} \rangle$) =	{ -,COIN-lit,DIME-lit }
272	First ($\langle \text{PARCH-lit} \rangle \langle \text{eq-op} \rangle \langle \text{parch-val} \rangle$) =		{ PARCH-lit }
273	First ($\langle \text{SCROLL-lit} \rangle \langle \text{scr-char} \rangle \langle \text{eq-op} \rangle \langle \text{scroll-val} \rangle$) =		{ SCROLL-lit }
274	First ($\langle \text{eq-op} \rangle \langle \text{bool-ope-arr} \rangle$) =	First ($\langle \text{eq-op} \rangle$) =	{ ==,!= }
275	First (λ) =	Follow ($\langle \text{bool-eq-arr} \rangle$) =	{ !=,.,<,<=,=,==,>,>=,{ }
276	First ($\langle \text{rel} \rangle \langle \text{bool-arith2-arr} \rangle \langle \text{bool-eq-arr} \rangle$) =	First ($\langle \text{rel} \rangle$) =	{ <,>,<=,>= }

277	First (<code><eq-digit><bool-arith3-arr></code>) =	First (<code><eq-digit></code>) =	{ <code>==,!=</code> }
278	First (<code><dime-arith><bool-arith2-arr></code>) =	First (<code><dime-arith></code>) =	{ <code>%,*,+,-,/,<^</code> }
279	First (λ) =	Follow (<code><bool-arith2-arr></code>) =	{ [] }
280	First (<code><dime-arith><bool-arith3-arr></code>) =	First (<code><dime-arith></code>) =	{ <code>%,*,+,-,/,<^</code> }
281	First (λ) =	Follow (<code><bool-arith3-arr></code>) =	{ [] }
282	First (<code><dime-arith><bool-arith><releq-arr></code>) =	First (<code><dime-arith></code>) =	{ <code>%,*,+,-,/,<^</code> }
283	First (<code><rel><bool-arith2-arr><bool-eq-arr></code>) =	First (<code><rel></code>) =	{ <code><,>,<=,>=</code> }
284	First (<code><eq-op><eq-ope-arr></code>) =	First (<code><eq-op></code>) =	{ <code>==,!=</code> }
285	First (λ) =	Follow (<code><bool-exp2-arr></code>) =	{ <code>=,[</code> }
286	First (<code>id<id-tail><bool-arith4-arr><bool-rel-arr></code>) =		{ <code>id</code> }
287	First (<code>((<value-grp>)<bool-arith4-arr><bool-rel-arr>)</code>) =		{ () }
288	First (<code><digit><bool-arith4-arr><bool-rel-arr></code>) =	First (<code><digit></code>) =	{ <code>-,COIN-lit,DIME-lit</code> }
289	First (<code><eq-ope1></code>) =	First (<code><eq-ope1></code>) =	{ <code>PARCH-lit,SCROLL-lit,AYE,NAY,!,<#</code> }
290	First (<code><dime-arith><bool-arith4-arr></code>) =	First (<code><dime-arith></code>) =	{ <code>%,*,+,-,/,<^</code> }
291	First (λ) =	Follow (<code><bool-arith4-arr></code>) =	{ <code>!,!#,!=,%,(*,+,-,/,<,<=,==,>,>=,AYE,COIN-lit,DIME-lit,NAY,PARCH-lit,SCROLL-lit,],^,id</code> }
292	First (<code><rel><bool-arith3-arr></code>) =	First (<code><rel></code>) =	{ <code><,>,<=,>=</code> }
293	First (λ) =	Follow (<code><bool-rel-arr></code>) =	{ <code>!!</code> }
294	First (<code><log-op><bool-ope-arr><bool-exp-arr></code>) =	First (<code><log-op></code>) =	{ <code>&&, </code> }
295	First (λ) =	Follow (<code><bool-exp-arr></code>) =	{ [] }

296	First (=[<bool-arr2>]) =		{ = }
297	First (λ) =	Follow (<bool-arr2-tail>) =	{ }
298	First ($\text{[<bool-arr1>]<bav2-tail>}$) =		{ }
299	First (, <bool-arr2>) =		{ {,} }
300	First (λ) =	Follow (<bav2-tail>) =	{ !! }
301	First ($\text{(<params> [<local-dec> <ret-stmnts> BACK <bool-ret-val>!!] <sub-func>)}$) =		{ (}
302	First ($\text{<bool-ope-ret><bool-exp-ret>}$) =	First (<bool-ope-ret>) =	{ -,id,(,!,!#,AYE,NAY,COIN-lit,DIME-lit,PARC H-lit,SCROLL-lit }
303	First ($\text{id<id-tail><bool-exp2-ret>}$) =		{ id }
304	First ($\text{(<value-grp><bool-exp2-ret>}$) =		{ (}
305	First ($\text{<bool><bool-eq-ret>}$) =	First (<bool>) =	{ AYE,NAY,!,!# }
306	First ($\text{<digit><bool-arith><rel-eq-ret>}$) =	First (<digit>) =	{ -,COIN-lit,DIME-lit }
307	First ($\text{PARCH-lit<eq-op><parc-val>}$) =		{ PARCH-lit }
308	First ($\text{SCROLL-lit<scr-char><eq-op><scroll-val>}$) =		{ SCROLL-lit }
309	First ($\text{<eq-op><bool-ope-ret>}$) =	First (<eq-op>) =	{ ==,! = }
310	First (λ) =	Follow (<bool-eq-ret>) =	{ !!,%,*+,,-,/ ,^ }
311	First ($\text{<rel><bool-arith2-ret><bool-eq-ret>}$) =	First (<rel>) =	{ <,>,<=,>= }
312	First ($\text{<eq-digit><bool-arith3-ret>}$) =	First (<eq-digit>) =	{ ==,! = }

313	First ($\langle \text{dime-arith} \rangle \langle \text{bool-arith2-ret} \rangle$) =	First ($\langle \text{dime-arith} \rangle$) =	{ %, *, +, -, /, ^ }
314	First (λ) =	Follow ($\langle \text{bool-arith2-ret} \rangle$) =	{ !, ,, }
315	First ($\langle \text{dime-arith} \rangle \langle \text{bool-arith3-ret} \rangle$) =	First ($\langle \text{dime-arith} \rangle$) =	{ %, *, +, -, /, ^ }
316	First (λ) =	Follow ($\langle \text{bool-arith3-ret} \rangle$) =	{ !, ,, }
317	First ($\langle \text{dime-arith} \rangle \langle \text{bool-arith} \rangle \langle \text{releq-ret} \rangle$) =	First ($\langle \text{dime-arith} \rangle$) =	{ %, *, +, -, /, ^ }
318	First ($\langle \text{rel} \rangle \langle \text{bool-arith2-ret} \rangle \langle \text{bool-eq-ret} \rangle$) =	First ($\langle \text{rel} \rangle$) =	{ <, >, <=, >= }
319	First ($\langle \text{eq-op} \rangle \langle \text{eq-ope-ret} \rangle$) =	First ($\langle \text{eq-op} \rangle$) =	{ ==, != }
320	First (λ) =	Follow ($\langle \text{bool-exp2-ret} \rangle$) =	{ !, ,, BOOL, COIN, DIME, PARCH, SCROLL }
321	First ($\langle \text{id-id-tail} \rangle \langle \text{bool-arith4-ret} \rangle \langle \text{bool-rel-ret} \rangle$) =		{ id }
322	First ($\langle \langle \text{value-grp} \rangle \rangle \langle \text{bool-arith4-ret} \rangle \langle \text{bool-rel-ret} \rangle$) =		{ (}
323	First ($\langle \text{digit} \rangle \langle \text{bool-arith4-ret} \rangle \langle \text{bool-rel-ret} \rangle$) =	First ($\langle \text{digit} \rangle$) =	{ -, COIN-lit, DIME-lit }
324	First ($\langle \text{eq-ope1} \rangle$) =	First ($\langle \text{eq-ope1} \rangle$) =	{ PARCH-lit, SCROLL-lit, AYE, NAY, !, !# }
325	First ($\langle \text{dime-arith} \rangle \langle \text{bool-arith4-ret} \rangle$) =	First ($\langle \text{dime-arith} \rangle$) =	{ %, *, +, -, /, ^ }
326	First (λ) =	Follow ($\langle \text{bool-arith4-ret} \rangle$) =	{ ! }
327	First ($\langle \text{rel} \rangle \langle \text{bool-arith3-ret} \rangle$) =	First ($\langle \text{rel} \rangle$) =	{ <, >, <=, >= }
328	First (λ) =	Follow ($\langle \text{bool-rel-ret} \rangle$) =	{) }
329	First ($\langle \text{log-op} \rangle \langle \text{bool-ope-ret} \rangle \langle \text{bool-exp-ret} \rangle$) =	First ($\langle \text{log-op} \rangle$) =	{ &&, }
330	First (λ) =	Follow ($\langle \text{bool-exp-ret} \rangle$) =	{) }
331	First ($\langle \text{d-type} \rangle$ id $\langle \text{param-mult} \rangle$) =	First ($\langle \text{d-type} \rangle$) =	{ COIN, DIME, PARCH, SCROLL, BOOL }
332	First (λ) =	Follow ($\langle \text{params} \rangle$) =	{ id }
333	First (, $\langle \text{params} \rangle$) =		{ , }
334	First (λ) =	Follow ($\langle \text{param-mult} \rangle$) =	{ BACK }

335	First (COIN) =		{ COIN }
336	First (DIME) =		{ DIME }
337	First (PARCH) =		{ PARCH }
338	First (SCROLL) =		{ SCROLL }
339	First (BOOL) =		{ BOOL }
340	First (<statements><ret-st mnts>) =	First (<statements>) =	{ id,ASK,ECHO,LOOK,CHART,HOIST,HEAV E,HAUL,+#,-# }
341	First (λ) =	Follow (<ret-stmnts>) =	{ !,!,!#,!=,%,&&,&,(,),*,+,+,#,,,,-,#/,<,<=,==, >,>=,ASK,AYE,BOOL,CHART,COIN,COIN- lit,DIME,DIME-lit,ECHO,HAUL,HEAVE,HOI ST,LOCKE,LOOK,NAY,PARCH,PARCH-lit, SCROLL,SCROLL-lit,[,],^,id,{, }
342	First ({<index><elmt2>) =		{ { }
343	First (\$id) =		{ \$ }
344	First ((<args>)) =		{ (}
345	First (λ) =	Follow (<id-tail>) =	{) }
346	First ({<index>}) =		{ { }
347	First (λ) =	Follow (<elmt2>) =	{) }
348	First (<value><args-mult>) =	First (<value>) =	{ -,id,(,!,!#,AYE,NAY,COIN-lit,DIME-lit,PARC H-lit,SCROLL-lit }
349	First (λ) =	Follow (<args>) =	{ !=,%,),*,+,-,/,<,<=,==,>,>=,^ }
350	First (, <args>) =		{ , }
351	First (λ) =	Follow (<args-mult>) =	{ !=,%,),*,+,-,/,<,<=,==,>,>=,^ }
352	First (id<id-tail><exp>) =		{ id }
353	First ((<value-grp><exp>)) =		{ (}
354	First (<digit><digit-tail>) =	First (<digit>) =	{ -,COIN-lit,DIME-lit }
355	First (PARCH-lit<parch-tail >) =		{ PARCH-lit }
356	First (SCROLL-lit<scr-char ><scroll-tail>) =		{ SCROLL-lit }
357	First (<bool><logeq-var>) =	First (<bool>) =	{ AYE,NAY,!,!# }
358	First (<arith-var><releq-var >) =	First (<arith-var>) =	{ %,*,+,-,/,^ }
359	First (λ) =	Follow (<digit-tail>) =	{ !=,%,),*,+,-,/,<,<=,==,>,>=,^ }

360	First (<code><dime-arith><arith-var></code>) =	First (<code><dime-arith></code>) =	{ %, *, +, -, /, ^ }
361	First (λ) =	Follow (<code><arith-var></code>) =	{ !=, %, &, &&, *, +, -, /, <, <=, ==, >, >=, ^, }
362	First (<code><rel><arith-var2><log-eq-var></code>) =	First (<code><rel></code>) =	{ <, >, <=, >= }
363	First (<code><eq-digit><arith-var3><log-var></code>) =	First (<code><eq-digit></code>) =	{ ==, != }
364	First (λ) =	Follow (<code><releq-var></code>) =	{ !=, %, &, *, +, -, /, <, <=, ==, >, >=, ^ }
365	First (<code><dime-arith><arith-var2></code>) =	First (<code><dime-arith></code>) =	{ %, *, +, -, /, ^ }
366	First (λ) =	Follow (<code><arith-var2></code>) =	{ !=, %, *, +, -, /, <, <=, ==, >, >=, ^ }
367	First (<code><dime-arith><arith-var3></code>) =	First (<code><dime-arith></code>) =	{ %, *, +, -, /, ^ }
368	First (λ) =	Follow (<code><arith-var3></code>) =	{ !=, <, <=, ==, >, >=, { }
369	First (<code><logeq-op><log-ope><log-var></code>) =	First (<code><logeq-op></code>) =	{ &&, , !=, == }
370	First (λ) =	Follow (<code><logeq-var></code>) =	{ !=, %, *, +, -, /, <, <=, ==, >, >=, ^ }
371	First (<code><log-op></code>) =	First (<code><log-op></code>) =	{ &&, }
372	First (<code><eq-op></code>) =	First (<code><eq-op></code>) =	{ ==, != }
373	First (<code><log-op><log-ope><log-var></code>) =	First (<code><log-op></code>) =	{ &&, }
374	First (λ) =	Follow (<code><log-var></code>) =	{ !=, %, *, +, -, /, <, <=, ==, >, >=, ^ }
375	First (<code><eq-op>PARCH-lit<log-var></code>) =	First (<code><eq-op></code>) =	{ ==, != }
376	First (λ) =	Follow (<code><parch-tail></code>) =	{ !=, %, *, +, -, /, <, <=, ==, >, >=, ^ }
377	First (<code><concat-var></code>) =	First (<code><concat-var></code>) =	{ & }
378	First (<code><eq-op><scroll-val><log-var></code>) =	First (<code><eq-op></code>) =	{ ==, != }
379	First (λ) =	Follow (<code><scroll-tail></code>) =	{ !=, %, *, +, -, /, <, <=, ==, >, >=, ^ }
380	First (<code>& <scroll-val><concat-var></code>) =		{ & }
381	First (λ) =	Follow (<code><concat-var></code>) =	{ !=, %, &, *, +, -, /, <, <=, ==, >, >=, ^ }
382	First (<code><dime-arith><arith-var><releq-var></code>) =	First (<code><dime-arith></code>) =	{ %, *, +, -, /, ^ }

383	First ($\langle \text{rel} \rangle \langle \text{arith-var2} \rangle \langle \text{log-eq-var} \rangle$) =	First ($\langle \text{rel} \rangle$) =	{ $\langle \text{,} \rangle, \langle \text{=}, \rangle, \langle \text{>}, \rangle$ }
384	First ($\langle \text{log-op} \rangle \langle \text{log-ope} \rangle \langle \text{log-var} \rangle$) =	First ($\langle \text{log-op} \rangle$) =	{ $\&, $ }
385	First ($\langle \text{eq-op} \rangle \langle \text{eq-var} \rangle \langle \text{log-var} \rangle$) =	First ($\langle \text{eq-op} \rangle$) =	{ $\text{==}, \text{!=}$ }
386	First ($\&$ $\langle \text{scroll-val} \rangle \langle \text{concat-var} \rangle$) =		{ $\&$ }
387	First (λ) =	Follow ($\langle \text{exp} \rangle$) =	{ $!=, \%, \&, *, +, -, /, \langle \text{,} \rangle, \langle \text{=}, \rangle, \langle \text{>}, \rangle, \text{^}$ }
388	First ($\text{id} \langle \text{id-tail} \rangle \langle \text{log-exp} \rangle$) =		{ id }
389	First ($\langle \langle \text{value-grp} \rangle \rangle \langle \text{log-exp} \rangle$) =		{ $\langle \text{ } \rangle$ }
390	First ($\langle \text{bool} \rangle \langle \text{log-booleq} \rangle$) =	First ($\langle \text{bool} \rangle$) =	{ $\text{AYE}, \text{NAY}, \text{!}, \text{!#}$ }
391	First ($\langle \text{digit} \rangle \langle \text{bool-arith} \rangle \langle \text{log-releq} \rangle$) =	First ($\langle \text{digit} \rangle$) =	{ $\text{-}, \text{COIN-lit}, \text{DIME-lit}$ }
392	First ($\text{PARCH-lit} \langle \text{eq-op} \rangle \langle \text{parch-val} \rangle$) =		{ PARCH-lit }
393	First ($\text{SCROLL-lit} \langle \text{scr-char} \rangle \langle \text{eq-op} \rangle \langle \text{scroll-val} \rangle$) =		{ SCROLL-lit }
394	First ($\langle \text{eq-op} \rangle \langle \text{log-ope} \rangle$) =	First ($\langle \text{eq-op} \rangle$) =	{ $\text{==}, \text{!=}$ }
395	First (λ) =	Follow ($\langle \text{log-booleq} \rangle$) =	{ $!=, \%, \&, *, +, -, /, \langle \text{,} \rangle, \langle \text{=}, \rangle, \langle \text{>}, \rangle, \text{^}$ }
396	First ($\langle \text{rel} \rangle \langle \text{arith-var2} \rangle \langle \text{log-booleq} \rangle$) =	First ($\langle \text{rel} \rangle$) =	{ $\langle \text{,} \rangle, \langle \text{=}, \rangle, \langle \text{>}, \rangle$ }
397	First ($\langle \text{eq-digit} \rangle \langle \text{arith-var3} \rangle$) =	First ($\langle \text{eq-digit} \rangle$) =	{ $\text{==}, \text{!=}$ }
398	First ($\langle \text{dime-arith} \rangle \langle \text{arith-var} \rangle \langle \text{log-releq} \rangle$) =	First ($\langle \text{dime-arith} \rangle$) =	{ $\%, *, +, -, /, \text{^}$ }
399	First ($\langle \text{rel} \rangle \langle \text{arith-var2} \rangle \langle \text{log-booleq} \rangle$) =	First ($\langle \text{rel} \rangle$) =	{ $\langle \text{,} \rangle, \langle \text{=}, \rangle, \langle \text{>}, \rangle$ }

400	First (<code><eq-op><eq-var></code>) =	First (<code><eq-op></code>) =	{ <code>==,!=</code> }
401	First (λ) =	Follow (<code><log-exp></code>) =	{ <code>!=,%,&,)*,+,-,/,<,<=,==,>,>=,^</code> }
402	First (<code>id<id-tail><arith-var4><rel-var></code>) =		{ <code>id</code> }
403	First (<code>(<value-grp><arith-var4><rel-var></code>) =		{ <code>()</code> }
404	First (<code><digit><arith-var4><rel-var></code>) =	First (<code><digit></code>) =	{ <code>-,COIN-lit,DIME-lit</code> }
405	First (<code><eq-ope1></code>) =	First (<code><eq-ope1></code>) =	{ <code>PARCH-lit,SCROLL-lit,AYE,NAY,!,#</code> }
406	First (<code><dime-arith><arith-var4></code>) =	First (<code><dime-arith></code>) =	{ <code>%,*,+,-,/,^</code> }
407	First (λ) =	Follow (<code><arith-var4></code>) =	{ <code>)</code> }
408	First (<code><rel><arith-var3></code>) =	First (<code><rel></code>) =	{ <code><,>,<=,>=</code> }
409	First (λ) =	Follow (<code><rel-var></code>) =	{ <code>)</code> }
410	First (<code>id<id-tail><exp-grp></code>) =		{ <code>id</code> }
411	First (<code>(<value-grp><exp-grp></code>) =		{ <code>()</code> }
412	First (<code><digit><digit-tail-grp></code>) =	First (<code><digit></code>) =	{ <code>-,COIN-lit,DIME-lit</code> }
413	First (<code>PARCH-lit<parch-tail-grp></code>) =		{ <code>PARCH-lit</code> }
414	First (<code>SCROLL-lit<scr-char><scroll-tail-grp></code>) =		{ <code>SCROLL-lit</code> }
415	First (<code><bool><logeq-grp></code>) =	First (<code><bool></code>) =	{ <code>AYE,NAY,!,#</code> }
416	First (<code><arith-grp><releq-grp></code>) =	First (<code><arith-grp></code>) =	{ <code>%,*,+,-,/,^</code> }
417	First (λ) =	Follow (<code><digit-tail-grp></code>) =	{ <code>!=,&,),=,==,{</code> }
418	First (<code><dime-arith><arith-grp></code>) =	First (<code><dime-arith></code>) =	{ <code>%,*,+,-,/,^</code> }
419	First (λ) =	Follow (<code><arith-grp></code>) =	{ <code>),=,{</code> }

420	First ($\langle \text{rel} \rangle \langle \text{arith-grp2} \rangle \langle \text{logeq-grp} \rangle$) =	First ($\langle \text{rel} \rangle$) =	{ $<, >, <=, >=$ }
421	First ($\langle \text{eq-digit} \rangle \langle \text{arith-grp3} \rangle \langle \text{bool-exp-grp} \rangle$) =	First ($\langle \text{eq-digit} \rangle$) =	{ $=, !=$ }
422	First (λ) =	Follow ($\langle \text{releqvar-grp} \rangle$) =	$!=, \&,), =, ==, \{, !!$
423	First ($\langle \text{dime-arith} \rangle \langle \text{arith-grp2} \rangle$) =	First ($\langle \text{dime-arith} \rangle$) =	{ $\%, *, +, -, /, ^$ }
424	First (λ) =	Follow ($\langle \text{arith-grp2} \rangle$) =	{ $), =, \{$ }
425	First ($\langle \text{dime-arith} \rangle \langle \text{arith-grp3} \rangle$) =	First ($\langle \text{dime-arith} \rangle$) =	{ $\%, *, +, -, /, ^$ }
426	First (λ) =	Follow ($\langle \text{arith-grp3} \rangle$) =	{ $)$ }
427	First ($\langle \text{logeq-op} \rangle \langle \text{bool-op-grp} \rangle \langle \text{bool-exp-grp} \rangle$) =	First ($\langle \text{logeq-op} \rangle$) =	{ $\&\&, , !=, ==$ }
428	First (λ) =	Follow ($\langle \text{logeq-grp} \rangle$) =	{ $)$ }
429	First ($\langle \text{eq-op} \rangle \text{PARCH-lit} \langle \text{bool-exp-grp} \rangle$) =	First ($\langle \text{eq-op} \rangle$) =	{ $=, !=$ }
430	First (λ) =	Follow ($\langle \text{parch-tail-grp} \rangle$) =	{ $)$ }
431	First ($\langle \text{concat-str} \rangle$) =	First ($\langle \text{concat-str} \rangle$) =	{ $\&$ }
432	First ($\langle \text{eq-op} \rangle \langle \text{scroll-val} \rangle \langle \text{bool-exp-grp} \rangle$) =	First ($\langle \text{eq-op} \rangle$) =	{ $=, !=$ }
433	First (λ) =	Follow ($\langle \text{scroll-tail-grp} \rangle$) =	{ $)$ }
434	First ($\&$ $\langle \text{scroll-val} \rangle$ $\langle \text{concat-grp} \rangle$) =		{ $\&$ }
435	First (λ) =	Follow ($\langle \text{concat-grp} \rangle$) =	{ $!!$ }
436	First ($\langle \text{dime-arith} \rangle \langle \text{arith-grp} \rangle \langle \text{releq-grp} \rangle$) =	First ($\langle \text{dime-arith} \rangle$) =	{ $\%, *, +, -, /, ^$ }
437	First ($\langle \text{rel} \rangle \langle \text{arith-grp2} \rangle \langle \text{logeq-grp} \rangle$) =	First ($\langle \text{rel} \rangle$) =	{ $<, >, <=, >=$ }
438	First ($\langle \text{log-op} \rangle \langle \text{bool-op-grp} \rangle \langle \text{bool-exp-grp} \rangle$) =	First ($\langle \text{log-op} \rangle$) =	{ $\&\&, $ }
439	First ($\langle \text{eq-op} \rangle \langle \text{eq-op-grp} \rangle \langle \text{bool-exp-grp} \rangle$) =	First ($\langle \text{eq-op} \rangle$) =	{ $=, !=$ }

440	First (& <scroll-val><concat-grp>) =		{ & }
441	First (λ) =	Follow (<exp-grp>) =	{ !! }
442	First (LOCKE <const-init>!!) =		{ LOCKE }
443	First (COIN <coin-locke><coin-locke-mult>) =		{ COIN }
444	First (DIME <dime-locke><dime-locke-mult>) =		{ DIME }
445	First (PARCH <parch-locke><parch-locke-mult>) =		{ PARCH }
446	First (SCROLL <scroll-locke><scroll-locke-mult>) =		{ SCROLL }
447	First (BOOL <bool-locke><bool-locke-mult>) =		{ BOOL }
448	First (id = COIN-lit) =		{ id }
449	First (, <coin-locke><coin-locke-mult>) =		{ , }
450	First (λ) =	Follow (<coin-locke-mult>) =	{ !!, }
451	First (id = <locke-digit>) =		{ id }
452	First (COIN-lit) =		{ COIN-lit }
453	First (DIME-lit) =		{ DIME-lit }
454	First (, <dime-locke><dime-locke-mult>) =		{ , }
455	First (λ) =	Follow (<dime-locke-mult>) =	{ !! }
456	First (id = PARCH-lit) =		{ id }
457	First (, <parch-locke><parch-locke-mult>) =		{ , }
458	First (λ) =	Follow (<parch-locke-mult>) =	{ },,AYE,NAY }
459	First (id = SCROLL-lit<scr-id>) =		{ id }
460	First ({COIN-lit}) =		{ { }
461	First (λ) =	Follow (<scr-id>) =	{ BOOL,COIN,DIME,PARCH,SCROLL }
462	First (, <scroll-locke> <scroll-locke-mult>) =	First (, <scroll-locke> <scroll-locke-mult>) =	{ , }

463	First (λ) =	Follow (<scroll-locke-mult>)	{ BOOL, COIN, DIME, PARCH, SCROLL }
464	First (id = <locke-bool>) =		{ id }
465	First (AYE) =		{ AYE }
466	First (NAY) =		{ NAY }
467	First (, <bool-locke> <bool-locke-mult>) =		{ , }
468	First (λ) =	Follow (<bool-locke-mult>)	{ BOOL, COIN, DIME, PARCH, SCROLL }
469	First (MAST id [<mem-dec> <mem-dec-tail>] !! <struct>) =		{ MAST }
470	First (λ) =	Follow (<struct>) =	{ !! }
471	First (<d-type> id <mem-mult>!!) =	First (<d-type>) =	{ COIN, DIME, PARCH, SCROLL, BOOL }
472	First (, id <mem-mult>) =		{ , }
473	First (λ) =	Follow (<mem-mult>) =	{ ABYSS }
474	First (<mem-dec> <mem-dec-tail>) =	First (<mem-dec>) =	{ COIN, DIME, PARCH, SCROLL, BOOL }
475	First (λ) =	Follow (<mem-dec-tail>) =	{ ABYSS }
476	First (<return-func>) =	First (<return-func>) =	{ COIN, DIME, PARCH, SCROLL, BOOL }
477	First (<nonreturn-func>) =	First (<nonreturn-func>) =	{ ABYSS }
478	First (λ) =	Follow (<sub-func>) =	{ ABYSS }
479	First (COIN id<coin-func>) =		{ COIN }
480	First (DIME id<dime-func>) =		{ DIME }
481	First (PARCH id<parch-func>) =		{ PARCH }
482	First (SCROLL id<scroll-func>) =		{ SCROLL }
483	First (BOOL id<bool-func>) =		{ BOOL }
484	First (ABYSS id <params> [<ahoy-local-dec> <nonret-stmnts> <nonret-back>] <sub-func>) =		{ ABYSS }
485	First (<nonret-stmnt><nonr et-tail>) =	First (<nonret-stmnt>) =	{ id, ASK, ECHO, LOOK, CHART, HOIST, HEAV E, HAUL, +#, -# }

486	First ($\langle \text{nonret-stmnts} \rangle$) =	First ($\langle \text{nonret-stmnts} \rangle$) =	{ id,ASK,ECHO,LOOK,CHART,HOIST,HEAV E,HAUL,+#,-# }
487	First (λ) =	Follow ($\langle \text{nonret-tail} \rangle$) =	{ id,ASK,ECHO,LOOK,CHART,HOIST,HEAV E,HAUL,+#,-# }
488	First (BACK!!) =		{ BACK }
489	First (λ) =	Follow ($\langle \text{nonret-back} \rangle$) =	{ id,ASK,ECHO,LOOK,CHART,HOIST,HEAV E,HAUL,+#,-# }
490	First (id $\langle \text{assign-tail} \rangle$!!) =		{ id }
491	First (ASK (SCROLL-lit, $\langle \text{addr} \rangle$) !!) =		{ ASK }
492	First (ECHO (SCROLL-lit $\langle \text{args-mult} \rangle$) !!) =		{ ECHO }
493	First (LOOK ($\langle \text{condition} \rangle$) [$\langle \text{look-body} \rangle$ $\langle \text{jump-stmnt} \rangle$] $\langle \text{nonret-look-tail} \rangle$) =		{ LOOK }
494	First (CHART ($\langle \text{chart-cond} \rangle$) [$\langle \text{courses} \rangle$ $\langle \text{course-tail} \rangle$ $\langle \text{adrift-case} \rangle$]) =		{ CHART }
495	First (HOIST ($\langle \text{hoist-init} \rangle$!! $\langle \text{hoist-cond} \rangle$!! $\langle \text{hoist-upd} \rangle$) [$\langle \text{look-body} \rangle$ $\langle \text{jump-stmnt} \rangle$]) =		{ HOIST }
496	First (HEAVE ($\langle \text{condition} \rangle$) [$\langle \text{look-body} \rangle$ $\langle \text{jump-stmnt} \rangle$]) =		{ HEAVE }
497	First (HAUL [$\langle \text{look-body} \rangle$ $\langle \text{jump-stmnt} \rangle$] HEAVE ($\langle \text{condition} \rangle$!!) =		{ HAUL }
498	First ($\langle \text{unary-exp} \rangle$!!) =	First ($\langle \text{unary-exp} \rangle$) =	{ +#,-# }

499	First (DROLOOK ($\langle$ condition $\rangle$) [$\langle$ look-body $\rangle$ $\langle$ jump-stmnt $\rangle$] $\langle$ nonret-look-tail $\rangle$) =		{ DROLOOK }
500	First (DROP [$\langle$ look-body $\rangle$ $\langle$ jump-stmnt $\rangle$]) =		{ DROP }
501	First (λ) =	Follow ($\langle$ nonret-look-tail $\rangle$) =	{ + #, #, ASK, BACK, CHART, ECHO, HAUL, HE AVE, HOIST, LOOK, MAST, id, { }
502	First ($\langle$ var-arr $\rangle$ $\langle$ local-dec $\rangle$) =	First ($\langle$ var-arr $\rangle$) =	{ COIN, DIME, PARCH, SCROLL, BOOL }
503	First ($\langle$ struct-dec $\rangle$) =	First ($\langle$ struct-dec $\rangle$) =	{ MAST }
504	First (λ) =	Follow ($\langle$ local-dec $\rangle$) =	{ BACK, { }
505	First (COIN id $\langle$ coin-var-arr $\rangle$!!) =		{ COIN }
506	First (DIME id $\langle$ dime-var-arr $\rangle$!!) =		{ DIME }
507	First (PARCH id $\langle$ parch-var-arr $\rangle$!!) =		{ PARCH }
508	First (SCROLL id $\langle$ scroll-var-arr $\rangle$!!) =		{ SCROLL }
509	First (BOOL id $\langle$ bool-var-arr $\rangle$!!) =		{ BOOL }
510	First (MAST id id $\langle$ str-dec-init $\rangle$!! $\langle$ struct-dec $\rangle$) =		{ MAST }
511	First (λ) =	Follow ($\langle$ struct-dec $\rangle$) =	{ !! }
512	First (, id $\langle$ str-dec-tail $\rangle$) =		{ , }
513	First (= $\langle$ str-val $\rangle$ $\langle$ str-val-tai l $\rangle$) =		{ = }
514	First (λ) =	Follow ($\langle$ str-dec-init $\rangle$) =	{ !=, %, *, +, -, /, <, <=, ==, >, >=,], ^ }
515	First (, id $\langle$ str-dec-tail $\rangle$) =		{ , }
516	First (λ) =	Follow ($\langle$ str-dec-tail $\rangle$) =	{] }
517	First ($\langle$ value-str $\rangle$) =	First ($\langle$ value-str $\rangle$) =	{ -, id, (, !, ! #, AYE, NAY, COIN-lit, DIME-lit, PARC H-lit, SCROLL-lit }
518	First (\$id = $\langle$ value-str $\rangle$) =		{ \$ }
519	First (, $\langle$ str-val $\rangle$ $\langle$ str-val-tail $\rangle$) =		{ , }

520	First (λ) =	Follow (<str-val-tail>) =	{ !=, %, *, +, -, /, <, <=, ==, >, >=,], ^ }
521	First (id<id-tail><exp-str>) =		{ id }
522	First ((<value-grp><exp-str>) =		{ (}
523	First (<digit><digit-tail-str>) =	First (<digit>) =	{ -, COIN-lit, DIME-lit }
524	First (PARCH-lit<parch-tail-str>) =		{ PARCH-lit }
525	First (SCROLL-lit<scr-char><scroll-tail-str>) =		{ SCROLL-lit }
526	First (<bool><logeq-str>) =	First (<bool>) =	{ AYE, NAY, !, !# }
527	First (<arith-str><releq-str>) =	First (<arith-str>) =	{ %, *, +, -, /, ^ }
528	First (λ) =	Follow (<digit-tail-str>) =	{ !=, %, *, +, -, /, <, <=, ==, >, >=,], ^ }
529	First (<dime-arith><arith-str>) =	First (<dime-arith>) =	{ %, *, +, -, /, ^ }
530	First (λ) =	Follow (<arith-str>) =	{ !=, %, &, *, +, -, /, <, <=, ==, >, >=,], ^ }
531	First (<rel><arith-str2><logeq-str>) =	First (<rel>) =	{ <, >, <=, >= }
532	First (<eq-digit><arith-str3><bool-exp-arr>) =	First (<eq-digit>) =	{ ==, != }
533	First (λ) =	Follow (<releq-str>) =	{ [}
534	First (<dime-arith><arith-str2>) =	First (<dime-arith>) =	{ %, *, +, -, /, ^ }
535	First (λ) =	Follow (<arith-str2>) =	{ !=, %, *, +, -, /, <, <=, ==, >, >=,], ^ }
536	First (<dime-arith><arith-str3>) =	First (<dime-arith>) =	{ %, *, +, -, /, ^ }
537	First (λ) =	Follow (<arith-str3>) =	{ !=, %, *, +, -, /, <, <=, ==, >, >=,], ^ }
538	First (<logeq-op><log-op-str><bool-exp-arr>) =	First (<logeq-op>) =	{ &&, , !=, == }
539	First (λ) =	Follow (<logeq-str>) =	{ !=, %, *, +, -, /, <, <=, ==, >, >=,], ^ }

540	First (<eq-op>PARCH-lit<bool-exp-arr>) =	First (<eq-op>) =	{ ==,!= }
541	First (λ) =	Follow (<parch-tail-str>) =	{ !=,%,),*,+,-,/,<,<=,=>,>=,],^ }
542	First (<concat-str>) =	First (<concat-str>) =	{ & }
543	First (<eq-op><scroll-val><bool-exp-arr>) =	First (<eq-op>) =	{ ==,!= }
544	First (λ) =	Follow (<scroll-tail-str>) =	{ !=,%,),*,+,-,/,<,<=,=>,>=,],^ }
545	First (& <scroll-val><concat-str>) =	First (& <scroll-val> <concat-str>) =	{ & }
546	First (λ) =	Follow (<concat-str>) =	{ +,#, #,ADRIFT,ASK,BACK,CHART,COIN,C OURSE,DROP,DROPLOOK,ECHO,HAUL, HEAVE,HOIST,LAND,LOOK,id,{ }
547	First (<dime-arith><arith-str><releq-str>) =	First (<dime-arith>) =	{ %,*,+,-,/,<,<=,=>,>=,],^ }
548	First (<rel><arith-str2><log-eq-str>) =	First (<rel>) =	{ <,>,<=,>= }
549	First (<log-op><bool-ope-arr><bool-exp-arr>) =	First (<log-op>) =	{ &&, }
550	First (<eq-op><eq-ope-arr><bool-exp-arr>) =	First (<eq-op>) =	{ ==,!= }
551	First (& <scroll-val><concat-str>) =	First (& <scroll-val><concat-str>) =	{ & }
552	First (λ) =	Follow (<exp-str>) =	{ +,#, #,ADRIFT,ASK,BACK,CHART,COIN,C OURSE,DROP,DROPLOOK,ECHO,HAUL, HEAVE,HOIST,LAND,LOOK,id,{ }
553	First (<assign-stmnt>) =	First (<assign-stmnt>) =	{ id }
554	First (<ask-stmnt>) =	First (<ask-stmnt>) =	{ ASK }
555	First (<echo-stmnt>) =	First (<echo-stmnt>) =	{ ECHO }
556	First (<look-stmnt>) =	First (<look-stmnt>) =	{ LOOK }
557	First (<chart-stmnt>) =	First (<chart-stmnt>) =	{ CHART }
558	First (<hoist-stmnt>) =	First (<hoist-stmnt>) =	{ HOIST }
559	First (<heave-stmnt>) =	First (<heave-stmnt>) =	{ HEAVE }
560	First (<haul-stmnt>) =	First (<haul-stmnt>) =	{ HAUL }

561	First (<unary-exp>!!) =	First (<unary-exp>) =	{ +#, -# }
562	First (id<assign-tail>!!) =	First (id<assign-tail>!!) =	{ id }
563	First (<arr-str><assign-bod y>) =	First (<arr-str>) =	{ \$, { }
564	First ((<args>)) =	First ((<args>) =	{ (}
565	First ({<index>}<arr-str-tail >) =	First ({<index>}<arr-str-tail> =	{ { }
566	First (\$id) =	First (\$id) =	{ \$ }
567	First (λ) =	Follow (<arr-str>) =	{ !! }
568	First ({<index>}) =	First ({<index>}) =	{ { }
569	First (λ) =	Follow (<arr-str-tail>) =	{ !! }
570	First (= <value-as>) =	First (= <value-as>) =	{ -, id, (, !, !#, AYE, NAY, COIN-lit, DIME-lit, PARC H-lit, SCROLL-lit }
571	First (<arith-assign-op><di me-ret-val>) =	First (<arith-assign-op>) =	{ +, -, *, /, %, ^ }
572	First (id<id-tail><exp-as>) =	First (id<id-tail><exp-as>) =	{ id }
573	First ((<value-grp><exp-as >) =	First ((<value-grp>) =	{ (}
574	First (<digit><digit-tail-as>) =	First (<digit>) =	{ -, COIN-lit, DIME-lit }
575	First (PARCH-lit<parch-tail- as>) =		{ PARCH-lit }
576	First (SCROLL-lit<scr-char ><scroll-tail-as>) =		{ SCROLL-lit }
577	First (<bool><logeq-as>) =	First (<bool>) =	{ AYE, NAY, !, !# }
578	First (<arith-as><releq-as>) =	First (<arith-as>) =	{ %, *, +, -, /, ^ }
579	First (λ) =	Follow (<digit-tail-as>) =	{ !! }
580	First (<dime-arith><arith-as >) =	First (<dime-arith>) =	{ %, *, +, -, /, ^ }
581	First (λ) =	Follow (<arith-as>) =	{ !!, !=, &, == }

582	First (<code><rel><arith-as2><log eq-as></code>) =	First (<code><rel></code>) =	{ <code><, >, <=, >=</code> }
583	First (<code><eq-digit><arith-as3> <bool-exp-ret></code>) =	First (<code><eq-digit></code>) =	{ <code>=, !=</code> }
584	First (<code>λ</code>) =	Follow (<code><releq-as></code>) =	{ <code>!!,,</code> }
585	First (<code><dime-arith><arith-as 2></code>) =	First (<code><dime-arith></code>) =	{ <code>%, *, +, -, /, ^</code> }
586	First (<code>λ</code>) =	Follow (<code><arith-as2></code>) =	{ <code>!!</code> }
587	First (<code><dime-arith><arith-as 3></code>) =	First (<code><dime-arith></code>) =	{ <code>%, *, +, -, /, ^</code> }
588	First (<code>λ</code>) =	Follow (<code><arith-as3></code>) =	{ <code>!!</code> }
589	First (<code><logeq-op><bool-ope -ret><bool-exp-ret></code>) =	First (<code><logeq-op></code>) =	{ <code>&&, , !=, ==</code> }
590	First (<code>λ</code>) =	Follow (<code><logeq-as></code>) =	{ <code>!!</code> }
591	First (<code><eq-op>PARCH-lit<lo g-as></code>) =	First (<code><eq-op></code>) =	{ <code>=, !=</code> }
592	First (<code>λ</code>) =	Follow (<code><parch-tail-as></code>) =	{ <code>!!</code> }
593	First (<code><concat-as></code>) =	First (<code><concat-as></code>) =	{ <code>&</code> }
594	First (<code><eq-op><scroll-val>< bool-exp-ret></code>) =	First (<code><eq-op></code>) =	{ <code>=, !=</code> }
595	First (<code>λ</code>) =	Follow (<code><scroll-tail-as></code>) =	{ <code>!!</code> }
596	First (<code>& <scroll-val> <concat-as></code>) =		{ <code>&</code> }
597	First (<code>λ</code>) =	Follow (<code><concat-as></code>) =	{ <code>!!,(,,=,{</code> }
598	First (<code><dime-arith><arith-as ><releq-as></code>) =	First (<code><dime-arith></code>) =	{ <code>%, *, +, -, /, ^</code> }
599	First (<code><rel><arith-as2><log eq-as></code>) =	First (<code><rel></code>) =	{ <code><, >, <=, >=</code> }
600	First (<code><log-op><bool-ope-r et><bool-exp-ret></code>) =	First (<code><log-op></code>) =	{ <code>&&, </code> }
601	First (<code><eq-op><eq-ope-ret> <bool-exp-ret></code>) =	First (<code><eq-op></code>) =	{ <code>=, !=</code> }
602	First (<code>& <scroll-val><concat-a s></code>) =		{ <code>&</code> }

603	First (λ) =	Follow (<exp-as>) =	{ + #, - #, ADRIFT, ASK, BACK, CHART, COIN, C OURSE, DROP, DROPLOOK, ECHO, HAUL, HEAVE, HOIST, LAND, LOOK, id, { }
604	First (+ =) =		{ + = }
605	First (- =) =		{ - = }
606	First (* =) =		{ * = }
607	First (/ =) =		{ / = }
608	First (% =) =		{ % = }
609	First (^ =) =		{ ^ = }
610	First (ASK (SCROLL-lit, <addr>) !!) =		{ ASK }
611	First (@id<arr-str><addr-tai l>) =		{ @ }
612	First (, <addr>) =		{ , }
613	First (λ) =	Follow (<addr-tail>) =	{ + #, - #, ADRIFT, ASK, BACK, CHART, COIN, C OURSE, DROP, DROPLOOK, ECHO, HAUL, HEAVE, HOIST, LAND, LOOK, id, { }
614	First (ECHO (SCROLL-lit <args-mult>) !!) =		{ ECHO }
615	First (LOOK (<condition>) [<look-body> <jump-stmnt>] <look-tail>) =		{ LOOK }
616	First (<bool-grp-val>) =	First (<bool-grp-val>) =	{ - , id, (, ! , # , AYE, NAY, COIN-lit, DIME-lit, PARC H-lit, SCROLL-lit }
617	First (<statements><look-b ody>) =	First (<statements>) =	{ id, ASK, ECHO, LOOK, CHART, HOIST, HEAV E, HAUL, + #, - # }
618	First (λ) =	Follow (<look-body>) =	{ + #, - #, ADRIFT, ASK, BACK, CHART, COIN, C OURSE, DROP, DROPLOOK, ECHO, HAUL, HEAVE, HOIST, LAND, LOOK, id, { }
619	First (SAIL!!) =		{ SAIL }
620	First (LAND!!) =		{ LAND }
621	First (λ) =	Follow (<jump-stmnt>) =	{ + #, - #, ADRIFT, ASK, BACK, CHART, COIN, C OURSE, DROP, DROPLOOK, ECHO, HAUL, HEAVE, HOIST, LAND, LOOK, id, { }

622	First (DROPLOOK ($\langle$ condition $\rangle$) [$\langle$ look-body $\rangle$ $\langle$ jump-stmnt $\rangle$] $\langle$ look-tail $\rangle$) =		{ DROPLOOK }
623	First (DROP [$\langle$ look-body $\rangle$ $\langle$ jump-stmnt $\rangle$]) =		{ DROP }
624	First (λ) =	Follow ($\langle$ look-tail $\rangle$) =	{ }
625	First (CHART ($\langle$ chart-cond $\rangle$) [$\langle$ courses $\rangle$ $\langle$ course-tail $\rangle$ $\langle$ adrift-case $\rangle$]) =		{ CHART }
626	First (id $\langle$ id-tail $\rangle$) =	First (id $\langle$ id-tail $\rangle$) =	{ id }
627	First ($\langle$ chart-const $\rangle$) =	First ($\langle$ chart-const $\rangle$) =	{ COIN-lit,PARCH-lit,SCROLL-lit }
628	First (COIN-lit) =		{ COIN-lit }
629	First (PARCH-lit) =		{ PARCH-lit }
630	First (SCROLL-lit $\langle$ scr-id $\rangle$) =		{ SCROLL-lit }
631	First (COURSE $\langle$ chart-const $\rangle$: $\langle$ course-body $\rangle$ $\langle$ course-jmp $\rangle$) =		{ COURSE }
632	First ($\langle$ statements $\rangle$ $\langle$ cours e-body $\rangle$) =	First ($\langle$ statements $\rangle$) =	{ id,ASK,ECHO,LOOK,CHART,HOIST,HEAV E,HAUL,+,-,# }
633	First (λ) =	Follow ($\langle$ course-body $\rangle$) =	{ HOIST }
634	First (SAIL!!) =		{ SAIL }
635	First (LAND!!) =		{ LAND }
636	First (λ) =	Follow ($\langle$ course-jmp $\rangle$) =	{ }
637	First ($\langle$ courses $\rangle$ $\langle$ course-ta il $\rangle$) =	First ($\langle$ courses $\rangle$) =	{ COURSE }
638	First (λ) =	Follow ($\langle$ course-tail $\rangle$) =	{ LAND }
639	First (ADRIFT : $\langle$ adrift-body $\rangle$ LAND!!) =		{ ADRIFT }
640	First (λ) =	Follow ($\langle$ adrift-case $\rangle$) =	{ +,-,#,ADRIFT,ASK,BACK,CHART,COIN,C OURSE,DROP,DROPLOOK,ECHO,HAUL, HEAVE,HOIST,LAND,LOOK,id,{ }
641	First ($\langle$ statements $\rangle$ $\langle$ adrift- body $\rangle$) =	First ($\langle$ statements $\rangle$) =	{ id,ASK,ECHO,LOOK,CHART,HOIST,HEAV E,HAUL,+,-,# }

642	First (λ) =	Follow (<adrift-body>) =	{ !! }
643	First (HOIST (<hoist-init>!! <hoist-cond>!! <hoist-upd>) [<look-body> <jump-stmnt>]) =		{ HOIST }
644	First (COIN id = COIN-lit <init1-mult>) =		{ COIN }
645	First (id<arr-str> = COIN-lit <init2-mult>) =		{ id }
646	First (λ) =	Follow (<hoist-init>) =	{ !! }
647	First (, id = COIN-lit <init1-mult>) =		{ , }
648	First (λ) =	Follow (<init1-mult>) =	{ !! }
649	First (, id<arr-str> = COIN-lit <init2-mult>) =		{ , }
650	First (λ) =	Follow (<init2-mult>) =	{ !,!#,%,(*,+,-,/ ,AYE,COIN-lit,DIME-lit,NAY,P ARCH-lit,SCROLL-lit,^,id }
651	First (<dime-val><bool-arit h><releq-op><dime-v al><bool-arith3-grp>< hoist-log>) =	First (<dime-val>) =	{ (,-,COIN-lit,DIME-lit,id }
652	First (<rel-op>) =	First (<rel-op>) =	{ <,>,<=,>= }
653	First (<eq-op>) =	First (<eq-op>) =	{ ==,!= }
654	First (<log-op><hoist-cond >) =	First (<log-op>) =	{ &&, }
655	First (λ) =	Follow (<hoist-log>) =	{ HAUL }
656	First (<upd><upd-mult>) =	First (<upd>) =	{ +#,-#,id }
657	First (<unary-op>id<arr-str >) =	First (<unary-op>) =	{ +#,-# }
658	First (id<arr-str><arith-assi gn-op><dime-grp-val>) =		{ id }
659	First (, <upd><upd-mult>) =		{ , }

660	First (λ) =	Follow (<upd-mult>) =	{ + #, - #, ADRIFT, ASK, BACK, CHART, COIN, C OURSE, DROP, DROPLOOK, ECHO, HAUL, HEAVE, HOIST, LAND, LOOK, id, { }
661	First (HEAVE <condition> [<look-body> <jump-stmnt>]) =		{ HEAVE }
662	First (HAUL [<look-body> <jump-stmnt>] HEAVE <condition>!!) =		{ HAUL }
663	First <unary-op>id<arr-str >) =	First (<unary-op>) =	{ + #, - # }
664	First (+ #) =		{ + # }
665	First (- #) =		{ - # }
666	First <var-arr><ahoy-local- dec>) =	First (<var-arr>) =	{ COIN, DIME, PARCH, SCROLL, BOOL }
667	First <ahoy-struct-dec>) =	First (<ahoy-struct-dec>) =	{ MAST }
668	First (λ) =	Follow (<ahoy-local-dec>) =	{id, ASK, ECHO, LOOK, CHART, HOIST, HEAVE, HAUL, + #, - #}
669	First (MAST id id <str-dec-init>!! <ahoy-struct-dec>) =		{ MAST }
670	First (λ) =	Follow (<ahoy-struct-dec>) =	{id, ASK, ECHO, LOOK, CHART, HOIST, HEAVE, HAUL, + #, - #}
671	First <ahoy-stmnt><ahoy-t ail>) =	First (<ahoy-stmnt>) =	{ id, ASK, ECHO, LOOK, CHART, HOIST, HEAV E, HAUL, + #, - # }
672	First (<ahoy-stmnts>) =	First (<ahoy-stmnts>) =	{ id, ASK, ECHO, LOOK, CHART, HOIST, HEAV E, HAUL, + #, - # }
673	First (λ) =	Follow (<ahoy-tail>) =	{ DROP, DROPLOOK }
674	First (id<assign-tail>!!)) =		{ id }
675	First (ASK (SCROLL-lit, <addr>) !!) =		{ ASK }
676	First (ECHO (SCROLL-lit <args-mult>) !!) =		{ ECHO }



677	First (LOOK ($\langle$ condition $\rangle$) [$\langle$ look-body $\rangle$ $\langle$ jump-stmnt $\rangle$] $\langle$ ahoy-look-tail $\rangle$) =		{ LOOK }
678	First (CHART ($\langle$ chart-cond $\rangle$) [$\langle$ courses $\rangle$ $\langle$ course-tail $\rangle$ $\langle$ adrift-case $\rangle$]) =		{ CHART }
679	First (HOIST ($\langle$ hoist-init $\rangle$!! $\langle$ hoist-cond $\rangle$!! $\langle$ hoist-upd $\rangle$) [$\langle$ look-body $\rangle$ $\langle$ jump-stmnt $\rangle$]) =		{ HOIST }
680	First (HEAVE ($\langle$ condition $\rangle$) [$\langle$ look-body $\rangle$ $\langle$ jump-stmnt $\rangle$]) =		{ HEAVE }
681	First (HAUL [$\langle$ look-body $\rangle$ $\langle$ jump-stmnt $\rangle$] HEAVE ($\langle$ condition $\rangle$!!) =		{ HAUL }
682	First ($\langle$ unary-exp $\rangle$!!) =	First ($\langle$ unary-exp $\rangle$) =	{ +#, -# }
683	First (DROPLOOK ($\langle$ condition $\rangle$) [$\langle$ look-body $\rangle$ $\langle$ jump-st mnt $\rangle$] $\langle$ ahoy-look-tail $\rangle$) =		{ DROPLOOK }
684	First (DROP [$\langle$ look-body $\rangle$ $\langle$ jump-st mnt $\rangle$]) =		{ DROP }
685	First (λ) =	Follow ($\langle$ ahoy-look-tail $\rangle$) =	{id, ASK, ECHO, LOOK, CHART, HOIST, HEAVE, HAUL, +#, -#, }

XI. TEST SCRIPTS

LEXICAL ANALYSIS TEST SCRIPT

Test Case/Test Scenario	Expected Output	Actual Output	Test Result
1234567890123456	No Lexical Error	No Lexical Error	✓
1000000000000000	No Lexical Error	No Lexical Error	✓
12345678901234567	With Lexical Error	With Lexical Error	✓
0	No Lexical Error	No Lexical Error	✓
0123	With Lexical Error	With Lexical Error	✓
123456h89	With Lexical Error	With Lexical Error	✓
-0	With Lexical Error	With Lexical Error	✓
-123456789	No Lexical Error	No Lexical Error	✓
123)	No Lexical Error	No Lexical Error	✓
(123	No Lexical Error	No Lexical Error	✓
1234567890123456.12345678	No Lexical Error	No Lexical Error	✓
1234567890123456.123456789	With Lexical Error	With Lexical Error	✓
0.123	No Lexical Error	No Lexical Error	✓
123.123	No Lexical Error	No Lexical Error	✓
123.10000000	No Lexical Error	No Lexical Error	✓
123.100000000	With Lexical Error	With Lexical Error	✓
-0.123	No Lexical Error	No Lexical Error	✓
-12,345	No Lexical Error	No Lexical Error	✓
1,23	No Lexical Error	No Lexical Error	✓
1.234.567	With Lexical Error	With Lexical Error	✓
'a'	No Lexical Error	No Lexical Error	✓
"	With Lexical Error	With Lexical Error	✓
' '	No Lexical Error	No Lexical Error	✓
'\d'	With Lexical Error	With Lexical Error	✓
'\s'	No Lexical Error	No Lexical Error	✓
""	With Lexical Error	With Lexical Error	✓
'asf'sasd'sad'	With Lexical Error	With Lexical Error	✓
'sasas's'ds''d'd's'	With Lexical Error	With Lexical Error	✓
'\t'	No Lexical Error	No Lexical Error	✓
'\n'	With Lexical Error	With Lexical Error	✓
"hello"	No Lexical Error	No Lexical Error	✓
"\d"	No Lexical Error	No Lexical Error	✓
"\s"	With Lexical Error	With Lexical Error	✓
""	With Lexical Error	With Lexical Error	✓
""	With Lexical Error	With Lexical Error	✓
"hello"&"hi"	No Lexical Error	No Lexical Error	✓
("Good" & "Morning") & "Boss"	No Lexical Error	No Lexical Error	✓
"hjrlejrkg""hgjhgj"); bvng	With Lexical Error	With Lexical Error	✓
""&"&"\d"&"\n\t"!!	With Lexical Error	With Lexical Error	✓
"Hello, everyone!\n Calculator\t Sum"	No Lexical Error	No Lexical Error	✓
NAY	No Lexical Error	No Lexical Error	✓
(AYE)	No Lexical Error	No Lexical Error	✓
NAY!!	No Lexical Error	No Lexical Error	✓
Aye	With Lexical Error	With Lexical Error	✓
TRUE	With Lexical Error	With Lexical Error	✓
AHOY[	With Lexical Error	With Lexical Error	✓
ABYSS x	No Lexical Error	No Lexical Error	✓
BACK;	With Lexical Error	With Lexical Error	✓
CHART[	With Lexical Error	With Lexical Error	✓

DIME()	With Lexical Error	With Lexical Error	✓
LOOK()	No Lexical Error	No Lexical Error	✓
MAST Mast	With Lexical Error	With Lexical Error	✓
HAULHEAVE	With Lexical Error	With Lexical Error	✓
SAIL!!	No Lexical Error	No Lexical Error	✓
DROP{	With Lexical Error	With Lexical Error	✓
coin	No Lexical Error	No Lexical Error	✓
coin_123	No Lexical Error	No Lexical Error	✓
coinSum	With Lexical Error	With Lexical Error	✓
thisisatwentycharid_	No Lexical Error	No Lexical Error	✓
array2d[j]{k}	No Lexical Error	No Lexical Error	✓
array{3}	No Lexical Error	No Lexical Error	✓
"Hello"(i)	No Lexical Error	No Lexical Error	✓
"hi"{0}	No Lexical Error	No Lexical Error	✓
notanarray[3]	With Lexical Error	With Lexical Error	✓
invalidarray{}	With Lexical Error	With Lexical Error	✓
3+6+8	No Lexical Error	No Lexical Error	✓
1++2	With Lexical Error	With Lexical Error	✓
+#x	No Lexical Error	No Lexical Error	✓
y+#	With Lexical Error	With Lexical Error	✓
4^=5	No Lexical Error	No Lexical Error	✓
6>>3	With Lexical Error	With Lexical Error	✓
3<6	No Lexical Error	No Lexical Error	✓
7>3<6	No Lexical Error	No Lexical Error	✓
4<=8	No Lexical Error	No Lexical Error	✓
9!==4	With Lexical Error	With Lexical Error	✓
3 > 5 && 2 < 6	No Lexical Error	No Lexical Error	✓
"Hello"&&"hi"	No Lexical Error	No Lexical Error	✓
x AYE	No Lexical Error	No Lexical Error	✓
a b&&c+d=f)	No Lexical Error	No Lexical Error	✓
(a < b && b <= c) (d > e && e != f)	No Lexical Error	No Lexical Error	✓
(x * 5) == (y + 10) && (z / 2) > 100	No Lexical Error	No Lexical Error	✓
ECHO("%S %S %S", first, middle, last)!!	No Lexical Error	No Lexical Error	✓
COIN result = multiply(5, 3)!!	No Lexical Error	No Lexical Error	✓
COIN arr{5} = [1, 2, 3, 4, 5, 6]!!num{2}!!	No Lexical Error	No Lexical Error	✓
DIME Coords{2}{3} = [[1.5, -4.2, 0.0], [99.9, -1.1, 3.1415]]!!	With Lexical Error	With Lexical Error	✓
DIME temperature_readings{3}{3} = [[-10.5, 0.0, 32.125], [98.6, -273.15, 100.0], [0.00001, -0.99, 55.5]]!!	No Lexical Error	No Lexical Error	✓
MAST shop_item [COIN item_code!! SCROLL description!! DIME unit_price!! COIN stock_count!! BOOL is_available!!]!!	No Lexical Error	No Lexical Error	✓
MAST cargo_manifest { SCROLL item_name!! DIME total_weight!! COIN quantity!! }!!	With Lexical Error	With Lexical Error	✓
MAST book ref_a = ["C_Lang", 1972], ref_b = ["SeaStack", 2024]!!	No Lexical Error	No Lexical Error	✓
LOOK(x > 5) { ECHO("x is greater")!! } DROP(x < 5) { ECHO("x is lesser")!! }	With Lexical Error	With Lexical Error	✓

SYNTAX ANALYSIS TEST SCRIPT

Test Case/Test Scenario	Expected Output	Actual Output	Test Result
AHOY() [DIME x = 10.5, y = 3.2, z = 2.0!! DIME complex = (x * y + z) / (x - y) + (y ^ z)!! ECHO("Complex: %D", complex)!!]	No Syntax Error	No Syntax Error	✓
AHOY() [COIN arr{3} = [10, 20, 30]!! COIN calc = (arr{0} + arr{1}) * arr{2} / (arr{0} - 5)!! ECHO("Calc: %C", calc)!!]	No Syntax Error	No Syntax Error	✓
AHOY() [DIME matrix{2}{2} = [[1.5, 2.5], [3.5, 4.5]]!! DIME sum = (matrix{0}{0} + matrix{1}{1}) * (matrix{0}{1} + matrix{1}{0})!! ECHO("Sum: %D", sum)!!]	No Syntax Error	No Syntax Error	✓
AHOY() [DIME values{3} = [2.5, 3.5, 4.5]!! DIME avg = (values{0} + values{1} + values{2}) / 3.0!! ECHO("Average: %D\n", avg)!!]	No Syntax Error	No Syntax Error	✓
AHOY() [COIN arr{3} = [10, 20, 30]!! COIN array_calc = arr{0} + arr{1} * arr{2} / 2!! ECHO("Array calc: %C\n", array_calc)!!]	No Syntax Error	No Syntax Error	✓
AHOY() [DIME a = 3.5, b = 2.1!! BOOL check = (a + b) >= (a * b)!! ECHO("Check: %B\n", check)!!]	No Syntax Error	No Syntax Error	✓
AHOY() [COIN x = 5, y = 10!! BOOL result = x < y!! ECHO("Result: %B\n", result)!!]	No Syntax Error	No Syntax Error	✓
AHOY() [SCROLL name1 = "Alice", name2 = "Bob"!! BOOL same = (name1 == name2)!! BOOL different = (name1 != name2)!! ECHO("Same: %B, Different: %B", same, different)!!]	No Syntax Error	No Syntax Error	✓
COIN get_value() [COIN val = 50!! BACK val!!] AHOY() [BOOL compare = (get_value() * 2) > (get_value() + 75)!! ECHO("Compare: %B", compare)!!]	No Syntax Error	No Syntax Error	✓
AHOY() [BOOL flag1 = AYE, flag2 = NAY!! BOOL comp1 = (flag1 == AYE)!! BOOL comp2 = (flag2 != AYE)!! BOOL result = comp1 && comp2!! ECHO("Result: %B", result)!!]	No Syntax Error	No Syntax Error	✓

AHOY() [COIN a = 10, b = 5!! COIN result = a + + b!! ECHO("Invalid\n")!!]	With Syntax Error	With Syntax Error	✓
AHOY() [DIME x = 5.5!! DIME wrong = * x + 5.0!! ECHO("Invalid\n")!!]	With Syntax Error	With Syntax Error	✓
AHOY() [DIME y = 5.5!! DIME wrong = (y +)!! ECHO("Invalid\n")!!]	With Syntax Error	With Syntax Error	✓
AHOY() [COIN matrix{2}{2} = [[10, 20], [30, 40]]!! BOOL diagonal = (matrix{0}{0} + matrix{1}{1}) == (matrix{0}{1} + matrix{1}{0})!! ECHO("Diagonal equal: %B", diagonal)!!]	No Syntax Error	No Syntax Error	✓
AHOY() [PARCH grade1 = 'A', grade2 = 'B'!! BOOL is_higher = (grade1 == grade2)!! BOOL not_equal = (grade1 != grade2)!! ECHO("Higher: %B, Not equal: %B", is_higher, not_equal)!!]	No Syntax Error	No Syntax Error	✓
AHOY() [PARCH ch = 'A'!! BOOL bad = ch > 'Z'!! ECHO("Invalid\n")!!]	With Syntax Error	With Syntax Error	✓
AHOY() [COIN x = 10, y = 20!! BOOL chained = 5 < x < y!! ECHO("Invalid\n")!!]	With Syntax Error	With Syntax Error	✓
AHOY() [DIME gpa = 3.5!! COIN age = 20!! BOOL eligible = ((gpa >= 3.0) && (age >= 18)) ((gpa >= 3.5) && (age >= 17))!! ECHO("Eligible: %B", eligible)!!]	No Syntax Error	No Syntax Error	✓
AHOY() [BOOL flags{3} = [AYE, NAY, AYE]!! BOOL combined = (flags{0} && flags{2}) (!flags{1} && flags{0})!! ECHO("Combined: %B", combined)!!]	No Syntax Error	No Syntax Error	✓
AHOY() [COIN a = 10, b = 20, c = 30!! BOOL nested = ((a < b) && (b < c)) ((a + b) > c && !(b == 20))!! ECHO("Nested: %B", nested)!!]	No Syntax Error	No Syntax Error	✓
AHOY() [BOOL a = AYE!! BOOL result = !#a && !NAY AYE && !#NAY!! ECHO("Result: %B", result)!!]	No Syntax Error	No Syntax Error	✓

AHOY() [COIN score = 85, attendance = 90!! BOOL pass = (score >= 60 && attendance >= 75) (score >= 90 attendance == 100)!! ECHO("Pass: %B", pass)!!]	No Syntax Error	No Syntax Error	✓
AHOY() [SCROLL name = "Admin"!! COIN level = 5!! BOOL access = (name == "Admin" && level >= 3) (name != "Guest" && level > 0)!! ECHO("Access: %B", access)!!]	No Syntax Error	No Syntax Error	✓
AHOY() [SCROLL name = "Test"!! BOOL wrong = name > "Admin"!! ECHO("Invalid\n")!!]	With Syntax Error	With Syntax Error	✓
AHOY() [COIN counter = 0!! HOIST (COIN i = 0!! i < 5!! +#i) [+#counter!! ECHO("Counter: %C", counter)!!]]	No Syntax Error	No Syntax Error	✓
AHOY() [COIN nums[5] = [10, 20, 30, 40, 50]!! COIN index = 0!! HEAVE (index < 5) [ECHO("Num: %C", nums[index])!! +#index!!]]	No Syntax Error	No Syntax Error	✓
AHOY() [COIN x = 100!! HEAVE (x > 0) [-#x!! LOOK (x % 10 == 0) [ECHO("Decade: %C", x)!!]]]	No Syntax Error	No Syntax Error	✓
AHOY() [COIN a = 5, b = 10!! +#a!! -#b!! COIN result = a + b!! ECHO("Result: %C", result)!!]	With Syntax Error	With Syntax Error	✓
AHOY() [COIN loop_var = 1!! HOIST (loop_var!! loop_var <= 5!! +#loop_var) [ECHO("Loop: %C\n", loop_var)!!]]	With Syntax Error	With Syntax Error	✓
AHOY() [DIME gwa, COIN score!! ASK("%D%C", @gwa, @score)!!]	With Syntax Error	With Syntax Error	✓

AHOY() [COIN score1, score2!! ASK("%C%C", @score1, @score2)!!]	No Syntax Error	No Syntax Error	✓
AHOY() [SCROLL first, middle, last!! ASK("%S%S%S", @first, @middle, @last)!!]	No Syntax Error	No Syntax Error	✓
AHOY() [DIME gwa!! COIN score!! ASK("%D%C", @gwa, @score)!!]	No Syntax Error	No Syntax Error	✓
AHOY() [COIN total = 50!! ECHO(total, "Total: %C")!!]	With Syntax Error	With Syntax Error	✓
AHOY() [ECHO("Hello: ",)!!]	No Syntax Error	No Syntax Error	✓
AHOY() [ECHO("Result: %C", ECHO("Inner"))!!]	With Syntax Error	With Syntax Error	✓
AHOY() [PARCH initial = 'B'!! PARCH grade = 'A'!! ECHO("Student Initial: %P", initial)!! ECHO("Final Grade: %P", 'A')!!]	With Syntax Error	With Syntax Error	✓
AHOY() [SCROLL greeting = "Hi"!! ECHO("Hello")]	With Syntax Error	With Syntax Error	✓
AHOY() [COIN score = 50!! ECHO("Score: %C", score)!!]	No Syntax Error	No Syntax Error	✓
AHOY() [COIN score = 99!! ECHO("Score: %C", score)!!]	No Syntax Error	No Syntax Error	✓
AHOY() [DIME price!! price = 99.99!!]	No Syntax Error	No Syntax Error	✓
AHOY() [DIME wallet, item_price!! COIN quantity!! BOOL afford!! wallet = 1000.00!! item_price = 299.50!! quantity = 3!! wallet -= item_price * quantity!! afford = wallet > 0.00!!]	No Syntax Error	No Syntax Error	✓

MAST student [SCROLL name!!]!!			
AHOY() [SCROLL name!! MAST student student1 = ["Luffy"]!! name = student1\$name!!]	No Syntax Error	No Syntax Error	✓
AHOY() [BOOL system_online = AYE!! LOOK (system_online == AYE) [ECHO("All systems GO!!")!!]]	No Syntax Error	No Syntax Error	✓
AHOY() [DROLOOK (AYE) [ECHO("Error: Missing LOOK")!!]]	With Syntax Error	With Syntax Error	✓
AHOY() [COIN current_followers = 990000!! LOOK (current_followers >= 1000000) [ECHO("1 Million reached!!")!!] DROLOOK (current_followers >= 900000) [ECHO("Almost at 1 Million!!")!!]]	No Syntax Error	No Syntax Error	✓
AHOY() [LOOK (AYE) [ECHO("Valid")!!] DROP (AYE) [ECHO("Error: DROP has condition")!!]]	With Syntax Error	With Syntax Error	✓
AHOY() [COIN player_hp = 0!! BOOL is_dead = NAY!! LOOK (player_hp <= 0) [is_dead = AYE!! ECHO("Game Over!")!!]]	No Syntax Error	No Syntax Error	✓
AHOY() [DROLOOK (xp > 10) [ECHO("Fail")!!]]	With Syntax Error	With Syntax Error	✓
AHOY() [BOOL sensor_active = AYE!! LOOK (sensor_active) [ECHO("System Monitoring Active...")!!] DROP [ECHO("Warning: Sensor Offline!")!!]]	No Syntax Error	No Syntax Error	✓

ALISWAG, K. V. J.
 ALTEZA, J. R. D.
 CAYACAP, F. A. S.

DEL ROSARIO, J. M. A.
 MAGAAN, F. M. V.
 MULLENO, J. M. A.



<pre>AHOY() [COIN choice = 1!! CHART (choice) [COURSE 1: ECHO("First Option")!! LAND!! ADRIFT: ECHO("Default")!! LAND!!]]</pre>	No Syntax Error	No Syntax Error	✓
<pre>AHOY() [COIN score = 85!! LOOK (score >= 90) [ECHO("Grade: A")!!] DROLOOK (score >= 80) [ECHO("Grade: B")!!] DROLOOK (score >= 70) [ECHO("Grade: C")!!] DROP [ECHO("Grade: F")!!]]</pre>	No Syntax Error	No Syntax Error	✓
<pre>AHOY() [COIN hp!! BOOL poisoned, cure!! hp = 15!! poisoned = AYE!! cure = NAY!! LOOK ((hp < 20 poisoned) && !cure) [ECHO("Danger: Critical Status!")!!]]</pre>	No Syntax Error	No Syntax Error	✓
<pre>AHOY() [COIN grade_score = 5!! CHART (grade_score) [COURSE 5: ECHO("Excellent!")!! LAND!! COURSE 4: ECHO("Good!")!! LAND!! ADRIFT: ECHO("Unknown Grade")!! LAND!!]]</pre>	No Syntax Error	No Syntax Error	✓

SEMANTIC ANALYSIS TEST SCRIPT

Test Case/Test Scenario	Expected Output	Actual Output	Test Result
AHOY() [COIN a = 5, b = 10, sum = 0!! sum = a + b!!]	No Semantic Error	No Semantic Error	✓
AHOY() [DIME x = 1.5, y = 2.0, prod = 0.0!! prod = x * y!!]	No Semantic Error	No Semantic Error	✓
AHOY() [COIN a = 10, b = 0!! SCROLL text = "hello"!! b = a + text!!]	With Semantic Error	With Semantic Error	✓
AHOY() [DIME a = 5.5, b = 0.0!! BOOL flag = AYE!! b = a - flag!!]	With Semantic Error	With Semantic Error	✓
AHOY() [COIN a, b!! b = a + 5!!]	With Semantic Error	With Semantic Error	✓
AHOY() [COIN a = 10, b = 5!! BOOL res = NAY!! res = a > b!!]	No Semantic Error	No Semantic Error	✓
AHOY() [SCROLL word1 = "Hi", word2 = "Hi"!! BOOL check = NAY!! check = word1 == word2!!]	No Semantic Error	No Semantic Error	✓
AHOY() [SCROLL a = "A", b = "B"!! BOOL res = NAY!! res = a > b!!]	With Semantic Error	With Semantic Error	✓
AHOY() [COIN num = 5!! SCROLL text = "5"!! BOOL res = NAY!! res = num == text!!]	With Semantic Error	With Semantic Error	✓
AHOY() [BOOL a = AYE, b = NAY, res = NAY!! res = a <= b!!]	With Semantic Error	With Semantic Error	✓
AHOY() [BOOL a = AYE, b = NAY, res = NAY!! res = a && b!!]	No Semantic Error	No Semantic Error	✓
AHOY() [COIN x = 10, y = 5!! BOOL res = NAY!! res = !(x > y)!!]	No Semantic Error	No Semantic Error	✓

AHOY() [COIN a = 1, b = 0!! BOOL res = NAY!! res = a && b!!]	With Semantic Error	With Semantic Error	✓
AHOY() [SCROLL text = "AYE"!! BOOL b = NAY, res = NAY!! res = text b!!]	With Semantic Error	With Semantic Error	✓
AHOY() [DIME val = 10.5!! BOOL res = NAY!! res = !val!!]	With Semantic Error	With Semantic Error	✓
AHOY() [COIN count = 0!! +#count!!]	No Semantic Error	No Semantic Error	✓
AHOY() [HOIST (COIN i = 1!! i <= 3!! +#i) [ECHO("Count: %C\n", i)!!]]	No Semantic Error	No Semantic Error	✓
AHOY() [DIME val = 5.5!! +#val!!]	With Semantic Error	With Semantic Error	✓
AHOY() [SCROLL text = "Hello"!! -#text!!]	With Semantic Error	With Semantic Error	✓
AHOY() [BOOL flag = AYE!! +#flag!!]	With Semantic Error	With Semantic Error	✓
AHOY() [SCROLL a = "Sea", b = "Stack", c = " "!! c = a & b!!]	No Semantic Error	No Semantic Error	✓
AHOY() [SCROLL greeting = " "!! greeting = "Hello" & " World"!!]	No Semantic Error	No Semantic Error	✓
AHOY() [SCROLL msg = " "!! COIN age = 18!! msg = "Age: " & age!!]	With Semantic Error	With Semantic Error	✓
AHOY() [SCROLL msg = " "!! PARCH letter = 'A'!! msg = "Grade: " & letter!!]	With Semantic Error	With Semantic Error	✓
AHOY() [SCROLL a, b = " "!! b = a & "!!"!!]	With Semantic Error	With Semantic Error	✓

AHOY() [SCROLL word = "SeaStack", letter = " "!! letter = word{0}!!]	No Semantic Error	No Semantic Error	✓
AHOY() [SCROLL text = "Hello", res = " "!! COIN idx = 1!! res = text{idx}!!]	No Semantic Error	No Semantic Error	✓
AHOY() [SCROLL msg = "Hi", res = " "!! res = msg{5}!!]	With Semantic Error	With Semantic Error	✓
AHOY() [SCROLL word = "Stack"!! PARCH char = 'a'!! char = word{2}!!]	With Semantic Error	With Semantic Error	✓
AHOY() [SCROLL text = "Test", idx = "1", res = " "!! res = text{idx}!!]	With Semantic Error	With Semantic Error	✓
MAST person [COIN age!!]!! AHOY() [COIN my_age = 0!! MAST person p1 = [2]!! my_age = p1\$age!!]	No Semantic Error	No Semantic Error	✓
MAST item [DIME price!!]!! AHOY() [MAST item i1 = [10.5]!! i1\$price += 5.0!!]	No Semantic Error	No Semantic Error	✓
AHOY() [COIN age = 20, res = 0!! res = age\$val!!]	With Semantic Error	With Semantic Error	✓
MAST student [COIN grade!!]!! AHOY() [MAST student s1 = [3]!! s1\$grade = "A"!!]	With Semantic Error	With Semantic Error	✓
MAST car [SCROLL brand!!]!! AHOY() [COIN year = 0!! MAST car c1 = ["Toyota"]!! year = c1\$year!!]	With Semantic Error	With Semantic Error	✓

<pre>AHOY() [COIN val = 0!! COIN arr{3} = [4, -6]!! val = arr{1}!!]</pre>	No Semantic Error	No Semantic Error	✓
<pre>AHOY() [DIME grid{2}{2} = [[1.1, 2.2], [3.3, 4.4]]!! COIN row = 0, col = 1!! grid{row}{col} = 5.5!!]</pre>	No Semantic Error	No Semantic Error	✓
<pre>AHOY() [DIME idx = 1.0!! COIN nums{2} = [2, 7]!! val = 0!! val = nums{idx}!!]</pre>	With Semantic Error	With Semantic Error	✓
<pre>AHOY() [COIN scores{2} = [3, 8]!! scores{0} = "Failed"!!]</pre>	With Semantic Error	With Semantic Error	✓
<pre>AHOY() [COIN number = 100, res = 0!! res = number{0}!!]</pre>	With Semantic Error	With Semantic Error	✓
<pre>AHOY() [DIME p, r, t, amount!! DIME e = 2.71828!! ASK("Enter Principal, Rate, and Time: %D %D %D", @p, @r, @t)!! amount = p * (e ^ r * t)!! ECHO("Continuous Compounding Total: %D\n", amount)!!]</pre>	No Semantic Error	No Semantic Error	✓
<pre>AHOY() [COIN base, modulus!! COIN exponent, result!! ASK("Enter base and modulus: %D %D", @base, @modulus)!! result = (base^exponent) % modulus!! ECHO("Modular Exponentiation Result: %D\n", result)!!]</pre>	With Semantic Error	With Semantic Error	✓
<pre>AHOY() [DIME v_initial, accel, time, displacement!! ASK("Enter initial velocity, acceleration, and time: %D %D %D", @v_initial, @accel, @time)!! displacement = (v_initial * time) + (0.5 * accel * (time ^ 2.0))!! ECHO("Total Displacement: %D\n", displacement)!!]</pre>	No Semantic Error	No Semantic Error	✓
<pre>AHOY() [DIME radius, volume, remainder!! ASK("Enter sphere radius: %C", @radius)!! volume = (4.0 / 3.0) * 3.14159 * (radius ^ 3.0)!! remainder = volume % 2.0!! ECHO("Volume remainder: %D\n", remainder)!!]</pre>	With Semantic Error	With Semantic Error	✓

<pre>AHOY() [DIME capacity = 10000.0, init_pop = 150.0, rate = 0.15, time, e, current_pop!! ASK("Enter time elapsed: %D", @time)!! e = 2.71828!! current_po = capacity / (1.0 + (((capacity - init_pop) / init_pop) * (e ^ (0.0 - (rate * time))))!! ECHO("Estimated Population: %D\n", current_pop)!!]</pre>	With Semantic Error	With Semantic Error	✓
<pre>AHOY() [DIME vec_a{3} = [2.5, 4.1, -1.0]!! DIME vec_b{3}!! DIME scalar_proj = 0.0, dot = 0.0, mag_a = 0.0!! ECHO("Enter Vector B components (x y z): ")!! ASK("%D%D%D", @vec_b{0}, @vec_b{1}, @vec_b{2})!! dot = (vec_a{0} * vec_b{0}) + (vec_a{1} * vec_b{1}) + (vec_a{2} * vec_b{2})!! mag_a = ((vec_a{0} ^ 2.0) + (vec_a{1} ^ 2.0) + (vec_a{2} ^ 2.0)) ^ 0.5!! scalar_proj = dot / mag_a!! ECHO("Scalar Projection: %D\n", scalar_proj)!!]</pre>	No Semantic Error	No Semantic Error	✓
<pre>AHOY() [COIN coeffs{4} = [1, -5, 6, 2]!! COIN x = 0, y = 0!! ECHO("Enter x to evaluate: ")!! ASK("%C", @x, @y)!! y = (coeffs{3} * (x ^ 3)) + coeffs{0}!! ECHO("Polynomial Result: %C\n", y)!!]</pre>	With Semantic Error	With Semantic Error	✓
<pre>AHOY() [DIME sensor_data{5} = [12.4, 15.1, 14.8, 16.2, 18.0]!! DIME new_val = 0.0, avg = 0.0!! ECHO("Enter new sensor reading: ")!! ASK("%D", @new_val)!! ~ SEMANTIC ERROR: Attempting to access sensor_data{5} avg = (sensor_data{3} + sensor_data{4} + sensor_data{5} + new_val) / 4.0!! ECHO("Latest SMA: %D\n", avg)!!]</pre>	With Semantic Error	With Semantic Error	✓
<pre>AHOY() [COIN memory_addresses{3} = [12]!! COIN offset = 0, target = 0!! ECHO("Enter memory offset: ")!! ASK("%C", @offset)!! target = memory_addresses{3} + offset!! ECHO("Target Address: %C\n", target)!!]</pre>	With Semantic Error	With Semantic Error	✓

<pre>AHOY() [DIME u{3}!! DIME v{3}!! DIME cx = 0.0, cy = 0.0, cz = 0.0, mag_sq = 0.0!! ECHO("Enter U vector (x y z): ")!! ASK("%D%D%D", @u{0}, @u{1}, @u{2})!! ECHO("Enter V vector (x y z): ")!! ASK("%D%D%D", @v{0}, @v{1}, @v{2})!! cx = (u{1} * v{2}) - (u{2} * v{1})!! cy = (u{2} * v{0}) - (u{0} * v{2})!! cz = (u{0} * v{1}) - (u{1} * v{0})!! mag_sq = (cx ^ 2.0) + (cy ^ 2.0) + (cz ^ 2.0)!! ECHO("Cross Product Mag Squared: %D\n", mag_sq)!!]</pre>	No Semantic Error	No Semantic Error	✓
<pre>MAST satellite [DIME mass!! DIME radius_orbit!! DIME semi_major_axis!!]!! AHOY() [DIME mu = 398600.0, velocity = 0.0!! MAST satellite sat_one!! ECHO("Enter radius and semi-major axis: ")!! ASK("%D%D", @sat_one\$radius_orbit, @sat_one\$semi_major_axis)!! velocity = (mu * ((2.0 / sat_one\$radius_orbit) - (1.0 / sat_one\$semi_major_axis))) ^ 0.5!! ECHO("Orbital Velocity: %D\n", velocity)!!]</pre>	No Semantic Error	No Semantic Error	✓
<pre>MAST ledger [COIN account_id!! DIME balance!!]!! AHOY() [DIME next_year = 0.0!! MAST ledger my_account!! ECHO("Enter ID and Deposit: ")!! ASK("%C%D", @my_account\$account_id, @my_account\$balance)!! next_year = my_account\$balance * (1.0 + my_account\$interest_rate)!! ECHO("Projected Balance: %D\n", next_year)!!]</pre>	With Semantic Error	With Semantic Error	✓

<pre> MAST sphere [DIME cx, cy, cz!! DIME radius!!]!! AHOY() [DIME ray_dx = 0.0, ray_dy = 0.0, ray_dz = -1.0!! DIME oc_x = 0.0, oc_y = 0.0, oc_z = 0.0!! DIME b = 0.0, c = 0.0, discriminant = 0.0!! MAST sphere target!! ECHO("Enter center X, Y, Z and Radius: ")!! ASK("%D%D%D%D", @target\$cx, @target\$cy, @target\$cz, @target\$radius)!! oc_x = 0.0 - target\$cx!! oc_y = 0.0 - target\$cy!! oc_z = 0.0 - target\$cz!! b = 2.0 * ((oc_x * ray_dx) + (oc_y * ray_dy) + (oc_z * ray_dz))!! c = ((oc_x ^ 2.0) + (oc_y ^ 2.0) + (oc_z ^ 2.0)) - (target\$diameter ^ 2.0)!! discriminant = (b ^ 2.0) - (4.0 * 1.0 * c)!! ECHO("Intersection Discriminant: %D\n", discriminant)!!] </pre>	With Semantic Error	With Semantic Error	✓
<pre> MAST configuration [COIN max_retries!! DIME timeout_ms!!]!! AHOY() [DIME total_wait = 0.0!! MAST configuration network_cfg!! ECHO("Enter timeout (ms): ")!! ASK("%D", @network_cfg\$timeout_ms)!! network_cfg\$max_retries = 3!! total_wait = network_cfg\$max_retries * network_cfg\$timeout_ms!! ECHO("Max wait time: %D\n", total_wait)!!] </pre>	No Semantic Error	No Semantic Error	✓
<pre> MAST student_record [SCROLL name!! COIN age!! DIME gwa!! BOOL is_enrolled!!]!! AHOY() [MAST student_record student_one = ["Alice", 20, 1.25, AYE]!! student_two\$gwa = 2.50!! ECHO("--- Student 1 ---\n")!! ECHO("Name: %S Age: %C GWA: %D Enrolled: %B\n", student_one\$name, student_one\$age, student_one\$gwa, student_one\$is_enrolled)!!] </pre>	With Semantic Error	With Semantic Error	✓

SOURCE CODE

lexer.py

```
from backend.lexical.token import Token
from backend.lexical.handlers.comments import Comments
from backend.lexical.handlers.digits import Digits
from backend.lexical.handlers.identifiers import Identifiers
from backend.lexical.handlers.reserved_words import ReservedWords
from backend.lexical.handlers.texts import Texts
from backend.lexical.handlers.symbols import Symbols
from backend.lexical.handlers.delimiters import Delimiters
from backend.lexical.lexer_errors import LexerErrors

class Lexer(Comments, Digits, Identifiers, ReservedWords, Texts, Symbols, LexerErrors):
    def __init__(self, text):
        self.text = text
        self.pos = 0
        self.line = 1
        self.col = 1
        self.current_char = self.text[self.pos] if self.pos < len(self.text) else None
        self.identifier_table = {}
        self.tokens = []
        self.errors = []

        self.token_start_pos = 0
        self.token_start_line = 1
        self.token_start_col = 1

        # =====
        # LEXER HELPERS
        # =====

    def mark_token_start(self):
        self.token_start_pos = self.pos
        self.token_start_line = self.line
        self.token_start_col = self.col

    def current_token_text(self):
        return self.text[self.token_start_pos:self.pos]

    # consumes char
    def advance(self):
        if self.current_char == "\n":
            self.line += 1
            self.col = 1
        else:
            self.col += 1

        self.pos += 1
        if self.pos < len(self.text):
            self.current_char = self.text[self.pos]
        else:
            self.current_char = None

    # tokenize chars
    def tokenize(self):
        while self.current_char is not None:
            self.mark_token_start()
            tok = self.state0()
            if tok:
                self.tokens.append(tok)
```

```
        return self.tokens,
self.errors

    # validate delimiter
    def
_is_valid_delimiter(self,
delim_set_name):
    char =
self.current_char

    if char is None: return
True

    delims =
Delimiters.delims()
    if delim_set_name in
delims: return char in
delims[delim_set_name]

    return False

#
=====
# ENTRY POINT -
STATE 0
#
=====

def state0(self):
    if self.current_char is
None:
        return None
    char =
self.current_char

    # Reserved Words
    if char == 'A': return
self.rw1()
    if char == 'B': return
self.rw23()

SeaStack
```

```
    if char == 'C': return
self.rw32()
    if char == 'D': return
self.rw47()
    if char == 'E': return
self.rw61()
    if char == 'H': return
self.rw66()
    if char == 'L': return
self.rw81()
    if char == 'M': return
self.rw94()
    if char == 'N': return
self.rw99()
    if char == 'P': return
self.rw103()
    if char == 'S': return
self.rw109()

    # Symbols
    if char == "+": return
self.rs120()
    if char == "-": return
self.rs126()
    if char == "*": return
self.rs133()
    if char == "/": return
self.rs137()
    if char == "%": return
self.rs141()
    if char == "^": return
self.rs145()

    if char == "=": return
self.rs149()

    if char == "!": return
self.rs153()
    if char == "<": return
self.rs161()
    if char == ">": return
self.rs165()
```

```
    if char == "&": return
self.rs169()
    if char == "|": return
self.rs173()

    if char == ":": return
self.rs176()
    if char == "@": return
self.rs178()
    if char == "$": return
self.rs180()
    if char == ",": return
self.rs182()

    if char == "{": return
self.rs184()
    if char == "}": return
self.rs186()
    if char == "(": return
self.rs188()
    if char == ")": return
self.rs190()
    if char == "[": return
self.rs192()
    if char == "]": return
self.rs194()

    # Identifiers
    if char.islower():
return self.i196()

    # COIN and DIME
literals
    if char == "0": return
self.c236()
    if char in
"123456789": return
self.c238()

    # PARCH and
SCROLL Literals
```

<pre> if char == "": return self.p287() if char == "": return self.s292() # Comments if char == "~": return self.cm297() </pre>	<pre> # Whitespaces if char.isspace(): l, c = self.line, self.col self.advance() return Token("whitespace", char, l, c) </pre>	<pre> # Unknown characters self.advance() return self.error() </pre>
--	--	--

token.py

<pre> class Token: def __init__(self, type, value, line, col, error_msg=None): self.type = type # identified token shown in the token column </pre>	<pre> self.value = value # actual sequence of characters shown in the lexeme column self.line = line # line no. of lexeme self.col = col # column no. of lexeme self.error_msg = error_msg # error messages </pre>	<pre> def __repr__(self): # token formatting return f"Token({self.type}, '{self.value}', L{self.line}:C{self.col})" # e.g. Token(id, 'total', L1:C1) </pre>
--	--	--

reserved_words.py

<pre> from backend.lexical.token import Token from backend.lexical.handlers.delimiters import Delimiters # ===== ===== ===== ===== ===== # RESERVED WORDS: must be uppercase letter only # ===== ===== </pre>	<pre> ===== ===== ===== class ReservedWords: # ===== ===== # check char if valid as rw </pre>	<pre> def check_rw(self, char, next_state): self.advance() if self.current_char == char: return next_state() return self.error() # check delimiter if valid def check_rwdelim(self, valid_delims, next_state): self.advance() char = self.current_char if char in valid_delims or char is None: return next_state() return self.error() </pre>
--	--	---

```
# finalize reserved word
token
def finalize_rw(self,
token_type):
    return
Token(token_type,
self.current_token_text(),
self.token_start_line,
self.token_start_col)

#
=====
===
# RESERVED WORDS
"A"
#
=====
===

def rw1(self):
    self.advance()
    char = self.current_char

    if char == 'B': return
self.rw2()
    if char == 'D': return
self.rw7()
    if char == 'H': return
self.rw13()
    if char == 'S': return
self.rw17()
    if char == 'Y': return
self.rw20()

    return self.error()

# ABYSS
def rw2(self): return
self.check_rw('Y', self.rw3)
    def rw3(self): return
self.check_rw('S', self.rw4)
```

SeaStack

```
def rw4(self): return
self.check_rw('S', self.rw5)
    def rw5(self): return
self.check_rwdelim(Delimiters.delims()["WHITESPACE"], self.rw6)
    def rw6(self): return
self.finalize_rw("ABYSS")

# ADRIFT
def rw7(self): return
self.check_rw('R', self.rw8)
    def rw8(self): return
self.check_rw('I', self.rw9)
    def rw9(self): return
self.check_rw('F', self.rw10)
    def rw10(self): return
self.check_rw('T', self.rw11)
    def rw11(self):
        valid =
Delimiters.delims()["WHITESPACE"] | set(':')
        return
self.check_rwdelim(valid,
self.rw12)
    def rw12(self): return
self.finalize_rw("ADRIFT")

# AHOY
def rw13(self): return
self.check_rw('O', self.rw14)
    def rw14(self): return
self.check_rw('Y', self.rw15)
    def rw15(self): return
self.check_rw('(', self.rw16)
    def rw16(self): return
self.finalize_rw("AHOY")

# ASK
def rw17(self): return
self.check_rw('K', self.rw18)
```

```
def rw18(self): return
self.check_rw('(', self.rw19)
    def rw19(self): return
self.finalize_rw("ASK")

# AYE
def rw20(self): return
self.check_rw('E', self.rw21)
    def rw21(self): return
self.check_rwdelim(Delimiters.delims()["BOOL_DELIM"], self.rw22)
    def rw22(self): return
self.finalize_rw("AYE")

#
=====
===
# RESERVED WORDS
"B"
#
=====
===

def rw23(self):
    self.advance()
    char = self.current_char

    if char == 'A': return
self.rw24()
    if char == 'O': return
self.rw28()

    return self.error()

# BACK
def rw24(self): return
self.check_rw('C', self.rw25)
    def rw25(self): return
self.check_rw('K', self.rw26)
    def rw26(self):
```

```
        valid =
Delimiters.delims()["WHIT
SPACE"] | set("!")
        return
self.check_rwdelim(valid,
self.rw27)
        def rw27(self): return
self.finalize_rw("BACK")

        # BOOL
        def rw28(self): return
self.check_rw('O', self.rw29)
        def rw29(self): return
self.check_rw('L', self.rw30)
        def rw30(self): return
self.check_rwdelim(Delimit
rs.delims()["WHITESPACE"
], self.rw31)
        def rw31(self): return
self.finalize_rw("BOOL")

        #
=====
====
        # RESERVED WORDS
"C"
        #
=====
=====

        def rw32(self):
            self.advance()
            char = self.current_char

            if char == 'H': return
self.rw33()
            if char == 'O': return
self.rw38()

            return self.error()
```

```
        # CHART
        def rw33(self): return
self.check_rw('A', self.rw34)
        def rw34(self): return
self.check_rw('R', self.rw35)
        def rw35(self): return
self.check_rw('T', self.rw36)
        def rw36(self):
            valid =
Delimiters.delims()["WHIT
SPACE"] | set('(')
            return
self.check_rwdelim(valid,
self.rw37)
        def rw37(self): return
self.finalize_rw("CHART")

        # COIN/COURSE
        def rw38(self):
            self.advance()
            char = self.current_char

            if char == 'I': return
self.rw39()
            if char == 'U': return
self.rw42()

            return self.error()

        # COIN
        def rw39(self): return
self.check_rw('N', self.rw40)
        def rw40(self): return
self.check_rwdelim(Delimit
rs.delims()["WHITESPACE"
], self.rw41)
        def rw41(self): return
self.finalize_rw("COIN")

        # COURSE
        def rw42(self): return
self.check_rw('R', self.rw43)
```

```
        def rw43(self): return
self.check_rw('S', self.rw44)
        def rw44(self): return
self.check_rw('E', self.rw45)
        def rw45(self): return
self.check_rwdelim(Delimit
rs.delims()["WHITESPACE"
], self.rw46)
        def rw46(self): return
self.finalize_rw("COURSE")

        #
=====
====
        # RESERVED WORDS
"D"
        #
=====
=====

        def rw47(self):
            self.advance()
            char = self.current_char

            if char == 'I': return
self.rw48()
            if char == 'R': return
self.rw52()

            return self.error()

        # DIME
        def rw48(self): return
self.check_rw('M',
self.rw49)
        def rw49(self): return
self.check_rw('E', self.rw50)
        def rw50(self): return
self.check_rwdelim(Delimit
rs.delims()["WHITESPACE"
], self.rw51)
```

```

def rw51(self): return
self.finalize_rw("DIME")

# DROP
def rw52(self): return
self.check_rw('O', self.rw53)
def rw53(self): return
self.check_rw('P', self.rw54)
def rw54(self):
    self.advance()
    char = self.current_char

    if char == 'L': return
self.rw56()

    valid =
Delimiters.delims()["WHIT
SPACE"] | set(['I'])
    if char in valid or char
is None: return self.rw55()

def rw55(self): return
self.finalize_rw("DROP")

# DROLOOK
def rw56(self): return
self.check_rw('O', self.rw57)
def rw57(self): return
self.check_rw('O', self.rw58)
def rw58(self): return
self.check_rw('K', self.rw59)
def rw59(self):
    valid =
Delimiters.delims()["WHIT
SPACE"] | set(['I'])
    return
self.check_rwdelim(valid,
self.rw60)
    def rw60(self): return
self.finalize_rw("DROPLOO
K")

def rw61(self): return
self.check_rw('C', self.rw62)
def rw62(self): return
self.check_rw('H', self.rw63)
def rw63(self): return
self.check_rw('O', self.rw64)
def rw64(self): return
self.check_rw('(', self.rw65)
def rw65(self): return
self.finalize_rw("ECHO")

#
=====
# RESERVED WORDS
"H"
#
=====

def rw66(self):
    self.advance()
    char = self.current_char

    if char == 'A': return
self.rw67()
    if char == 'E': return
self.rw71()
    if char == 'O': return
self.rw76()

return self.error()

# HAUL
def rw67(self): return
self.check_rw('U', self.rw68)
def rw68(self): return
self.check_rw('L', self.rw69)
def rw69(self):
    valid =
Delimiters.delims()["WHIT
SPACE"] | set(['I'])
    return
self.check_rwdelim(valid,
self.rw70)
    def rw70(self): return
self.finalize_rw("HAUL")

# HEAVE
def rw71(self): return
self.check_rw('A', self.rw72)
def rw72(self): return
self.check_rw('V', self.rw73)
def rw73(self): return
self.check_rw('E', self.rw74)
def rw74(self):
    valid =
Delimiters.delims()["WHIT
SPACE"] | set(['I'])
    return
self.check_rwdelim(valid,
self.rw75)
    def rw75(self): return
self.finalize_rw("HEAVE")

# HOIST
def rw76(self): return
self.check_rw('I', self.rw77)
def rw77(self): return
self.check_rw('S', self.rw78)
def rw78(self): return
self.check_rw('T', self.rw79)
def rw79(self):

```

```

    valid =
Delimiters.delims()["WHIT
SPACE"] | set('(')
    return
self.check_rwdelim(valid,
self.rw80)
    def rw80(self): return
self.finalize_rw("HOIST")

#
=====
===
# RESERVED WORDS
"L"
#
=====
===

def rw81(self):
    self.advance()
    char = self.current_char

    if char == 'A': return
self.rw82()
    if char == 'O': return
self.rw86()

    return self.error()

# LAND
def rw82(self): return
self.check_rw('N', self.rw83)
    def rw83(self): return
self.check_rw('D', self.rw84)
    def rw84(self):
        valid =
Delimiters.delims()["WHIT
SPACE"] | set('(')
        return
self.check_rwdelim(valid,
self.rw85)

```

```

def rw85(self): return
self.finalize_rw("LAND")

# LOCKE/LOOK
def rw86(self):
    self.advance()
    char = self.current_char

    if char == 'C': return
self.rw87()
    if char == 'O': return
self.rw91()

    return self.error()

# LOCKE
def rw87(self): return
self.check_rw('K', self.rw88)
    def rw88(self): return
self.check_rw('E', self.rw89)
    def rw89(self): return
self.check_rwdelim(Delimiters.delims()["WHITESPACE"
], self.rw90)
    def rw90(self): return
self.finalize_rw("LOCKE")

# LOOK
def rw91(self): return
self.check_rw('K', self.rw92)
    def rw92(self):
        valid =
Delimiters.delims()["WHIT
SPACE"] | set('(')
        return
self.check_rwdelim(valid,
self.rw93)
    def rw93(self): return
self.finalize_rw("LOOK")

```

```

#
=====
===
# RESERVED WORD
"MAST"
#
=====
===
def rw94(self): return
self.check_rw('A', self.rw95)
    def rw95(self): return
self.check_rw('S', self.rw96)
    def rw96(self): return
self.check_rw('T', self.rw97)
    def rw97(self): return
self.check_rwdelim(Delimiters.delims()["WHITESPACE"
], self.rw98)
    def rw98(self): return
self.finalize_rw("MAST")

#
=====
===
# RESERVED WORD
"NAY"
#
=====
===
def rw99(self): return
self.check_rw('A',
self.rw100)
    def rw100(self): return
self.check_rw('Y',
self.rw101)
    def rw101(self): return
self.check_rwdelim(Delimiters.delims()["BOOL_DELIM
"], self.rw102)
    def rw102(self): return
self.finalize_rw("NAY")

```

```
#
=====
===
# RESERVED WORD
"PARCH"
#
=====
===
def rw103(self): return
self.check_rw('A',
self.rw104)
def rw104(self): return
self.check_rw('R',
self.rw105)
def rw105(self): return
self.check_rw('C',
self.rw106)
def rw106(self): return
self.check_rw('H',
self.rw107)
def rw107(self): return
self.check_rwdelim(Delimiters.delims()["WHITESPACE"], self.rw108)
def rw108(self): return
self.finalize_rw("PARCH")
```

```
#
=====
===
# RESERVED WORDS
"S"
#
=====
===
def rw109(self):
self.advance()
char = self.current_char

if char == 'A': return
self.rw110()
if char == 'C': return
self.rw114()

return self.error()

# SAIL
def rw110(self): return
self.check_rw('I', self.rw111)
def rw111(self): return
self.check_rw('L',
self.rw112)
def rw112(self):
valid =
Delimiters.delims()["WHITESPACE"] | set('!')
```

```
return
self.check_rwdelim(valid,
self.rw113)
def rw113(self): return
self.finalize_rw("SAIL")

# SCROLL
def rw114(self): return
self.check_rw('R',
self.rw115)
def rw115(self): return
self.check_rw('O',
self.rw116)
def rw116(self): return
self.check_rw('L',
self.rw117)
def rw117(self): return
self.check_rw('L',
self.rw118)
def rw118(self): return
self.check_rwdelim(Delimiters.delims()["WHITESPACE"], self.rw119)
def rw119(self): return
self.finalize_rw("SCROLL")
```

symbols.py

```
from backend.lexical.token
import Token
from
backend.lexical.handlers.delimiters import Delimiters
from
backend.lexical.lexer_errors
import LexerErrors
```

```
class Symbols(LexerErrors):

# check if delimiter is
valid
def check_delim(self,
delim_key):
return
self.is_valid_delimiter(delim_key)
```

```
def rs120(self):
self.advance()
if
self.check_delim("GEN_OP_DELIM"): return
self.rs121()
if self.current_char ==
"#": return self.rs122()
```

<pre> if self.current_char == "=": return self.rs124() return self.error() def rs121(self): return Token("+", self.current_token_text(), self.token_start_line, self.token_start_col) # INCREMENT '+' def rs122(self): self.advance() if self.check_delim("LOWLET "): return self.rs123() return self.error() def rs123(self): return Token("+#", self.current_token_text(), self.token_start_line, self.token_start_col) # ADD ASSIGN '+=' def rs124(self): self.advance() if self.check_delim("GEN_OP _DELIM"): return self.rs125() return self.error() def rs125(self): return Token("+=", self.current_token_text(), self.token_start_line, self.token_start_col)</pre>	<pre> # SUBTRACT '-' def rs126(self): self.advance() if self.check_delim("MINUS_ _DELIM"): return self.rs127() if self.current_char == "#": return self.rs128() if self.current_char == "=": return self.rs130() if self.current_char == "0": return self.rs132() if self.current_char is not None and self.current_char in "123456789": return self.c238() return self.error() def rs127(self): return Token("-", self.current_token_text(), self.token_start_line, self.token_start_col) # DECREMENT '-#' def rs128(self): self.advance() if self.check_delim("LOWLET "): return self.rs129() return self.error() def rs129(self): return Token("-#", self.current_token_text(), self.token_start_line, self.token_start_col)</pre>	<pre> # SUBTRACT ASSIGN '._=' def rs130(self): self.advance() if self.check_delim("GEN_OP _DELIM"): return self.rs131() return self.error() def rs131(self): return Token("-=", self.current_token_text(), self.token_start_line, self.token_start_col) # NEGATIVE DECIMAL '-0.' def rs132(self): self.advance() if self.current_char == ".": return self.d270() return self.error() # MULTIPLY '*' def rs133(self): self.advance() if self.check_delim("GEN_OP _DELIM"): return self.rs134() if self.current_char == "=": return self.rs135() return self.error() def rs134(self): return Token("*", self.current_token_text(),</pre>
--	---	---

```
self.token_start_line,
self.token_start_col)

def rs135(self):
    self.advance()
    if
self.check_delim("GEN_OP
_DELIM"): return
self.rs136()

    return self.error()

def rs136(self): return
Token("*= ",
self.current_token_text(),
self.token_start_line,
self.token_start_col)

    # DIVIDE '/'
    def rs137(self):
        self.advance()
        if
self.check_delim("GEN_OP
_DELIM"): return
self.rs138()
        if self.current_char ==
"=": return self.rs139()

    return self.error()

def rs138(self): return
Token("/",
self.current_token_text(),
self.token_start_line,
self.token_start_col)

    def rs139(self):
        self.advance()
        if
self.check_delim("GEN_OP
_DELIM"): return
self.rs140()

self.token_start_line,
self.token_start_col)

        return self.error()

def rs140(self): return
Token("/=",
self.current_token_text(),
self.token_start_line,
self.token_start_col)

        # MODULO '%'
        def rs141(self):
            self.advance()
            if
self.check_delim("GEN_OP
_DELIM"): return
self.rs142()
            if self.current_char ==
"=": return self.rs143()

        return self.error()

def rs142(self): return
Token("% ",
self.current_token_text(),
self.token_start_line,
self.token_start_col)

        # MODULO ASSIGN
        '%='
        def rs143(self):
            self.advance()
            if
self.check_delim("GEN_OP
_DELIM"): return
self.rs144()

        return self.error()

def rs144(self): return
Token("%=",
self.current_token_text(),

self.token_start_line,
self.token_start_col)

        # EXPONENT '^'
        def rs145(self):
            self.advance()
            if
self.check_delim("GEN_OP
_DELIM"): return
self.rs146()
            if self.current_char ==
"=": return self.rs147()

        return self.error()

def rs146(self): return
Token("^ ",
self.current_token_text(),
self.token_start_line,
self.token_start_col)

        # EXPONENT ASSIGN
        '^='
        def rs147(self):
            self.advance()
            if
self.check_delim("GEN_OP
_DELIM"): return
self.rs148()

        return self.error()

def rs148(self): return
Token("^=",
self.current_token_text(),
self.token_start_line,
self.token_start_col)

        # ASSIGN '='
        def rs149(self):
            self.advance()
```

```
if
self.check_delim("ASSIGN_
DELIM"): return self.rs150()
    if self.current_char ==
"=": return self.rs151()

    return self.error()

def rs150(self): return
Token("=",
self.current_token_text(),
self.token_start_line,
self.token_start_col)

# EQUAL '='
def rs151(self):
    self.advance()
    if
self.check_delim("LOG_OP
_DELIM"): return
self.rs152()

    return self.error()

def rs152(self): return
Token("==",
self.current_token_text(),
self.token_start_line,
self.token_start_col)

def rs153(self):
    self.advance()
    if
self.check_delim("NOT_DE
LIM"): return self.rs154()
        if self.current_char ==
"!": return self.rs155()
        if self.current_char ==
"#": return self.rs157()
        if self.current_char ==
"=": return self.rs159()
```

```
return self.error()

def rs154(self): return
Token("!",
self.current_token_text(),
self.token_start_line,
self.token_start_col)

# TERMINATOR '!'
def rs155(self):
    self.advance()
    if
self.check_delim("TERMIN
ATOR_DELIM"): return
self.rs156()

    return self.error()

def rs156(self): return
Token("!!",
self.current_token_text(),
self.token_start_line,
self.token_start_col)

# DOUBLE NOT '!'
def rs157(self):
    self.advance()
    if
self.check_delim("NOT_DE
LIM"): return self.rs158()

    return self.error()

def rs158(self): return
Token("!#",
self.current_token_text(),
self.token_start_line,
self.token_start_col)

# NOT EQUAL '!='
def rs159(self):
    self.advance()
```

```
if
self.check_delim("LOG_OP
_DELIM"): return
self.rs160()

    return self.error()

def rs160(self): return
Token("!=",
self.current_token_text(),
self.token_start_line,
self.token_start_col)

# LESS THAN '<'
def rs161(self):
    self.advance()
    if
self.check_delim("GEN_OP
_DELIM"): return
self.rs162()
        if self.current_char ==
"=": return self.rs163()

    return self.error()

def rs162(self): return
Token("<",
self.current_token_text(),
self.token_start_line,
self.token_start_col)

def rs163(self):
    self.advance()
    if
self.check_delim("GEN_OP
_DELIM"): return
self.rs164()

    return self.error()

def rs164(self): return
Token("<=",
```

```
self.current_token_text(),
self.token_start_line,
self.token_start_col)

def rs165(self):
    self.advance()
    if
self.check_delim("GEN_OP
_DELIM"): return
self.rs166()
    if self.current_char ==
"=": return self.rs167()

    return self.error()

def rs166(self): return
Token(">",
self.current_token_text(),
self.token_start_line,
self.token_start_col)

def rs167(self):
    self.advance()
    if
self.check_delim("GEN_OP
_DELIM"): return
self.rs168()

    return self.error()

def rs168(self): return
Token(">=",
self.current_token_text(),
self.token_start_line,
self.token_start_col)

# CONCATENATE '&'
def rs169(self):
    self.advance()
    if
self.check_delim("CONCAT
```

```
_DELIM"): return
self.rs170()
    if self.current_char ==
"&": return self.rs171()

    return self.error()

def rs170(self): return
Token("&",
self.current_token_text(),
self.token_start_line,
self.token_start_col)

def rs171(self):
    self.advance()
    if
self.check_delim("LOG_OP
_DELIM"): return
self.rs172()

    return self.error()

def rs172(self): return
Token("&&",
self.current_token_text(),
self.token_start_line,
self.token_start_col)

def rs173(self):
    self.advance()
    if self.current_char ==
"|": return self.rs174()
    return self.error()

def rs174(self):
    self.advance()
    if
self.check_delim("LOG_OP
_DELIM"): return
self.rs175()

    return self.error()
```

```
def rs175(self): return
Token("||",
self.current_token_text(),
self.token_start_line,
self.token_start_col)

def rs176(self):
    self.advance()
    if
self.check_delim("COLON_
DELIM"): return self.rs177()

    return self.error()

def rs177(self): return
Token(":",
self.current_token_text(),
self.token_start_line,
self.token_start_col)

def rs178(self):
    self.advance()
    if
self.check_delim("LOWLET
"): return self.rs179()

    return self.error()

def rs179(self): return
Token("@",
self.current_token_text(),
self.token_start_line,
self.token_start_col)

def rs180(self):
    self.advance()
    if
self.check_delim("LOWLET
"): return self.rs181()

    return self.error()
```

```
def rs181(self): return
Token("$",
self.current_token_text(),
self.token_start_line,
self.token_start_col)

def rs182(self):
self.advance()
if
self.check_delim("COMMA
_DELIM"): return
self.rs183()

return self.error()

def rs183(self): return
Token(",",
self.current_token_text(),
self.token_start_line,
self.token_start_col)

def rs184(self):
self.advance()
if
self.check_delim("ALPHAN
UMERIC"): return
self.rs185()

return self.error()

def rs185(self): return
Token("{",
self.current_token_text(),
self.token_start_line,
self.token_start_col)

def rs186(self):
self.advance()
if
self.check_delim("CLOSEC
B_DELIM"): return
self.rs187()

return self.error()

def rs187(self): return
Token("}",
self.current_token_text(),
self.token_start_line,
self.token_start_col)

def rs188(self):
self.advance()
if
self.check_delim("OPENP
_DELIM"): return self.rs189()

return self.error()

def rs189(self): return
Token("(",
self.current_token_text(),
self.token_start_line,
self.token_start_col)

def rs190(self):
self.advance()
if
self.check_delim("CLOSEP
_DELIM"): return
self.rs191()

return self.error()

def rs191(self): return
Token(")",
self.current_token_text(),
self.token_start_line,
self.token_start_col)

def rs192(self):
self.advance()
if
self.check_delim("OPENS
B_DELIM"): return
self.rs193()

return self.error()

def rs193(self): return
Token("[",
self.current_token_text(),
self.token_start_line,
self.token_start_col)

def rs194(self):
self.advance()
if
self.check_delim("CLOSES
B_DELIM"): return
self.rs195()

return self.error()

def rs195(self): return
Token("]",
self.current_token_text(),
self.token_start_line,
self.token_start_col)
```

identifiers.py

```
from backend.lexical.token
import Token
from
backend.lexical.handlers.delimiters import Delimiters

class Identifiers:

    # check char if valid as
    identifier
    def check_id(self):
        char = self.current_char
        return char is not None
        and (char.islower() or
        char.isdigit() or char == "_")

    # finalize identifier token
    def finalize_id(self):
        result =
self.current_token_text()
        line, col = self.line,
self.col

        if
self._is_valid_delimiter("ID
_DELIM"):

            if result not in
self.identifier_table:

self.identifier_table[result] =
f'id{len(self.identifier_table)
+ 1}'

            token_type =
self.identifier_table[result]
            return
Token(token_type, result,
self.token_start_line,
self.token_start_col)
            return self.error()

# char 1
def i196(self):
    self.advance()
    if self.check_id():
return self.i198()
    return self.i197()

    def i197(self): return
self.finalize_id()

# char 2
def i198(self):
    self.advance()
    if self.check_id():
return self.i200()
    return self.i199()

    def i199(self): return
self.finalize_id()

# char 3
def i200(self):
    self.advance()
    if self.check_id():
return self.i202()
    return self.i201()

    def i201(self): return
self.finalize_id()

# char 4
def i202(self):
    self.advance()
    if self.check_id():
return self.i204()
    return self.i203()

    def i203(self): return
self.finalize_id()

# char 5
def i204(self):

self.advance()
    if self.check_id():
return self.i206()
    return self.i205()

    def i205(self): return
self.finalize_id()

# char 6
def i206(self):
    self.advance()
    if self.check_id():
return self.i208()
    return self.i207()

    def i207(self): return
self.finalize_id()

# char 7
def i208(self):
    self.advance()
    if self.check_id():
return self.i210()
    return self.i209()

    def i209(self): return
self.finalize_id()

# char 8
def i210(self):
    self.advance()
    if self.check_id():
return self.i212()
    return self.i211()

    def i211(self): return
self.finalize_id()

# char 9
def i212(self):
    self.advance()
```

```
        if self.check_id():
return self.i214()
        return self.i213()

    def i213(self): return
self.finalize_id()

    # char 10
    def i214(self):
        self.advance()
        if self.check_id():
return self.i216()
        return self.i215()

    def i215(self): return
self.finalize_id()

    # char 11
    def i216(self):
        self.advance()
        if self.check_id():
return self.i218()
        return self.i217()

    def i217(self): return
self.finalize_id()

    # char 12
    def i218(self):
        self.advance()
        if self.check_id():
return self.i220()
        return self.i219()

    def i219(self): return
self.finalize_id()

    # char 13
    def i220(self):
```

```
        self.advance()
        if self.check_id():
return self.i222()
        return self.i221()

    def i221(self): return
self.finalize_id()

    # char 14
    def i222(self):
        self.advance()
        if self.check_id():
return self.i224()
        return self.i223()

    def i223(self): return
self.finalize_id()

    # char 15
    def i224(self):
        self.advance()
        if self.check_id():
return self.i226()
        return self.i225()

    def i225(self): return
self.finalize_id()

    # char 16
    def i226(self):
        self.advance()
        if self.check_id():
return self.i228()
        return self.i227()

    def i227(self): return
self.finalize_id()

    # char 17
```

digits.py

```
    def i228(self):
        self.advance()
        if self.check_id():
return self.i230()
        return self.i229()

    def i229(self): return
self.finalize_id()

    # char 18
    def i230(self):
        self.advance()
        if self.check_id():
return self.i232()
        return self.i231()

    def i231(self): return
self.finalize_id()

    # char 19
    def i232(self):
        self.advance()
        if self.check_id():
return self.i234()
        return self.i233()

    def i233(self): return
self.finalize_id()

    # char 20
    def i234(self):
        self.advance()
        if self.check_id():
return self.error()
        return self.i235()

    def i235(self): return
self.finalize_id()
```

```
from backend.lexical.token
import Token
from
backend.lexical.handlers.delimiters import Delimiters
from
backend.lexical.lexer_errors
import LexerErrors

class Digits(LexerErrors):

    # check char if valid as
    digit
    def check_digit(self):
        return self.current_char
    is not None and
    self.current_char.isdigit()

    # finalize COIN-lit or
    DIME-lit token
    def finalize_coin(self):
        if
    self._is_valid_delimiter("DIGIT_DELIM"):
        return
    Token("COIN-lit",
    self.current_token_text(),
    self.token_start_line,
    self.token_start_col)
        return self.error()

    def finalize_dime(self):
        if
    self._is_valid_delimiter("DIGIT_DELIM"):
        return
    Token("DIME-lit",
    self.current_token_text(),
    self.token_start_line,
    self.token_start_col)
        return self.error()

    # lone 0
    def c236(self):
        self.advance()
        if self.current_char ==
    ".": return self.d270()
        return self.c237()

    def c237(self): return
    self.finalize_coin()

    # digit 1
    def c238(self):
        self.advance()
        if self.check_digit():
    return self.c240()
        if self.current_char ==
    ".": return self.d270()
        return self.c239()

    def c239(self): return
    self.finalize_coin()

    # digit 2
    def c240(self):
        self.advance()
        if self.check_digit():
    return self.c242()
        if self.current_char ==
    ".": return self.d270()
        return self.c241()

    def c241(self): return
    self.finalize_coin()

    # digit 3
    def c242(self):
        self.advance()
        if self.check_digit():
    return self.c244()
        if self.current_char ==
    ".": return self.d270()
        return self.c243()

    def c243(self): return
    self.finalize_coin()

    # digit 4
    def c244(self):
        self.advance()
        if self.check_digit():
    return self.c246()
        if self.current_char ==
    ".": return self.d270()
        return self.c245()

    def c245(self): return
    self.finalize_coin()

    # digit 5
    def c246(self):
        self.advance()
        if self.check_digit():
    return self.c248()
        if self.current_char ==
    ".": return self.d270()
        return self.c247()

    def c247(self): return
    self.finalize_coin()

    # digit 6
    def c248(self):
        self.advance()
        if self.check_digit():
    return self.c250()
        if self.current_char ==
    ".": return self.d270()
        return self.c249()

    def c249(self): return
    self.finalize_coin()

    # digit 7
    def c250(self):
```



```
self.advance()
if self.check_digit():
return self.c252()
    if self.current_char ==
".": return self.d270()
    return self.c251()

def c251(self): return
self.finalize_coin()

# digit 8
def c252(self):
    self.advance()
    if self.check_digit():
return self.c254()
    if self.current_char ==
".": return self.d270()
    return self.c253()

def c253(self): return
self.finalize_coin()

# digit 9
def c254(self):
    self.advance()
    if self.check_digit():
return self.c256()
    if self.current_char ==
".": return self.d270()
    return self.c255()

def c255(self): return
self.finalize_coin()

# digit 10
def c256(self):
    self.advance()
    if self.check_digit():
return self.c258()
    if self.current_char ==
".": return self.d270()
    return self.c257()
```

```
def c257(self): return
self.finalize_coin()

# digit 11
def c258(self):
    self.advance()
    if self.check_digit():
return self.c260()
    if self.current_char ==
".": return self.d270()
    return self.c259()

def c259(self): return
self.finalize_coin()

# digit 12
def c260(self):
    self.advance()
    if self.check_digit():
return self.c262()
    if self.current_char ==
".": return self.d270()
    return self.c261()

def c261(self): return
self.finalize_coin()

# digit 13
def c262(self):
    self.advance()
    if self.check_digit():
return self.c264()
    if self.current_char ==
".": return self.d270()
    return self.c263()

def c263(self): return
self.finalize_coin()

# digit 14
def c264(self):
```

```
self.advance()
if self.check_digit():
return self.c266()
    if self.current_char ==
".": return self.d270()
    return self.c265()

def c265(self): return
self.finalize_coin()

# digit 15
def c266(self):
    self.advance()
    if self.check_digit():
return self.c268()
    if self.current_char ==
".": return self.d270()
    return self.c267()

def c267(self): return
self.finalize_coin()

# digit 16
def c268(self):
    self.advance()
    if self.check_digit():
return self.error()
    if self.current_char ==
".": return self.d270()
    return self.c269()

def c269(self): return
self.finalize_coin()

# decimal point
def d270(self):
    self.advance()
    if self.check_digit():
return self.d271()
    return self.error()

# decimal 1
```

```
def d271(self):
    self.advance()
    if self.check_digit():
return self.d273()
    return self.d272()

def d272(self): return
self.finalize_dime()

# decimal 2
def d273(self):
    self.advance()
    if self.check_digit():
return self.d275()
    return self.d274()

def d274(self): return
self.finalize_dime()

# decimal 3
def d275(self):
    self.advance()
    if self.check_digit():
return self.d277()
    return self.d276()
```

```
def d276(self): return
self.finalize_dime()

# decimal 4
def d277(self):
    self.advance()
    if self.check_digit():
return self.d279()
    return self.d278()

def d278(self): return
self.finalize_dime()

# decimal 5
def d279(self):
    self.advance()
    if self.check_digit():
return self.d281()
    return self.d280()

def d280(self): return
self.finalize_dime()

# decimal 6
def d281(self):
    self.advance()
```

```
if self.check_digit():
return self.d283()
    return self.d282()

def d282(self): return
self.finalize_dime()

# decimal 7
def d283(self):
    self.advance()
    if self.check_digit():
return self.d285()
    return self.d284()

def d284(self): return
self.finalize_dime()

# decimal 8
def d285(self):
    self.advance()
    if self.check_digit():
return self.error()
    return self.d286()

def d286(self): return
self.finalize_dime()
```

texts.py

```
from backend.lexical.token
import Token
from
backend.lexical.handlers.deli
miters import Delimiters
from
backend.lexical.lexer_errors
import LexerErrors

class Texts(LexerErrors):

    # check delimiter if valid
    def check_pdelim(self):

SeaStack
```

```
return
self._is_valid_delimiter("PAR
CH_DELIM")

def check_sdelim(self):
    return
self._is_valid_delimiter("SC
ROLL_DELIM")

# check char if valid as
PARCH or SCROLL literal
def check_parch(self):
```

```
allowed =
Delimiters.delims()["ASCII"
] - set("\n\r")
    return self.current_char
is not None and
self.current_char in allowed

def check_scroll(self):
    allowed =
Delimiters.delims()["ASCII"
] - set("\n\r")
```

```
        return self.current_char
    is not None and
    self.current_char in allowed

    def p287(self):
        self.advance()
        char = self.current_char

        if char == "": return
        self.error()

        if char == '\n' or char is
        None: return self.error()
        if char == '\\': return
        self.p291()
        if self.check_parch():
            return self.p288()

        return self.error()

    # valid PARCH char
    def p288(self):
        self.advance()
        if self.current_char ==
        "": return self.p289()
        return self.error()

    # closing single quote
    def p289(self):
        self.advance()
        if self.check_pdelim():
            return self.p290()
        return self.error()

    # tokenize PARCH literal
    def p290(self): return
    Token("PARCH-lit",
    self.current_token_text(),
```

```
        self.token_start_line,
        self.token_start_col)

        # PARCH escape
        sequences
        def p291(self):
            self.advance()
            valid_escapes = ['s', 'n',
            't', '0', '\\']

            if self.current_char in
            valid_escapes: return
            self.p288()
            return self.error()

        def s292(self):
            self.advance()
            char = self.current_char

            if char == "": return
            self.error()
            if char == '\n' or char is
            None: return self.error()
            if char == '\\': return
            self.s296()
            if self.check_scroll():
                return self.s293()

            return self.error()

        # valid SCROLL chars
        def s293(self):
            self.advance()
            char = self.current_char

            if char == "": return
            self.s294()
```

```
        if char == '\n' or char is
        None: return self.error()
        if char == '\\': return
        self.s296()
        if self.check_scroll():
            return self.s293()

        return self.error()

    # closing double quote
    def s294(self):
        self.advance()
        if self.check_sdelim():
            return self.s295()
        return self.error()

    # tokenize SCROLL
    literal
    def s295(self): return
    Token("SCROLL-lit",
    self.current_token_text(),
    self.token_start_line,
    self.token_start_col)

    # SCROLL escape
    sequences
    def s296(self):
        self.advance()
        valid_escapes = ['d', 'n',
        't', '0', '\\']

        if self.current_char in
        valid_escapes: return
        self.s293()
        return self.error()
```

comments.py

```

from backend.lexical.token      )      self.advance()
import Token                    if self.current_char ==
from                            # body of single-Line
backend.lexical.lexer_errors    comment
import LexerErrors              def cm299(self):
                                self.advance()

class                            # close or continue
Comments(LexerErrors):          multi-line comment
    def cm297(self):              def cm302(self):
        self.advance()            self.advance()

        if self.current_char ==    if self.current_char ==
        '\n' or self.current_char is '\n' or self.current_char is
        None: return self.cm298()  '~': return self.cm303()
        if self.current_char ==    if self.current_char is
        '(': return self.cm300()    not None: return
        if self.current_char is    self.cm301()
        not None: return
        self.cm299()

        return self.error()

        # tokenize single-line
comment
    def cm298(self):
        if self.current_char is
        not None: self.advance()
        return Token(
            "single-comment",

self.current_token_text(),
    self.token_start_line,
    self.token_start_col

```

syn_parser.py

```

from syntax.predict_set
import PREDICT

```

SeaStack

```
from backend.syntax.syntax_error
s import SyntaxErrors

class Parser:
    def __init__(self, tokens,
source_code):
        ignored_types = [
            "whitespace",
            "newline",
            "single-comment",
            "multi-comment",
        ]

        self.tokens = [t for t in
tokens if t.type not in
ignored_types]

        for t in self.tokens:
            if
t.type.startswith("id") and
t.type[2:].isdigit():
                t.type = "id"

        self.pos = 0
        self.current_token =
self.tokens[self.pos] if
self.tokens else None
        self.errors = []
        self.err_handler =
SyntaxErrors(source_code)

    def advance(self):
        self.pos += 1
        if self.pos <
len(self.tokens):
            self.current_token =
self.tokens[self.pos]
        else:
            self.current_token =
None

    def eat(self, token_type):
        if self.current_token
and self.current_token.type
== token_type:
            self.advance()
        else:
            raise
Exception(self.err_handler.g
et_missing_token_error(
self.current_token,
token_type,
))

    def get_production(self,
non_terminal):
        if not
self.current_token:
            return None
        productions =
PREDICT.get(non_terminal,
{})
        return
productions.get(self.current_
token.type)

    def
error_invalid_token(self,
non_terminal):
        expected =
list(PREDICT.get(non_termi
nal, {}).keys())
        raise
Exception(self.err_handler.g
et_invalid_token_error(self.c
urrent_token, expected))

    def parse(self):
        try:
            if not self.tokens:
                raise
Exception(self.err_handler.g
et_missing_start_error())

            self.program()

            if self.current_token
is not None:
                raise
Exception(self.err_handler.g
et_expected_eof_error(self.c
urrent_token))

        except Exception as e:
            if e.args and
isinstance(e.args[0], dict):
                self.errors.append(e.args[0])
            else:
                self.errors.append({
                    "type": "Parser
Crash",
                    "line": "?",
                    "col": "?",
                    "found":
"CRASH",
                    "expected": [],
                    "message":
str(e),
                })

        return self.errors

    def program(self):
        # <program>
        prod =
self.get_production('<progra
m>')
        if prod == 1:
            self.global_dec()
            self.eat('AHOY')
            self.eat('(')
            self.eat(')')
            self.eat('[')
```

```
self.ahoy_local_dec()
self.ahoy_stmts()
self.eat(']')
else:
self.error_invalid_token('<pr
ogram>')

def global_dec(self):
    # <global-dec>
    prod =
self.get_production('<global-
dec>')
    if prod == 2:
        self.var_arr_func()
    elif prod == 3:
        self.const()
        self.global_dec()
    elif prod == 4:
        self.struct()
        self.sub_func()
    elif prod == 5:
        self.nonreturn_func()
    elif prod == 6:
        pass # Lambda
    else:
self.error_invalid_token('<gl
obal-dec>')

def var_arr_func(self):
    # <var-arr-func>
    prod =
self.get_production('<var-arr
-func>')
    if prod == 7:
        self.eat('COIN')
        self.eat('id')

self.coin_var_arr_func()
    elif prod == 8:
        self.eat('DIME')
```

```
self.eat('id')
self.dime_var_arr_func()
    elif prod == 9:
        self.eat('PARCH')
        self.eat('id')

self.parch_var_arr_func()
    elif prod == 10:
        self.eat('SCROLL')
        self.eat('id')

self.scroll_var_arr_func()
    elif prod == 11:
        self.eat('BOOL')
        self.eat('id')

self.bool_var_arr_func()
    else:
self.error_invalid_token('<va
r-arr-func>')

def
coin_var_arr_func(self):
    # <coin-var-arr-func>
    prod =
self.get_production('<coin-v
ar-arr-func>')
    if prod == 12:
        self.coin_var_arr()
        self.eat('!!')
        self.global_dec()
    elif prod == 13:
        self.coin_func()
    else:
self.error_invalid_token('<co
in-var-arr-func>')

def coin_var_arr(self):
    # <coin-var-arr>
```

```
prod =
self.get_production('<coin-v
ar-arr>')
    if prod == 14:
        self.coin_var()
    elif prod == 15:
        self.coin_arr()
    else:
self.error_invalid_token('<co
in-var-arr>')

def coin_var(self):
    # <coin-var>
    prod =
self.get_production('<coin-v
ar>')
    if prod == 16:
        self.coin_init()
        self.coin_init_mult()
    else:
self.error_invalid_token('<co
in-var>')

def coin_init(self):
    # <coin-init>
    prod =
self.get_production('<coin-in
it>')
    if prod == 17:
        self.eat('=')
        self.coin_val()
        self.coin_exp()
    elif prod == 18:
        pass # Lambda
    else:
self.error_invalid_token('<co
in-init>')

def coin_init_mult(self):
```

```
# <coin-init-mult>
prod =
self.get_production('<coin-in
it-mult>')
if prod == 19:
    self.eat(',')
    self.eat('id')
    self.coin_init()
    self.coin_init_mult()
elif prod == 20:
    pass # Lambda
else:
    self.error_invalid_token('<co
in-init-mult>')

def coin_val(self):
    # <coin-val>
    prod =
self.get_production('<coin-v
al>')
    if prod == 21:
        self.eat('COIN-lit')
    elif prod == 22:
        self.neg()
        self.neg_coin_val()
    else:
        self.error_invalid_token('<co
in-val>')

def neg(self):
    # <neg>
    prod =
self.get_production('<neg>')
    if prod == 23:
        self.eat('-')
    elif prod == 24:
        pass # Lambda
    else:
```

```
self.error_invalid_token('<ne
g>')

def neg_coin_val(self):
    # <neg-coin-val>
    prod =
self.get_production('<neg-co
in-val>')
    if prod == 25:
        self.eat('id')
    if
self.current_token.type in
['{', '$', '(']:
        self.id_tail()
    elif prod == 26:
        self.eat('(')
        self.coin_val_grp()
        self.eat(')')
    else:
        self.error_invalid_token('<ne
g-coin-val>')

def coin_exp(self):
    # <coin-exp>
    prod =
self.get_production('<coin-e
xp>')
    if prod == 27:
        self.coin_arith()
        self.coin_exp()
    elif prod == 28:
        pass # Lambda
    else:
        self.error_invalid_token('<co
in-exp>')

def coin_arith(self):
    # <coin-arith>
```

```
prod =
self.get_production('<coin-ar
ith>')
    if prod == 29:
        self.arith_op()
        self.coin_val()
    else:
        self.error_invalid_token('<co
in-arith>')

def coin_val_grp(self):
    # <coin-val-grp>
    prod =
self.get_production('<coin-v
al-grp>')
    if prod == 30:
        self.coin_val()
        self.coin_grp_exp()
    else:
        self.error_invalid_token('<co
in-val-grp>')

def coin_grp_exp(self):
    # <coin-grp-exp>
    prod =
self.get_production('<coin-gr
p-exp>')
    if prod == 31:
        self.coin_arith()
        self.coin_grp_exp()
    elif prod == 32:
        pass # Lambda
    else:
        self.error_invalid_token('<co
in-grp-exp>')

def arith_op(self):
    # <arith-op>
```

```
prod =
self.get_production('<arith-op>')
if prod == 33:
    self.eat('+')
elif prod == 34:
    self.eat('-')
elif prod == 35:
    self.eat('*')
elif prod == 36:
    self.eat('/')
elif prod == 37:
    self.eat('%')
elif prod == 38:
    self.eat('^')
else:
    self.error_invalid_token('<arith-op>')

def coin_arr(self):
    # <coin-arr>
    prod =
self.get_production('<coin-arr>')

if prod == 39:
    self.eat('{')
    self.eat('COIN-lit')
    self.eat('}')
    self.coin_arr_tail()
else:
    self.error_invalid_token('<coin-arr>')

def coin_arr_tail(self):
    # <coin-arr-tail>
    prod =
self.get_production('<coin-arr-tail>')

if prod == 40:
    self.eat('=')
```

```
self.eat('[')
self.coin_arr1()
self.eat(']')
elif prod == 41:
    self.eat('{')
    self.eat('COIN-lit')
    self.eat('}')
    self.coin_arr2_tail()
elif prod == 42:
    pass # Lambda
else:
    self.error_invalid_token('<coin-arr-tail>')

def coin_arr1(self):
    # <coin-arr1>
    prod =
self.get_production('<coin-arr1>')

if prod == 43:
    self.coin_arr_val()
    self.cav_tail()
else:
    self.error_invalid_token('<coin-arr1>')

def coin_arr_val(self):
    # <coin-arr-val>
    prod =
self.get_production('<coin-arr-val>')

if prod == 44:
    self.coin_val()
    self.coin_arr_exp()
else:
    self.error_invalid_token('<coin-arr-val>')

def coin_arr_exp(self):
```

```
# <coin-arr-exp>
prod =
self.get_production('<coin-arr-exp>')

if prod == 45:
    self.coin_arith()
    self.coin_arr_exp()
elif prod == 46:
    pass # Lambda
else:
    self.error_invalid_token('<coin-arr-exp>')

def cav_tail(self):
    # <cav-tail>
    prod =
self.get_production('<cav-tail>')

if prod == 47:
    self.eat(',')
    self.coin_arr1()
elif prod == 48:
    pass # Lambda
else:
    self.error_invalid_token('<cav-tail>')

def coin_arr2_tail(self):
    # <coin-arr2-tail>
    prod =
self.get_production('<coin-arr2-tail>')

if prod == 49:
    self.eat('=')
    self.eat('[')
    self.coin_arr2()
    self.eat(']')
elif prod == 50:
    pass # Lambda
else:
```

```

self.error_invalid_token('<coin-arr2-tail>')

def coin_arr2(self):
    # <coin-arr2>
    prod =
self.get_production('<coin-arr2>')
    if prod == 51:
        self.eat('[')
        self.coin_arr1()
        self.eat(']')
        self.cav2_tail()
    else:

self.error_invalid_token('<coin-arr2>')

def cav2_tail(self):
    # <cav2-tail>
    prod =
self.get_production('<cav2-tail>')
    if prod == 52:
        self.eat(',')
        self.coin_arr2()
    elif prod == 53:
        pass # Lambda
    else:

self.error_invalid_token('<cav2-tail>')

def coin_func(self):
    # <coin-func>
    prod =
self.get_production('<coin-func>')
    if prod == 54:
        self.eat('(')
        self.params()

        self.eat(')')
        self.eat('[')
        self.local_dec()
        self.ret_stmts()
        self.eat('BACK')
        self.coin_ret_val()
        self.eat('!!')
        self.eat(']')
        self.sub_func()
    else:

self.error_invalid_token('<coin-func>')

def coin_ret_val(self):
    # <coin-ret-val>
    prod =
self.get_production('<coin-ret-val>')
    if prod == 55:
        self.coin_val()
        self.coin_ret_exp()
    else:

self.error_invalid_token('<coin-ret-val>')

def coin_ret_exp(self):
    # <coin-ret-exp>
    prod =
self.get_production('<coin-ret-exp>')
    if prod == 56:
        self.coin_arith()
        self.coin_ret_exp()
    elif prod == 57:
        pass # Lambda
    else:

self.error_invalid_token('<coin-ret-exp>')

def
dime_var_arr_func(self):
    # <dime-var-arr-func>
    prod =
self.get_production('<dime-var-arr-func>')
    if prod == 58:
        self.dime_var_arr()
        self.eat('!!')
        self.global_dec()
    elif prod == 59:
        self.dime_func()
    else:

self.error_invalid_token('<dime-var-arr-func>')

def dime_var_arr(self):
    # <dime-var-arr>
    prod =
self.get_production('<dime-var-arr>')
    if prod == 60:
        self.dime_var()
    elif prod == 61:
        self.dime_arr()
    else:

self.error_invalid_token('<dime-var-arr>')

def dime_var(self):
    # <dime-var>
    prod =
self.get_production('<dime-var>')
    if prod == 62:
        self.dime_init()
        self.dime_init_mult()
    else:

```

```
self.error_invalid_token('<di
me-var>')

def dime_init(self):
    # <dime-init>
    prod =
self.get_production('<dime-i
nit>')
    if prod == 63:
        self.eat('=')
        self.dime_init_val()
    elif prod == 64:
        pass # Lambda
    else:

self.error_invalid_token('<di
me-init>')

def dime_init_mult(self):
    # <dime-init-mult>
    prod =
self.get_production('<dime-i
nit-mult>')
    if prod == 65:
        self.eat(',')
        self.eat('id')
        self.dime_init()
        self.dime_init_mult()
    elif prod == 66:
        pass # Lambda
    else:

self.error_invalid_token('<di
me-init-mult>')

def dime_init_val(self):
    # <dime-init-val>
    prod =
self.get_production('<dime-i
nit-val>')
    if prod == 67:

self.dime_val()
self.dime_exp()
else:

self.error_invalid_token('<di
me-init-val>')

def dime_val(self):
    # <dime-val>
    prod =
self.get_production('<dime-v
al>')
    if prod == 68:
        self.eat('DIME-lit')
    elif prod == 69:
        self.eat('COIN-lit')
    elif prod == 70:
        self.neg()
        self.neg_dime_val()
    else:

self.error_invalid_token('<di
me-val>')

def neg_dime_val(self):
    # <neg-dime-val>
    prod =
self.get_production('<neg-di
me-val>')
    if prod == 71:
        self.eat('id')
        if
self.current_token.type in
['{', '$', '(']:
            self.id_tail()
        elif prod == 72:
            self.eat('(')
            self.dime_grp_val()
            self.eat(')')
        else:

self.dime_exp(self):
    # <dime-exp>
    prod =
self.get_production('<dime-e
xp>')
    if prod == 73:
        self.dime_arith()
        self.dime_exp()
    elif prod == 74:
        pass # Lambda
    else:

self.error_invalid_token('<di
me-exp>')

def dime_arith(self):
    # <dime-arith>
    prod =
self.get_production('<dime-a
rith>')
    if prod == 75:
        self.arith_op()
        self.dime_val()
    else:

self.error_invalid_token('<di
me-arith>')

def dime_grp_val(self):
    # <dime-grp-val>
    prod =
self.get_production('<dime-g
rp-val>')
    if prod == 76:
        self.dime_val()
        self.dime_grp_exp()
    else:
```

```
self.error_invalid_token('<dime-grp-val>')

def dime_grp_exp(self):
    # <dime-grp-exp>
    prod =
self.get_production('<dime-grp-exp>')
    if prod == 77:
        self.dime_arith()
        self.dime_grp_exp()
    elif prod == 78:
        pass # Lambda
    else:

self.error_invalid_token('<dime-grp-exp>')

def dime_arr(self):
    # <dime-arr>
    prod =
self.get_production('<dime-arr>')
    if prod == 79:
        self.eat('{')
        self.eat('COIN-lit')
        self.eat('}')
        self.dime_arr_tail()
    else:

self.error_invalid_token('<dime-arr>')

def dime_arr_tail(self):
    # <dime-arr-tail>
    prod =
self.get_production('<dime-arr-tail>')
    if prod == 80:
        self.eat('=')
        self.eat('[')

self.dime_arr1()
self.eat(']')
elif prod == 81:
    self.eat('{')
    self.eat('COIN-lit')
    self.eat('}')
    self.dime_arr2_tail()
elif prod == 82:
    pass # Lambda
else:

self.error_invalid_token('<dime-arr-tail>')

def dime_arr1(self):
    # <dime-arr1>
    prod =
self.get_production('<dime-arr1>')
    if prod == 83:
        self.dime_arr_val()
        self.dav_tail()
    else:

self.error_invalid_token('<dime-arr1>')

def dime_arr_val(self):
    # <dime-arr-val>
    prod =
self.get_production('<dime-arr-val>')
    if prod == 84:
        self.dime_val()
        self.dime_arr_exp()
    else:

self.error_invalid_token('<dime-arr-val>')

def dime_arr_exp(self):
    # <dime-arr-exp>

prod =
self.get_production('<dime-arr-exp>')
    if prod == 85:
        self.dime_arith()
        self.dime_arr_exp()
    elif prod == 86:
        pass # Lambda
    else:

self.error_invalid_token('<dime-arr-exp>')

def dav_tail(self):
    # <dav-tail>
    prod =
self.get_production('<dav-tail>')
    if prod == 87:
        self.eat(',')
        self.dime_arr1()
    elif prod == 88:
        pass # Lambda
    else:

self.error_invalid_token('<dav-tail>')

def dime_arr2_tail(self):
    # <dime-arr2-tail>
    prod =
self.get_production('<dime-arr2-tail>')
    if prod == 89:
        self.eat('=')
        self.eat('[')
        self.dime_arr2()
        self.eat(']')
    elif prod == 90:
        pass # Lambda
    else:
```

```
self.error_invalid_token('<dime-arr2-tail>')

def dime_arr2(self):
    # <dime-arr2>
    prod =
self.get_production('<dime-arr2>')
    if prod == 91:
        self.eat('[')
        self.dime_arr1()
        self.eat(']')
        self.dav2_tail()
    else:

self.error_invalid_token('<dime-arr2>')

def dav2_tail(self):
    # <dav2-tail>
    prod =
self.get_production('<dav2-tail>')
    if prod == 92:
        self.eat(',')
        self.dime_arr2()
    elif prod == 93:
        pass # Lambda
    else:

self.error_invalid_token('<dime-arr2-tail>')

def dime_func(self):
    # <dime-func>
    prod =
self.get_production('<dime-func>')
    if prod == 94:
        self.eat('(')
        self.params()

        self.eat(')')
        self.eat('[')
        self.local_dec()
        self.ret_stmts()
        self.eat('BACK')
        self.dime_ret_val()
        self.eat('!!')
        self.eat(']')
        self.sub_func()
    else:

self.error_invalid_token('<dime-func>')

def dime_ret_val(self):
    # <dime-ret-val>
    prod =
self.get_production('<dime-ret-val>')
    if prod == 95:
        self.dime_val()
        self.dime_ret_exp()
    else:

self.error_invalid_token('<dime-ret-val>')

def dime_ret_exp(self):
    # <dime-ret-exp>
    prod =
self.get_production('<dime-ret-exp>')
    if prod == 96:
        self.dime_arith()
        self.dime_ret_exp()
    elif prod == 97:
        pass # Lambda
    else:

self.error_invalid_token('<dime-ret-exp>')

def
parch_var_arr_func(self):
    # <parch-var-arr-func>
    prod =
self.get_production('<parch-var-arr-func>')
    if prod == 98:
        self.parch_var_arr()
        self.eat('!!')
        self.global_dec()
    elif prod == 99:
        self.parch_func()
    else:

self.error_invalid_token('<parch-var-arr-func>')

def parch_var_arr(self):
    # <parch-var-arr>
    prod =
self.get_production('<parch-var-arr>')
    if prod == 100:
        self.parch_var()
    elif prod == 101:
        self.parch_arr()
    else:

self.error_invalid_token('<parch-var-arr>')

def parch_var(self):
    # <parch-var>
    prod =
self.get_production('<parch-var>')
    if prod == 102:
        self.parch_init()

self.parch_init_mult()
    else:
```

```
self.error_invalid_token('<parch-var>')

def parch_init(self):
    # <parch-init>
    prod =
self.get_production('<parch-init>')
    if prod == 103:
        self.eat('=')
        self.parch_val()
    elif prod == 104:
        pass # Lambda
    else:

self.error_invalid_token('<parch-init>')

def parch_init_mult(self):
    # <parch-init-mult>
    prod =
self.get_production('<parch-init-mult>')
    if prod == 105:
        self.eat(',')
        self.eat('id')
        self.parch_init()

self.parch_init_mult()
    elif prod == 106:
        pass # Lambda
    else:

self.error_invalid_token('<parch-init-mult>')

def parch_val(self):
    # <parch-val>
    prod =
self.get_production('<parch-val>')

    if prod == 107:
        self.eat('PARCH-lit')
    elif prod == 108:
        self.eat('id')
        if
self.current_token.type in
['{', '$', '(']:
            self.id_tail()
        else:

self.error_invalid_token('<parch-val>')

def parch_arr(self):
    # <parch-arr>
    prod =
self.get_production('<parch-arr>')
    if prod == 109:
        self.eat('{')
        self.eat('COIN-lit')
        self.eat('}')
        self.parch_arr_tail()
    else:

self.error_invalid_token('<parch-arr>')

def parch_arr_tail(self):
    # <parch-arr-tail>
    prod =
self.get_production('<parch-arr-tail>')
    if prod == 110:
        self.eat('=')
        self.eat('[')
        self.parch_arr1()
        self.eat(']')
    elif prod == 111:
        self.eat('{')
        self.eat('COIN-lit')
        self.eat('}')

    self.parch_arr2_tail()
    elif prod == 112:
        pass # Lambda
    else:

self.error_invalid_token('<parch-arr-tail>')

def parch_arr1(self):
    # <parch-arr1>
    prod =
self.get_production('<parch-arr1>')
    if prod == 113:
        self.parch_val()
        self.pav_tail()
    else:

self.error_invalid_token('<parch-arr1>')

def pav_tail(self):
    # <pav-tail>
    prod =
self.get_production('<pav-tail>')
    if prod == 114:
        self.eat(',')
        self.parch_arr1()
    elif prod == 115:
        pass # Lambda
    else:

self.error_invalid_token('<pav-tail>')

def parch_arr2_tail(self):
    # <parch-arr2-tail>
    prod =
self.get_production('<parch-arr2-tail>')
    if prod == 116:
```

```
self.eat('=')
self.eat('[')
self.parch_arr2()
self.eat(']')
elif prod == 117:
    pass # Lambda
else:

self.error_invalid_token('<parch-arr2-tail>')

def parch_arr2(self):
    # <parch-arr2>
    prod =
self.get_production('<parch-arr2>')
    if prod == 118:
        self.eat('[')
        self.parch_arr1()
        self.eat(']')
        self.pav2_tail()
    else:

self.error_invalid_token('<parch-arr2>')

def pav2_tail(self):
    # <pav2-tail>
    prod =
self.get_production('<pav2-tail>')
    if prod == 119:
        self.eat(',')
        self.parch_arr2()
    elif prod == 120:
        pass # Lambda
    else:

self.error_invalid_token('<pav2-tail>')

def parch_func(self):

SeaStack
```

```
# <parch-func>
prod =
self.get_production('<parch-func>')
if prod == 121:
    self.eat('[')
    self.params()
    self.eat(')')
    self.eat('[')
    self.local_dec()
    self.ret_stmts()
    self.eat('BACK')
    self.parch_val()
    self.eat('!!')
    self.eat(']')
    self.sub_func()
else:

self.error_invalid_token('<parch-func>')

def
scroll_var_arr_func(self):
    # <scroll-var-arr-func>
    prod =
self.get_production('<scroll-var-arr-func>')
    if prod == 122:
        self.scroll_var_arr()
        self.eat('!!')
        self.global_dec()
    elif prod == 123:
        self.scroll_func()
        self.sub_func()
    else:

self.error_invalid_token('<scroll-var-arr-func>')

def scroll_var_arr(self):
    # <scroll-var-arr>
```

```
prod =
self.get_production('<scroll-var-arr>')
if prod == 124:
    self.scroll_var()
elif prod == 125:
    self.scroll_arr()
else:

self.error_invalid_token('<scroll-var-arr>')

def scroll_var(self):
    # <scroll-var>
    prod =
self.get_production('<scroll-var>')
    if prod == 126:
        self.scroll_init()

self.scroll_init_mult()
else:

self.error_invalid_token('<scroll-var>')

def scroll_init(self):
    # <scroll-init>
    prod =
self.get_production('<scroll-init>')
    if prod == 127:
        self.eat('=')
        self.scroll_init_val()
    elif prod == 128:
        pass # Lambda
    else:

self.error_invalid_token('<scroll-init>')

def scroll_init_mult(self):
```

```
# <scroll-init-mult>
prod =
self.get_production('<scroll-i
nit-mult>')
    if prod == 129:
        self.eat(',')
        self.eat('id')
        self.scroll_init()

self.scroll_init_mult()
    elif prod == 130:
        pass # Lambda
    else:

self.error_invalid_token('<sc
roll-init-mult>')

def scroll_init_val(self):
    # <scroll-init-val>
    prod =
self.get_production('<scroll-i
nit-val>')
    if prod == 131:
        self.scroll_val()
        self.scroll_exp()
    else:

self.error_invalid_token('<sc
roll-init-val>')

def scroll_val(self):
    # <scroll-val>
    prod =
self.get_production('<scroll-
val>')
    if prod == 132:

self.eat('SCROLL-lit')
    if
self.current_token.type in
['{']:
        self.scr_char()

        elif prod == 133:
            self.eat('id')
            if
self.current_token.type in
['{', '$', '(']:
                self.id_tail()
            elif prod == 134:
                self.eat('(')
                self.scroll_grp_val()
                self.eat(')')
            else:

self.error_invalid_token('<sc
roll-val>')

def scr_char(self):
    # <scr-char>
    prod =
self.get_production('<scr-cha
r>')
    if prod == 135:
        self.eat('{')
        self.index()
        self.eat('}')
    elif prod == 136:
        pass # Lambda
    else:

self.error_invalid_token('<sc
r-char>')

def index(self):
    # <index>
    prod =
self.get_production('<index>
')
    if prod == 137:
        self.eat('id')
    elif prod == 138:
        self.eat('COIN-lit')
    else:

self.error_invalid_token('<in
dex>')

def scroll_exp(self):
    # <scroll-exp>
    prod =
self.get_production('<scroll-
exp>')
    if prod == 139:
        self.scroll_concat()
        self.scroll_exp()
    elif prod == 140:
        pass # Lambda
    else:

self.error_invalid_token('<sc
roll-exp>')

def scroll_concat(self):
    # <scroll-concat>
    prod =
self.get_production('<scroll-
concat>')
    if prod == 141:
        self.eat('&')
        self.scroll_val()
    else:

self.error_invalid_token('<sc
roll-concat>')

def scroll_grp_val(self):
    # <scroll-grp-val>
    prod =
self.get_production('<scroll-
grp-val>')
    if prod == 142:
        self.scroll_val()
        self.scroll_grp_exp()
    else:
```

```
self.error_invalid_token('<scroll-grp-val>')

def scroll_grp_exp(self):
    # <scroll-grp-exp>
    prod =
self.get_production('<scroll-grp-exp>')
    if prod == 143:
        self.scroll_concat()
        self.scroll_grp_exp()
    elif prod == 144:
        pass # Lambda
    else:

self.error_invalid_token('<scroll-grp-exp>')

def scroll_arr(self):
    # <scroll-arr>
    prod =
self.get_production('<scroll-arr>')
    if prod == 145:
        self.eat('{')
        self.eat('COIN-lit')
        self.eat('}')
        self.scroll_arr_tail()
    else:

self.error_invalid_token('<scroll-arr>')

def scroll_arr_tail(self):
    # <scroll-arr-tail>
    prod =
self.get_production('<scroll-arr-tail>')
    if prod == 146:
        self.eat('=')
        self.eat('[')
```

```
self.scroll_arr1()
self.eat(']')
elif prod == 147:
    self.eat('{')
    self.eat('COIN-lit')
    self.eat('}')
    self.scroll_arr2_tail()
elif prod == 148:
    pass # Lambda
else:

self.error_invalid_token('<scroll-arr-tail>')

def scroll_arr1(self):
    # <scroll-arr1>
    prod =
self.get_production('<scroll-arr1>')
    if prod == 149:
        self.scroll_arr_val()
        self.sav_tail()
    else:

self.error_invalid_token('<scroll-arr1>')

def scroll_arr_val(self):
    # <scroll-arr-val>
    prod =
self.get_production('<scroll-arr-val>')
    if prod == 150:
        self.scroll_val()
        self.scroll_arr_exp()
    else:

self.error_invalid_token('<scroll-arr-val>')

def scroll_arr_exp(self):
    # <scroll-arr-exp>
```

```
prod =
self.get_production('<scroll-arr-exp>')
    if prod == 151:
        self.scroll_concat()
        self.scroll_arr_exp()
    elif prod == 152:
        pass # Lambda
    else:

self.error_invalid_token('<scroll-arr-exp>')

def sav_tail(self):
    # <sav-tail>
    prod =
self.get_production('<sav-tail>')
    if prod == 153:
        self.eat(',')
        self.scroll_arr1()
    elif prod == 154:
        pass # Lambda
    else:

self.error_invalid_token('<sav-tail>')

def scroll_arr2_tail(self):
    # <scroll-arr2-tail>
    prod =
self.get_production('<scroll-arr2-tail>')
    if prod == 155:
        self.eat('=')
        self.eat('[')
        self.scroll_arr2()
        self.eat(']')
    elif prod == 156:
        pass # Lambda
    else:
```

```
self.error_invalid_token('<scroll-arr2-tail>')

def scroll_arr2(self):
    # <scroll-arr2>
    prod =
self.get_production('<scroll-arr2>')
    if prod == 157:
        self.eat('[')
        self.scroll_arr1()
        self.eat(']')
        self.sav2_tail()
    else:

self.error_invalid_token('<scroll-arr2>')

def sav2_tail(self):
    # <sav2-tail>
    prod =
self.get_production('<sav2-tail>')
    if prod == 158:
        self.eat(',')
        self.scroll_arr2()
    elif prod == 159:
        pass # Lambda
    else:

self.error_invalid_token('<sav2-tail>')

def scroll_func(self):
    # <scroll-func>
    prod =
self.get_production('<scroll-func>')
    if prod == 160:
        self.eat('(')
        self.params()

        self.eat(')')
        self.eat('[')
        self.local_dec()
        self.ret_stmts()
        self.eat('BACK')
        self.scroll_ret_val()
        self.eat('!!')
        self.eat(']')
        self.sub_func()
    else:

self.error_invalid_token('<scroll-func>')

def scroll_ret_val(self):
    # <scroll-ret-val>
    prod =
self.get_production('<scroll-ret-val>')
    if prod == 161:
        self.scroll_val()
        self.scroll_ret_exp()
    else:

self.error_invalid_token('<scroll-ret-val>')

def scroll_ret_exp(self):
    # <scroll-ret-exp>
    prod =
self.get_production('<scroll-ret-exp>')
    if prod == 162:
        self.scroll_concat()
        self.scroll_ret_exp()
    elif prod == 163:
        pass # Lambda
    else:

self.error_invalid_token('<scroll-ret-exp>')

def
bool_var_arr_func(self):
    # <bool-var-arr-func>
    prod =
self.get_production('<bool-var-arr-func>')
    if prod == 164:
        self.bool_var_arr()
        self.eat('!!')
        self.global_dec()
    elif prod == 165:
        self.bool_func()
    else:

self.error_invalid_token('<bool-var-arr-func>')

def bool_var_arr(self):
    # <bool-var-arr>
    prod =
self.get_production('<bool-var-arr>')
    if prod == 166:
        self.bool_var()
    elif prod == 167:
        self.bool_arr()
    else:

self.error_invalid_token('<bool-var-arr>')

def bool_var(self):
    # <bool-var>
    prod =
self.get_production('<bool-var>')
    if prod == 168:
        self.bool_init()
        self.bool_init_mult()
    else:
```

<pre> self.error_invalid_token('<bo ol-var>') def bool_init(self): # <bool-init> prod = self.get_production('<bool-in it>') if prod == 169: self.eat('=') self.bool_init_val() elif prod == 170: pass # Lambda else: self.error_invalid_token('<bo ol-init>') def bool_init_mult(self): # <bool-init-mult> prod = self.get_production('<bool-in it-mult>') if prod == 171: self.eat(',') self.eat('id') self.bool_init() self.bool_init_mult() elif prod == 172: pass # Lambda else: self.error_invalid_token('<bo ol-init-mult>') def bool_init_val(self): # <bool-init-val> prod = self.get_production('<bool-in it-val>') if prod == 173: </pre>	<pre> self.bool_val() self.bool_exp() else: self.error_invalid_token('<bo ol-init-val>') def bool_val(self): # <bool-val> prod = self.get_production('<bool-v al>') if prod == 174: self.eat('id') if self.current_token.type in ['{', '\$', '(']: self.id_tail() self.bool_exp2() elif prod == 175: self.eat('(') self.value_grp() self.eat(')') self.bool_exp2() elif prod == 176: self.bool() self.bool_eq() elif prod == 177: self.digit() self.bool_arith() self.rel_eq() elif prod == 178: self.eat('PARCH-lit') self.eq_op() self.parch_val() elif prod == 179: </pre>	<pre> self.eq_op() self.scroll_val() else: self.error_invalid_token('<bo ol-val>') def bool(self): # <bool> prod = self.get_production('<bool>') if prod == 180: self.bool_lit() elif prod == 181: self.not_op() self.not_ope() else: self.error_invalid_token('<bo ol>') def bool_lit(self): # <bool-lit> prod = self.get_production('<bool-li t>') if prod == 182: self.eat('AYE') elif prod == 183: self.eat('NAY') else: self.error_invalid_token('<bo ol-lit>') def not_op(self): # <not-op> prod = self.get_production('<not-op >') if prod == 184: self.eat('!') </pre>
--	---	--

```
elif prod == 185:
    self.eat('!#')
else:
    self.error_invalid_token('<not-op>')

def not_ope(self):
    # <not-ope>
    prod =
self.get_production('<not-ope>')
    if prod == 186:
        self.eat('id')
        if
self.current_token.type in
['{', '$', '(']:
            self.id_tail()
        elif prod == 187:
            self.eat('(')
            self.bool_grp_val()
            self.eat(')')
        elif prod == 188:
            self.bool_lit()
        else:
            self.error_invalid_token('<not-ope>')

def bool_eq(self):
    # <bool-eq>
    prod =
self.get_production('<bool-eq>')
    if prod == 189:
        self.eq_op()
        self.bool_val()
    elif prod == 190:
        pass # Lambda
    else:
        self.error_invalid_token('<not-ope>')

def digit(self):
    # <digit>
    prod =
self.get_production('<digit>')
    if prod == 191:
        self.eat('COIN-lit')
    elif prod == 192:
        self.eat('DIME-lit')
    elif prod == 193:
        self.eat('-')
        self.neg_digit()
    else:
        self.error_invalid_token('<digit>')

def neg_digit(self):
    # <neg-digit>
    prod =
self.get_production('<neg-digit>')
    if prod == 194:
        self.eat('id')
        if
self.current_token.type in
['{', '$', '(']:
            self.id_tail()
        elif prod == 195:
            self.eat('(')
            self.dime_grp_val()
            self.eat(')')
        else:
            self.error_invalid_token('<neg-digit>')

def bool_arith(self):
    # <bool-arith>
    prod =
self.get_production('<bool-arith>')
    if prod == 196:
        self.dime_arith()
        self.bool_arith()
    elif prod == 197:
        pass # Lambda
    else:
        self.error_invalid_token('<bool-arith>')

def rel_eq(self):
    # <rel-eq>
    prod =
self.get_production('<rel-eq>')
    if prod == 198:
        self.rel()
        self.bool_arith2()
        self.bool_eq()
    elif prod == 199:
        self.eq_digit()
        self.bool_arith3()
    else:
        self.error_invalid_token('<rel-eq>')

def rel(self):
    # <rel>
    prod =
self.get_production('<rel>')
    if prod == 200:
        self.rel_op()
        self.dime_val()
    else:
        self.error_invalid_token('<rel>')
```

```

        else:
            self.rel_eq()
            elif prod == 213:
                self.rel()
                self.bool_arith2()
                self.bool_eq()
            elif prod == 214:
                self.eq_op()
                self.eq_ope()
            elif prod == 215:
                pass # Lambda
            else:
                self.error_invalid_token('<bol-exp2>')

        def eq_ope(self):
            # <eq-ope>
            prod =
            self.get_production('<eq-ope>')
            if prod == 216:
                self.eat('id')
                if
                self.current_token.type in
                ['{', '$', '(']:
                    self.id_tail()
                    self.bool_arith4()
                    self.bool_rel()
                elif prod == 217:
                    self.eat('(')
                    self.value_grp()
                    self.eat(')')
                    self.bool_arith4()
                    self.bool_rel()
                elif prod == 218:
                    self.digit()
                    self.bool_arith4()
                    self.bool_rel()
                elif prod == 219:
                    self.eq_ope1()
            else:

def rel_op(self):
    # <rel-op>
    prod =
    self.get_production('<rel-op>')
    if prod == 201:
        self.eat('<')
    elif prod == 202:
        self.eat('>')
    elif prod == 203:
        self.eat('<=')
    elif prod == 204:
        self.eat('>=')
    else:

self.error_invalid_token('<rel-op>')

def bool_arith2(self):
    # <bool-arith2>
    prod =
    self.get_production('<bool-arith2>')
    if prod == 205:
        self.dime_arith()
        self.bool_arith2()
    elif prod == 206:
        pass # Lambda
    else:

self.error_invalid_token('<bool-arith2>')

def eq_digit(self):
    # <eq-digit>
    prod =
    self.get_production('<eq-digit>')
    if prod == 207:
        self.eq_op()
        self.dime_val()

        else:
            self.error_invalid_token('<eq-digit>')

        def eq_op(self):
            # <eq-op>
            prod =
            self.get_production('<eq-op>')
            if prod == 208:
                self.eat('==')
            elif prod == 209:
                self.eat('!=')
            else:

self.error_invalid_token('<eq-op>')

def bool_arith3(self):
    # <bool-arith3>
    prod =
    self.get_production('<bool-arith3>')
    if prod == 210:
        self.dime_arith()
        self.bool_arith3()
    elif prod == 211:
        pass # Lambda
    else:

self.error_invalid_token('<bool-arith3>')

def bool_exp2(self):
    # <bool-exp2>
    prod =
    self.get_production('<bool-exp2>')
    if prod == 212:
        self.dime_arith()
        self.bool_arith()

```

```
self.error_invalid_token('<eq
-ope>')

def eq_ope1(self):
    # <eq-ope1>
    prod =
self.get_production('<eq-ope
1>')
    if prod == 220:
        self.eat('PARCH-lit')
    elif prod == 221:

self.eat('SCROLL-lit')
    if
self.current_token.type in
['{']:
        self.scr_char()
    elif prod == 222:
        self.bool()
    else:

self.error_invalid_token('<eq
-ope1>')

def bool_arith4(self):
    # <bool-arith4>
    prod =
self.get_production('<bool-ar
ith4>')
    if prod == 223:
        self.dime_arith()
        self.bool_arith4()
    elif prod == 224:
        pass # Lambda
    else:

self.error_invalid_token('<bo
ol-arith4>')

def bool_rel(self):
    # <bool-rel>
```

```
prod =
self.get_production('<bool-re
l>')
    if prod == 225:
        self.rel()
        self.bool_arith3()
    elif prod == 226:
        pass # Lambda
    else:

self.error_invalid_token('<bo
ol-rel>')

def bool_exp(self):
    # <bool-exp>
    prod =
self.get_production('<bool-e
xp>')
    if prod == 227:
        self.log_op()
        self.bool_val()
        self.bool_exp()
    elif prod == 228:
        pass # Lambda
    else:

self.error_invalid_token('<bo
ol-exp>')

def log_op(self):
    # <log-op>
    prod =
self.get_production('<log-op
>')
    if prod == 229:
        self.eat('||')
    elif prod == 230:
        self.eat('&&')
    else:

self.error_invalid_token('<lo
g-op>')
```

```
def bool_grp_val(self):
    # <bool-grp-val>
    prod =
self.get_production('<bool-g
rp-val>')
    if prod == 231:
        self.bool_ope_grp()
        self.bool_exp_grp()
    else:

self.error_invalid_token('<bo
ol-grp-val>')

def bool_ope_grp(self):
    # <bool-ope-grp>
    prod =
self.get_production('<bool-o
pe-grp>')
    if prod == 232:
        self.eat('id')
        if
self.current_token.type in
['{', '$', '(']:
            self.id_tail()
            self.bool_exp2_grp()
    elif prod == 233:
        self.eat('(')
        self.value_grp()
        self.eat(')')
        self.bool_exp2_grp()
    elif prod == 234:
        self.bool()
        self.bool_eq_grp()
    elif prod == 235:
        self.digit()
        self.bool_arith()
        self.releq_grp()
    elif prod == 236:
        self.eat('PARCH-lit')
        self.eq_op()
        self.parch_val()
```

```
elif prod == 237:

self.eat('SCROLL-lit')
    if
self.current_token.type in
['{']:
        self.scr_char()
        self.eq_op()
        self.scroll_val()
    else:

self.error_invalid_token('<bo
ol-ope-grp>')

def bool_eq_grp(self):
    # <bool-eq-grp>
    prod =
self.get_production('<bool-e
q-grp>')
    if prod == 238:
        self.eq_op()
        self.bool_ope_grp()
    elif prod == 239:
        pass # Lambda
    else:

self.error_invalid_token('<bo
ol-eq-grp>')

def releq_grp(self):
    # <releq-grp>
    prod =
self.get_production('<releq-g
rp>')
    if prod == 240:
        self.rel()

self.bool_arith2_grp()
    self.bool_eq_grp()
    elif prod == 241:
        self.eq_digit()
```

```
self.bool_arith3_grp()
    else:

self.error_invalid_token('<re
leq-grp>')

def bool_arith2_grp(self):
    # <bool-arith2-grp>
    prod =
self.get_production('<bool-ar
ith2-grp>')
    if prod == 242:
        self.dime_arith()

self.bool_arith2_grp()
    elif prod == 243:
        pass # Lambda
    else:

self.error_invalid_token('<bo
ol-arith2-grp>')

def bool_arith3_grp(self):
    # <bool-arith3-grp>
    prod =
self.get_production('<bool-ar
ith3-grp>')
    if prod == 244:
        self.dime_arith()

self.bool_arith3_grp()
    elif prod == 245:
        pass # Lambda
    else:

self.error_invalid_token('<bo
ol-arith3-grp>')

def bool_exp2_grp(self):
    # <bool-exp2-grp>
```

```
prod =
self.get_production('<bool-e
xp2-grp>')
    if prod == 246:
        self.dime_arith()
        self.bool_arith()
        self.releq_grp()
    elif prod == 247:
        self.rel()

self.bool_arith2_grp()
    self.bool_eq_grp()
    elif prod == 248:
        self.eq_op()
        self.eq_ope_grp()
    elif prod == 249:
        pass # Lambda
    else:

self.error_invalid_token('<bo
ol-exp2-grp>')

def eq_ope_grp(self):
    # <eq-ope-grp>
    prod =
self.get_production('<eq-ope
-grp>')
    if prod == 250:
        self.eat('id')
        if
self.current_token.type in
['{', '$', '(']:
            self.id_tail()

self.bool_arith4_grp()
    self.bool_rel_grp()
    elif prod == 251:
        self.eat('(')
        self.value_grp()
        self.eat(')')

self.bool_arith4_grp()
```

```
        self.bool_rel_grp()
    elif prod == 252:
        self.digit()

self.bool_arith4_grp()
    self.bool_rel_grp()
    elif prod == 253:
        self.eq_opel()
    else:

self.error_invalid_token('<eq
-opel-grp>')

    def bool_arith4_grp(self):
        # <bool-arith4-grp>
        prod =
self.get_production('<bool-ar
ith4-grp>')
        if prod == 254:
            self.dime_arith()

self.bool_arith4_grp()
    elif prod == 255:
        pass # Lambda
    else:

self.error_invalid_token('<bo
ol-arith4-grp>')

    def bool_rel_grp(self):
        # <bool-rel-grp>
        prod =
self.get_production('<bool-re
l-grp>')
        if prod == 256:
            self.rel()

self.bool_arith3_grp()
    elif prod == 257:
        pass # Lambda
    else:

        self.error_invalid_token('<bo
ol-rel-grp>')

    def bool_exp_grp(self):
        # <bool-exp-grp>
        prod =
self.get_production('<bool-e
xp-grp>')
        if prod == 258:
            self.log_op()
            self.bool_ope_grp()
            self.bool_exp_grp()
        elif prod == 259:
            pass # Lambda
        else:

self.error_invalid_token('<bo
ol-exp-grp>')

    def bool_arr(self):
        # <bool-arr>
        prod =
self.get_production('<bool-ar
r>')
        if prod == 260:
            self.eat('{')
            self.eat('COIN-lit')
            self.eat('}')
            self.bool_arr_tail()
        else:

self.error_invalid_token('<bo
ol-arr>')

    def bool_arr_tail(self):
        # <bool-arr-tail>
        prod =
self.get_production('<bool-ar
r-tail>')
        if prod == 261:
            self.eat('=')

        self.eat('[')
        self.bool_arr1()
        self.eat(']')
        elif prod == 262:
            self.eat('{')
            self.eat('COIN-lit')
            self.eat('}')
            self.bool_arr2_tail()
        elif prod == 263:
            pass # Lambda
        else:

self.error_invalid_token('<bo
ol-arr-tail>')

    def bool_arr1(self):
        # <bool-arr1>
        prod =
self.get_production('<bool-ar
r1>')
        if prod == 264:
            self.bool_arr_val()
            self.bav_tail()
        else:

self.error_invalid_token('<bo
ol-arr1>')

    def bav_tail(self):
        # <bav-tail>
        prod =
self.get_production('<bav-tai
l>')
        if prod == 265:
            self.eat(',')
            self.bool_arr1()
        elif prod == 266:
            pass # Lambda
        else:

self.error_invalid_token('<ba
v-tail>')
```

```
elif prod == 273:

def bool_arr_val(self):
    # <bool-arr-val>
    prod =
self.get_production('<bool-ar
r-val>')
    if prod == 267:
        self.bool_ope_arr()
        self.bool_exp_arr()
    else:

self.error_invalid_token('<bo
ol-arr-val>')

def bool_ope_arr(self):
    # <bool-ope-arr>
    prod =
self.get_production('<bool-o
pe-arr>')
    if prod == 268:
        self.eat('id')
        if
self.current_token.type in
['{', '$', '(']:
            self.id_tail()
            self.bool_exp2_arr()
        elif prod == 269:
            self.eat('(')
            self.value_grp()
            self.eat(')')
            self.bool_exp2_arr()
        elif prod == 270:
            self.bool()
            self.bool_eq_arr()
        elif prod == 271:
            self.digit()
            self.bool_arith()
            self.releq_arr()
        elif prod == 272:
            self.eat('PARCH-lit')
            self.eq_op()
            self.parch_val()

elif prod == 273:
    self.eat('SCROLL-lit')
    if
self.current_token.type in
['{']:
        self.scr_char()
        self.eq_op()
        self.scroll_val()
    else:

self.error_invalid_token('<bo
ol-ope-arr>')

def bool_eq_arr(self):
    # <bool-eq-arr>
    prod =
self.get_production('<bool-e
q-arr>')
    if prod == 274:
        self.eq_op()
        self.bool_ope_arr()
    elif prod == 275:
        pass # Lambda
    else:

self.error_invalid_token('<bo
ol-eq-arr>')

def releq_arr(self):
    # <releq-arr>
    prod =
self.get_production('<releq-a
rr>')
    if prod == 276:
        self.rel()
        self.bool_arith2_arr()
        self.bool_eq_arr()
    elif prod == 277:
        self.eq_digit()
        self.bool_arith3_arr()
    else:

self.error_invalid_token('<re
leq-arr>')

def bool_arith2_arr(self):
    # <bool-arith2-arr>
    prod =
self.get_production('<bool-ar
ith2-arr>')
    if prod == 278:
        self.dime_arith()
        self.bool_arith2_arr()
    elif prod == 279:
        pass # Lambda
    else:

self.error_invalid_token('<bo
ol-arith2-arr>')

def bool_arith3_arr(self):
    # <bool-arith3-arr>
    prod =
self.get_production('<bool-ar
ith3-arr>')
    if prod == 280:
        self.dime_arith()
        self.bool_arith3_arr()
    elif prod == 281:
        pass # Lambda
    else:

self.error_invalid_token('<bo
ol-arith3-arr>')

def bool_exp2_arr(self):
    # <bool-exp2-arr>
    prod =
self.get_production('<bool-e
xp2-arr>')
    if prod == 282:
        self.dime_arith()
        self.bool_arith()
```



```
        self.releq_arr()
    elif prod == 283:
        self.rel()
        self.bool_arith2_arr()
        self.bool_eq_arr()
    elif prod == 284:
        self.eq_op()
        self.eq_ope_arr()
    elif prod == 285:
        pass # Lambda
    else:
self.error_invalid_token('<bo
ol-exp2-arr>')

    def eq_ope_arr(self):
        # <eq-ope-arr>
        prod =
self.get_production('<eq-ope
-arr>')
        if prod == 286:
            self.eat('id')
            if
self.current_token.type in
['{', '$', '[']:
                self.id_tail()
                self.bool_arith4_arr()
                self.bool_rel_arr()
            elif prod == 287:
                self.eat('(')
                self.value_grp()
                self.eat(')')
                self.bool_arith4_arr()
                self.bool_rel_arr()
            elif prod == 288:
                self.digit()
                self.bool_arith4_arr()
                self.bool_rel_arr()
            elif prod == 289:
                self.eq_ope1()
            else:
self.error_invalid_token('<eq
-ope-arr>')

        def bool_arith4_arr(self):
            # <bool-arith4-arr>
            prod =
self.get_production('<bool-ar
ith4-arr>')
            if prod == 290:
                self.dime_arith()
                self.bool_arith4_arr()
            elif prod == 291:
                pass # Lambda
            else:
self.error_invalid_token('<bo
ol-arith4-arr>')

        def bool_rel_arr(self):
            # <bool-rel-arr>
            prod =
self.get_production('<bool-re
l-arr>')
            if prod == 292:
                self.rel()
                self.bool_arith3_arr()
            elif prod == 293:
                pass # Lambda
            else:
self.error_invalid_token('<bo
ol-rel-arr>')

        def bool_exp_arr(self):
            # <bool-exp-arr>
            prod =
self.get_production('<bool-e
xp-arr>')
            if prod == 294:
                self.log_op()
                self.bool_ope_arr()
                self.bool_exp_arr()
            elif prod == 295:
                pass # Lambda
            else:
self.error_invalid_token('<bo
ol-exp-arr>')

        def bool_arr2_tail(self):
            # <bool-arr2-tail>
            prod =
self.get_production('<bool-ar
r2-tail>')
            if prod == 296:
                self.eat('=')
                self.eat('[')
                self.bool_arr2()
                self.eat(']')
            elif prod == 297:
                pass # Lambda
            else:
self.error_invalid_token('<bo
ol-arr2-tail>')

        def bool_arr2(self):
            # <bool-arr2>
            prod =
self.get_production('<bool-ar
r2>')
            if prod == 298:
                self.eat('[')
                self.bool_arr1()
                self.eat(']')
                self.bav2_tail()
            else:
self.error_invalid_token('<bo
ol-arr2>')

        def bav2_tail(self):
            # <bav2-tail>
```

```

    prod =
self.get_production('<bav2-tail>')
    if prod == 299:
        self.eat(',')
        self.bool_arr2()
    elif prod == 300:
        pass # Lambda
    else:
self.error_invalid_token('<bav2-tail>')

    def bool_func(self):
        # <bool-func>
        prod =
self.get_production('<bool-func>')
        if prod == 301:
            self.eat('(')
            self.params()
            self.eat(')')
            self.eat('[')
            self.local_dec()
            self.ret_stmnts()
            self.eat('BACK')
            self.bool_ret_val()
            self.eat('!!')
            self.eat(']')
            self.sub_func()
        else:
self.error_invalid_token('<bool-func>')

    def bool_ret_val(self):
        # <bool-ret-val>
        prod =
self.get_production('<bool-ret-val>')
        if prod == 302:
            self.bool_ope_ret()

```

```

        self.bool_exp_ret()
    else:
self.error_invalid_token('<bool-ret-val>')

    def bool_ope_ret(self):
        # <bool-ope-ret>
        prod =
self.get_production('<bool-ope-ret>')
        if prod == 303:
            self.eat('id')
            if
self.current_token.type in
['{', '$', '(']:
                self.id_tail()
                self.bool_exp2_ret()
            elif prod == 304:
                self.eat('(')
                self.value_grp()
                self.eat(')')
                self.bool_exp2_ret()
            elif prod == 305:
                self.bool()
                self.bool_eq_ret()
            elif prod == 306:
                self.digit()
                self.bool_arith()
                self.releq_ret()
            elif prod == 307:
                self.eat('PARCH-lit')
                self.eq_op()
                self.parch_val()
            elif prod == 308:
self.eat('SCROLL-lit')
                if
self.current_token.type in
['{']:
                    self.scr_char()
                    self.eq_op()

```

```

        self.scroll_val()
    else:
self.error_invalid_token('<bool-ope-ret>')

    def bool_eq_ret(self):
        # <bool-eq-ret>
        prod =
self.get_production('<bool-eq-ret>')
        if prod == 309:
            self.eq_op()
            self.bool_ope_ret()
        elif prod == 310:
            pass # Lambda
        else:
self.error_invalid_token('<bool-eq-ret>')

    def releq_ret(self):
        # <releq-ret>
        prod =
self.get_production('<releq-ret>')
        if prod == 311:
            self.rel()
            self.bool_arith2_ret()
            self.bool_eq_ret()
        elif prod == 312:
            self.eq_digit()
            self.bool_arith3_ret()
        else:
self.error_invalid_token('<releq-ret>')

    def bool_arith2_ret(self):
        # <bool-arith2-ret>

```

```
prod =
self.get_production('<bool-ar
ith2-ret>')
    if prod == 313:
        self.dime_arith()
        self.bool_arith2_ret()
    elif prod == 314:
        pass # Lambda
    else:

self.error_invalid_token('<bo
ol-arith2-ret>')

def bool_arith3_ret(self):
    # <bool-arith3-ret>
    prod =
self.get_production('<bool-ar
ith3-ret>')
    if prod == 315:
        self.dime_arith()
        self.bool_arith3_ret()
    elif prod == 316:
        pass # Lambda
    else:

self.error_invalid_token('<bo
ol-arith3-ret>')

def bool_exp2_ret(self):
    # <bool-exp2-ret>
    prod =
self.get_production('<bool-e
xp2-ret>')
    if prod == 317:
        self.dime_arith()
        self.bool_arith()
        self.releq_ret()
    elif prod == 318:
        self.rel()
        self.bool_arith2_ret()
        self.bool_eq_ret()
    elif prod == 319:
```

```
self.eq_op()
self.eq_ope_ret()
elif prod == 320:
    pass # Lambda
else:

self.error_invalid_token('<bo
ol-exp2-ret>')

def eq_ope_ret(self):
    # <eq-ope-ret>
    prod =
self.get_production('<eq-ope
-ret>')
    if prod == 321:
        self.eat('id')
        if
self.current_token.type in
['{', '$', '(']:
            self.id_tail()
            self.bool_arith4_ret()
            self.bool_rel_ret()
        elif prod == 322:
            self.eat('(')
            self.value_grp()
            self.eat(')')
            self.bool_arith4_ret()
            self.bool_rel_ret()
        elif prod == 323:
            self.digit()
            self.bool_arith4_ret()
            self.bool_rel_ret()
        elif prod == 324:
            self.eq_ope1()
        else:

self.error_invalid_token('<eq
-ope-ret>')

def bool_arith4_ret(self):
    # <bool-arith4-ret>
```

```
prod =
self.get_production('<bool-ar
ith4-ret>')
    if prod == 325:
        self.dime_arith()
        self.bool_arith4_ret()
    elif prod == 326:
        pass # Lambda
    else:

self.error_invalid_token('<bo
ol-arith4-ret>')

def bool_rel_ret(self):
    # <bool-rel-ret>
    prod =
self.get_production('<bool-re
l-ret>')
    if prod == 327:
        self.rel()
        self.bool_arith3_ret()
    elif prod == 328:
        pass # Lambda
    else:

self.error_invalid_token('<bo
ol-rel-ret>')

def bool_exp_ret(self):
    # <bool-exp-ret>
    prod =
self.get_production('<bool-e
xp-ret>')
    if prod == 329:
        self.log_op()
        self.bool_ope_ret()
        self.bool_exp_ret()
    elif prod == 330:
        pass # Lambda
    else:
```

```

self.error_invalid_token('<bol-exp-ret>')

def params(self):
    # <params>
    prod =
self.get_production('<params>')

    if prod == 331:
        self.d_type()
        self.eat('id')
        self.param_mult()
    elif prod == 332:
        pass # Lambda
    else:

self.error_invalid_token('<params>')

def param_mult(self):
    # <param-mult>
    prod =
self.get_production('<param-mult>')

    if prod == 333:
        self.eat(',')
        self.d_type()
        self.eat('id')
        self.param_mult()
    elif prod == 334:
        pass # Lambda
    else:

self.error_invalid_token('<param-mult>')

def d_type(self):
    # <d-type>
    prod =
self.get_production('<d-type>')

    if prod == 335:
        self.eat('COIN')
    elif prod == 336:
        self.eat('DIME')
    elif prod == 337:
        self.eat('PARCH')
    elif prod == 338:
        self.eat('SCROLL')
    elif prod == 339:
        self.eat('BOOL')
    else:

self.error_invalid_token('<d-type>')

def ret_stmnts(self):
    # <ret-stmnts>
    prod =
self.get_production('<ret-stmnts>')

    if prod == 340:
        self.statements()
        self.ret_stmnts()
    elif prod == 341:
        pass # Lambda
    else:

self.error_invalid_token('<ret-stmnts>')

def id_tail(self):
    # <id-tail>
    prod =
self.get_production('<id-tail>')

    if prod == 342:
        self.eat('{')
        self.index()
        self.eat('}')
    if
self.current_token.type in
['{']:

    self.elmt2()
    elif prod == 343:
        self.eat('$')
        self.eat('id')
    elif prod == 344:
        self.eat('(')
        self.args()
        self.eat(')')
    elif prod == 345:
        pass # Lambda
    else:

self.error_invalid_token('<id-tail>')

def elmt2(self):
    # <elmt2>
    prod =
self.get_production('<elmt2>')

    if prod == 346:
        self.eat('{')
        self.index()
        self.eat('}')
    elif prod == 347:
        pass # Lambda
    else:

self.error_invalid_token('<elmt2>')

def args(self):
    # <args>
    prod =
self.get_production('<args>')

    if prod == 348:
        self.value()
        self.args_mult()
    elif prod == 349:
        pass # Lambda
    else:

```

```
self.error_invalid_token('<ar
gs>')

def args_mult(self):
    # <args-mult>
    prod =
self.get_production('<args-m
ult>')
    if prod == 350:
        self.eat(',')
        self.value()
        self.args_mult()
    elif prod == 351:
        pass # Lambda
    else:

self.error_invalid_token('<ar
gs-mult>')

def value(self):
    # <value>
    prod =
self.get_production('<value>'
)
    if prod == 352:
        self.eat('id')
        if
self.current_token.type in
['{', '$', '(']:
            self.id_tail()
            self.exp()
        elif prod == 353:
            self.eat('(')
            self.value_grp()
            self.eat(')')
            self.exp()
        elif prod == 354:
            self.digit()
            self.digit_tail()
        elif prod == 355:
            self.eat('PARCH-lit')

            self.parch_tail()
            elif prod == 356:

self.eat('SCROLL-lit')
            if
self.current_token.type in
['{']:
                self.scr_char()
                self.scroll_tail()
            elif prod == 357:
                self.bool()
                self.logeq_var()
            else:

self.error_invalid_token('<va
lue>')

def digit_tail(self):
    # <digit-tail>
    prod =
self.get_production('<digit-ta
il>')
    if prod == 358:
        self.arith_var()
        self.releq_var()
    elif prod == 359:
        pass # Lambda
    else:

self.error_invalid_token('<di
git-tail>')

def arith_var(self):
    # <arith-var>
    prod =
self.get_production('<arith-v
ar>')
    if prod == 360:
        self.dime_arith()
        self.arith_var()
    elif prod == 361:
        pass # Lambda

        else:

self.error_invalid_token('<re
leq-var>')

def releq_var(self):
    # <releq-var>
    prod =
self.get_production('<releq-v
ar>')
    if prod == 362:
        self.rel()
        self.arith_var2()
        self.logeq_var()
    elif prod == 363:
        self.eq_digit()
        self.arith_var3()
        self.log_var()
    elif prod == 364:
        pass # Lambda
    else:

self.error_invalid_token('<re
leq-var>')

def arith_var2(self):
    # <arith-var2>
    prod =
self.get_production('<arith-v
ar2>')
    if prod == 365:
        self.dime_arith()
        self.arith_var2()
    elif prod == 366:
        pass # Lambda
    else:

self.error_invalid_token('<ar
ith-var2>')

def arith_var3(self):
    # <arith-var3>
```

```
        prod =
self.get_production('<arith-v
ar3>')
    if prod == 367:
        self.dime_arith()
        self.arith_var3()
    elif prod == 368:
        pass # Lambda
    else:

self.error_invalid_token('<ar
ith-var3>')

    def logeq_var(self):
        # <logeq-var>
        prod =
self.get_production('<logeq-
var>')
    if prod == 369:
        self.logeq_op()
        self.log_ope()
        self.log_var()
    elif prod == 370:
        pass # Lambda
    else:

self.error_invalid_token('<lo
geq-var>')

    def logeq_op(self):
        # <logeq-op>
        prod =
self.get_production('<logeq-
op>')
    if prod == 371:
        self.log_op()
    elif prod == 372:
        self.eq_op()
    else:

self.error_invalid_token('<lo
geq-op>')
```

```
    def log_var(self):
        # <log-var>
        prod =
self.get_production('<log-var
>')
    if prod == 373:
        self.log_op()
        self.log_ope()
        self.log_var()
    elif prod == 374:
        pass # Lambda
    else:

self.error_invalid_token('<lo
g-var>')

    def parch_tail(self):
        # <parch-tail>
        prod =
self.get_production('<parch-t
ail>')
    if prod == 375:
        self.eq_op()
        self.eat('PARCH-lit')
        self.log_var()
    elif prod == 376:
        pass # Lambda
    else:

self.error_invalid_token('<pa
rch-tail>')

    def scroll_tail(self):
        # <scroll-tail>
        prod =
self.get_production('<scroll-t
ail>')
    if prod == 377:
        self.concat_var()
    elif prod == 378:
        self.eq_op()
```

```
        self.scroll_val()
        self.log_var()
    elif prod == 379:
        pass # Lambda
    else:

self.error_invalid_token('<sc
roll-tail>')

    def concat_var(self):
        # <concat-var>
        prod =
self.get_production('<concat
-var>')
    if prod == 380:
        self.eat('&')
        self.scroll_val()
        self.concat_var()
    elif prod == 381:
        pass # Lambda
    else:

self.error_invalid_token('<co
ncat-var>')

    def exp(self):
        # <exp>
        prod =
self.get_production('<exp>')
    if prod == 382:
        self.dime_arith()
        self.arith_var()
        self.releq_var()
    elif prod == 383:
        self.rel()
        self.arith_var2()
        self.logeq_var()
    elif prod == 384:
        self.log_op()
        self.log_ope()
        self.log_var()
    elif prod == 385:
```

```
        self.eq_op()
        self.eq_var()
        self.log_var()
    elif prod == 386:
        self.eat('&')
        self.scroll_val()
        self.concat_var()
    elif prod == 387:
        pass # Lambda
    else:
        self.error_invalid_token('<exp>')

    def log_ope(self):
        # <log-ope>
        prod =
    self.get_production('<log-ope>')
        if prod == 388:
            self.eat('id')
            if
    self.current_token.type in
    ['{', '$', '(']:
            self.id_tail()
            self.log_exp()
        elif prod == 389:
            self.eat('(')
            self.value_grp()
            self.eat(')')
            self.log_exp()
        elif prod == 390:
            self.bool()
            self.log_booleq()
        elif prod == 391:
            self.digit()
            self.bool_arith()
            self.log_releq()
        elif prod == 392:
            self.eat('PARCH-lit')
            self.eq_op()
            self.parch_val()

        elif prod == 393:
            self.eat('SCROLL-lit')
            if
            self.current_token.type in
            ['{']:
                self.scr_char()
                self.eq_op()
                self.scroll_val()
            else:
                self.error_invalid_token('<log-ope>')

    def log_booleq(self):
        # <log-booleq>
        prod =
    self.get_production('<log-booleq>')
        if prod == 394:
            self.eq_op()
            self.log_ope()
        elif prod == 395:
            pass # Lambda
        else:
            self.error_invalid_token('<log-booleq>')

    def log_releq(self):
        # <log-releq>
        prod =
    self.get_production('<log-rel
eq>')
        if prod == 396:
            self.rel()
            self.arith_var2()
            self.log_booleq()
        elif prod == 397:
            self.eq_digit()
            self.arith_var3()
        else:

self.error_invalid_token('<log-exp>')

    def log_exp(self):
        # <log-exp>
        prod =
    self.get_production('<log-exp>')
        if prod == 398:
            self.dime_arith()
            self.arith_var()
            self.log_releq()
        elif prod == 399:
            self.rel()
            self.arith_var2()
            self.log_booleq()
        elif prod == 400:
            self.eq_op()
            self.eq_var()
        elif prod == 401:
            pass # Lambda
        else:
            self.error_invalid_token('<log-exp>')

    def eq_var(self):
        # <eq-var>
        prod =
    self.get_production('<eq-var>')
        if prod == 402:
            self.eat('id')
            if
    self.current_token.type in
    ['{', '$', '(']:
                self.id_tail()
                self.arith_var4()
                self.rel_var()
            elif prod == 403:
                self.eat('(')
```

```
        self.value_grp()
        self.eat('(')
        self.arith_var4()
        self.rel_var()
    elif prod == 404:
        self.digit()
        self.arith_var4()
        self.rel_var()
    elif prod == 405:
        self.eq_opel()
    else:

self.error_invalid_token('<eq
-var>')

    def arith_var4(self):
        # <arith-var4>
        prod =
self.get_production('<arith-v
ar4>')
        if prod == 406:
            self.dime_arith()
            self.arith_var4()
        elif prod == 407:
            pass # Lambda
        else:

self.error_invalid_token('<ar
ith-var4>')

    def rel_var(self):
        # <rel-var>
        prod =
self.get_production('<rel-var
>')
        if prod == 408:
            self.rel()
            self.arith_var3()
        elif prod == 409:
            pass # Lambda
        else:

        self.error_invalid_token('<re
l-var>')

        def value_grp(self):
            # <value-grp>
            prod =
self.get_production('<value-
grp>')
            if prod == 410:
                self.eat('id')
                if

self.current_token.type in
['{', '$', '(']:
                    self.id_tail()
                    self.exp_grp()
                elif prod == 411:
                    self.eat('(')
                    self.value_grp()
                    self.eat(')')
                    self.exp_grp()
                elif prod == 412:
                    self.digit()
                    self.digit_tail_grp()
                elif prod == 413:
                    self.eat('PARCH-lit')
                    self.parch_tail_grp()
                elif prod == 414:

self.eat('SCROLL-lit')
                if

self.current_token.type in
['{']:
                    self.scr_char()
                    self.scroll_tail_grp()
                elif prod == 415:
                    self.bool()
                    self.logeq_grp()
                else:

self.error_invalid_token('<va
lue-grp>')

    def digit_tail_grp(self):
        # <digit-tail-grp>
        prod =
self.get_production('<digit-ta
il-grp>')
        if prod == 416:
            self.arith_grp()
            self.releqvar_grp()
        elif prod == 417:
            pass # Lambda
        else:

self.error_invalid_token('<di
git-tail-grp>')

    def arith_grp(self):
        # <arith-grp>
        prod =
self.get_production('<arith-g
rp>')
        if prod == 418:
            self.dime_arith()
            self.arith_grp()
        elif prod == 419:
            pass # Lambda
        else:

self.error_invalid_token('<ar
ith-grp>')

    def releqvar_grp(self):
        # <releqvar-grp>
        prod =
self.get_production('<releqv
ar-grp>')
        if prod == 420:
            self.rel()
            self.arith_grp2()
            self.logeq_grp()
        elif prod == 421:
            self.eq_digit()
```

```

    self.arith_grp3()
    self.bool_exp_grp()
    elif prod == 422:
        pass # Lambda
    else:

self.error_invalid_token('<re
leqvar-grp>')

    def arith_grp2(self):
        # <arith-grp2>
        prod =
self.get_production('<arith-g
rp2>')
        if prod == 423:
            self.dime_arith()
            self.arith_grp2()
        elif prod == 424:
            pass # Lambda
        else:

self.error_invalid_token('<ar
ith-grp2>')

    def arith_grp3(self):
        # <arith-grp3>
        prod =
self.get_production('<arith-g
rp3>')
        if prod == 425:
            self.dime_arith()
            self.arith_grp3()
        elif prod == 426:
            pass # Lambda
        else:

self.error_invalid_token('<ar
ith-grp3>')

    def logeq_grp(self):
        # <logeq-grp>

        prod =
self.get_production('<logeq-
grp>')
        if prod == 427:
            self.logeq_op()
            self.bool_ope_grp()
            self.bool_exp_grp()
        elif prod == 428:
            pass # Lambda
        else:

self.error_invalid_token('<lo
geq-grp>')

    def parch_tail_grp(self):
        # <parch-tail-grp>
        prod =
self.get_production('<parch-t
ail-grp>')
        if prod == 429:
            self.eq_op()
            self.eat('PARCH-lit')
            self.bool_exp_grp()
        elif prod == 430:
            pass # Lambda
        else:

self.error_invalid_token('<pa
rch-tail-grp>')

    def scroll_tail_grp(self):
        # <scroll-tail-grp>
        prod =
self.get_production('<scroll-t
ail-grp>')
        if prod == 431:
            self.concat_str()
        elif prod == 432:
            self.eq_op()
            self.scroll_val()
            self.bool_exp_grp()
        elif prod == 433:

        pass # Lambda
    else:

self.error_invalid_token('<sc
roll-tail-grp>')

    def concat_grp(self):
        # <concat-grp>
        prod =
self.get_production('<concat
-grp>')
        if prod == 434:
            self.eat('&')
            self.scroll_val()
            self.concat_grp()
        elif prod == 435:
            pass # Lambda
        else:

self.error_invalid_token('<co
ncat-grp>')

    def exp_grp(self):
        # <exp-grp>
        prod =
self.get_production('<exp-gr
p>')
        if prod == 436:
            self.dime_arith()
            self.arith_grp()
            self.releqvar_grp()
        elif prod == 437:
            self.rel()
            self.arith_grp2()
            self.logeq_grp()
        elif prod == 438:
            self.log_op()
            self.bool_ope_grp()
            self.bool_exp_grp()
        elif prod == 439:
            self.eq_op()
            self.eq_ope_grp()

```

```

    self.bool_exp_grp()
    elif prod == 440:
        self.eat('&')
        self.scroll_val()
        self.concat_grp()
    elif prod == 441:
        pass # Lambda
    else:

self.error_invalid_token('<exp-grp>')

    def const(self):
        # <const>
        prod =
self.get_production('<const>')
    )
        if prod == 442:
            self.eat('LOCKE')
            self.const_init()
            self.eat('!!')
        else:

self.error_invalid_token('<const>')

    def const_init(self):
        # <const-init>
        prod =
self.get_production('<const-init>')
        if prod == 443:
            self.eat('COIN')
            self.coin_locke()

self.coin_locke_mult()
        elif prod == 444:
            self.eat('DIME')
            self.dime_locke()

self.dime_locke_mult()
        elif prod == 445:

    self.eat('PARCH')
    self.parch_locke()

self.parch_locke_mult()
        elif prod == 446:
            self.eat('SCROLL')
            self.scroll_locke()

self.scroll_locke_mult()
        elif prod == 447:
            self.eat('BOOL')
            self.bool_locke()

self.bool_locke_mult()
        else:

self.error_invalid_token('<const-init>')

    def coin_locke(self):
        # <coin-locke>
        prod =
self.get_production('<coin-locke>')
        if prod == 448:
            self.eat('id')
            self.eat('=')
            self.eat('COIN-lit')
        else:

self.error_invalid_token('<coin-locke>')

    def coin_locke_mult(self):
        # <coin-locke-mult>
        prod =
self.get_production('<coin-locke-mult>')
        if prod == 449:
            self.eat(',')
            self.coin_locke()

self.coin_locke_mult()
        elif prod == 450:
            pass # Lambda
        else:

self.error_invalid_token('<coin-locke-mult>')

    def dime_locke(self):
        # <dime-locke>
        prod =
self.get_production('<dime-locke>')
        if prod == 451:
            self.eat('id')
            self.eat('=')
            self.locke_digit()
        else:

self.error_invalid_token('<dime-locke>')

    def locke_digit(self):
        # <locke-digit>
        prod =
self.get_production('<locke-digit>')
        if prod == 452:
            self.eat('COIN-lit')
        elif prod == 453:
            self.eat('DIME-lit')
        else:

self.error_invalid_token('<locke-digit>')

    def
dime_locke_mult(self):
        # <dime-locke-mult>

```

```
prod =
self.get_production('<dime-l
ocke-mult>')
    if prod == 454:
        self.eat(',')
        self.dime_locke()

self.dime_locke_mult()
    elif prod == 455:
        pass # Lambda
    else:

self.error_invalid_token('<di
me-locke-mult>')

def parch_locke(self):
    # <parch-locke>
    prod =
self.get_production('<parch-l
ocke>')
    if prod == 456:
        self.eat('id')
        self.eat('=')
        self.eat('PARCH-lit')
    else:

self.error_invalid_token('<pa
rch-locke>')

def
parch_locke_mult(self):
    # <parch-locke-mult>
    prod =
self.get_production('<parch-l
ocke-mult>')
    if prod == 457:
        self.eat(',')
        self.parch_locke()

self.parch_locke_mult()
    elif prod == 458:
        pass # Lambda
```

```
else:

self.error_invalid_token('<pa
rch-locke-mult>')

def scroll_locke(self):
    # <scroll-locke>
    prod =
self.get_production('<scroll-l
ocke>')
    if prod == 459:
        self.eat('id')
        self.eat('=')

self.eat('SCROLL-lit')
    if
self.current_token.type in
['{']:
        self.scr_id()
    else:

self.error_invalid_token('<sc
roll-locke>')

def scr_id(self):
    # <scr-id>
    prod =
self.get_production('<scr-id>
')
    if prod == 460:
        self.eat('{')
        self.eat('COIN-lit')
        self.eat('}')
    elif prod == 461:
        pass # Lambda
    else:

self.error_invalid_token('<sc
r-id>')

def
scroll_locke_mult(self):
```

```
# <scroll-locke-mult>
prod =
self.get_production('<scroll-l
ocke-mult>')
    if prod == 462:
        self.eat(',')
        self.scroll_locke()

self.scroll_locke_mult()
    elif prod == 463:
        pass # Lambda
    else:

self.error_invalid_token('<sc
roll-locke-mult>')

def bool_locke(self):
    # <bool-locke>
    prod =
self.get_production('<bool-lo
cke>')
    if prod == 464:
        self.eat('id')
        self.eat('=')
        self.locke_bool()
    else:

self.error_invalid_token('<bo
ol-locke>')

def locke_bool(self):
    # <locke-bool>
    prod =
self.get_production('<locke-
bool>')
    if prod == 465:
        self.eat('AYE')
    elif prod == 466:
        self.eat('NAY')
    else:
```

```
self.error_invalid_token('<locke-bool>')

def bool_locke_mult(self):
    # <bool-locke-mult>
    prod =
self.get_production('<bool-locke-mult>')
    if prod == 467:
        self.eat(',')
        self.bool_locke()

self.bool_locke_mult()
    elif prod == 468:
        pass # Lambda
    else:

self.error_invalid_token('<bool-locke-mult>')

def struct(self):
    # <struct>
    prod =
self.get_production('<struct>')
    if prod == 469:
        self.eat('MAST')
        self.eat('id')
        self.eat('[')
        self.mem_dec()
        self.mem_dec_tail()
        self.eat(']')
        self.eat('!!')
        self.struct()
    elif prod == 470:
        pass # Lambda
    else:

self.error_invalid_token('<struct>')

def mem_dec(self):
    # <mem-dec>
    prod =
self.get_production('<mem-dec>')
    if prod == 471:
        self.d_type()
        self.eat('id')
        self.mem_mult()
        self.eat('!!')
    else:

self.error_invalid_token('<mem-dec>')

def mem_mult(self):
    # <mem-mult>
    prod =
self.get_production('<mem-mult>')
    if prod == 472:
        self.eat(',')
        self.eat('id')
        self.mem_mult()
    elif prod == 473:
        pass # Lambda
    else:

self.error_invalid_token('<mem-mult>')

def mem_dec_tail(self):
    # <mem-dec-tail>
    prod =
self.get_production('<mem-dec-tail>')
    if prod == 474:
        self.mem_dec()
        self.mem_dec_tail()
    elif prod == 475:
        pass # Lambda
    else:

self.error_invalid_token('<mem-dec-tail>')

def sub_func(self):
    # <sub-func>
    prod =
self.get_production('<sub-func>')
    if prod == 476:
        self.return_func()
    elif prod == 477:
        self.nonreturn_func()
    elif prod == 478:
        pass # Lambda
    else:

self.error_invalid_token('<sub-func>')

def return_func(self):
    # <return-func>
    prod =
self.get_production('<return-func>')
    if prod == 479:
        self.eat('COIN')
        self.eat('id')
        self.coin_func()
    elif prod == 480:
        self.eat('DIME')
        self.eat('id')
        self.dime_func()
    elif prod == 481:
        self.eat('PARCH')
        self.eat('id')
        self.parch_func()
    elif prod == 482:
        self.eat('SCROLL')
        self.eat('id')
        self.scroll_func()
    elif prod == 483:
```

```
        self.eat('BOOL')
        self.eat('id')
        self.bool_func()
    else:

self.error_invalid_token('<return-func>')

    def nonreturn_func(self):
        # <nonreturn-func>
        prod =
self.get_production('<nonreturn-func>')
        if prod == 484:
            self.eat('ABYSS')
            self.eat('id')
            self.eat('(')
            self.params()
            self.eat(')')
            self.eat('[')
            self.ahoy_local_dec()
            self.nonret_stmts()
            self.nonret_back()
            self.eat(']')
            self.sub_func()
        else:

self.error_invalid_token('<nonreturn-func>')

    def nonret_stmts(self):
        # <nonret-stmts>
        prod =
self.get_production('<nonret-stmts>')
        if prod == 485:
            self.nonret_stmt()
            self.nonret_tail()
        else:

self.error_invalid_token('<nonret-stmts>')

def nonret_tail(self):
    # <nonret-tail>
    prod =
self.get_production('<nonret-tail>')
    if prod == 486:
        self.nonret_stmts()
    elif prod == 487:
        pass # Lambda
    else:

self.error_invalid_token('<nonret-tail>')

    def nonret_back(self):
        # <nonret-back>
        prod =
self.get_production('<nonret-back>')
        if prod == 488:
            self.eat('BACK')
            self.eat('!!')
        elif prod == 489:
            pass # Lambda
        else:

self.error_invalid_token('<nonret-back>')

    def nonret_stmt(self):
        # <nonret-stmnt>
        prod =
self.get_production('<nonret-stmnt>')
        if prod == 490:
            self.eat('id')
            self.assign_tail()
            self.eat('!!')
        elif prod == 491:
            self.eat('ASK')
            self.eat('(')

self.eat('SCROLL-lit')
    self.eat(',')
    self.addr()
    self.eat(')')
    self.eat('!!')
    elif prod == 492:
        self.eat('ECHO')
        self.eat('(')

self.eat('SCROLL-lit')
    self.args_mult()
    self.eat(')')
    self.eat('!!')
    elif prod == 493:
        self.eat('LOOK')
        self.eat('(')
        self.condition()
        self.eat(')')
        self.eat('[')
        self.look_body()
        self.jump_stmt()
        self.eat(']')

self.nonret_look_tail()
    elif prod == 494:
        self.eat('CHART')
        self.eat('(')
        self.chart_cond()
        self.eat(')')
        self.eat('[')
        self.courses()
        self.course_tail()
        self.adrift_case()
        self.eat(']')
    elif prod == 495:
        self.eat('HOIST')
        self.eat('(')
        self.hoist_init()
        self.eat('!!')
        self.hoist_cond()
        self.eat('!!')
```

```

    self.hoist_upd()
    self.eat('')
    self.eat('[')
    self.look_body()
    self.jump_stmnt()
    self.eat(']')
    elif prod == 496:
        self.eat('HEAVE')
        self.eat('')
        self.condition()
        self.eat('')
        self.eat('[')
        self.look_body()
        self.jump_stmnt()
        self.eat(']')
    elif prod == 497:
        self.eat('HAUL')
        self.eat('[')
        self.look_body()
        self.jump_stmnt()
        self.eat(']')
        self.eat('HEAVE')
        self.eat('')
        self.condition()
        self.eat('')
        self.eat('!!')
    elif prod == 498:
        self.unary_exp()
        self.eat('!!')
    else:
        self.error_invalid_token('<nonret-stmnt>')

    def nonret_look_tail(self):
        # <nonret-look-tail>
        prod =
        self.get_production('<nonret-look-tail>')
        if prod == 499:

self.eat('DROPLOOK')

    self.eat('(')
    self.condition()
    self.eat(')')
    self.eat('[')
    self.look_body()
    self.jump_stmnt()
    self.eat(']')

    self.nonret_look_tail()
    elif prod == 500:
        self.eat('DROP')
        self.eat('[')
        self.look_body()
        self.jump_stmnt()
        self.eat(']')
    elif prod == 501:
        pass # Lambda
    else:
        self.error_invalid_token('<nonret-look-tail>')

    def local_dec(self):
        # <local-dec>
        prod =
        self.get_production('<local-dec>')
        if prod == 502:
            self.var_arr()
            self.local_dec()
        elif prod == 503:
            self.struct_dec()
        elif prod == 504:
            pass # Lambda
        else:
            self.error_invalid_token('<local-dec>')

    def var_arr(self):
        # <var-arr>

    prod =
    self.get_production('<var-arr>')
    if prod == 505:
        self.eat('COIN')
        self.eat('id')
        self.coin_var_arr()
        self.eat('!!')
    elif prod == 506:
        self.eat('DIME')
        self.eat('id')
        self.dime_var_arr()
        self.eat('!!')
    elif prod == 507:
        self.eat('PARCH')
        self.eat('id')
        self.parch_var_arr()
        self.eat('!!')
    elif prod == 508:
        self.eat('SCROLL')
        self.eat('id')
        self.scroll_var_arr()
        self.eat('!!')
    elif prod == 509:
        self.eat('BOOL')
        self.eat('id')
        self.bool_var_arr()
        self.eat('!!')
    else:
        self.error_invalid_token('<var-arr>')

    def struct_dec(self):
        # <struct-dec>
        prod =
        self.get_production('<struct-dec>')
        if prod == 510:
            self.eat('MAST')
            self.eat('id')
            self.eat('id')

```

```
        self.str_dec_init()
        self.eat('!!')
        self.struct_dec()
    elif prod == 511:
        pass # Lambda
    else:

self.error_invalid_token('<struct-dec>')

    def str_dec_init(self):
        # <str-dec-init>
        prod =
self.get_production('<str-dec-init>')
        if prod == 512:
            self.eat(',')
            self.eat('id')
            self.str_dec_tail()
        elif prod == 513:
            self.eat('=')
            self.eat('[')
            self.str_val()
            self.str_val_tail()
            self.eat(']')
        elif prod == 514:
            pass # Lambda
        else:

self.error_invalid_token('<str-dec-init>')

    def str_dec_tail(self):
        # <str-dec-tail>
        prod =
self.get_production('<str-dec-tail>')
        if prod == 515:
            self.eat(',')
            self.eat('id')
            self.str_dec_tail()
        elif prod == 516:

        pass # Lambda
    else:

self.error_invalid_token('<str-dec-tail>')

    def str_val(self):
        # <str-val>
        prod =
self.get_production('<str-val>')
        if prod == 517:
            self.value_str()
        elif prod == 518:
            self.eat('$')
            self.eat('id')
            self.eat('=')
            self.value_str()
        else:

self.error_invalid_token('<str-val>')

    def str_val_tail(self):
        # <str-val-tail>
        prod =
self.get_production('<str-val-tail>')
        if prod == 519:
            self.eat(',')
            self.str_val()
            self.str_val_tail()
        elif prod == 520:
            pass # Lambda
        else:

self.error_invalid_token('<str-val-tail>')

    def value_str(self):
        # <value-str>

        prod =
self.get_production('<value-str>')
        if prod == 521:
            self.eat('id')
            if
self.current_token.type in
['{', '$', '(']:
                self.id_tail()
                self.exp_str()
            elif prod == 522:
                self.eat('(')
                self.value_grp()
                self.eat(')')
                self.exp_str()
            elif prod == 523:
                self.digit()
                self.digit_tail_str()
            elif prod == 524:
                self.eat('PARCH-lit')
                self.parch_tail_str()
            elif prod == 525:

self.eat('SCROLL-lit')
            if
self.current_token.type in
['{']:
                self.scr_char()
                self.scroll_tail_str()
            elif prod == 526:
                self.bool()
                self.logseq_str()
            else:

self.error_invalid_token('<value-str>')

    def digit_tail_str(self):
        # <digit-tail-str>
        prod =
self.get_production('<digit-tail-str>')
```

```
if prod == 527:
    self.arith_str()
    self.releq_str()
elif prod == 528:
    pass # Lambda
else:
    self.error_invalid_token('<digit-tail-str>')

def arith_str(self):
    # <arith-str>
    prod =
self.get_production('<arith-str>')

if prod == 529:
    self.dime_arith()
    self.arith_str()
elif prod == 530:
    pass # Lambda
else:
    self.error_invalid_token('<arith-str>')

def releq_str(self):
    # <releq-str>
    prod =
self.get_production('<releq-str>')

if prod == 531:
    self.rel()
    self.arith_str2()
    self.logeq_str()
elif prod == 532:
    self.eq_digit()
    self.arith_str3()
    self.bool_exp_arr()
elif prod == 533:
    pass # Lambda
else:
    self.error_invalid_token('<releq-str>')

def arith_str2(self):
    # <arith-str2>
    prod =
self.get_production('<arith-str2>')

if prod == 534:
    self.dime_arith()
    self.arith_str2()
elif prod == 535:
    pass # Lambda
else:
    self.error_invalid_token('<arith-str2>')

def arith_str3(self):
    # <arith-str3>
    prod =
self.get_production('<arith-str3>')

if prod == 536:
    self.dime_arith()
    self.arith_str3()
elif prod == 537:
    pass # Lambda
else:
    self.error_invalid_token('<arith-str3>')

def logeq_str(self):
    # <logeq-str>
    prod =
self.get_production('<logeq-str>')

if prod == 538:
    self.logeq_op()
    self.log_ope_str()

self.bool_exp_arr()
elif prod == 539:
    pass # Lambda
else:
    self.error_invalid_token('<logeq-str>')

def parch_tail_str(self):
    # <parch-tail-str>
    prod =
self.get_production('<parch-tail-str>')

if prod == 540:
    self.eq_op()
    self.eat('PARCH-lit')
    self.bool_exp_arr()
elif prod == 541:
    pass # Lambda
else:
    self.error_invalid_token('<parch-tail-str>')

def scroll_tail_str(self):
    # <scroll-tail-str>
    prod =
self.get_production('<scroll-tail-str>')

if prod == 542:
    self.concat_str()
elif prod == 543:
    self.eq_op()
    self.scroll_val()
    self.bool_exp_arr()
elif prod == 544:
    pass # Lambda
else:
    self.error_invalid_token('<scroll-tail-str>')
```

```
def concat_str(self):
    # <concat-str>
    prod =
self.get_production('<concat
-str>')
    if prod == 545:
        self.eat('&')
        self.scroll_val()
        self.concat_str()
    elif prod == 546:
        pass # Lambda
    else:

self.error_invalid_token('<co
ncat-str>')

def exp_str(self):
    # <exp-str>
    prod =
self.get_production('<exp-str
>')
    if prod == 547:
        self.dime_arith()
        self.arith_str()
        self.releq_str()
    elif prod == 548:
        self.rel()
        self.arith_str2()
        self.logeq_str()
    elif prod == 549:
        self.log_op()
        self.bool_ope_arr()
        self.bool_exp_arr()
    elif prod == 550:
        self.eq_op()
        self.eq_ope_arr()
        self.bool_exp_arr()
    elif prod == 551:
        self.eat('&')
        self.scroll_val()
        self.concat_str()
    elif prod == 552:

pass # Lambda
    else:

self.error_invalid_token('<ex
p-str>')

def statements(self):
    # <statements>
    prod =
self.get_production('<statem
ents>')
    if prod == 553:
        self.assign_stmnt()
    elif prod == 554:
        self.ask_stmnt()
    elif prod == 555:
        self.echo_stmnt()
    elif prod == 556:
        self.look_stmnt()
    elif prod == 557:
        self.chart_stmnt()
    elif prod == 558:
        self.hoist_stmnt()
    elif prod == 559:
        self.heave_stmnt()
    elif prod == 560:
        self.haul_stmnt()
    elif prod == 561:
        self.unary_exp()
        self.eat('!!')
    else:

self.error_invalid_token('<st
atements>')

def assign_stmnt(self):
    # <assign-stmnt>
    prod =
self.get_production('<assign-
stmnt>')
    if prod == 562:
        self.eat('id')

self.assign_tail()
    self.eat('!!')
    else:

self.error_invalid_token('<as
sign-stmnt>')

def assign_tail(self):
    # <assign-tail>
    prod =
self.get_production('<assign-
tail>')
    if prod == 563:
        if
self.current_token.type in
['{', '$']:
            self.arr_str()
            self.assign_body()
        elif prod == 564:
            self.eat('(')
            self.args()
            self.eat(')')
        else:

self.error_invalid_token('<as
sign-tail>')

def arr_str(self):
    # <arr-str>
    prod =
self.get_production('<arr-str
>')
    if prod == 565:
        self.eat('{')
        self.index()
        self.eat('}')
        if
self.current_token.type in
['{']:
            self.arr_str_tail()
        elif prod == 566:
            self.eat('$')
```

```
        self.eat('id')
    elif prod == 567:
        pass # Lambda
    else:

self.error_invalid_token('<ar
r-str>')

def arr_str_tail(self):
    # <arr-str-tail>
    prod =
self.get_production('<arr-str-
tail>')
    if prod == 568:
        self.eat('{')
        self.index()
        self.eat('}')
    elif prod == 569:
        pass # Lambda
    else:

self.error_invalid_token('<ar
r-str-tail>')

def assign_body(self):
    # <assign-body>
    prod =
self.get_production('<assign-
body>')
    if prod == 570:
        self.eat('=')
        self.value_as()
    elif prod == 571:
        self.arith_assign_op()
        self.dime_ret_val()
    else:

self.error_invalid_token('<as
sign-body>')

def value_as(self):
    # <value-as>
```

```
    prod =
self.get_production('<value-
as>')
    if prod == 572:
        self.eat('id')
        if
self.current_token.type in
['{', '$', '(']:
            self.id_tail()
            self.exp_as()
        elif prod == 573:
            self.eat('(')
            self.value_grp()
            self.exp_as()
        elif prod == 574:
            self.digit()
            self.digit_tail_as()
        elif prod == 575:
            self.eat('PARCH-lit')
            self.parch_tail_as()
        elif prod == 576:

self.eat('SCROLL-lit')
        if
self.current_token.type in
['{']:
            self.scr_char()
            self.scroll_tail_as()
        elif prod == 577:
            self.bool()
            self.logeq_as()
        else:

self.error_invalid_token('<va
lue-as>')

def digit_tail_as(self):
    # <digit-tail-as>
    prod =
self.get_production('<digit-ta
il-as>')
```

```
    if prod == 578:
        self.arith_as()
        self.releq_as()
    elif prod == 579:
        pass # Lambda
    else:

self.error_invalid_token('<di
git-tail-as>')

def arith_as(self):
    # <arith-as>
    prod =
self.get_production('<arith-a
s>')
    if prod == 580:
        self.dime_arith()
        self.arith_as()
    elif prod == 581:
        pass # Lambda
    else:

self.error_invalid_token('<ar
ith-as>')

def releq_as(self):
    # <releq-as>
    prod =
self.get_production('<releq-a
s>')
    if prod == 582:
        self.rel()
        self.arith_as2()
        self.logeq_as()
    elif prod == 583:
        self.eq_digit()
        self.arith_as3()
        self.bool_exp_ret()
    elif prod == 584:
        pass # Lambda
    else:
```

```
self.error_invalid_token('<releq-as>')

def arith_as2(self):
    # <arith-as2>
    prod =
self.get_production('<arith-as2>')
    if prod == 585:
        self.dime_arith()
        self.arith_as2()
    elif prod == 586:
        pass # Lambda
    else:

self.error_invalid_token('<arith-as2>')

def arith_as3(self):
    # <arith-as3>
    prod =
self.get_production('<arith-as3>')
    if prod == 587:
        self.dime_arith()
        self.arith_as3()
    elif prod == 588:
        pass # Lambda
    else:

self.error_invalid_token('<arith-as3>')

def logeq_as(self):
    # <logeq-as>
    prod =
self.get_production('<logeq-as>')
    if prod == 589:
        self.logeq_op()
        self.bool_ope_ret()

        self.bool_exp_ret()
    elif prod == 590:
        pass # Lambda
    else:

self.error_invalid_token('<logeq-as>')

def parch_tail_as(self):
    # <parch-tail-as>
    prod =
self.get_production('<parch-tail-as>')
    if prod == 591:
        self.eq_op()
        self.eat('PARCH-lit')
        self.log_as()
    elif prod == 592:
        pass # Lambda
    else:

self.error_invalid_token('<parch-tail-as>')

def scroll_tail_as(self):
    # <scroll-tail-as>
    prod =
self.get_production('<scroll-tail-as>')
    if prod == 593:
        self.concat_as()
    elif prod == 594:
        self.eq_op()
        self.scroll_val()
        self.bool_exp_ret()
    elif prod == 595:
        pass # Lambda
    else:

self.error_invalid_token('<scroll-tail-as>')

def concat_as(self):
    # <concat-as>
    prod =
self.get_production('<concat-as>')
    if prod == 596:
        self.eat('&')
        self.scroll_val()
        self.concat_as()
    elif prod == 597:
        pass # Lambda
    else:

self.error_invalid_token('<concat-as>')

def exp_as(self):
    # <exp-as>
    prod =
self.get_production('<exp-as>')
    if prod == 598:
        self.dime_arith()
        self.arith_as()
        self.releq_as()
    elif prod == 599:
        self.rel()
        self.arith_as2()
        self.logeq_as()
    elif prod == 600:
        self.log_op()
        self.bool_ope_ret()
        self.bool_exp_ret()
    elif prod == 601:
        self.eq_op()
        self.eq_ope_ret()
        self.bool_exp_ret()
    elif prod == 602:
        self.eat('&')
        self.scroll_val()
        self.concat_as()
    elif prod == 603:
```

```

    pass # Lambda
    else:

self.error_invalid_token('<exp-as>')

    def arith_assign_op(self):
        # <arith-assign-op>
        prod =
self.get_production('<arith-assign-op>')
        if prod == 604:
            self.eat('+=')
        elif prod == 605:
            self.eat('-=')
        elif prod == 606:
            self.eat('*=')
        elif prod == 607:
            self.eat('/=')
        elif prod == 608:
            self.eat('%=')
        elif prod == 609:
            self.eat('^=')
        else:

self.error_invalid_token('<arith-assign-op>')

    def ask_stmtnt(self):
        # <ask-stmnt>
        prod =
self.get_production('<ask-stmnt>')
        if prod == 610:
            self.eat('ASK')
            self.eat('(')

self.eat('SCROLL-lit')
    self.eat(',')
    self.addr()
    self.eat(')')
    self.eat('!!')
```

```

    else:

self.error_invalid_token('<ask-stmnt>')

    def addr(self):
        # <addr>
        prod =
self.get_production('<addr>')
        if prod == 611:
            self.eat('@')
            self.eat('id')
            if

self.current_token.type in
['{', '$']:
                self.arr_str()
                self.addr_tail()
            else:

self.error_invalid_token('<addr>')

    def addr_tail(self):
        # <addr-tail>
        prod =
self.get_production('<addr-tail>')
        if prod == 612:
            self.eat(',')
            self.addr()
        elif prod == 613:
            pass # Lambda
        else:

self.error_invalid_token('<addr-tail>')

    def echo_stmtnt(self):
        # <echo-stmnt>
        prod =
self.get_production('<echo-stmnt>')
```

```

    if prod == 614:
        self.eat('ECHO')
        self.eat('(')

self.eat('SCROLL-lit')
        self.args_mult()
        self.eat(')')
        self.eat('!!')
    else:

self.error_invalid_token('<echo-stmnt>')

    def look_stmtnt(self):
        # <look-stmnt>
        prod =
self.get_production('<look-stmnt>')
        if prod == 615:
            self.eat('LOOK')
            self.eat('(')
            self.condition()
            self.eat(')')
            self.eat('[')
            self.look_body()
            self.jump_stmtnt()
            self.eat(']')
            self.look_tail()
        else:

self.error_invalid_token('<look-stmnt>')

    def condition(self):
        # <condition>
        prod =
self.get_production('<condition>')
        if prod == 616:
            self.bool_grp_val()
        else:
```

```

self.error_invalid_token('<condition>')

    def look_body(self):
        # <look-body>
        prod =
self.get_production('<look-body>')
        if prod == 617:
            self.statements()
            self.look_body()
        elif prod == 618:
            pass # Lambda
        else:

self.error_invalid_token('<look-body>')

    def jump_stmnt(self):
        # <jump-stmnt>
        prod =
self.get_production('<jump-stmnt>')
        if prod == 619:
            self.eat('SAIL')
            self.eat('!!')
        elif prod == 620:
            self.eat('LAND')
            self.eat('!!')
        elif prod == 621:
            pass # Lambda
        else:

self.error_invalid_token('<jump-stmnt>')

    def look_tail(self):
        # <look-tail>
        prod =
self.get_production('<look-tail>')

```

```

if prod == 622:
    self.eat('DROPLOOK')
    self.eat('(')
    self.condition()
    self.eat(')')
    self.eat('[')
    self.look_body()
    self.jump_stmnt()
    self.eat(']')
    self.look_tail()
elif prod == 623:
    self.eat('DROP')
    self.eat('[')
    self.look_body()
    self.jump_stmnt()
    self.eat(']')
elif prod == 624:
    pass # Lambda
else:

self.error_invalid_token('<look-tail>')

def chart_stmnt(self):
    # <chart-stmnt>
    prod =
self.get_production('<chart-stmnt>')
    if prod == 625:
        self.eat('CHART')
        self.eat('(')
        self.chart_cond()
        self.eat(')')
        self.eat('[')
        self.courses()
        self.course_tail()
        self.adrift_case()
        self.eat(']')
    else:

```

```

self.error_invalid_token('<chart-stmnt>')

def chart_cond(self):
    # <chart-cond>
    prod =
self.get_production('<chart-cond>')
    if prod == 626:
        self.eat('id')
        if
self.current_token.type in
['{', '$', '[':
            self.id_tail()
        elif prod == 627:
            self.chart_const()
        else:

self.error_invalid_token('<chart-cond>')

def chart_const(self):
    # <chart-const>
    prod =
self.get_production('<chart-const>')
    if prod == 628:
        self.eat('COIN-lit')
    elif prod == 629:
        self.eat('PARCH-lit')
    elif prod == 630:

self.eat('SCROLL-lit')
    if
self.current_token.type in
['{']:
        self.scr_id()
    else:

self.error_invalid_token('<chart-const>')

```

```

def courses(self):
    # <courses>
    prod =
self.get_production('<course
s>')
    if prod == 631:
        self.eat('COURSE')
        self.chart_const()
        self.eat(':')
        self.course_body()
        self.course_jump()
    else:

self.error_invalid_token('<co
urses>')

    def course_body(self):
        # <course-body>
        prod =
self.get_production('<course
-body>')
        if prod == 632:
            self.statements()
            self.course_body()
        elif prod == 633:
            pass # Lambda
        else:

self.error_invalid_token('<co
urse-body>')

    def course_jump(self):
        # <course-jump>
        prod =
self.get_production('<course
-jump>')
        if prod == 634:
            self.eat('SAIL')
            self.eat('!!')
        elif prod == 635:
            self.eat('LAND')

        self.eat('!!')
        elif prod == 636:
            pass # Lambda
        else:

self.error_invalid_token('<co
urse-jump>')

    def course_tail(self):
        # <course-tail>
        prod =
self.get_production('<course
-tail>')
        if prod == 637:
            self.courses()
            self.course_tail()
        elif prod == 638:
            pass # Lambda
        else:

self.error_invalid_token('<co
urse-tail>')

    def adrift_case(self):
        # <adrift-case>
        prod =
self.get_production('<adrift-
case>')
        if prod == 639:
            self.eat('ADRIFT')
            self.eat(':')
            self.adrift_body()
            self.eat('LAND')
            self.eat('!!')
        elif prod == 640:
            pass # Lambda
        else:

self.error_invalid_token('<ad
rift-case>')

    def adrift_body(self):

        self.eat('!!')
        prod =
self.get_production('<adrift-
body>')
        if prod == 641:
            self.statements()
            self.adrift_body()
        elif prod == 642:
            pass # Lambda
        else:

self.error_invalid_token('<ad
rift-body>')

    def hoist_stmnt(self):
        # <hoist-stmnt>
        prod =
self.get_production('<hoist-s
tmnt>')
        if prod == 643:
            self.eat('HOIST')
            self.eat('(')
            self.hoist_init()
            self.eat('!!')
            self.hoist_cond()
            self.eat('!!')
            self.hoist_upd()
            self.eat(')')
            self.eat('[')
            self.look_body()
            self.jump_stmnt()
            self.eat(']')
        else:

self.error_invalid_token('<ho
ist-stmnt>')

    def hoist_init(self):
        # <hoist-init>
        prod =
self.get_production('<hoist-i
nit>')

```

```
if prod == 644:
    self.eat('COIN')
    self.eat('id')
    self.eat('=')
    self.coin_val()
    self.init1_mult()
elif prod == 645:
    self.eat('id')
    if
self.current_token.type in
['{', '$']:
    self.arr_str()
    self.eat('=')
    self.coin_val()
    self.init2_mult()
elif prod == 646:
    pass # Lambda
else:

self.error_invalid_token('<hoist-init>')

def init1_mult(self):
    # <init1-mult>
    prod =
self.get_production('<init1-mult>')
    if prod == 647:
        self.eat(',')
        self.eat('id')
        self.eat('=')
        self.coin_val()
        self.init1_mult()
    elif prod == 648:
        pass # Lambda
    else:

self.error_invalid_token('<init1-mult>')

def init2_mult(self):
    # <init2-mult>
```

SeaStack

```
prod =
self.get_production('<init2-mult>')
    if prod == 649:
        self.eat(',')
        self.eat('id')
        if
self.current_token.type in
['{', '$']:
    self.arr_str()
    self.eat('=')
    self.coin_val()
    self.init2_mult()
elif prod == 650:
    pass # Lambda
else:

self.error_invalid_token('<init2-mult>')

def hoist_cond(self):
    # <hoist-cond>
    prod =
self.get_production('<hoist-cond>')
    if prod == 651:
        self.dime_val()
        self.bool_arith()
        self.releq_op()
        self.dime_val()
    self.bool_arith3_grp()
    self.hoist_log()
    else:

self.error_invalid_token('<hoist-cond>')

def releq_op(self):
    # <releq-op>
```

```
prod =
self.get_production('<releq-op>')
    if prod == 652:
        self.rel_op()
    elif prod == 653:
        self.eq_op()
    else:

self.error_invalid_token('<releq-op>')

def hoist_log(self):
    # <hoist-log>
    prod =
self.get_production('<hoist-log>')
    if prod == 654:
        self.log_op()
        self.hoist_cond()
    elif prod == 655:
        pass # Lambda
    else:

self.error_invalid_token('<hoist-log>')

def hoist_upd(self):
    # <hoist-upd>
    prod =
self.get_production('<hoist-upd>')
    if prod == 656:
        self.upd()
        self.upd_mult()
    else:

self.error_invalid_token('<hoist-upd>')

def upd(self):
    # <upd>
```

```
prod =
self.get_production('<upd>')
if prod == 657:
    self.unary_op()
    self.eat('id')
    if
self.current_token.type in
['{', '$']:
    self.arr_str()
    elif prod == 658:
        self.eat('id')
        if
self.current_token.type in
['{', '$']:
    self.arr_str()
    self.arith_assign_op()
    self.dime_grp_val()
else:
self.error_invalid_token('<upd>')

def upd_mult(self):
    # <upd-mult>
    prod =
self.get_production('<upd-m
ult>')
    if prod == 659:
        self.eat(',')
        self.upd()
        self.upd_mult()
    elif prod == 660:
        pass # Lambda
    else:
self.error_invalid_token('<up
d-mult>')

def heave_stmnt(self):
    # <heave-stmnt>
```

```
prod =
self.get_production('<heave-
stmnt>')
if prod == 661:
    self.eat('HEAVE')
    self.eat('(')
    self.condition()
    self.eat(')')
    self.eat '[')
    self.look_body()
    self.jump_stmnt()
    self.eat(']')
else:
self.error_invalid_token('<he
ave-stmnt>')

def haul_stmnt(self):
    # <haul-stmnt>
    prod =
self.get_production('<haul-st
mnt>')
    if prod == 662:
        self.eat('HAUL')
        self.eat '[')
        self.look_body()
        self.jump_stmnt()
        self.eat(']')
        self.eat('HEAVE')
        self.eat('(')
        self.condition()
        self.eat(')')
        self.eat('!!')
    else:
self.error_invalid_token('<ha
ul-stmnt>')

def unary_exp(self):
    # <unary-exp>
```

```
prod =
self.get_production('<unary-
exp>')
if prod == 663:
    self.unary_op()
    self.eat('id')
    if
self.current_token.type in
['{', '$']:
        self.arr_str()
    else:
self.error_invalid_token('<un
ary-exp>')

def unary_op(self):
    # <unary-op>
    prod =
self.get_production('<unary-
op>')
    if prod == 664:
        self.eat('+#')
    elif prod == 665:
        self.eat('-#')
    else:
self.error_invalid_token('<un
ary-op>')

def ahoy_local_dec(self):
    # <ahoy-local-dec>
    prod =
self.get_production('<ahoy-l
ocal-dec>')
    if prod == 666:
        self.var_arr()
        self.ahoy_local_dec()
    elif prod == 667:
self.ahoy_struct_dec()
    elif prod == 668:
        pass # Lambda
```

```
else:
    prod =
self.error_invalid_token('<ahoy-local-dec>')
    if prod == 672:
        self.ahoy_stmnts()
    elif prod == 673:
        pass # Lambda
    else:
self.error_invalid_token('<ahoy-tail>')
    def ahoy_stmnt(self):
        # <ahoy-stmnt>
        prod =
self.get_production('<ahoy-struct-dec>')
    if prod == 669:
        self.eat('MAST')
        self.eat('id')
        self.eat('id')
        self.str_dec_init()
        self.eat('!!')
self.ahoy_struct_dec()
    elif prod == 670:
        pass # Lambda
    else:
self.error_invalid_token('<ahoy-struct-dec>')
    def ahoy_stmnts(self):
        # <ahoy-stmnts>
        prod =
self.get_production('<ahoy-stmnts>')
    if prod == 671:
        self.ahoy_stmnt()
        self.ahoy_tail()
    else:
self.error_invalid_token('<ahoy-stmnts>')
    def ahoy_tail(self):
        # <ahoy-tail>
        prod =
self.get_production('<ahoy-tail>')
    if prod == 672:
        self.ahoy_stmnts()
    elif prod == 673:
        pass # Lambda
    else:
self.error_invalid_token('<ahoy-tail>')
    def ahoy_stmnt(self):
        # <ahoy-stmnt>
        prod =
self.get_production('<ahoy-stmnt>')
    if prod == 674:
        self.eat('id')
        self.assign_tail()
        self.eat('!!')
    elif prod == 675:
        self.eat('ASK')
        self.eat('(')
self.eat('SCROLL-lit')
        self.eat(',')
        self.addr()
        self.eat(')')
        self.eat('!!')
    elif prod == 676:
        self.eat('ECHO')
        self.eat('(')
self.eat('SCROLL-lit')
        self.args_mult()
        self.eat(')')
        self.eat('!!')
    elif prod == 677:
        self.eat('LOOK')
        self.eat('(')
        self.condition()
self.eat(')')
self.eat('[')
self.look_body()
self.jump_stmnt()
self.eat(']')
elif prod == 678:
    self.eat('CHART')
    self.eat('(')
    self.chart_cond()
    self.eat(')')
    self.eat('[')
    self.courses()
    self.course_tail()
    self.adrift_case()
    self.eat(']')
elif prod == 679:
    self.eat('HOIST')
    self.eat('(')
    self.hoist_init()
    self.eat('!!')
    self.hoist_cond()
    self.eat('!!')
    self.hoist_upd()
    self.eat(')')
    self.eat('[')
    self.look_body()
    self.jump_stmnt()
    self.eat(']')
elif prod == 680:
    self.eat('HEAVE')
    self.eat('(')
    self.condition()
    self.eat(')')
    self.eat('[')
    self.look_body()
    self.jump_stmnt()
    self.eat(']')
elif prod == 681:
    self.eat('HAUL')
    self.eat('[')
    self.look_body()
```

```

    self.jump_stmnt()
    self.eat(']')
    self.eat('HEAVE')
    self.eat('(')
    self.condition()
    self.eat(')')
    self.eat('!!')
    elif prod == 682:
        self.unary_exp()
        self.eat('!!')
    else:

self.error_invalid_token('<ah
oy-stmnt>')

```

```

def ahoy_look_tail(self):
    # <ahoy-look-tail>
    prod =
self.get_production('<ahoy-l
ook-tail>')
    if prod == 683:

self.eat('DROPLOOK')
    self.eat('(')
    self.condition()
    self.eat(')')
    self.eat('[')
    self.look_body()
    self.jump_stmnt()
    self.eat(']')

```

```

    self.ahoy_look_tail()
    elif prod == 684:
        self.eat('DROP')
        self.eat('[')
        self.look_body()
        self.jump_stmnt()
        self.eat(']')
    elif prod == 685:
        pass # Lambda
    else:

self.error_invalid_token('<ah
oy-look-tail>')

```

predict_set.py

```

PREDICT = {
    '<program>': {
        'ABYSS': 1, 'AHOY': 1,
        'BOOL': 1, 'COIN': 1,
        'DIME': 1, 'LOCKE': 1,
        'MAST': 1, 'PARCH': 1,
        'SCROLL': 1
    },
    '<global-dec>': {
        'BOOL': 2, 'COIN': 2,
        'DIME': 2, 'PARCH': 2,
        'SCROLL': 2,
        'LOCKE': 3,
        'MAST': 4,
        'ABYSS': 5,
        'AHOY': 6
    },
    '<var-arr-func>': {
        'COIN': 7,
        'DIME': 8,
        'PARCH': 9,
        'SCROLL': 10,
        'BOOL': 11
    },

```

```

    '<coin-var-arr-func>': {
        '!!': 12, ' ': 12, '=': 12,
        '{': 12,
        '(': 13
    },
    '<coin-var-arr>': {
        '!!': 14, ' ': 14, '=': 14,
        '{': 15
    },
    '<coin-var>': {
        '!!': 16, ' ': 16, '=': 16
    },
    '<coin-init>': {
        '=': 17,
        '!!': 18, ' ': 18
    },
    '<coin-init-mult>': {
        ' ': 19,
        '!!': 20
    },
    '<coin-val>': {
        'COIN-lit': 21,
        '(': 22, ' ': 22, 'id': 22
    },

```

```

    '<neg>': {
        ' ': 23,
        '(': 24, 'id': 24
    },
    '<neg-coin-val>': {
        'id': 25,
        '(': 26
    },
    '<coin-exp>': {
        '%': 27, '*': 27, '+': 27,
        '-': 27, '/': 27, '^': 27,
        '!!': 28, ' ': 28
    },
    '<coin-arith>': {
        '%': 29, '*': 29, '+': 29,
        '-': 29, '/': 29, '^': 29
    },
    '<coin-val-grp>': {
        '(': 30, ' ': 30, 'COIN-lit':
        30, 'id': 30
    },
    '<coin-grp-exp>': {
        '%': 31, '*': 31, '+': 31,
        '-': 31, '/': 31, '^': 31,

```

')': 32	'<cav2-tail>': {	'COIN-lit': 69,
},	'!': 52,	'(: 70, '!': 70, 'id': 70
'<arith-op>': {	']': 53	},
'+' : 33,	},	'<neg-dime-val>': {
'-' : 34,	'<coin-func>': {	'id': 71,
'*' : 35,	'(' : 54	'(' : 72
'/' : 36,	},	},
'%' : 37,	'<coin-ret-val>': {	'<dime-exp>': {
'^' : 38	'(' : 55, '!': 55, 'COIN-lit':	'%' : 73, '*' : 73, '+' : 73,
},	55, 'id': 55	'-' : 73, '/' : 73, '^' : 73,
'<coin-arr>': {	},	'!!' : 74, '!' : 74
'{' : 39	'<coin-ret-exp>': {	},
},	'%' : 56, '*' : 56, '+' : 56,	'<dime-arith>': {
'<coin-arr-tail>': {	'-' : 56, '/' : 56, '^' : 56,	'%' : 75, '*' : 75, '+' : 75,
'=' : 40,	'!!' : 57	'-' : 75, '/' : 75, '^' : 75
'{' : 41,	},	},
'!!' : 42	'<dime-var-arr-func>': {	'<dime-grp-val>': {
},	'!!' : 58, '!' : 58, '=' : 58,	'(' : 76, '!': 76, 'COIN-lit':
'<coin-arr1>': {	'{' : 58,	76, 'DIME-lit': 76, 'id': 76
'(' : 43, '!': 43, 'COIN-lit':	'(' : 59	},
43, 'id': 43	},	'<dime-grp-exp>': {
},	'<dime-var-arr>': {	'%' : 77, '*' : 77, '+' : 77,
'<coin-arr-val>': {	'!!' : 60, '!' : 60, '=' : 60,	'-' : 77, '/' : 77, '^' : 77,
'(' : 44, '!': 44, 'COIN-lit':	'{' : 61	')' : 78, '!' : 78
44, 'id': 44	},	},
},	'<dime-var>': {	'<dime-arr>': {
'<coin-arr-exp>': {	'!!' : 62, '!' : 62, '=' : 62	'{' : 79
'%' : 45, '*' : 45, '+' : 45,	},	},
'-' : 45, '/' : 45, '^' : 45,	'<dime-init>': {	'<dime-arr-tail>': {
'!' : 46, ']' : 46	'=' : 63,	'=' : 80,
},	'!!' : 64, '!' : 64	'{' : 81,
'<cav-tail>': {	},	'!!' : 82
'!' : 47,	'<dime-init-mult>': {	},
']' : 48	'!' : 65,	'<dime-arr1>': {
},	'!!' : 66	'(' : 83, '!': 83, 'COIN-lit':
'<coin-arr2-tail>': {	},	83, 'DIME-lit': 83, 'id': 83
'=' : 49,	'<dime-init-val>': {	},
'!!' : 50	'(' : 67, '!': 67, 'COIN-lit':	'<dime-arr-val>': {
},	67, 'DIME-lit': 67, 'id': 67	'(' : 84, '!': 84, 'COIN-lit':
'<coin-arr2>': {	},	84, 'DIME-lit': 84, 'id': 84
'[' : 51	'<dime-val>': {	},
},	'DIME-lit': 68,	'<dime-arr-exp>': {

'%': 85, '*': 85, '+': 85,	'!': 102, ',': 102, '=': 102	(': 121
'-': 85, '/': 85, '^': 85,	},	},
':': 86, ']'': 86	'<parch-init>': {	'<scroll-var-arr-func>': {
},	'=: 103,	'!': 122, ',': 122, '=':
'<dav-tail>': {	'!': 104, ',': 104	122, ':': 122,
':': 87,	},	(': 123
']': 88	'<parch-init-mult>': {	},
},	':': 105,	'<scroll-var-arr>': {
'<dime-arr2-tail>': {	'!': 106	'!': 124, ',': 124, '=':
'=: 89,	},	124,
'!': 90	'<parch-val>': {	'{: 125
},	'PARCH-lit': 107,	},
'<dime-arr2>': {	'id': 108	'<scroll-var>': {
'[': 91	},	'!': 126, ',': 126, '=': 126
},	'<parch-arr>': {	},
'<dav2-tail>': {	'{: 109	'<scroll-init>': {
':': 92,	},	'=: 127,
']': 93	'<parch-arr-tail>': {	'!': 128, ',': 128
},	'=: 110,	},
'<dime-func>': {	'{: 111,	'<scroll-init-mult>': {
(': 94	'!': 112	':': 129,
},	},	'!': 130
'<dime-ret-val>': {	'<parch-arr1>': {	},
(': 95, '-': 95, 'COIN-lit':	'PARCH-lit': 113, 'id':	'<scroll-init-val>': {
95, 'DIME-lit': 95, 'id': 95	113	(': 131, 'SCROLL-lit':
},	},	131, 'id': 131
'<dime-ret-exp>': {	'<pav-tail>': {	},
'%': 96, '*': 96, '+': 96,	':': 114,	'<scroll-val>': {
'-': 96, '/': 96, '^': 96,	']': 115	'SCROLL-lit': 132,
'!': 97	},	'id': 133,
},	'<parch-arr2-tail>': {	(': 134
'<parch-var-arr-func>': {	'=: 116,	},
'!': 98, ',': 98, '=': 98,	'!': 117	'<scr-char>': {
'{: 98,	},	'{: 135,
(': 99	'<parch-arr2>': {	'!': 136, '!=': 136, '&':
},	'[': 118	136, '&&': 136, ')': 136, ',':
'<parch-var-arr>': {	},	136, '==': 136, ']'': 136, ' ':
'!': 100, ',': 100, '=':	'<pav2-tail>': {	136
100,	':': 119,	},
'{: 101	']': 120	'<index>': {
},	},	'id': 137,
'<parch-var>': {	'<parch-func>': {	'COIN-lit': 138

},	'!': 156	'!': 173, '!#': 173, '(:
'<scroll-exp>': {	},	173, '-': 173, 'AYE': 173,
'&': 139,	'<scroll-arr2>': {	'COIN-lit': 173, 'DIME-lit':
'!': 140, ',': 140	'[': 157	173, 'NAY': 173,
},	},	'PARCH-lit': 173,
'<scroll-concat>': {	'<sav2-tail>': {	'SCROLL-lit': 173, 'id': 173
'&': 141	',': 158,	},
},	']': 159	'<bool-val>': {
'<scroll-grp-val>': {	},	'id': 174,
'(: 142, 'SCROLL-lit':	'<scroll-func>': {	'(: 175,
142, 'id': 142	'(: 160	'!': 176, '!#': 176, 'AYE':
},	},	176, 'NAY': 176,
'<scroll-grp-exp>': {	'<scroll-ret-val>': {	'-': 177, 'COIN-lit': 177,
'&': 143,	'(: 161, 'SCROLL-lit':	'DIME-lit': 177,
')': 144	161, 'id': 161	'PARCH-lit': 178,
},	},	'SCROLL-lit': 179
'<scroll-arr>': {	'<scroll-ret-exp>': {	},
'{': 145	'&': 162,	'<bool>': {
},	'!': 163	'AYE': 180, 'NAY': 180,
'<scroll-arr-tail>': {	},	'!': 181, '!#': 181
'=: 146,	'<bool-var-arr-func>': {	},
'{': 147,	'!': 164, ',': 164, '=:	'<bool-lit>': {
'!': 148	164, '{': 164,	'AYE': 182,
},	'(: 165	'NAY': 183
'<scroll-arr1>': {	},	},
'(: 149, 'SCROLL-lit':	'<bool-var-arr>': {	'<not-op>': {
149, 'id': 149	'!': 166, ',': 166, '=:	'!': 184,
},	166,	'!#': 185
'<scroll-arr-val>': {	'{': 167	},
'(: 150, 'SCROLL-lit':	},	'<not-ope>': {
150, 'id': 150	'<bool-var>': {	'id': 186,
},	'!': 168, ',': 168, '=: 168	'(: 187,
'<scroll-arr-exp>': {	},	'AYE': 188, 'NAY': 188
'&': 151,	'<bool-init>': {	},
',': 152, ']' : 152	'=: 169,	'<bool-eq>': {
},	'!': 170, ',': 170	'!': 189, '==': 189,
'<sav-tail>': {	},	'!': 190, '&&': 190, ',':
',': 153,	'<bool-init-mult>': {	190, ']' : 190
]' : 154	',': 171,	},
},	'!': 172	'<digit>': {
'<scroll-arr2-tail>': {	},	'COIN-lit': 191,
'=: 155,	'<bool-init-val>': {	'DIME-lit': 192,

```
'-': 193
},
'<neg-digit>': {
  'id': 194,
  '(': 195
},
'<bool-arith>': {
  '%': 196, '*': 196, '+':
196, '-': 196, '/': 196, '^': 196,
  '!=': 197, '<': 197, '<=':
197, '==': 197, '>': 197, '>=':
197
},
'<rel-eq>': {
  '<': 198, '<=': 198, '>':
198, '>=': 198,
  '!=': 199, '==': 199
},
'<rel>': {
  '<': 200, '<=': 200, '>':
200, '>=': 200
},
'<rel-op>': {
  '<': 201,
  '>': 202,
  '<=': 203,
  '>=': 204
},
'<bool-arith2>': {
  '%': 205, '*': 205, '+':
205, '-': 205, '/': 205, '^': 205,
  '!!': 206, '!=': 206, '&&':
206, '': 206, '==': 206, '||':
206
},
'<eq-digit>': {
  '!=': 207, '==': 207
},
'<eq-op>': {
  '==': 208,
  '!=': 209
},

```

```
'<bool-arith3>': {
  '%': 210, '*': 210, '+':
210, '-': 210, '/': 210, '^': 210,
  '!!': 211, '&&': 211, '':
211, '||': 211
},
'<bool-exp2>': {
  '%': 212, '*': 212, '+':
212, '-': 212, '/': 212, '^': 212,
  '<': 213, '<=': 213, '>':
213, '>=': 213,
  '!=': 214, '==': 214,
  '!!': 215, '&&': 215, '':
215, '||': 215
},
'<eq-ope>': {
  'id': 216,
  '(': 217,
  '-': 218, 'COIN-lit': 218,
  'DIME-lit': 218,
  '!'': 219, '!'#: 219, 'AYE':
219, 'NAY': 219,
  'PARCH-lit': 219,
  'SCROLL-lit': 219
},
'<eq-ope1>': {
  'PARCH-lit': 220,
  'SCROLL-lit': 221,
  '!'': 222, '!'#: 222, 'AYE':
222, 'NAY': 222
},
'<bool-arith4>': {
  '%': 223, '*': 223, '+':
223, '-': 223, '/': 223, '^': 223,
  '!!': 224, '&&': 224, '':
224, '<': 224, '<=': 224, '>':
224, '>=': 224, '||': 224
},
'<bool-rel>': {
  '<': 225, '<=': 225, '>':
225, '>=': 225,

```

```
'!'': 226, '&&': 226, '':
226, '||': 226
},
'<bool-exp>': {
  '&&': 227, '||': 227,
  '!'': 228, '': 228
},
'<log-op>': {
  '||': 229,
  '&&': 230
},
'<bool-grp-val>': {
  '!'': 231, '!'#: 231, '(':
231, '-': 231, 'AYE': 231,
  'COIN-lit': 231, 'DIME-lit':
231, 'NAY': 231,
  'PARCH-lit': 231,
  'SCROLL-lit': 231, 'id': 231
},
'<bool-ope-grp>': {
  'id': 232,
  '(': 233,
  '!'': 234, '!'#: 234, 'AYE':
234, 'NAY': 234,
  '-': 235, 'COIN-lit': 235,
  'DIME-lit': 235,
  'PARCH-lit': 236,
  'SCROLL-lit': 237
},
'<bool-eq-grp>': {
  '!=': 238, '==': 238,
  '&&': 239, '): 239, '||':
239
},
'<releq-grp>': {
  '<': 240, '<=': 240, '>':
240, '>=': 240,
  '!=': 241, '==': 241
},
'<bool-arith2-grp>': {
  '%': 242, '*': 242, '+':
242, '-': 242, '/': 242, '^': 242,

```

'!': 243, '&&': 243, ')':
243, '==': 243, '||': 243
,
'<bool-arith3-grp>': {
 '%' : 244, '*' : 244, '+' :
244, '-' : 244, '/' : 244, '^' : 244,
 '!' : 245, '&&': 245, ')':
245, '||': 245
,
'<bool-exp2-grp>': {
 '%' : 246, '*' : 246, '+' :
246, '-' : 246, '/' : 246, '^' : 246,
 '<' : 247, '<=' : 247, '>':
247, '>=' : 247,
 '!' : 248, '==': 248,
 '&&': 249, ')': 249, '||':
249
,
'<eq-ope-grp>': {
 'id' : 250,
 '(' : 251,
 '-' : 252, 'COIN-lit' : 252,
'DIME-lit' : 252,
 '!' : 253, '!'# : 253, 'AYE':
253, 'NAY' : 253,
'PARCH-lit' : 253,
'SCROLL-lit' : 253
,
'<bool-arith4-grp>': {
 '%' : 254, '*' : 254, '+' :
254, '-' : 254, '/' : 254, '^' : 254,
 '&&': 255, ')': 255, '<':
255, '<=' : 255, '>': 255, '>=' :
255, '||': 255
,
'<bool-rel-grp>': {
 '<' : 256, '<=' : 256, '>':
256, '>=' : 256,
 '&&': 257, ')': 257, '||':
257
,
'<bool-exp-grp>': {

'&&': 258, '||': 258,
)': 259
,
'<bool-arr>': {
 '{' : 260
,
'<bool-arr-tail>': {
 '=' : 261,
 '{' : 262,
 '!' : 263
,
'<bool-arr1>': {
 '!' : 264, '!'# : 264, '(':
264, '-' : 264, 'AYE' : 264,
'COIN-lit' : 264, 'DIME-lit':
264, 'NAY' : 264,
'PARCH-lit' : 264,
'SCROLL-lit' : 264, 'id' : 264
,
'<bav-tail>': {
 '!' : 265,
 ']' : 266
,
'<bool-arr-val>': {
 '!' : 267, '!'# : 267, '(':
267, '-' : 267, 'AYE' : 267,
'COIN-lit' : 267, 'DIME-lit':
267, 'NAY' : 267,
'PARCH-lit' : 267,
'SCROLL-lit' : 267, 'id' : 267
,
'<bool-ope-arr>': {
 'id' : 268,
 '(' : 269,
 '!' : 270, '!'# : 270, 'AYE':
270, 'NAY' : 270,
 '-' : 271, 'COIN-lit' : 271,
'DIME-lit' : 271,
'PARCH-lit' : 272,
'SCROLL-lit' : 273
,
'<bool-eq-arr>': {

'!': 274, '==': 274,
'&&': 275, '!' : 275, ']' :
275, '||': 275
,
'<releq-arr>': {
 '<' : 276, '<=' : 276, '>':
276, '>=' : 276,
 '!' : 277, '==': 277
,
'<bool-arith2-arr>': {
 '%' : 278, '*' : 278, '+' :
278, '-' : 278, '/' : 278, '^' : 278,
 '!' : 279, '&&': 279, '!' :
279, '==': 279, ']' : 279, '||':
279
,
'<bool-arith3-arr>': {
 '%' : 280, '*' : 280, '+' :
280, '-' : 280, '/' : 280, '^' : 280,
 '&&': 281, '!' : 281, ']' :
281, '||': 281
,
'<bool-exp2-arr>': {
 '%' : 282, '*' : 282, '+' :
282, '-' : 282, '/' : 282, '^' : 282,
 '<' : 283, '<=' : 283, '>':
283, '>=' : 283,
 '!' : 284, '==': 284,
 '&&': 285, '!' : 285, ']' :
285, '||': 285
,
'<eq-ope-arr>': {
 'id' : 286,
 '(' : 287,
 '-' : 288, 'COIN-lit' : 288,
'DIME-lit' : 288,
 '!' : 289, '!'# : 289, 'AYE':
289, 'NAY' : 289,
'PARCH-lit' : 289,
'SCROLL-lit' : 289
,
'<bool-arith4-arr>': {

```
'%': 290, '*': 290, '+':
290, '-': 290, '/': 290, '^': 290,
'&&': 291, ':': 291, '<':
291, '<=': 291, '>': 291, '>=':
291, ']'': 291, '||': 291
},
'<bool-rel-arr>': {
'<': 292, '<=': 292, '>':
292, '>=': 292,
'&&': 293, ':': 293, ']'':
293, '||': 293
},
'<bool-exp-arr>': {
'&&': 294, '||': 294,
':': 295, ']'': 295
},
'<bool-arr2-tail>': {
'=': 296,
'!'': 297
},
'<bool-arr2>': {
'[': 298
},
'<bav2-tail>': {
':': 299,
']'': 300
},
'<bool-func>': {
'(': 301
},
'<bool-ret-val>': {
'!': 302, '!#': 302, '(':
302, '-': 302, 'AYE': 302,
'COIN-lit': 302, 'DIME-lit':
302, 'NAY': 302,
'PARCH-lit': 302,
'SCROLL-lit': 302, 'id': 302
},
'<bool-ope-ret>': {
'id': 303,
'(': 304,
```

```
'!': 305, '!#': 305, 'AYE':
305, 'NAY': 305,
'-': 306, 'COIN-lit': 306,
'DIME-lit': 306,
'PARCH-lit': 307,
'SCROLL-lit': 308
},
'<bool-eq-ret>': {
'!=': 309, '==': 309,
'!'': 310, '&&': 310, '||':
310
},
'<releq-ret>': {
'<': 311, '<=': 311, '>':
311, '>=': 311,
'!=': 312, '==': 312
},
'<bool-arith2-ret>': {
'%': 313, '*': 313, '+':
313, '-': 313, '/': 313, '^': 313,
'!'': 314, '!=': 314, '&&':
314, '==': 314, '||': 314
},
'<bool-arith3-ret>': {
'%': 315, '*': 315, '+':
315, '-': 315, '/': 315, '^': 315,
'!'': 316, '&&': 316, '||':
316
},
'<bool-exp2-ret>': {
'%': 317, '*': 317, '+':
317, '-': 317, '/': 317, '^': 317,
'<': 318, '<=': 318, '>':
318, '>=': 318,
'!=': 319, '==': 319,
'!'': 320, '&&': 320, '||':
320
},
'<eq-ope-ret>': {
'id': 321,
'(': 322,
```

```
'-': 323, 'COIN-lit': 323,
'DIME-lit': 323,
'!': 324, '!#': 324, 'AYE':
324, 'NAY': 324,
'PARCH-lit': 324,
'SCROLL-lit': 324
},
'<bool-arith4-ret>': {
'%': 325, '*': 325, '+':
325, '-': 325, '/': 325, '^': 325,
'!'': 326, '&&': 326, '<':
326, '<=': 326, '>': 326, '>=':
326, '||': 326
},
'<bool-rel-ret>': {
'<': 327, '<=': 327, '>':
327, '>=': 327,
'!'': 328, '&&': 328, '||':
328
},
'<bool-exp-ret>': {
'&&': 329, '||': 329,
'!'': 330
},
'<params>': {
'BOOL': 331, 'COIN':
331, 'DIME': 331, 'PARCH':
331, 'SCROLL': 331,
')': 332
},
'<param-mult>': {
':': 333,
')': 334
},
'<d-type>': {
'COIN': 335,
'DIME': 336,
'PARCH': 337,
'SCROLL': 338,
'BOOL': 339
},
'<ret-stmnts>': {
```


'+#': 340, '-#': 340,
 'ASK': 340, 'CHART': 340,
 'ECHO': 340, 'HAUL': 340,
 'HEAVE': 340, 'HOIST': 340,
 'LOOK': 340, 'id': 340,
 'BACK': 341
 },
 '<id-tail>': {
 '(': 342,
 '\$': 343,
 '(': 344,
 '!!': 345, '!=': 345, '%':
 345, '&': 345, '&&': 345, ')':
 345, '*': 345, '+': 345, ',': 345,
 '-': 345, '/': 345, '<': 345, '<=':
 345, '==': 345, '>': 345, '>=':
 345, ']'': 345, '^': 345, '||': 345
 },
 '<elmt2>': {
 '(': 346,
 '!!': 347, '!=': 347, '%':
 347, '&': 347, '&&': 347, ')':
 347, '*': 347, '+': 347, ',': 347,
 '-': 347, '/': 347, '<': 347, '<=':
 347, '==': 347, '>': 347, '>=':
 347, ']'': 347, '^': 347, '||': 347
 },
 '<args>': {
 '(': 348, '!'': 348, '(':
 348, '-': 348, 'AYE': 348,
 'COIN-lit': 348, 'DIME-lit':
 348, 'NAY': 348,
 'PARCH-lit': 348,
 'SCROLL-lit': 348, 'id': 348,
 ')': 349
 },
 '<args-mult>': {
 ',': 350,
 ')': 351
 },
 '<value>': {
 'id': 352,

(': 353,
 '-': 354, 'COIN-lit': 354,
 'DIME-lit': 354,
 'PARCH-lit': 355,
 'SCROLL-lit': 356,
 '!'': 357, '!'': 357, 'AYE':
 357, 'NAY': 357
 },
 '<digit-tail>': {
 '!=': 358, '%': 358, '*':
 358, '+': 358, '-': 358, '/': 358,
 '<': 358, '<=': 358, '==': 358,
 '>': 358, '>=': 358, '^': 358,
 ')': 359, ',': 359
 },
 '<arith-var>': {
 '%': 360, '*': 360, '+':
 360, '-': 360, '/': 360, '^': 360,
 '!=': 361, ')': 361, ',':
 361, '<': 361, '<=': 361, '==':
 361, '>': 361, '>=': 361
 },
 '<releq-var>': {
 '<': 362, '<=': 362, '>':
 362, '>=': 362,
 '!=': 363, '==': 363,
 ')': 364, ',': 364
 },
 '<arith-var2>': {
 '%': 365, '*': 365, '+':
 365, '-': 365, '/': 365, '^': 365,
 '!=': 366, '&&': 366, ')':
 366, ',': 366, '==': 366, '||':
 366
 },
 '<arith-var3>': {
 '%': 367, '*': 367, '+':
 367, '-': 367, '/': 367, '^': 367,
 '&&': 368, ')': 368, ',':
 368, '||': 368
 },
 '<logeq-var>': {

'!=': 369, '&&': 369,
 '==': 369, '||': 369,
 ')': 370, ',': 370
 },
 '<logeq-op>': {
 '&&': 371, '||': 371,
 '!=': 372, '==': 372
 },
 '<log-var>': {
 '&&': 373, '||': 373,
 ')': 374, ',': 374
 },
 '<parch-tail>': {
 '!=': 375, '==': 375,
 ')': 376, ',': 376
 },
 '<scroll-tail>': {
 '&': 377,
 '!=': 378, '==': 378,
 ')': 379, ',': 379
 },
 '<concat-var>': {
 '&': 380,
 ')': 381, ',': 381
 },
 '<exp>': {
 '%': 382, '*': 382, '+':
 382, '-': 382, '/': 382, '^': 382,
 '<': 383, '<=': 383, '>':
 383, '>=': 383,
 '&&': 384, '||': 384,
 '!=': 385, '==': 385,
 '&': 386,
 ')': 387, ',': 387
 },
 '<log-ope>': {
 'id': 388,
 '(': 389,
 '!'': 390, '!'': 390, 'AYE':
 390, 'NAY': 390,
 '-': 391, 'COIN-lit': 391,
 'DIME-lit': 391,

'PARCH-lit': 392, 'SCROLL-lit': 393 }, '<log-booleq>': { '!=': 394, '==': 394, '&&': 395, ')': 395, ' ': 395, ' ': 395 }, '<log-releq>': { '<': 396, '<=': 396, '>': 396, '>=': 396, '!=': 397, '==': 397 }, '<log-exp>': { '%' : 398, '*' : 398, '+': 398, '-' : 398, '/' : 398, '^': 398, '<': 399, '<=': 399, '>': 399, '>=': 399, '!=': 400, '==': 400, '&&': 401, ')': 401, ' ': 401, ' ': 401 }, '<eq-var>': { 'id': 402, '(': 403, '-': 404, 'COIN-lit': 404, 'DIME-lit': 404, '!': 405, '!#': 405, 'AYE': 405, 'NAY': 405, 'PARCH-lit': 405, 'SCROLL-lit': 405 }, '<arith-var4>': { '%' : 406, '*' : 406, '+': 406, '-' : 406, '/' : 406, '^': 406, '&&': 407, ')': 407, ' ': 407, '<': 407, '<=': 407, '>': 407, '>=': 407, ' ': 407 }, '<rel-var>': { '<': 408, '<=': 408, '>': 408, '>=': 408,	'&&': 409, ')': 409, ' ': 409, ' ': 409 }, '<value-grp>': { 'id': 410, '(': 411, '-': 412, 'COIN-lit': 412, 'DIME-lit': 412, 'PARCH-lit': 413, 'SCROLL-lit': 414, '!': 415, '!#': 415, 'AYE': 415, 'NAY': 415 }, '<digit-tail-grp>': { '!=': 416, '%' : 416, '*' : 416, '+' : 416, '-' : 416, '/' : 416, '<': 416, '<=': 416, '==': 416, '>': 416, '>=': 416, '^': 416, ')': 417 }, '<arith-grp>': { '%' : 418, '*' : 418, '+': 418, '-' : 418, '/' : 418, '^': 418, '!=': 419, ')': 419, '<': 419, '<=': 419, '==': 419, '>': 419, '>=': 419 }, '<releqvar-grp>': { '<': 420, '<=': 420, '>': 420, '>=': 420, '!=': 421, '==': 421, ')': 422 }, '<arith-grp2>': { '%' : 423, '*' : 423, '+': 423, '-' : 423, '/' : 423, '^': 423, '!=': 424, '&&': 424, ')': 424, '==': 424, ' ': 424 }, '<arith-grp3>': { '%' : 425, '*' : 425, '+': 425, '-' : 425, '/' : 425, '^': 425,	'&&': 426, ')': 426, ' ': 426 }, '<logeq-grp>': { '!=': 427, '&&': 427, '==': 427, ' ': 427, ')': 428 }, '<parch-tail-grp>': { '!=': 429, '==': 429, ')': 430 }, '<scroll-tail-grp>': { '&': 431, '!=': 432, '==': 432, ')': 433 }, '<concat-grp>': { '&': 434, ')': 435 }, '<exp-grp>': { '%' : 436, '*' : 436, '+': 436, '-' : 436, '/' : 436, '^': 436, '<': 437, '<=': 437, '>': 437, '>=': 437, '&&': 438, ' ': 438, '!=': 439, '==': 439, '&': 440, ')': 441 }, '<const>': { 'LOCKE': 442 }, '<const-init>': { 'COIN': 443, 'DIME': 444, 'PARCH': 445, 'SCROLL': 446, 'BOOL': 447 }, '<coin-locke>': {
---	--	--

'id': 448	},	'ABYSS': 484
},	'<bool-locke-mult>': {	},
'<coin-locke-mult>': {	'!': 467,	'<nonret-stmnts>': {
'!': 449,	'!!': 468	'+#': 485, '-#': 485,
'!!': 450	},	'ASK': 485, 'CHART': 485,
},	'<struct>': {	'ECHO': 485, 'HAUL': 485,
'<dime-locke>': {	'MAST': 469,	'HEAVE': 485, 'HOIST': 485,
'id': 451	'ABYSS': 470, 'AHOY':	'LOOK': 485, 'id': 485
},	470, 'BOOL': 470, 'COIN':	},
'<locke-digit>': {	470, 'DIME': 470, 'PARCH':	'<nonret-tail>': {
'COIN-lit': 452,	470, 'SCROLL': 470	'+#': 486, '-#': 486,
'DIME-lit': 453	},	'ASK': 486, 'CHART': 486,
},	'<mem-dec>': {	'ECHO': 486, 'HAUL': 486,
'<dime-locke-mult>': {	'BOOL': 471, 'COIN':	'HEAVE': 486, 'HOIST': 486,
'!': 454,	471, 'DIME': 471, 'PARCH':	'LOOK': 486, 'id': 486,
'!!': 455	471, 'SCROLL': 471	'!': 487, 'BACK': 487
},	},	},
'<parch-locke>': {	'<mem-mult>': {	'<nonret-back>': {
'id': 456	'!': 472,	'BACK': 488,
},	'!!': 473	'!': 489
'<parch-locke-mult>': {	},	},
'!': 457,	'<mem-dec-tail>': {	'<nonret-stmnt>': {
'!!': 458	'BOOL': 474, 'COIN':	'id': 490,
},	474, 'DIME': 474, 'PARCH':	'ASK': 491,
'<scroll-locke>': {	474, 'SCROLL': 474,	'ECHO': 492,
'id': 459	'!': 475	'LOOK': 493,
},	},	'CHART': 494,
'<scr-id>': {	'<sub-func>': {	'HOIST': 495,
'!': 460,	'BOOL': 476, 'COIN':	'HEAVE': 496,
'!!': 461, '): 461, '): 461,	476, 'DIME': 476, 'PARCH':	'HAUL': 497,
'!': 461	476, 'SCROLL': 476,	'+#': 498, '-#': 498
},	'ABYSS': 477,	},
'<scroll-locke-mult>': {	'AHOY': 478	'<nonret-look-tail>': {
'!': 462,	},	'DROLOOK': 499,
'!!': 463	'<return-func>': {	'DROP': 500,
},	'COIN': 479,	'+#': 501, '-#': 501, '!':
'<bool-locke>': {	'DIME': 480,	501, 'ASK': 501, 'BACK':
'id': 464	'PARCH': 481,	501, 'CHART': 501, 'ECHO':
},	'SCROLL': 482,	501, 'HAUL': 501, 'HEAVE':
'<locke-bool>': {	'BOOL': 483	501, 'HOIST': 501, 'LOOK':
'AYE': 465,	},	501, 'id': 501
'NAY': 466	'<nonreturn-func>': {	},

'<local-dec>': { 'BOOL': 502, 'COIN': 502, 'DIME': 502, 'PARCH': 502, 'SCROLL': 502, 'MAST': 503, '+'#: 504, '-'#: 504, 'ASK': 504, 'BACK': 504, 'CHART': 504, 'ECHO': 504, 'HAUL': 504, 'HEAVE': 504, 'HOIST': 504, 'LOOK': 504, 'id': 504 }, '<var-arr>': { 'COIN': 505, 'DIME': 506, 'PARCH': 507, 'SCROLL': 508, 'BOOL': 509 }, '<struct-dec>': { 'MAST': 510, '+'#: 511, '-'#: 511, 'ASK': 511, 'BACK': 511, 'CHART': 511, 'ECHO': 511, 'HAUL': 511, 'HEAVE': 511, 'HOIST': 511, 'LOOK': 511, 'id': 511 }, '<str-dec-init>': { '!': 512, '=': 513, '!!': 514 }, '<str-dec-tail>': { '!': 515, '!!': 516 }, '<str-val>': { '!': 517, '!'#: 517, '(!': 517, '!'': 517, 'AYE': 517, 'COIN-lit': 517, 'DIME-lit': 517, 'NAY': 517,	'PARCH-lit': 517, 'SCROLL-lit': 517, 'id': 517, '\$': 518 }, '<str-val-tail>': { '!': 519, ']': 520 }, '<value-str>': { 'id': 521, '(!': 522, '!'': 523, 'COIN-lit': 523, 'DIME-lit': 523, 'PARCH-lit': 524, 'SCROLL-lit': 525, '!'': 526, '!'#: 526, 'AYE': 526, 'NAY': 526 }, '<digit-tail-str>': { '!'': 527, '!'#: 527, '!'': 527, '!'': 527, '!'': 527, '!'': 527, '!'': 527, '!'': 527, '!'': 527, '!'': 527, '!'': 527, '!'': 528, '!'': 528 }, '<arith-str>': { '!'#: 529, '!'': 529, '!'': 529, '!'': 529, '!'': 529, '!'': 529, '!'': 530, '!'': 530, '!'': 530, '!'': 530, '!'': 530, '!'': 530, '!'': 530, '!'': 530 }, '<releq-str>': { '!'': 531, '!'': 531, '!'': 531, '!'': 531, '!'': 532, '!'': 532, '!'': 533, '!'': 533 }, '<arith-str2>': { '!'#: 534, '!'': 534, '!'': 534, '!'': 534, '!'': 534, '!'': 534, '!'': 534, '!'': 534,	'!'': 535, '!'': 535, '!'': 535, '!'': 535, '!'': 535, '!'': 535 }, '<arith-str3>': { '!'#: 536, '!'': 536, '!'': 536, '!'': 536, '!'': 536, '!'': 536, '!'': 537, '!'': 537, '!'': 537, '!'': 537 }, '<logeq-str>': { '!'': 538, '!'': 538, '!'': 538, '!'': 538, '!'': 539, '!'': 539 }, '<parch-tail-str>': { '!'': 540, '!'': 540, '!'': 541, '!'': 541 }, '<scroll-tail-str>': { '!'': 542, '!'': 543, '!'': 543, '!'': 544, '!'': 544 }, '<concat-str>': { '!'': 545, '!)': 546, '!'': 546, '!'': 546 }, '<exp-str>': { '!'#: 547, '!'': 547, '!'': 547, '!'': 547, '!'': 547, '!'': 547, '!'': 548, '!'': 548, '!'': 548, '!'': 548, '!'': 549, '!'': 549, '!'': 550, '!'': 550, '!'': 551, '!'': 552, '!'': 552 }, '<statements>': { 'id': 553, 'ASK': 554, 'ECHO': 555,
--	---	---

'LOOK': 556,	'-': 574, 'COIN-lit': 574,	'!!!': 590
'CHART': 557,	'DIME-lit': 574,	},
'HOIST': 558,	'PARCH-lit': 575,	'<parch-tail-as>': {
'HEAVE': 559,	'SCROLL-lit': 576,	'!=': 591, '==': 591,
'HAUL': 560,	'!': 577, '!#': 577, 'AYE':	'!!!': 592
'+#': 561, '-#': 561	577, 'NAY': 577	},
},	},	'<scroll-tail-as>': {
'<assign-stmnt>': {	'<digit-tail-as>': {	'&': 593,
'id': 562	'!=': 578, '%': 578, '*':	'!=': 594, '==': 594,
},	578, '+': 578, '-': 578, '/': 578,	'!!!': 595
'<assign-tail>': {	'<': 578, '<=': 578, '==': 578,	},
'\$': 563, '%': 563, '*':	'>': 578, '>=': 578, '^': 578,	'<concat-as>': {
563, '+': 563, '-': 563, '/':	'!!!': 579	'&': 596,
563, '=': 563, '^': 563, '{':	},	'!!!': 597
563,	'<arith-as>': {	},
'(': 564	'%': 580, '*': 580, '+':	'<exp-as>': {
},	580, '-': 580, '/': 580, '^': 580,	'%': 598, '*': 598, '+':
'<arr-str>': {	'!!!': 581, '!=': 581, '<':	598, '-': 598, '/': 598, '^': 598,
'{': 565,	581, '<=': 581, '==': 581, '>':	'<': 599, '<=': 599, '>':
'\$': 566,	581, '>=': 581	599, '>=': 599,
'!!!': 567, '%': 567, ')':	},	'&&': 600, ' ': 600,
567, '*': 567, '+': 567, ',':	'<releq-as>': {	'!=': 601, '==': 601,
567, '-': 567, '/': 567, '=':	'<': 582, '<=': 582, '>':	'&': 602,
567, '^': 567	582, '>=': 582,	'!!!': 603
},	'!=': 583, '==': 583,	},
'<arr-str-tail>': {	'!!!': 584	'<arith-assign-op>': {
'{': 568,	},	'+'': 604,
'!!!': 569, '%': 569, ')':	'<arith-as2>': {	'-': 605,
569, '*': 569, '+': 569, ',':	'%': 585, '*': 585, '+':	'*': 606,
569, '-': 569, '/': 569, '=':	585, '-': 585, '/': 585, '^': 585,	'/'': 607,
569, '^': 569	'!!!': 586, '!=': 586, '&&':	'%': 608,
},	586, '==': 586, ' ': 586	'^': 609
'<assign-body>': {	},	},
'=': 570,	'<arith-as3>': {	'<ask-stmnt>': {
'%': 571, '*': 571,	'%': 587, '*': 587, '+':	'ASK': 610
'+'': 571, '-': 571, '/': 571,	587, '-': 587, '/': 587, '^': 587,	},
'^': 571	'!!!': 588, '&&': 588, ' ':	'<addr>': {
},	588	'@': 611
'<value-as>': {	},	},
'id': 572,	'<logeq-as>': {	'<addr-tail>': {
'(': 573,	'!=': 589, '&&': 589,	',': 612,
	'==': 589, ' ': 589,	')': 613

},	'CHART': 625	'+#': 641, '-#': 641,
'<echo-stmnt>': {	},	'ASK': 641, 'CHART': 641,
'ECHO': 614	'<chart-cond>': {	'ECHO': 641, 'HAUL': 641,
},	'id': 626,	'HEAVE': 641, 'HOIST': 641,
'<look-stmnt>': {	'COIN-lit': 627,	'LOOK': 641, 'id': 641,
'LOOK': 615	'PARCH-lit': 627,	'LAND': 642
},	'SCROLL-lit': 627	},
'<condition>': {	},	'<hoist-stmnt>': {
'!': 616, '!#': 616, '(!':	'<chart-const>': {	'HOIST': 643
616, '-': 616, 'AYE': 616,	'COIN-lit': 628,	},
'COIN-lit': 616, 'DIME-lit':	'PARCH-lit': 629,	'<hoist-init>': {
616, 'NAY': 616,	'SCROLL-lit': 630	'COIN': 644,
'PARCH-lit': 616,	},	'id': 645,
'SCROLL-lit': 616, 'id': 616	'<courses>': {	'!!!': 646
},	'COURSE': 631	},
'<look-body>': {	},	'<init1-mult>': {
'+#': 617, '-#': 617,	'<course-body>': {	'!': 647,
'ASK': 617, 'CHART': 617,	'+#': 632, '-#': 632,	'!!!': 648
'ECHO': 617, 'HAUL': 617,	'ASK': 632, 'CHART': 632,	},
'HEAVE': 617, 'HOIST': 617,	'ECHO': 632, 'HAUL': 632,	'<init2-mult>': {
'LOOK': 617, 'id': 617,	'HEAVE': 632, 'HOIST': 632,	'!': 649,
']': 618, 'LAND': 618,	'LOOK': 632, 'id': 632,	'!!!': 650
'SAIL': 618	']': 633, 'ADRIFT': 633,	},
},	'COURSE': 633, 'LAND':	'<hoist-cond>': {
'<jump-stmnt>': {	633, 'SAIL': 633	'(!': 651, '-': 651,
'SAIL': 619,	},	'COIN-lit': 651, 'DIME-lit':
'LAND': 620,	'<course-jmp>': {	651, 'id': 651
']': 621	'SAIL': 634,	},
},	'LAND': 635,	'<releq-op>': {
'<look-tail>': {	']': 636, 'ADRIFT': 636,	'<': 652, '<=: 652, '>':
'DROPLOOK': 622,	'COURSE': 636	652, '>=: 652,
'DROP': 623,	},	'!=': 653, '==': 653
'+#': 624, '-#': 624, ']':	'<course-tail>': {	},
624, 'ADRIFT': 624, 'ASK':	'COURSE': 637,	'<hoist-log>': {
624, 'BACK': 624, 'CHART':	']': 638, 'ADRIFT': 638	'&&': 654, ' ': 654,
624, 'COURSE': 624,	},	'!!!': 655
'ECHO': 624, 'HAUL': 624,	'<adrift-case>': {	},
'HEAVE': 624, 'HOIST': 624,	'ADRIFT': 639,	'<hoist-upd>': {
'LAND': 624, 'LOOK': 624,	']': 640	'+#': 656, '-#': 656, 'id':
'SAIL': 624, 'id': 624	},	656
},	'<adrift-body>': {	},
'<chart-stmnt>': {		'<upd>': {

```

    '+#': 657, '-#': 657,
    'id': 658
  },
  '<upd-mult>': {
    '!': 659,
    ')': 660
  },
  '<heave-stmnt>': {
    'HEAVE': 661
  },
  '<haul-stmnt>': {
    'HAUL': 662
  },
  '<unary-exp>': {
    '+#': 663, '-#': 663
  },
  '<unary-op>': {
    '+#': 664,
    '-#': 665
  },
  '<ahoy-local-dec>': {
    'BOOL': 666, 'COIN':
666, 'DIME': 666, 'PARCH':
666, 'SCROLL': 666,
    'MAST': 667,
    '+#': 668, '-#': 668,
    'ASK': 668, 'CHART': 668,
    'ECHO': 668, 'HAUL': 668,
    'HEAVE': 668, 'HOIST': 668,
    'LOOK': 668, 'id': 668
  },
  '<ahoy-struct-dec>': {
    'MAST': 669,
    '+#': 670, '-#': 670,
    'ASK': 670, 'CHART': 670,
    'ECHO': 670, 'HAUL': 670,
    'HEAVE': 670, 'HOIST': 670,
    'LOOK': 670, 'id': 670
  },
  '<ahoy-stmnts>': {
    '+#': 671, '-#': 671,
    'ASK': 671, 'CHART': 671,
    'ECHO': 671, 'HAUL': 671,
    'HEAVE': 671, 'HOIST': 671,
    'LOOK': 671, 'id': 671
  },
  '<ahoy-tail>': {
    '+#': 672, '-#': 672,
    'ASK': 672, 'CHART': 672,
    'ECHO': 672, 'HAUL': 672,
    'HEAVE': 672, 'HOIST': 672,
    'LOOK': 672, 'id': 672,
    ']': 673
  },
  '<ahoy-stmnt>': {
    'id': 674,
    'ASK': 675,
    'ECHO': 676,
    'LOOK': 677,
    'CHART': 678,
    'HOIST': 679,
    'HEAVE': 680,
    'HAUL': 681,
    '+#': 682, '-#': 682
  },
  '<ahoy-look-tail>': {
    'DROPLOOK': 683,
    'DROP': 684,
    '+#': 685, '-#': 685, ']':
685, 'ASK': 685, 'CHART':
685, 'ECHO': 685, 'HAUL':
685, 'HEAVE': 685, 'HOIST':
685, 'LOOK': 685, 'id': 685
  },
  }

```

```

class ASTNode:
    resolved_type: str = None

    def __repr__(self):
        return
        f'{self.__class__.__name__}
        ({attrs})'

    def __init__(self, attrs):
        self.__dict__.update({k: v for k, v in
        attrs.items() if k != 'token'})

```

SeaStack

```
# program node -
global_decls + ahoy_body.
class
ProgramNode(ASTNode):
    def __init__(self,
global_decls, ahoy_body,
token=None):
        self.global_decls =
global_decls #
list[ASTNode]
        self.ahoy_body =
ahoy_body # AhoyNode
        self.token = token

# AHoy() [ local_decls
statements ].
class AhoyNode(ASTNode):
    def __init__(self,
local_decls, statements,
token=None):
        self.local_decls =
local_decls #
list[ASTNode]
        self.statements =
statements #
list[ASTNode]
        self.token = token

# constant declaration
class
ConstDeclNode(ASTNode):
    def __init__(self, dtype,
name, value, token=None):
        self.dtype = dtype #
str - dtype
        self.name = name #
str - constant id
        self.value = value #
LiteralNode
        self.token = token

# variable declaration
```

```
class
VarDeclNode(ASTNode):
    def __init__(self, dtype,
name, init_value,
token=None):
        self.dtype = dtype
# str - dtype
        self.name = name
# str - variable id
        self.init_value =
init_value # ASTNode |
None
        self.token = token

# array declaration
class
ArrayDeclNode(ASTNode):
    def __init__(self, dtype,
name, dimensions, is_2d,
init_values, token=None):
        self.dtype = dtype
# str - dtype
        self.name = name
# str - array id
        self.dimensions =
dimensions # list[int] (1d or
2d)
        self.is_2d = is_2d
# bool
        self.init_values =
init_values # list | list-of-lists
| None
        self.token = token

# structure definition
class
StructDefNode(ASTNode):
    def __init__(self, name,
members, token=None):
        self.name = name
# str - structure id
```

```
self.members =
members #
list[MemberDeclNode]
        self.token = token

# structure member
class
MemberDeclNode(ASTNode):
    def __init__(self, dtype,
name, token=None):
        self.dtype = dtype # str
- dtype
        self.name = name #
str - structure member id
        self.token = token

# structure variable
declaration
class
StructVarDeclNode(ASTNode):
    def __init__(self,
struct_type, var_name, inits,
token=None):
        self.struct_type =
struct_type # str - structure
id
        self.var_name =
var_name # str -
structure variable id
        self.inits = inits
#
list[PositionalInitNode|Name
dInitNode] | None
        self.token = token

# positional value in struct
initializer list
class
PositionalInitNode(ASTNode):
```



```
def __init__(self, value,
token=None):
    self.value = value #
expression node
    self.token = token

# member assignment:
$member = value
class
NamedInitNode(ASTNode):
    def __init__(self,
member_name, value,
token=None):
        self.member_name =
member_name # str -
structure member id
        self.value = value
# expression node
        self.token = token

# function definition
class
FuncDefNode(ASTNode):
    def __init__(self,
return_type, name, params,
local_decls, body,
return_expr, token=None):
        self.return_type =
return_type # str - rtype
        self.name = name
# str - function id
        self.params = params
# list[ParamNode]
        self.local_decls =
local_decls #
list[ASTNode]
        self.body = body
# list[ASTNode]
        self.return_expr =
return_expr # ASTNode |
None (None for ABYSS)
        self.token = token
```

```
# function parameter
class
ParamNode(ASTNode):
    def __init__(self, dtype,
name, token=None):
        self.dtype = dtype # str
- dtype
        self.name = name #
str - param id
        self.token = token

# assign statement
class
AssignNode(ASTNode):
    def __init__(self,
var_name, target_kind,
index1, index2, member,
value, token=None):
        self.var_name =
var_name # str - id
        self.target_kind =
target_kind # lhs type -
variable / array element /
structure member
        self.index1 = index1
# ASTNode | None
        self.index2 = index2
# ASTNode | None
        self.member = member
# str | None
        self.value = value
# ASTNode
        self.token = token

# compound assignment
class
CompoundAssignNode(AST
Node):
    def __init__(self,
var_name, target_kind,
```

```
index1, index2, member,
operator, value,
token=None):
        self.var_name =
var_name # str - id
        self.target_kind =
target_kind # lhs type -
variable / array element /
structure member
        self.index1 = index1
        self.index2 = index2
        self.member = member
        self.operator = operator
# compound assign operator
- +=, -=, *=, /=, %=, ^=
        self.value = value
# ASTNode
        self.token = token

# ASK statement
class AskNode(ASTNode):
    def __init__(self,
format_string, targets,
token=None):
        self.format_string =
format_string # str - format
string
        self.targets = targets
# list[AddressNode]
        self.token = token

# address of input
class
AddressNode(ASTNode):
    def __init__(self,
var_name, target_kind,
index1, index2, member,
token=None):
        self.var_name =
var_name # str - id
        self.target_kind =
target_kind # type - variable
```

```
/ array element / structure
member
    self.index1 = index1
    self.index2 = index2
    self.member = member
    self.token = token
```

```
# ECHO statement
class EchoNode(ASTNode):
    def __init__(self,
format_string, args,
token=None):
        self.format_string =
format_string # str - format
string
        self.args = args
# list[ASTNode]
        self.token = token
```

```
# LOOK statement
class LookNode(ASTNode):
    def __init__(self,
condition, body, droplooks,
drop_body, token=None):
        self.condition =
condition # ASTNode
(BOOL)
        self.body = body
# list[ASTNode]
        self.droplooks =
droplooks # list[(condition,
body)]
        self.drop_body =
drop_body #
list[ASTNode] | None
        self.token = token
```

```
# CHART statement
class ChartNode(ASTNode):
    def __init__(self, expr,
courses, adrift_body,
token=None):
```

```
        self.expr = expr
# ASTNode (COIN /
PARCH / SCROLL)
        self.courses = courses
# list[CourseNode]
        self.adrift_body =
adrift_body #
list[ASTNode] | None
        self.token = token
```

```
# COURSE statement
class
CourseNode(ASTNode):
    def __init__(self, value,
body, jump, token=None):
        self.value = value #
ASTNode (literal)
        self.body = body #
list[ASTNode]
        self.jump = jump #
'SAIL' | 'LAND' | None
        self.token = token
```

```
# HOIST statement
class HoistNode(ASTNode):
    def __init__(self, inits,
condition, updates, body,
jump, token=None):
        self.inits = inits #
list[HoistInitNode]
        self.condition =
condition # ASTNode
        self.updates = updates
# list[HoistUpdateNode]
        self.body = body
# list[ASTNode]
        self.jump = jump
# str | None
        self.token = token
```

```
# HOIST initialization
```

```
class
HoistInitNode(ASTNode):
    def __init__(self,
declares_new, var_name,
value, token=None):
        self.declares_new =
declares_new # BOOL
        self.var_name =
var_name # str
        self.value = value
# LiteralNode (COIN-lit)
        self.token = token
```

```
# HOIST update
class
HoistUpdateNode(ASTNode
):
    def __init__(self,
update_kind, var_name,
target_kind, index1,
member,
        unary_op,
compound_op, value,
token=None):
        self.update_kind =
update_kind # unary /
compound assignment
        self.var_name =
var_name
        self.target_kind =
target_kind # variable /
array element / member
        self.index1 = index1
        self.member = member
        self.unary_op =
unary_op # +# | -# |
None
        self.compound_op =
compound_op # str | None
        self.value = value
# ASTNode | None
        self.token = token
```

```

self.value = value
self.token = token

# HEAVE statement
class
HeaveNode(ASTNode):
    def __init__(self,
condition, body, jump,
token=None):
        self.condition =
condition
        self.body = body
        self.jump = jump
        self.token = token

# HAUL HEAVE statement
class
HaulHeaveNode(ASTNode):
    def __init__(self, body,
condition, jump,
token=None):
        self.body = body
        self.condition =
condition
        self.jump = jump
        self.token = token

# SAIL
class SailNode(ASTNode):
    def __init__(self,
token=None):
        self.token = token

# LAND
class LandNode(ASTNode):
    def __init__(self,
token=None):
        self.token = token

# return with value
class
ReturnNode(ASTNode):
    def __init__(self, value,
token=None):
        self.value = value
        self.token = token

# return with no value
class BackNode(ASTNode):
    def __init__(self,
token=None):
        self.token = token

# unary statement
class
UnaryStmtNode(ASTNode):
    def __init__(self, operator,
var_name, target_kind,
index1, index2, member,
token=None):
        self.operator = operator
        self.var_name =
var_name
        self.target_kind =
target_kind
        self.index1 = index1
        self.index2 = index2
        self.member = member
        self.token = token

# func()!! — function call as
statement (discards return).
class
FuncCallStmtNode(ASTNode):
    def __init__(self,
call_expr, token=None):
        self.call_expr =
call_expr # FuncCallNode
        self.token = token

# COIN-lit, DIME-lit,
PARCH-lit, SCROLL-lit,
AYE, NAY.
class
LiteralNode(ASTNode):
    def __init__(self, dtype,
value, token=None):
        self.dtype = dtype #
str
        self.value = value #
Python value
        self.token = token

class IdentNode(ASTNode):
    def __init__(self, name,
token=None):
        self.name = name
        self.token = token

# arr{i} or arr{i}{j} — 1D
or 2D array element access.
class
ArrayAccessNode(ASTNode):
    def __init__(self, name,
indices, token=None):
        self.name = name #
str
        self.indices = indices #
list[ASTNode] (length 1 or
2)
        self.token = token

class
MemberAccessNode(ASTNode):
    def __init__(self,
var_name, member_name,
token=None):
        self.var_name =
var_name
        self.member_name =
member_name
        self.token = token

```

```

class
ScrollCharAccessNode(AST
Node):
    def __init__(self,
scroll_expr, index,
token=None):
        self.scroll_expr =
scroll_expr # ASTNode
        self.index = index
# ASTNode (must be COIN)
        self.token = token

class
StringConcatNode(ASTNode):
    def __init__(self,
operands, token=None):
        self.operands =
operands # list[ASTNode]
        (all SCROLL)
        self.token = token

# func(arg1, arg2) —
function call expression.
class
FuncCallNode(ASTNode):
    def __init__(self, name,
args, token=None):
        self.name = name # str
        self.args = args #
list[ASTNode]
        self.token = token

# left OP right — arithmetic,
relational, equality, logical.

class
BinaryOpNode(ASTNode):
    def __init__(self, left,
operator, right,
token=None):
        self.left = left
        self.operator = operator
# str
        self.right = right
        self.token = token

class
UnaryOpNode(ASTNode):
    def __init__(self, operator,
operand, token=None):
        self.operator = operator
        self.operand = operand
        self.token = token

```

ast_parser.py

```

from syntax.predict_set
import PREDICT
from semantic.ast_nodes
import (
    ProgramNode,
    AhoyNode, ConstDeclNode,
    VarDeclNode,
    ArrayDeclNode,
    StructDefNode,
    MemberDeclNode,
    StructVarDeclNode,
    PositionalInitNode,
    NamedInitNode,
    FuncDefNode, ParamNode,
    AssignNode,
    CompoundAssignNode,
    AskNode, AddressNode,
    EchoNode, LookNode,
    ChartNode, CourseNode,

    HoistNode,
    HoistInitNode,
    HoistUpdateNode,
    HeaveNode,
    HaulHeaveNode,
    SailNode, LandNode,
    ReturnNode, BackNode,
    UnaryStmtNode,
    FuncCallStmtNode,
    LiteralNode, IdentNode,
    ArrayAccessNode,
    MemberAccessNode,
    ScrollCharAccessNode,
    StringConcatNode,
    FuncCallNode,
    BinaryOpNode,
    UnaryOpNode,
)
class ASTParser:

    def __init__(self, tokens,
source_code):
        ignored_types =
['whitespace', 'newline',
'single-comment',
'multi-comment']
        self.tokens = [t for t in
tokens if t.type not in
ignored_types]
        for t in self.tokens:
            if
t.type.startswith('id') and
t.type[2:].isdigit():
                t.type = 'id'
            self.pos = 0
            self.current_token =
self.tokens[0] if self.tokens
else None

        def advance(self):

```

```

        self.pos += 1
        self.current_token =
self.tokens[self.pos] if
self.pos < len(self.tokens)
else None

    def eat(self, token_type):
        if self.current_token
and self.current_token.type
== token_type:
            self.advance()
        else:
            found =
self.current_token.type if
self.current_token else 'EOF'
            raise RuntimeError(
                f'ASTParser bug:
expected {token_type!r}, got
{found!r} '
                f'at line
{getattr(self.current_token,
"line", "?")}')

    def get_production(self,
non_terminal):
        if not
self.current_token:
            return None
        return
PREDICT.get(non_terminal,
{}).get(self.current_token.ty
pe)

    def _arr1_values(self,
val_method):
        values = [val_method()]
        while self.current_token
and self.current_token.type
== ',':
            self.eat(',');
            values.append(val_method()
)

```

SeaStack

```

        return values

    def _arr2_rows(self,
val_method):
        rows = []
        while self.current_token
and self.current_token.type
== '[':
            self.eat('[');
            rows.append(self._arr1_valu
es(val_method)); self.eat(']')
            if self.current_token
and self.current_token.type
== ',':
                self.eat(',')
            return rows

    def build(self): return
self.program()

    # main program
    def program(self):
        tok = self.current_token
        global_decls =
self.global_dec()
        self.eat('AHOY');
        self.eat('('); self.eat(')');
        self.eat('[')
        local_decls =
self.ahoy_local_dec()
        statements =
self.ahoy_stmtnts()
        self.eat(']')
        return
ProgramNode(global_decls,
AhoyNode(local_decls,
statements, tok), tok)

    # global declarations
    def global_dec(self):
        nodes = []

```

```

        prod =
self.get_production('<global-
dec>')
        if prod == 2:
            nodes.extend(self.var_arr_fu
nc())
            elif prod == 3:
                nodes.extend(self.const())
            nodes.extend(self.global_dec
())
            elif prod == 4:
                nodes.extend(self.struct())
            nodes.extend(self.sub_func()
)
            elif prod == 5:
                nodes.append(self.nonreturn
_func())
            nodes.extend(self.sub_func()
)
            elif prod == 6:
                pass # λ
                return nodes

        # variable / array /
function
        def var_arr_func(self):
            prod =
self.get_production('<var-arr
-func>')
            if prod == 7:
                self.eat('COIN'); nt =
self.current_token;
                self.eat('id')

```

```
        return
self.coin_var_arr_func('COIN', nt)
    elif prod == 8:
        self.eat('DIME'); nt =
self.current_token;
self.eat('id')
        return
self.dime_var_arr_func('DIME', nt)
    elif prod == 9:
        self.eat('PARCH'); nt =
self.current_token;
self.eat('id')
        return
self.parch_var_arr_func('PARCH', nt)
    elif prod == 10:
        self.eat('SCROLL');
nt = self.current_token;
self.eat('id')
        return
self.scroll_var_arr_func('SCROLL', nt)
    elif prod == 11:
        self.eat('BOOL'); nt =
self.current_token;
self.eat('id')
        return
self.bool_var_arr_func('BOOL', nt)
    return []

def
coin_var_arr_func(self,
dtype, nt):
    prod =
self.get_production('<coin-var-arr-func>')
    nodes = []
    if prod == 12:
```

```
nodes.extend(self.coin_var_arr_func(
dtype, nt))
        self.eat('!!');
nodes.extend(self.global_dec
())
        elif prod == 13:
nodes.extend(self.coin_func(
dtype, nt))
        return nodes

def
dime_var_arr_func(self,
dtype, nt):
    prod =
self.get_production('<dime-var-arr-func>')
    nodes = []
    if prod == 58:
nodes.extend(self.dime_var_arr(
dtype, nt))
        self.eat('!!');
nodes.extend(self.global_dec
())
        elif prod == 59:
nodes.extend(self.dime_func(
dtype, nt))
        return nodes

def
parch_var_arr_func(self,
dtype, nt):
    prod =
self.get_production('<parch-var-arr-func>')
    nodes = []
    if prod == 98:
```

```
nodes.extend(self.parch_var_arr(
dtype, nt))
        self.eat('!!');
nodes.extend(self.global_dec
())
        elif prod == 99:
nodes.extend(self.parch_func(
dtype, nt))
        return nodes

def
scroll_var_arr_func(self,
dtype, nt):
    prod =
self.get_production('<scroll-var-arr-func>')
    nodes = []
    if prod == 122:
nodes.extend(self.scroll_var_arr(
dtype, nt))
        self.eat('!!');
nodes.extend(self.global_dec
())
        elif prod == 123:
nodes.extend(self.scroll_func(
dtype, nt))
        return nodes

def
bool_var_arr_func(self,
dtype, nt):
    prod =
self.get_production('<bool-var-arr-func>')
    nodes = []
    if prod == 164:
```

```

nodes.extend(self.bool_var_
arr(dtype, nt))
    self.eat('!!');
nodes.extend(self.global_dec
())
    elif prod == 165:

nodes.extend(self.bool_func(
dtype, nt))
    return nodes

# COIN variable / array
def coin_var_arr(self,
dtype, nt):
    prod =
self.get_production('<coin-v
ar-arr>')
    if prod == 14: return
self.coin_var(dtype, nt)
    elif prod == 15: return
[self.coin_arr(dtype, nt)]
    return []

def coin_var(self, dtype,
nt):
    init = self.coin_init()
    return
[VarDeclNode(dtype,
nt.value, init, nt)] +
self.coin_init_mult(dtype)

def coin_init(self):
    prod =
self.get_production('<coin-in
it>')
    if prod == 17:
        self.eat('='); return
self.coin_val()
    return None # prod 18:
λ

def coin_init_mult(self,
dtype):
    prod =
self.get_production('<coin-in
it-mult>')
    if prod == 19:
        self.eat(','); nt =
self.current_token;
self.eat('id')
        init = self.coin_init()
        return
[VarDeclNode(dtype,
nt.value, init, nt)] +
self.coin_init_mult(dtype)
    return [] # prod 20: λ

def coin_arr(self, dtype,
nt):
    self.eat('{'); d1t =
self.current_token;
self.eat('COIN-lit');
self.eat('{')
    return
self.coin_arr_tail(dtype, nt,
int(d1t.value))

def coin_arr_tail(self,
dtype, nt, d1):
    prod =
self.get_production('<coin-ar
r-tail>')
    if prod == 40:
        self.eat('=');
self.eat('['); vals =
self.coin_arr1(); self.eat(']')
    return
ArrayDeclNode(dtype,
nt.value, [d1], False, vals, nt)
    elif prod == 41:
        self.eat('{'); d2t =
self.current_token;
self.eat('COIN-lit');
self.eat('{')
    return
ArrayDeclNode(dtype,
nt.value, [d1, d2], True,
rows, nt)
    else: # prod 50: λ
        return
ArrayDeclNode(dtype,
nt.value, [d1, d2], True,
None, nt)

def coin_arr1(self):
    val = self.coin_arr_val()
    return [val] +
self.cav_tail()

def coin_arr_val(self):
    return self.coin_val()

def cav_tail(self):
    prod =
self.get_production('<cav-tai
l>')

```

```

        if prod == 47:
            self.eat(','); return
            self.coin_arr1()
            return []

        def coin_arr2(self):
            self.eat('['); row =
            self.coin_arr1(); self.eat(']')
            return [row] +
            self.cav2_tail()

        def cav2_tail(self):
            prod =
            self.get_production('<cav2-tail>')
            if prod == 52:
                self.eat(','); return
            self.coin_arr2()
            return []

        # DIME variable / array
        def dime_var_arr(self,
            dtype, nt):
            prod =
            self.get_production('<dime-var-arr>')
            if prod == 60: return
            self.dime_var(dtype, nt)
            elif prod == 61: return
            [self.dime_arr(dtype, nt)]
            return []

        def dime_var(self, dtype,
            nt):
            init = self.dime_init()
            return
            [VarDeclNode(dtype,
            nt.value, init, nt)] +
            self.dime_init_mult(dtype)

        def dime_init(self):

```

```

            prod =
            self.get_production('<dime-init>')
            if prod == 63:
                self.eat('='); return
                self.dime_val()
                return None

            def dime_init_mult(self,
            dtype):
                prod =
                self.get_production('<dime-init-mult>')
                if prod == 65:
                    self.eat(','); nt =
                    self.current_token;
                    self.eat('id')
                    init = self.dime_init()
                    return
                    [VarDeclNode(dtype,
                    nt.value, init, nt)] +
                    self.dime_init_mult(dtype)
                    return []

            def dime_arr(self, dtype,
            nt):
                self.eat('{'); d1t =
                self.current_token;
                self.eat('COIN-lit');
                self.eat('}')
                return
                self.dime_arr_tail(dtype, nt,
                int(d1t.value))

            def dime_arr_tail(self,
            dtype, nt, d1):
                prod =
                self.get_production('<dime-arr-tail>')
                if prod == 80:
                    self.eat('=');
                    self.eat('['); vals =

```

```

            self._arr1_values(self.dime_val); self.eat(']')
            return
            ArrayDeclNode(dtype,
            nt.value, [d1], False, vals, nt)
            elif prod == 81:
                self.eat('{'); d2t =
                self.current_token;
                self.eat('COIN-lit');
                self.eat('}')
                return
                self.dime_arr2_tail(dtype, nt,
                d1, int(d2t.value))
                else:
                    return
                    ArrayDeclNode(dtype,
                    nt.value, [d1], False, None,
                    nt)

            def dime_arr2_tail(self,
            dtype, nt, d1, d2):
                prod =
                self.get_production('<dime-arr2-tail>')
                if prod == 89:
                    self.eat('=');
                    self.eat('['); rows =
                    self._arr2_rows(self.dime_val); self.eat(']')
                    return
                    ArrayDeclNode(dtype,
                    nt.value, [d1, d2], True,
                    rows, nt)
                    else:
                        return
                        ArrayDeclNode(dtype,
                        nt.value, [d1, d2], True,
                        None, nt)

            # PARCH variable / array
            def parch_var_arr(self,
            dtype, nt):

```



```
prod =
self.get_production('<parch-
var-arr>')
    if prod == 100: return
self.parch_var(dtype, nt)
    elif prod == 101: return
[self.parch_arr(dtype, nt)]
    return []

def parch_var(self, dtype,
nt):
    init = self.parch_init()
    return
[VarDeclNode(dtype,
nt.value, init, nt)] +
self.parch_init_mult(dtype)

def parch_init(self):
    prod =
self.get_production('<parch-i
nit>')
    if prod == 103:
self.eat('='); return
self.parch_val()
    return None

def parch_init_mult(self,
dtype):
    prod =
self.get_production('<parch-i
nit-mult>')
    if prod == 105:
        self.eat(','); nt =
self.current_token;
self.eat('id')
        init =
self.parch_init()
        return
[VarDeclNode(dtype,
nt.value, init, nt)] +
self.parch_init_mult(dtype)
    return []
```

SeaStack

```
def parch_arr(self, dtype,
nt):
    self.eat('{'); d1t =
self.current_token;
self.eat('COIN-lit');
self.eat('}')
    return
self.parch_arr_tail(dtype, nt,
int(d1t.value))

def parch_arr_tail(self,
dtype, nt, d1):
    prod =
self.get_production('<parch-
arr-tail>')
    if prod == 110:
        self.eat('=');
self.eat('['); vals =
self._arr1_values(self.parch_
val); self.eat(']')
        return
ArrayDeclNode(dtype,
nt.value, [d1], False, vals, nt)
    elif prod == 111:
        self.eat('{'); d2t =
self.current_token;
self.eat('COIN-lit');
self.eat('}')
        return
self.parch_arr2_tail(dtype,
nt, d1, int(d2t.value))
    else:
        return
ArrayDeclNode(dtype,
nt.value, [d1], False, None,
nt)

def parch_arr2_tail(self,
dtype, nt, d1, d2):
```

```
prod =
self.get_production('<parch-
arr2-tail>')
    if prod == 116:
        self.eat('=');
self.eat('['); rows =
self._arr2_rows(self.parch_v
al); self.eat(']')
        return
ArrayDeclNode(dtype,
nt.value, [d1, d2], True,
rows, nt)
    else:
        return
ArrayDeclNode(dtype,
nt.value, [d1, d2], True,
None, nt)

# SCROLL variable /
array
def scroll_var_arr(self,
dtype, nt):
    prod =
self.get_production('<scroll-
var-arr>')
    if prod == 124: return
self.scroll_var(dtype, nt)
    elif prod == 125: return
[self.scroll_arr(dtype, nt)]
    return []

def scroll_var(self, dtype,
nt):
    init = self.scroll_init()
    return
[VarDeclNode(dtype,
nt.value, init, nt)] +
self.scroll_init_mult(dtype)

def scroll_init(self):
```

```
        prod =
self.get_production('<scroll-i
nit>')
        if prod == 127:
self.eat('='); return
self.scroll_val()
        return None

def scroll_init_mult(self,
dtype):
    prod =
self.get_production('<scroll-i
nit-mult>')
        if prod == 129:
            self.eat(','); nt =
self.current_token;
self.eat('id')
            init =
self.scroll_init()
            return
[VarDeclNode(dtype,
nt.value, init, nt)] +
self.scroll_init_mult(dtype)
        return []

def scroll_arr(self, dtype,
nt):
    self.eat('{'); d1t =
self.current_token;
self.eat('COIN-lit');
self.eat('{')
        return
self.scroll_arr_tail(dtype, nt,
int(d1t.value))

def scroll_arr_tail(self,
dtype, nt, d1):
    prod =
self.get_production('<scroll-
arr-tail>')
        if prod == 146:
```

```
        self.eat('=');
self.eat('['); vals =
self._arr1_values(self.scroll_
val); self.eat(']')
        return
ArrayDeclNode(dtype,
nt.value, [d1], False, vals, nt)
        elif prod == 147:
            self.eat('{'); d2t =
self.current_token;
self.eat('COIN-lit');
self.eat('{')
            return
self.scroll_arr2_tail(dtype,
nt, d1, int(d2t.value))
        else:
            return
ArrayDeclNode(dtype,
nt.value, [d1], False, None,
nt)

def scroll_arr2_tail(self,
dtype, nt, d1, d2):
    prod =
self.get_production('<scroll-
arr2-tail>')
        if prod == 155:
            self.eat('=');
self.eat('['); rows =
self._arr2_rows(self.scroll_v
al); self.eat(']')
            return
ArrayDeclNode(dtype,
nt.value, [d1, d2], True,
rows, nt)
        else:
            return
ArrayDeclNode(dtype,
nt.value, [d1, d2], True,
None, nt)

# BOOL variable / array
```

```
def bool_var_arr(self,
dtype, nt):
    prod =
self.get_production('<bool-v
ar-arr>')
        if prod == 166: return
self.bool_var(dtype, nt)
        elif prod == 167: return
[self.bool_arr(dtype, nt)]
        return []

def bool_var(self, dtype,
nt):
    init = self.bool_init()
    return
[VarDeclNode(dtype,
nt.value, init, nt)] +
self.bool_init_mult(dtype)

def bool_init(self):
    prod =
self.get_production('<bool-in
it>')
        if prod == 169:
self.eat('='); return
self.bool_val()
        return None

def bool_init_mult(self,
dtype):
    prod =
self.get_production('<bool-in
it-mult>')
        if prod == 171:
            self.eat(','); nt =
self.current_token;
self.eat('id')
            init = self.bool_init()
            return
[VarDeclNode(dtype,
nt.value, init, nt)] +
self.bool_init_mult(dtype)
```

```
        return []

    def bool_arr(self, dtype,
nt):
        self.eat('{'); d1t =
self.current_token;
self.eat('COIN-lit');
self.eat('}')
        return
self.bool_arr_tail(dtype, nt,
int(d1t.value))

    def bool_arr_tail(self,
dtype, nt, d1):
        prod =
self.get_production('<bool-ar
r-tail>')
        if prod == 261:
            self.eat('=');
self.eat('['); vals =
self._arr1_values(self.bool_v
al); self.eat(']')
            return
ArrayDeclNode(dtype,
nt.value, [d1], False, vals, nt)
        elif prod == 262:
            self.eat('{'); d2t =
self.current_token;
self.eat('COIN-lit');
self.eat('}')
            return
self.bool_arr2_tail(dtype, nt,
d1, int(d2t.value))
        else:
            return
ArrayDeclNode(dtype,
nt.value, [d1], False, None,
nt)

    def bool_arr2_tail(self,
dtype, nt, d1, d2):

        prod =
self.get_production('<bool-ar
r2-tail>')
        if prod == 296:
            self.eat('=');
self.eat('['); rows =
self._arr2_rows(self.bool_va
l); self.eat(']')
            return
ArrayDeclNode(dtype,
nt.value, [d1, d2], True,
rows, nt)
        else:
            return
ArrayDeclNode(dtype,
nt.value, [d1, d2], True,
None, nt)

        # constants
        def const(self):
            self.eat('LOCKE');
nodes = self.const_init();
self.eat('!!'); return nodes

        def const_init(self):
            prod =
self.get_production('<const-i
nit>')
            if prod == 443:
                self.eat('COIN'); return
self.coin_locke_list('COIN')
            elif prod == 444:
                self.eat('DIME'); return
self.dime_locke_list('DIME')
            elif prod == 445:
                self.eat('PARCH'); return
self.parch_locke_list('PARC
H')
            elif prod == 446:
                self.eat('SCROLL'); return
self.scroll_locke_list('SCRO
LL')

            elif prod == 447:
                self.eat('BOOL'); return
self.bool_locke_list('BOOL')
                return []

            def coin_locke_list(self,
dtype):
                nodes = []
                nt = self.current_token;
self.eat('id'); self.eat('=')
                vt = self.current_token;
self.eat('COIN-lit')

                nodes.append(ConstDeclNo
de(dtype, nt.value,
LiteralNode('COIN',
int(vt.value), vt), nt))
                while
self.get_production('<coin-lo
cke-mult>') == 449:
                    self.eat(','); nt =
self.current_token;
self.eat('id'); self.eat('=')
                    vt =
self.current_token;
self.eat('COIN-lit')

                nodes.append(ConstDeclNo
de(dtype, nt.value,
LiteralNode('COIN',
int(vt.value), vt), nt))
                return nodes

            def dime_locke_list(self,
dtype):
                nodes = []
                nt = self.current_token;
self.eat('id'); self.eat('=')
                vt = self.current_token
                if vt.type == 'COIN-lit':
```

```
        self.eat('COIN-lit');
lit = LiteralNode('COIN',
int(vt.value), vt)
    else:
        self.eat('DIME-lit');
lit = LiteralNode('DIME',
float(vt.value), vt)

nodes.append(ConstDeclNo
de(dtype, nt.value, lit, nt))
    while
self.get_production('<dime-l
ocke-mult>') == 454:
        self.eat(','); nt =
self.current_token;
self.eat('id'); self.eat('=')
        vt =
self.current_token
        if vt.type ==
'COIN-lit':

self.eat('COIN-lit'); lit =
LiteralNode('COIN',
int(vt.value), vt)
    else:

self.eat('DIME-lit'); lit =
LiteralNode('DIME',
float(vt.value), vt)

nodes.append(ConstDeclNo
de(dtype, nt.value, lit, nt))
    return nodes

def parch_locke_list(self,
dtype):
    nodes = []
    nt = self.current_token;
self.eat('id'); self.eat('=')
    vt = self.current_token;
self.eat('PARCH-lit')
```

```
nodes.append(ConstDeclNo
de(dtype, nt.value,
LiteralNode('PARCH',
vt.value, vt), nt))
    while
self.get_production('<parch-l
ocke-mult>') == 457:
        self.eat(','); nt =
self.current_token;
self.eat('id'); self.eat('=')
        vt =
self.current_token;
self.eat('PARCH-lit')

nodes.append(ConstDeclNo
de(dtype, nt.value,
LiteralNode('PARCH',
vt.value, vt), nt))
    return nodes

def scroll_locke_list(self,
dtype):
    nodes = []
    nt = self.current_token;
self.eat('id'); self.eat('=')
    st = self.current_token;
self.eat('SCROLL-lit')
    base =
LiteralNode('SCROLL',
st.value, st)
    val =
self._scr_id_const(base, st)

nodes.append(ConstDeclNo
de(dtype, nt.value, val, nt))
    while
self.get_production('<scroll-l
ocke-mult>') == 462:
        self.eat(','); nt =
self.current_token;
self.eat('id'); self.eat('=')
```

```
st =
self.current_token;
self.eat('SCROLL-lit')
    base =
LiteralNode('SCROLL',
st.value, st)
    val =
self._scr_id_const(base, st)

nodes.append(ConstDeclNo
de(dtype, nt.value, val, nt))
    return nodes

def _scr_id_const(self,
base, tok):
    if self.current_token
and self.current_token.type
== '{':
        self.eat('{'); it =
self.current_token;
self.eat('COIN-lit');
self.eat('}')
        return
ScrollCharAccessNode(base,
LiteralNode('COIN',
int(it.value), it), tok)
    return base

def bool_locke_list(self,
dtype):
    nodes = []
    nt = self.current_token;
self.eat('id'); self.eat('=')
    vt = self.current_token
    if vt.type == 'AYE':
self.eat('AYE'); lit =
LiteralNode('BOOL', True,
vt)
    else: self.eat('NAY'); lit
= LiteralNode('BOOL',
False, vt)
```

```

nodes.append(ConstDeclNode(dtype, nt.value, lit, nt))
    while
self.get_production('<bool-logic-mult>') == 467:
    self.eat(','); nt =
self.current_token;
self.eat('id'); self.eat('=')
    vt =
self.current_token
    if vt.type == 'AYE':
self.eat('AYE'); lit =
LiteralNode('BOOL', True,
vt)
    else: self.eat('NAY');
lit = LiteralNode('BOOL',
False, vt)

nodes.append(ConstDeclNode(dtype, nt.value, lit, nt))
    return nodes

# structure definition
def struct(self):
    nodes = []
    while self.current_token
and self.current_token.type
== 'MAST':
        prod =
self.get_production('<struct>')
        if prod != 469: break
        self.eat('MAST'); nt
= self.current_token;
self.eat('id'); self.eat('[')
            members =
self.mem_dec() +
self.mem_dec_tail()
            self.eat(']');
self.eat('!!!')

nodes.append(StructDefNode(nt.value, members, nt))
    return nodes

def mem_dec(self):
    dtype = self.d_type(); nt
= self.current_token;
self.eat('id')
    members =
[MemberDeclNode(dtype,
nt.value, nt)] +
self.mem_mult(dtype)
    self.eat('!!!'); return
members

def mem_mult(self,
dtype):
    prod =
self.get_production('<mem-mult>')
    if prod == 472:
        self.eat(','); nt =
self.current_token;
self.eat('id')
        return
[MemberDeclNode(dtype,
nt.value, nt)] +
self.mem_mult(dtype)
    return []

def mem_dec_tail(self):
    prod =
self.get_production('<mem-dec-tail>')
    if prod == 474: return
self.mem_dec() +
self.mem_dec_tail()
    return []

# function definition
def sub_func(self):
    nodes = []
    while True:
        prod =
self.get_production('<sub-func>')
        if prod == 476:
nodes.extend(self.return_func())
        elif prod == 477:
nodes.append(self.nonreturn_func())
        else:
            break
    return nodes

def return_func(self):
    prod =
self.get_production('<return-func>')
    dtype_map = {479:
'COIN', 480: 'DIME', 481:
'PARCH', 482: 'SCROLL',
483: 'BOOL'}
    dtype =
dtype_map[prod]
    self.eat(dtype); nt =
self.current_token;
self.eat('id')
    return
self._build_return_func(dtype, nt)

def
_build_return_func(self,
dtype, nt):
    self.eat('('); params =
self.params(); self.eat(')');
self.eat('[')
        local_decls =
self.local_dec()

```

```
        body = self.ret_stmnts()
        self.eat('BACK');
    return_expr =
    self._return_val_for(dtype)
        self.eat('!!'); self.eat(']')
        node =
    FuncDefNode(dtype,
    nt.value, params,
    local_decls, body,
    return_expr, nt)
        return [node] +
    self.sub_func()
```

```
        def coin_func(self, dtype,
    nt): return
    self._build_return_func(dtype
    e, nt)
        def dime_func(self, dtype,
    nt): return
    self._build_return_func(dtype
    e, nt)
        def parch_func(self,
    dtype, nt): return
    self._build_return_func(dtype
    e, nt)
        def scroll_func(self,
    dtype, nt): return
    self._build_return_func(dtype
    e, nt)
        def bool_func(self, dtype,
    nt): return
    self._build_return_func(dtype
    e, nt)
```

```
        def _return_val_for(self,
    dtype):
            if dtype == 'COIN':
    return self.coin_val()
            if dtype == 'DIME':
    return self.dime_val()
            if dtype == 'PARCH':
    return self.parch_val()
```

```
        if dtype == 'SCROLL':
    return self.scroll_val()
            if dtype == 'BOOL':
    return self.bool_val()

        def nonreturn_func(self):
            tok =
    self.current_token;
    self.eat('ABYSS')
            nt = self.current_token;
    self.eat('id')
            self.eat('('); params =
    self.params(); self.eat(')');
    self.eat('[')
            local_decls =
    self.ahoy_local_dec()
            body =
    self.nonret_stmnts()
            back =
    self.nonret_back()
            if back:
    body.append(back)
            self.eat(']')
            return
    FuncDefNode('ABYSS',
    nt.value, params,
    local_decls, body, None, tok)
```

```
        def nonret_stmnts(self):
            first =
    self.nonret_stmnt()
            return [first] +
    self.nonret_tail()
```

```
        def nonret_tail(self):
            prod =
    self.get_production('<nonret-
    tail>')
            if prod == 486: return
    self.nonret_stmnts()
            return []
```

```
        def nonret_back(self):
            prod =
    self.get_production('<nonret-
    back>')
            if prod == 488:
                tok =
    self.current_token;
    self.eat('BACK');
    self.eat('!!')
            return
    BackNode(tok)
            return None
```

```
        # parameters
        def params(self):
            prod =
    self.get_production('<params
    >')
            if prod == 331:
                dtype = self.d_type();
    nt = self.current_token;
    self.eat('id')
            return
    [ParamNode(dtype, nt.value,
    nt)] + self.param_mult()
            return []
```

```
        def param_mult(self):
            prod =
    self.get_production('<param-
    mult>')
            if prod == 333:
    self.eat(','); return
    self.params()
            return []
```

```
        def d_type(self):
            tok =
    self.current_token;
    self.eat(tok.type); return
    tok.type
```

```
# local declarations
def local_dec(self):
    nodes = []
    prod =
self.get_production('<local-dec>')
    while prod == 502:

nodes.extend(self.var_arr());
prod =
self.get_production('<local-dec>')
    if prod == 503:
nodes.extend(self.struct_dec(
))
    return nodes

def ahoy_local_dec(self):
    nodes = []
    prod =
self.get_production('<ahoy-local-dec>')
    while prod == 666:

nodes.extend(self.var_arr());
prod =
self.get_production('<ahoy-local-dec>')
    if prod == 667:
nodes.extend(self.ahoy_struct_dec())
    return nodes

def var_arr(self):
    prod =
self.get_production('<var-arr>')
    if prod == 505:
        self.eat('COIN'); nt =
self.current_token;
self.eat('id')
```

```
        n =
self.coin_var_arr('COIN',
nt); self.eat('!!'); return n
        elif prod == 506:
            self.eat('DIME'); nt =
self.current_token;
self.eat('id')
            n =
self.dime_var_arr('DIME',
nt); self.eat('!!'); return n
            elif prod == 507:
                self.eat('PARCH'); nt
= self.current_token;
self.eat('id')
                n =
self.parch_var_arr('PARCH',
nt); self.eat('!!'); return n
                elif prod == 508:
                    self.eat('SCROLL');
nt = self.current_token;
self.eat('id')
                    n =
self.scroll_var_arr('SCROLL',
nt); self.eat('!!'); return n
                    elif prod == 509:
                        self.eat('BOOL'); nt =
self.current_token;
self.eat('id')
                        n =
self.bool_var_arr('BOOL',
nt); self.eat('!!'); return n
                    return []

def struct_dec(self):
    nodes = []
    prod =
self.get_production('<struct-dec>')
    while prod == 510:
        self.eat('MAST'); tt =
self.current_token;
self.eat('id')
```

```
        vt =
self.current_token;
self.eat('id')
        inits, extras =
self.str_dec_init();
self.eat('!!')

nodes.append(StructVarDecl
Node(tt.value, vt.value, inits,
vt))
        for en in extras:
nodes.append(StructVarDecl
Node(tt.value, en, None, vt))
        prod =
self.get_production('<struct-dec>')
        return nodes

def ahoy_struct_dec(self):
    nodes = []
    prod =
self.get_production('<ahoy-struct-dec>')
    while prod == 669:
        self.eat('MAST'); tt =
self.current_token;
self.eat('id')
        vt =
self.current_token;
self.eat('id')
        inits, extras =
self.str_dec_init();
self.eat('!!')

nodes.append(StructVarDecl
Node(tt.value, vt.value, inits,
vt))
        for en in extras:
nodes.append(StructVarDecl
Node(tt.value, en, None, vt))
```

```
        prod =
self.get_production('<ahoy-str-
duct-dec>')
        return nodes

    def str_dec_init(self):
        prod =
self.get_production('<str-dec
-init>')
        if prod == 512:
            extras = [];
self.eat(',')
            nt =
self.current_token;
self.eat('id');
extras.append(nt.value)

        extras.extend(self.str_dec_tai
l()); return None, extras
        elif prod == 513:
            self.eat('=');
self.eat('['); inits =
self.str_val_list(); self.eat(']')
            return inits, []
            return None, []

    def str_dec_tail(self):
        prod =
self.get_production('<str-dec
-tail>')
        if prod == 515:
            self.eat(','); nt =
self.current_token;
self.eat('id')
            return [nt.value] +
self.str_dec_tail()
            return []

    def str_val_list(self):
        inits = [self.str_val()]
```

```
        prod =
self.get_production('<str-val-
tail>')
        while prod == 519:
            self.eat(',');
inits.append(self.str_val())
            prod =
self.get_production('<str-val-
tail>')
            return inits

    def str_val(self):
        prod =
self.get_production('<str-val
>')
        if prod == 518:
            self.eat('$'); nt =
self.current_token;
self.eat('id'); self.eat('=')
            return
NamedInitNode(nt.value,
self.value_str(), nt)
        else:
            return
PositionalInitNode(self.valu
e_str(), self.current_token)

        # expressions in structure
    def value_str(self):
        prod =
self.get_production('<value-s
tr>')
        if prod == 521:
            tok =
self.current_token;
self.eat('id')
            node =
self.id_tail(tok.value, tok)
            return
self._exp_str(node)
        elif prod == 522:
```

```
            self.eat('('); node =
self.value_grp(); self.eat(')')
            return
self._exp_str(node)
        elif prod == 523:
            node =
self.var_digit()
            return
self._digit_tail_str(node)
        elif prod == 524:
            tok =
self.current_token;
self.eat('PARCH-lit')
            return
self._parch_tail_str(LiteralN
ode('PARCH', tok.value,
tok))
        elif prod == 525:
            tok =
self.current_token;
self.eat('SCROLL-lit')
            base =
LiteralNode('SCROLL',
tok.value, tok)
            return
self._scroll_tail_str(self.scr
char_opt(base, tok))
        else:
            return self.bool_val()

    def _exp_str(self, left):
        return
self._exp_shared(left)

    def _digit_tail_str(self,
node):
        prod =
self.get_production('<digit-ta
il-str>')
        if prod == 527:
            node =
self.arith_fold(node)
```



```
        return
self._releq_str(node)
    return node

    def _releq_str(self, left):
        prod =
self.get_production('<releq-str>')
        if prod == 531:
            op_tok =
self.current_token; op =
self.rel_op()
            right =
self.arith_fold(self.dime_ope
())
            node =
BinaryOpNode(left, op,
right, op_tok)
            return
self._logeq_str(node)
            elif prod == 532:
                op_tok =
self.current_token; op =
self.eq_op()
                right =
self.arith_fold(self.dime_ope
())
                node =
BinaryOpNode(left, op,
right, op_tok)
                return
self.bool_exp_fold(node)
                return left

            def _logeq_str(self, left):
                prod =
self.get_production('<logeq-
str>')
                if prod == 538:
                    op_tok =
self.current_token; op =
self._logeq_op()
```

SeaStack

```
        right =
self.bool_ope()
        node =
BinaryOpNode(left, op,
right, op_tok)
        return
self.bool_exp_fold(node)
        return left

        def _parch_tail_str(self,
left):
            prod =
self.get_production('<parch-t
ail-str>')
            if prod == 540:
                op_tok =
self.current_token; op =
self.eq_op()
                rt =
self.current_token;
self.eat('PARCH-lit')
                node =
BinaryOpNode(left, op,
LiteralNode('PARCH',
rt.value, rt), op_tok)
                return
self.bool_exp_fold(node)
                return left

            def _scroll_tail_str(self,
left):
                prod =
self.get_production('<scroll-t
ail-str>')
                if prod == 542: return
self.scroll_concat_fold(left)
                elif prod == 543:
                    op_tok =
self.current_token; op =
self.eq_op()
                    right =
self.scroll_val()
```

```
        node =
BinaryOpNode(left, op,
right, op_tok)
        return
self.bool_exp_fold(node)
        return left

        # statements
        def ret_stmts(self):
            stmts = []
            prod =
self.get_production('<ret-stm
nts>')
            while prod == 340:

                stmts.append(self.statements
()); prod =
self.get_production('<ret-stm
nts>')
                return stmts

            def ahoy_stmts(self):
                stmts = []
                prod =
self.get_production('<ahoy-s
tmnts>')
                if prod == 671:

                    stmts.append(self.ahoy_stmn
t());
                    stmts.extend(self.ahoy_tail()
)
                    return stmts

            def ahoy_tail(self):
                prod =
self.get_production('<ahoy-t
ail>')
                if prod == 672: return
self.ahoy_stmts()
                return []
```

```
def ahoy_stmnt(self):
    prod =
self.get_production('<ahoy-s
tmnt>')
    if prod == 674:
        nt =
self.current_token;
self.eat('id')
        n =
self.assign_tail(nt);
self.eat('!!!'); return n
    elif prod == 675: return
self.ask_stmnt()
    elif prod == 676: return
self.echo_stmnt()
    elif prod == 677: return
self._look_stmnt_with_tail('a
hoy')
    elif prod == 678: return
self.chart_stmnt()
    elif prod == 679: return
self.hoist_stmnt()
    elif prod == 680: return
self.heave_stmnt()
    elif prod == 681: return
self.haul_stmnt()
    elif prod == 682:
        n = self.unary_exp();
self.eat('!!!'); return n

def nonret_stmnt(self):
    prod =
self.get_production('<nonret-
stmnt>')
    if prod == 490:
        nt =
self.current_token;
self.eat('id')
        n =
self.assign_tail(nt);
self.eat('!!!'); return n
```

```
    elif prod == 491: return
self.ask_stmnt()
    elif prod == 492: return
self.echo_stmnt()
    elif prod == 493: return
self._look_stmnt_with_tail('
nonret')
    elif prod == 494: return
self.chart_stmnt()
    elif prod == 495: return
self.hoist_stmnt()
    elif prod == 496: return
self.heave_stmnt()
    elif prod == 497: return
self.haul_stmnt()
    elif prod == 498:
        n = self.unary_exp();
self.eat('!!!'); return n

def statements(self):
    prod =
self.get_production('<statem
ents>')
    if prod == 553: return
self.assign_stmnt()
    elif prod == 554: return
self.ask_stmnt()
    elif prod == 555: return
self.echo_stmnt()
    elif prod == 556: return
self.look_stmnt()
    elif prod == 557: return
self.chart_stmnt()
    elif prod == 558: return
self.hoist_stmnt()
    elif prod == 559: return
self.heave_stmnt()
    elif prod == 560: return
self.haul_stmnt()
    elif prod == 561:
        n = self.unary_exp();
self.eat('!!!'); return n
```

```
# assignment
def assign_stmnt(self):
    nt = self.current_token;
self.eat('id')
    n = self.assign_tail(nt);
self.eat('!!!'); return n

def assign_tail(self, nt):
    prod =
self.get_production('<assign-
tail>')
    if prod == 563:
        if self.current_token
and self.current_token.type
in ('{', '$'):
            tk, i1, i2, m =
self.arr_str()
        else:
            tk, i1, i2, m = 'var',
None, None, None
        return
self.assign_body(nt, tk, i1,
i2, m)
    elif prod == 564:
        self.eat('('); args =
self.args(); self.eat(')')
        return
FuncCallStmtNode(FuncCall
Node(nt.value, args, nt), nt)

def arr_str(self):
    prod =
self.get_production('<arr-str
>')
    if prod == 565:
        self.eat('{'); idx1 =
self.index_expr();
self.eat('{')
        prod2 =
self.get_production('<arr-str-
tail>')
```

```
if prod2 == 568:
    self.eat('{'); idx2 =
self.index_expr();
self.eat('}')
    return 'array2d',
idx1, idx2, None
    return 'array1d', idx1,
None, None
    elif prod == 566:
        self.eat('$'); mt =
self.current_token;
self.eat('id')
        return 'member',
None, None, mt.value
    return 'var', None,
None, None
```

```
def assign_body(self, nt,
tk, i1, i2, m):
    prod =
self.get_production('<assign-
body>')
    if prod == 570:
        self.eat('='); val =
self.value_as()
        return
AssignNode(nt.value, tk, i1,
i2, m, val, nt)
    elif prod == 571:
        ot =
self.current_token; op =
self.arith_assign_op()
        val = self.dime_val()
        return
CompoundAssignNode(nt.va
lue, tk, i1, i2, m, op, val, ot)
```

```
# assignment value
def value_as(self):
    prod =
self.get_production('<value-
as>')
```

```
if prod == 572:
    tok =
self.current_token;
self.eat('id')
    node =
self.id_tail(tok.value, tok)
    return
self._exp_as(node)
    elif prod == 573:
        self.eat('('); node =
self.value_grp(); self.eat(')')
        return
self._exp_as(node)
    elif prod == 574:
        return
self._digit_tail_as(self.var_di
git())
    elif prod == 575:
        tok =
self.current_token;
self.eat('PARCH-lit')
        return
self._parch_tail_as(LiteralN
ode('PARCH', tok.value,
tok))
    elif prod == 576:
        tok =
self.current_token;
self.eat('SCROLL-lit')
        base =
LiteralNode('SCROLL',
tok.value, tok)
        return
self._scroll_tail_as(self.scr_c
har_opt(base, tok))
    else:
        return self.bool_val()
```

```
def _exp_as(self, left):
    return
self._exp_shared(left)
```

```
def _digit_tail_as(self,
node):
    prod =
self.get_production('<digit-ta
il-as>')
    if prod == 578:
        node =
self.arith_fold(node)
        return
self._releq_as(node)
    return node
```

```
def _releq_as(self, left):
    prod =
self.get_production('<releq-a
s>')
    if prod == 582:
        op_tok =
self.current_token; op =
self.rel_op()
        right =
self.arith_fold(self.dime_ope
())
        node =
BinaryOpNode(left, op,
right, op_tok)
        return
self._logeq_as(node)
    elif prod == 583:
        op_tok =
self.current_token; op =
self.eq_op()
        right =
self.arith_fold(self.dime_ope
())
        node =
BinaryOpNode(left, op,
right, op_tok)
        return
self.bool_exp_fold(node)
    return left
```

```
def _logeq_as(self, left):
    prod =
self.get_production('<logeq-
as>')
    if prod == 589:
        op_tok =
self.current_token; op =
self._logeq_op()
        right =
self.bool_ope()
        node =
BinaryOpNode(left, op,
right, op_tok)
        return
self.bool_exp_fold(node)
        return left

def _parch_tail_as(self,
left):
    prod =
self.get_production('<parch-t
ail-as>')
    if prod == 591:
        op_tok =
self.current_token; op =
self.eq_op()
        rt =
self.current_token;
self.eat('PARCH-lit')
        node =
BinaryOpNode(left, op,
LiteralNode('PARCH',
rt.value, rt), op_tok)
        return
self.bool_exp_fold(node)
        return left

def _scroll_tail_as(self,
left):
    prod =
self.get_production('<scroll-t
ail-as>')
```

```
    if prod == 593: return
self.scroll_concat_fold(left)
    elif prod == 594:
        op_tok =
self.current_token; op =
self.eq_op()
        right =
self.scroll_val()
        node =
BinaryOpNode(left, op,
right, op_tok)
        return
self.bool_exp_fold(node)
        return left

def arith_assign_op(self):
    tok =
self.current_token;
self.eat(tok.type); return
tok.type

# input / output
def ask_stmnt(self):
    tok = self.current_token
    self.eat('ASK');
self.eat('(')
    ft = self.current_token;
self.eat('SCROLL-lit');
self.eat(',')
    targets = self.addr_list()
    self.eat(')'); self.eat('!!!')
    return
AskNode(ft.value, targets,
tok)

def addr_list(self):
    targets = [self.addr()]
    while
self.get_production('<addr-ta
il>') == 612:
        self.eat(',');
targets.append(self.addr())
```

```
    return targets

def addr(self):
    tok =
self.current_token;
self.eat('@')
    nt = self.current_token;
self.eat('id')
    if self.current_token
and self.current_token.type
in ('{', '$'):
        tk, i1, i2, m =
self.arr_str()
    else:
        tk, i1, i2, m = 'var',
None, None, None
    return
AddressNode(nt.value, tk,
i1, i2, m, tok)

def echo_stmnt(self):
    tok = self.current_token
    self.eat('ECHO');
self.eat('(')
    ft = self.current_token;
self.eat('SCROLL-lit')
    args = self._echo_args()
    self.eat(')'); self.eat('!!!')
    return
EchoNode(ft.value, args,
tok)

def _echo_args(self):
    args = []
    while
self.get_production('<args-m
ult>') == 350:
        self.eat(',');
args.append(self.value())
    return args

# LOOK statement
```

```
def look_stmtnt(self):
    return
self._look_stmtnt_with_tail('ret')

def
_look_stmtnt_with_tail(self,
ctx):
    tok = self.current_token
    self.eat('LOOK');
self.eat('('); cond =
self.condition(); self.eat(')');
self.eat('[')
    body =
self.look_body(); jump =
self.jump_stmtnt()
    if jump:
body.append(jump)
    self.eat(']')
    if ctx == 'ahoy': dl, db
= self.ahoy_look_tail()
    elif ctx == 'nonret': dl,
db = self.nonret_look_tail()
    else: dl, db =
self.look_tail()
    return LookNode(cond,
body, dl, db, tok)

def condition(self):
    return
self.bool_grp_val()

def look_body(self):
    stmts = []
    prod =
self.get_production('<look-b
ody>')
    while prod == 617:

stmts.append(self.statements
()); prod =
```

```
self.get_production('<look-b
ody>')
    return stmts

def jump_stmtnt(self):
    prod =
self.get_production('<jump-s
tmnt>')
    tok = self.current_token
    if prod == 619:
self.eat('SAIL'); self.eat('!!');
return SailNode(tok)
    elif prod == 620:
self.eat('LAND');
self.eat('!!'); return
LandNode(tok)
    return None

def look_tail(self):
    droplooks, drop = [],
None
    prod =
self.get_production('<look-ta
il>')
    while prod == 622:

self.eat('DROLOOK');
self.eat('('); c =
self.condition(); self.eat(')');
self.eat('[')
        b = self.look_body();
j = self.jump_stmtnt()
        if j: b.append(j)
        self.eat(']');
droplooks.append((c, b))
        prod =
self.get_production('<look-ta
il>')
        if prod == 623:
            self.eat('DROP');
self.eat('[')
```

```
drop =
self.look_body(); j =
self.jump_stmtnt()
    if j: drop.append(j)
    self.eat(']')
    return droplooks, drop

def ahoy_look_tail(self):
    droplooks, drop = [],
None
    prod =
self.get_production('<ahoy-l
ook-tail>')
    while prod == 683:

self.eat('DROLOOK');
self.eat('('); c =
self.condition(); self.eat(')');
self.eat('[')
        b = self.look_body();
j = self.jump_stmtnt()
        if j: b.append(j)
        self.eat(']');
droplooks.append((c, b))
        prod =
self.get_production('<ahoy-l
ook-tail>')
        if prod == 684:
            self.eat('DROP');
self.eat('[')
            drop =
self.look_body(); j =
self.jump_stmtnt()
            if j: drop.append(j)
            self.eat(']')
            return droplooks, drop

def nonret_look_tail(self):
    droplooks, drop = [],
None
```

```
        prod =
self.get_production('<nonret-
look-tail>')
        while prod == 499:

self.eat('DROPLOOK');
self.eat('('); c =
self.condition(); self.eat(')');
self.eat('[')
        b = self.look_body();
j = self.jump_stmnt()
        if j: b.append(j)
        self.eat(']');
droplooks.append((c, b))
        prod =
self.get_production('<nonret-
look-tail>')
        if prod == 500:
            self.eat('DROP');
self.eat('[')
            drop =
self.look_body(); j =
self.jump_stmnt()
            if j: drop.append(j)
            self.eat(']')
            return droplooks, drop

# CHART statement
def chart_stmnt(self):
    tok = self.current_token
    self.eat('CHART');
self.eat('('); expr =
self.chart_cond();
self.eat(')'); self.eat('[')
        courses =
[self.courses()] +
self.course_tail()
        adrift =
self.adrift_case(); self.eat(']')
        return ChartNode(expr,
courses, adrift, tok)
```

```
def chart_cond(self):
    prod =
self.get_production('<chart-c
ond>')
    if prod == 626:
        tok =
self.current_token;
self.eat('id')
        if self.current_token
and self.current_token.type
in ('{', '$', '('):
            return
self.id_tail(tok.value, tok)
        return
IdentNode(tok.value, tok)
    else:
        return
self.chart_const()

def chart_const(self):
    prod =
self.get_production('<chart-c
onst>')
    tok = self.current_token
    if prod == 628:
        self.eat('COIN-lit');
return LiteralNode('COIN',
int(tok.value), tok)
    elif prod == 629:
        self.eat('PARCH-lit');
return LiteralNode('PARCH',
tok.value, tok)
    else: # prod 630

self.eat('SCROLL-lit'); base
= LiteralNode('SCROLL',
tok.value, tok)
        if self.current_token
and self.current_token.type
== '{':
            self.eat('{'); it =
self.current_token;
```

```
self.eat('COIN-lit');
self.eat('{}')
        return
ScrollCharAccessNode(base,
LiteralNode('COIN',
int(it.value), it), tok)
        return base

def courses(self):
    tok =
self.current_token;
self.eat('COURSE')
        val = self.chart_const();
self.eat(':')
        body =
self.course_body(); jump =
self.course_jump()
        if jump:
body.append(jump)
        jname =
jump.__class__.__name__.re
place('Node', '') if jump else
None
        return CourseNode(val,
body, jname, tok)

def course_body(self):
    stmts = []
    prod =
self.get_production('<course
-body>')
    while prod == 632:

stmts.append(self.statements
()); prod =
self.get_production('<course
-body>')
        return stmts

def course_jump(self):
```

```
prod =
self.get_production('<course
-jmp>')
tok = self.current_token
if prod == 634:
self.eat('SAIL'); self.eat('!!!');
return SailNode(tok)
elif prod == 635:
self.eat('LAND');
self.eat('!!!'); return
LandNode(tok)
return None

def course_tail(self):
courses = []
prod =
self.get_production('<course
-tail>')
while prod == 637:

courses.append(self.courses(
)); prod =
self.get_production('<course
-tail>')
return courses

def adrift_case(self):
prod =
self.get_production('<adrift-
case>')
if prod == 639:
self.eat('ADRIFT');
self.eat(':')
body =
self.adrift_body();
self.eat('LAND');
self.eat('!!!')
return body
return None

def adrift_body(self):
stmts = []
```

SeaStack

```
prod =
self.get_production('<adrift-
body>')
while prod == 641:

stmts.append(self.statements
()); prod =
self.get_production('<adrift-
body>')
return stmts

# looping statements
def hoist_stmtnt(self):
tok = self.current_token
self.eat('HOIST');
self.eat('(')
inits = self.hoist_init();
self.eat('!!!')
cond =
self.hoist_cond();
self.eat('!!!')
updates =
self.hoist_upd(); self.eat('');
self.eat('[')
body =
self.look_body(); jump =
self.jump_stmtnt()
if jump:
body.append(jump)
self.eat(']')
return HoistNode(inits,
cond, updates, body, None,
tok)

def hoist_init(self):
prod =
self.get_production('<hoist-i
nit>')
inits = []
if prod == 644:
```

```
self.eat('COIN'); nt =
self.current_token;
self.eat('id')
self.eat('='); val =
self.coin_val()

inits.append(HoistInitNode(
True, nt.value, val, nt))

inits.extend(self.init1_mult()
)

elif prod == 645:
nt =
self.current_token;
self.eat('id')
if self.current_token
and self.current_token.type
in ('{', '$'):
self.arr_str()
self.eat('='); val =
self.coin_val()

inits.append(HoistInitNode(
False, nt.value, val, nt))

inits.extend(self.init2_mult()
)

return inits

def init1_mult(self):
inits = []
while
self.get_production('<init1-
mult>') == 647:
self.eat(','); nt =
self.current_token;
self.eat('id')
self.eat('='); val =
self.coin_val()

inits.append(HoistInitNode(
True, nt.value, val, nt))
```

```

return inits

def init2_mult(self):
    inits = []
    while
self.get_production('<init2-
mult>') == 649:
        self.eat(','); nt =
self.current_token;
self.eat('id')
        if self.current_token
and self.current_token.type
in ('{', '$'):
            self.arr_str()
            self.eat('='); val =
self.coin_val()

inits.append(HoistInitNode(
False, nt.value, val, nt))
return inits

def hoist_cond(self):
    left =
self.arith_fold(self.dime_val(
))
    op_tok =
self.current_token; op =
self.releq_op()
    right =
self.arith_fold(self.dime_val(
))
    node =
BinaryOpNode(left, op,
right, op_tok)
    return
self._hoist_log_fold(node)

def releq_op(self):
    tok =
self.current_token;
self.eat(tok.type); return
tok.type

```

```

def _hoist_log_fold(self,
node):
    prod =
self.get_production('<hoist-l
og>')
    if prod == 654:
        op_tok =
self.current_token; op =
self.log_op()
        right =
self.hoist_cond()
        return
BinaryOpNode(node, op,
right, op_tok)
    return node

def hoist_upd(self):
    updates = [self.upd()]
    while
self.get_production('<upd-m
ult>') == 659:
        self.eat(',');
updates.append(self.upd())
    return updates

def upd(self):
    prod =
self.get_production('<upd>')
    if prod == 657:
        ot =
self.current_token; op = '+#'
if ot.type == '+#' else '-#'
        self.eat(ot.type); nt =
self.current_token;
self.eat('id')
        if self.current_token
and self.current_token.type
in ('{', '$'):
            tk, i1, _, m =
self.arr_str()

```

```

else: tk, i1, m = 'var',
None, None
        return
HoistUpdateNode('unary',
nt.value, tk, i1, m, op, None,
None, ot)
    else:
        nt =
self.current_token;
self.eat('id')
        if self.current_token
and self.current_token.type
in ('{', '$'):
            tk, i1, _, m =
self.arr_str()
            else: tk, i1, m = 'var',
None, None
            ot =
self.current_token; op =
self.arith_assign_op()
            val = self.dime_val()
            return
HoistUpdateNode('compoun
d', nt.value, tk, i1, m, None,
op, val, ot)

def heave_stmnt(self):
    tok = self.current_token
    self.eat('HEAVE');
    self.eat('('); cond =
self.condition(); self.eat(')');
    self.eat('[')
    body =
self.look_body(); jump =
self.jump_stmnt()
    if jump:
        body.append(jump)
        self.eat(']')
    return
HeaveNode(cond, body,
None, tok)

```



```
def haul_stmtnt(self):
    tok = self.current_token
    self.eat('HAUL');
self.eat('[')
    body =
self.look_body(); jump =
self.jump_stmtnt()
    if jump:
body.append(jump)
        self.eat(']');
self.eat('HEAVE');
self.eat('(')
        cond = self.condition();
self.eat(')'); self.eat('!!')
    return
HaulHeaveNode(body, cond,
None, tok)
```

```
def unary_exp(self):
    ot = self.current_token;
op = self.unary_op()
    nt = self.current_token;
self.eat('id')
    if self.current_token
and self.current_token.type
in ('{', '$'):
        tk, i1, i2, m =
self.arr_str()
    else: tk, i1, i2, m = 'var',
None, None, None
    return
UnaryStmtNode(op,
nt.value, tk, i1, i2, m, ot)
```

```
def unary_op(self):
    tok =
self.current_token;
self.eat(tok.type); return
tok.type
```

```
# expressions
def value(self):
```

SeaStack

```
prod =
self.get_production('<value>'
)
    if prod == 352:
        tok =
self.current_token;
self.eat('id')
        if self.current_token
and self.current_token.type
in ('{', '$', '('):
            node =
self.id_tail(tok.value, tok)
        else: node =
IdentNode(tok.value, tok)
    return
self._exp_shared(node)
    elif prod == 353:
        self.eat('('); node =
self.value_grp(); self.eat(')')
    return
self._exp_shared(node)
    elif prod == 354:
    return
self._digit_tail_var(self.var_
digit())
    elif prod == 355:
        tok =
self.current_token;
self.eat('PARCH-lit')
    return
self._parch_tail_var(LiteralN
ode('PARCH', tok.value,
tok))
    elif prod == 356:
        tok =
self.current_token;
self.eat('SCROLL-lit')
        base =
LiteralNode('SCROLL',
tok.value, tok)
```

```
return
self._scroll_tail_var(self.scr_
char_opt(base, tok))
    else: # prod 357: bool
    return self.bool_val()
```

```
def _digit_tail_var(self,
node):
    prod =
self.get_production('<digit-ta
il>')
    if prod == 358:
        node =
self.arith_fold(node)
    return
self._releq_var(node)
    return node
```

```
def _releq_var(self, left):
    prod =
self.get_production('<releq-v
ar>')
    if prod == 362:
        op_tok =
self.current_token; op =
self.rel_op()
        right =
self.arith_fold(self.dime_ope
())
    return
```

```
self._logeq_var(BinaryOpNo
de(left, op, right, op_tok))
    elif prod == 363:
        op_tok =
self.current_token; op =
self.eq_op()
        right =
self.arith_fold(self.dime_ope
())
    return
```

```
self._log_var(BinaryOpNode
(left, op, right, op_tok))
```

```

return left

def _logeq_var(self, left):
    prod =
self.get_production('<logeq-
var>')
    if prod == 369:
        op_tok =
self.current_token; op =
self._logeq_op()
        right =
self.bool_op()
        return
self._log_var(BinaryOpNode
(left, op, right, op_tok))
        return left

def _log_var(self, left):
    prod =
self.get_production('<log-var
>')
    if prod == 373:
        op_tok =
self.current_token; op =
self.log_op()
        right =
self.bool_op()
        return
self._log_var(BinaryOpNode
(left, op, right, op_tok))
        return left

def _parch_tail_var(self,
left):
    prod =
self.get_production('<parch-t
ail>')
    if prod == 375:
        op_tok =
self.current_token; op =
self.eq_op()

```

```

rt =
self.current_token;
self.eat('PARCH-lit')
    return
self._log_var(BinaryOpNode
(left, op,
LiteralNode('PARCH',
rt.value, rt), op_tok))
    return left

def _scroll_tail_var(self,
left):
    prod =
self.get_production('<scroll-t
ail>')
    if prod == 377: return
self.scroll_concat_fold(left)
    elif prod == 378:
        op_tok =
self.current_token; op =
self.eq_op()
        right =
self.scroll_val()
        return
self._log_var(BinaryOpNode
(left, op, right, op_tok))
        return left

def _exp_shared(self,
left):
    prod =
self.get_production('<exp>')
    if prod == 382:
        left =
self.arith_fold(left); return
self._releq_var(left)
    elif prod == 383:
        op_tok =
self.current_token; op =
self.rel_op()

```

```

right =
self.arith_fold(self.dime_ope
())
    return
self._logeq_var(BinaryOpNo
de(left, op, right, op_tok))
    elif prod == 384:
        op_tok =
self.current_token; op =
self.log_op()
        right =
self.bool_op()
        return
self._log_var(BinaryOpNode
(left, op, right, op_tok))
    elif prod == 385:
        op_tok =
self.current_token; op =
self.eq_op()
        right = self._eq_var()
        return
self._log_var(BinaryOpNode
(left, op, right, op_tok))
    elif prod == 386:
        return
self.scroll_concat_fold(left)
    return left # prod 387:
λ

def _eq_var(self):
    prod =
self.get_production('<eq-var
>')
    if prod == 402:
        tok =
self.current_token;
self.eat('id')
        if self.current_token
and self.current_token.type
in ('{', '$', '('):
            node =
self.id_tail(tok.value, tok)

```

```
        else: node =
IdentNode(tok.value, tok)
        node =
self.arith_fold(node); return
self._rel_var(node)
        elif prod == 403:
            self.eat('('); node =
self.value_grp(); self.eat(')')
            node =
self.arith_fold(node); return
self._rel_var(node)
        elif prod == 404:
            node =
self.arith_fold(self.var_digit(
)); return self._rel_var(node)
        else:
            return
self._eq_opel1()

def _rel_var(self, left):
    prod =
self.get_production('<rel-var
>')
    if prod == 408:
        op_tok =
self.current_token; op =
self.rel_op()
        right =
self.arith_fold(self.dime_ope
())
        return
BinaryOpNode(left, op,
right, op_tok)
        return left

# grouped value
def value_grp(self):
    prod =
self.get_production('<value-
grp>')
    if prod == 410:
```

```
        tok =
self.current_token;
self.eat('id')
        if self.current_token
and self.current_token.type
in ('{', '$', '('):
            node =
self.id_tail(tok.value, tok)
        else: node =
IdentNode(tok.value, tok)
        return
self._exp_grp(node)
        elif prod == 411:
            self.eat('('); node =
self.value_grp(); self.eat(')')
            return
self._exp_grp(node)
        elif prod == 412:
            return
self._digit_tail_grp(self.var_
digit())
        elif prod == 413:
            tok =
self.current_token;
self.eat('PARCH-lit')
            return
self._parch_tail_grp(Literal
Node('PARCH', tok.value,
tok))
        elif prod == 414:
            tok =
self.current_token;
self.eat('SCROLL-lit')
            base =
LiteralNode('SCROLL',
tok.value, tok)
            return
self._scroll_tail_grp(self.scr_
char_opt(base, tok))
        else:
            return self.bool_val()
```

```
def _exp_grp(self, left):
    prod =
self.get_production('<exp-gr
p>')
    if prod == 436:
        left =
self.arith_fold(left); return
self._releqvar_grp(left)
    elif prod == 437:
        op_tok =
self.current_token; op =
self.rel_op()
        right =
self.arith_fold(self.dime_ope
())
        return
self._logeq_grp(BinaryOpN
ode(left, op, right, op_tok))
    elif prod == 438:
        op_tok =
self.current_token; op =
self.log_op()
        right =
self.bool_grp_ope()
        return
self._bool_exp_grp(BinaryO
pNode(left, op, right,
op_tok))
    elif prod == 439:
        op_tok =
self.current_token; op =
self.eq_op()
        right =
self._eq_ope_grp()
        return
self._bool_exp_grp(BinaryO
pNode(left, op, right,
op_tok))
    elif prod == 440:
        return
self.scroll_concat_fold(left)
        return left
```

```

def _digit_tail_grp(self,
node):
    prod =
self.get_production('<digit-tail-grp>')
    if prod == 416:
        node =
self.arith_fold(node); return
self._releqvar_grp(node)
    return node

def _releqvar_grp(self,
left):
    prod =
self.get_production('<releqv
ar-grp>')
    if prod == 420:
        op_tok =
self.current_token; op =
self.rel_op()
        right =
self.arith_fold(self.dime_ope
())
    return
self._logeq_grp(BinaryOpN
ode(left, op, right, op_tok))
    elif prod == 421:
        op_tok =
self.current_token; op =
self.eq_op()
        right =
self.arith_fold(self.dime_ope
())
    node =
BinaryOpNode(left, op,
right, op_tok)
    return
self._bool_exp_grp(node)
    return left

def _logeq_grp(self, left):

```

SeaStack

```

    prod =
self.get_production('<logeq-
grp>')
    if prod == 427:
        op_tok =
self.current_token; op =
self._logeq_op()
        right =
self.bool_grp_ope()
    return
self._bool_exp_grp(BinaryO
pNode(left, op, right,
op_tok))
    return left

def _bool_exp_grp(self,
left):
    prod =
self.get_production('<bool-e
xp-grp>')
    if prod == 258:
        op_tok =
self.current_token; op =
self.log_op()
        right =
self.bool_grp_ope()
    return
self._bool_exp_grp(BinaryO
pNode(left, op, right,
op_tok))
    return left

def _parch_tail_grp(self,
left):
    prod =
self.get_production('<parch-t
ail-grp>')
    if prod == 429:
        op_tok =
self.current_token; op =
self.eq_op()

```

```

    rt =
self.current_token;
self.eat('PARCH-lit')
    return
self._bool_exp_grp(BinaryO
pNode(left, op,
LiteralNode('PARCH',
rt.value, rt), op_tok))
    return left

def _scroll_tail_grp(self,
left):
    prod =
self.get_production('<scroll-t
ail-grp>')
    if prod == 431: return
self.scroll_concat_fold(left)
    elif prod == 432:
        op_tok =
self.current_token; op =
self.eq_op()
        right =
self.scroll_val()
    return
self._bool_exp_grp(BinaryO
pNode(left, op, right,
op_tok))
    return left

def _eq_ope_grp(self):
    prod =
self.get_production('<eq-ope
-grp>')
    if prod == 250:
        tok =
self.current_token;
self.eat('id')
        if self.current_token
and self.current_token.type
in ('{', '$', '('):

```

```
        node =
self.id_tail(tok.value, tok)
    else: node =
IdentNode(tok.value, tok)
    return
self.arith_fold(node)
    elif prod == 251:
        self.eat('('); node =
self.value_grp(); self.eat(')')
    return
self.arith_fold(node)
    elif prod == 252:
        return
self.arith_fold(self.var_digit(
))
    else:
        return
self._eq_opel()

def _eq_opel(self):
    prod =
self.get_production('<eq-opel>')
    tok = self.current_token
    if prod == 220:
        self.eat('PARCH-lit');
return LiteralNode('PARCH',
tok.value, tok)
    elif prod == 221:

self.eat('SCROLL-lit'); base
= LiteralNode('SCROLL',
tok.value, tok)
    return
self.scr_char_opt(base, tok)
    else:
        return
self._bool_literal()

# id tail
def id_tail(self, name,
tok):
```

SeaStack

```
    prod =
self.get_production('<id-tail>')
    if prod == 342:
        self.eat('{'); idx1 =
self.index_expr();
self.eat('}')
        prod2 =
self.get_production('<elmt2>')
        if prod2 == 346:
            self.eat('{'); idx2 =
self.index_expr();
self.eat('}')
            return
ArrayAccessNode(name,
[idx1, idx2], tok)
        return
ArrayAccessNode(name,
[idx1], tok)
    elif prod == 343:
        self.eat('$'); mt =
self.current_token;
self.eat('id')
        return
MemberAccessNode(name,
mt.value, tok)
    elif prod == 344:
        self.eat('('); args =
self.args(); self.eat(')')
        return
FuncCallNode(name, args,
tok)
    else:
        return
IdentNode(name, tok)

def index_expr(self):
    prod =
self.get_production('<index>')
    tok = self.current_token
```

```
    if prod == 137:
self.eat('id'); return
IdentNode(tok.value, tok)
    else:
self.eat('COIN-lit'); return
LiteralNode('COIN',
int(tok.value), tok)

def args(self):
    prod =
self.get_production('<args>')
    if prod == 348:
        first = self.value()
        rest =
self._args_mult()
        return [first] + rest
    return []

def _args_mult(self):
    prod =
self.get_production('<args-mult>')
    if prod == 350:
self.eat(','); return self.args()
    return []

# data type values
def coin_val(self):
    prod =
self.get_production('<coin-val>')
    tok = self.current_token
    if prod == 21:
        self.eat('COIN-lit');
node = LiteralNode('COIN',
int(tok.value), tok)
    else:
        has_neg =
(self.current_token.type ==
'-')
        if has_neg:
self.eat('-')
```

```
node =
self.neg_coin_val()
    if has_neg: node =
UnaryOpNode('-', node, tok)
    return
self.arith_fold(node)

def neg_coin_val(self):
    prod =
self.get_production('<neg-co
in-val>')
    tok = self.current_token
    if prod == 25:
        self.eat('id')
        if self.current_token
and self.current_token.type
in ('{', '$', '('):
            return
self.id_tail(tok.value, tok)
        return
IdentNode(tok.value, tok)
    else:
        self.eat('('); val =
self.coin_val(); self.eat(')');
return val

def dime_val(self):
    prod =
self.get_production('<dime-v
al>')
    tok = self.current_token
    if prod == 68:
        self.eat('DIME-lit');
node = LiteralNode('DIME',
float(tok.value), tok)
        elif prod == 69:
            self.eat('COIN-lit');
node = LiteralNode('COIN',
int(tok.value), tok)
        else:
```

```
has_neg =
(self.current_token.type ==
'-')
    if has_neg:
self.eat('-')
        node =
self.neg_dime_val()
        if has_neg: node =
UnaryOpNode('-', node, tok)
        return
self.arith_fold(node)

def neg_dime_val(self):
    prod =
self.get_production('<neg-di
me-val>')
    tok = self.current_token
    if prod == 71:
        self.eat('id')
        if self.current_token
and self.current_token.type
in ('{', '$', '('):
            return
self.id_tail(tok.value, tok)
        return
IdentNode(tok.value, tok)
    else:
        self.eat('('); val =
self.dime_val(); self.eat(')');
return val

def dime_ope(self):
    return self.dime_val()

def _dime_atom(self):
    prod =
self.get_production('<dime-v
al>')
    tok = self.current_token
    if prod == 68:
```

```
self.eat('DIME-lit');
return LiteralNode('DIME',
float(tok.value), tok)
        elif prod == 69:
            self.eat('COIN-lit');
return LiteralNode('COIN',
int(tok.value), tok)
        else:
            has_neg =
(self.current_token.type ==
'-')
            if has_neg:
self.eat('-')
                node =
self.neg_dime_val()
                if has_neg: return
UnaryOpNode('-', node, tok)
                return node

def arith_fold(self, left):
    while self.current_token
and self.current_token.type
in ('+', '-', '*', '/', '%', '^'):
        op_tok =
self.current_token; op =
self.arith_op()
        right =
self._dime_atom()
        left =
BinaryOpNode(left, op,
right, op_tok)
        return left

def arith_op(self):
    tok =
self.current_token;
self.eat(tok.type); return
tok.type

def parch_val(self):
```

```

    prod =
self.get_production('<parch-
val>')
    tok = self.current_token
    if prod == 107:
        self.eat('PARCH-lit');
return LiteralNode('PARCH',
tok.value, tok)
    else:
        self.eat('id')
        if self.current_token
and self.current_token.type
in ('{', '$', '('):
            return
self.id_tail(tok.value, tok)
        return
IdentNode(tok.value, tok)

    def scroll_val(self):
    prod =
self.get_production('<scroll-
val>')
    tok = self.current_token
    if prod == 132:

self.eat('SCROLL-lit')
        first =
self.scr_char_opt(LiteralNod
e('SCROLL', tok.value, tok),
tok)
        elif prod == 133:
            self.eat('id')
            if self.current_token
and self.current_token.type
in ('{', '$', '('):
                first =
self.id_tail(tok.value, tok)
            else: first =
IdentNode(tok.value, tok)
        else: # prod 134
            self.eat('('); first =
self.scroll_val(); self.eat(')')

```

```

    return
self.scroll_concat_fold(first)

    def scr_char_opt(self,
base, tok):
        if self.current_token
and self.current_token.type
== '{':
            prod =
self.get_production('<scr-cha
r>')
            if prod == 135:
                self.eat('{'); idx =
self.index_expr();
self.eat('}')
            return
ScrollCharAccessNode(base,
idx, tok)
        return base

    def
scroll_concat_fold(self, left):
        if self.current_token
and self.current_token.type
== '&':
            tok =
self.current_token; operands
= [left]
            while
self.current_token and
self.current_token.type ==
'&':
                self.eat('&');
operands.append(self.scroll_
val())
            return
StringConcatNode(operands,
tok)
        return left

    def var_digit(self):

```

```

    prod =
self.get_production('<digit>'
)
    tok = self.current_token
    if prod == 191:
        self.eat('COIN-lit');
return LiteralNode('COIN',
int(tok.value), tok)
    elif prod == 192:
        self.eat('DIME-lit');
return LiteralNode('DIME',
float(tok.value), tok)
    else:
        self.eat('-'); inner =
self.neg_digit()
        return
UnaryOpNode('-', inner, tok)

    def neg_digit(self):
    prod =
self.get_production('<neg-di
git>')
    tok = self.current_token
    if prod == 194:
        self.eat('id')
        if self.current_token
and self.current_token.type
in ('{', '$', '('):
            return
self.id_tail(tok.value, tok)
        return
IdentNode(tok.value, tok)
    else:
        self.eat('('); val =
self.dime_val(); self.eat(')');
return val

    def bool_val(self):
    left = self.bool_ope()
    return
self.bool_exp_fold(left)

```

```
def bool_grp_val(self):
    left =
self.bool_grp_ope()
    return
self._bool_exp_grp(left)

def bool_grp_ope(self):
    prod =
self.get_production('<bool-ope-grp>')
    tok = self.current_token
    if prod == 232:
        self.eat('id')
        if self.current_token
and self.current_token.type
in ('{', '$', '('):
            node =
self.id_tail(tok.value, tok)
        else: node =
IdentNode(tok.value, tok)
        return
self._bool_exp2_grp(node)
    elif prod == 233:
        self.eat('('); node =
self.value_grp(); self.eat(')')
        return
self._bool_exp2_grp(node)
    elif prod == 234:
        node =
self._bool_literal()
        return
self._bool_eq_grp(node)
    elif prod == 235:
        left =
self.arith_fold(self.var_digit(
))
        return
self._releq_grp(left)
    elif prod == 236:
        self.eat('PARCH-lit')

        left =
LiteralNode('PARCH',
tok.value, tok)
        op_tok =
self.current_token; op =
self.eq_op()
        return
BinaryOpNode(left, op,
self.parch_val(), op_tok)
    elif prod == 237:
        self.eat('SCROLL-lit')
        left =
self.scr_char_opt(LiteralNod
e('SCROLL', tok.value, tok),
tok)
        op_tok =
self.current_token; op =
self.eq_op()
        return
BinaryOpNode(left, op,
self.scroll_val(), op_tok)

    def _bool_exp2_grp(self,
left):
        prod =
self.get_production('<bool-exp2-grp>')
        if prod == 246:
            left =
self.arith_fold(left); return
self._releq_grp(left)
        elif prod == 247:
            op_tok =
self.current_token; op =
self.rel_op()
            right =
self.arith_fold(self.dime_val(
))
            return
self._bool_eq_grp(BinaryOp
Node(left, op, right, op_tok))
        elif prod == 248:
            op_tok =
self.current_token; op =
self.eq_op()
            right =
self._eq_ope_grp()
            return
BinaryOpNode(left, op,
right, op_tok)
            return left

    def _bool_eq_grp(self,
left):
        prod =
self.get_production('<bool-eq-grp>')
        if prod == 238:
            op_tok =
self.current_token; op =
self.eq_op()
            return
BinaryOpNode(left, op,
self.bool_grp_ope(), op_tok)
            return left

    def _releq_grp(self, left):
        prod =
self.get_production('<releq-grp>')
        if prod == 240:
            op_tok =
self.current_token; op =
self.rel_op()
            right =
self.arith_fold(self.dime_val(
))
            return
self._bool_eq_grp(BinaryOp
Node(left, op, right, op_tok))
        elif prod == 241:
```



```

        op_tok =
self.current_token; op =
self.eq_op()
        right =
self.arith_fold(self.dime_val(
))
        return
BinaryOpNode(left, op,
right, op_tok)
        return left

def bool_ope(self):
    return
self.bool_grp_ope()

def _bool_literal(self):
    tok = self.current_token
    if tok.type == 'AYE':
        self.eat('AYE');
    return LiteralNode('BOOL',
True, tok)
        elif tok.type == 'NAY':
            self.eat('NAY');
    return LiteralNode('BOOL',
False, tok)
        else:
            op_tok =
self.current_token; op =
op_tok.type;
self.eat(op_tok.type)
            not_prod =
self.get_production('<not-op
e>')
            inner_tok =
self.current_token
            if not_prod == 186:
                self.eat('id')

```

```

import re
from semantic.symbol_table
import ( Symbol,
ArraySymbol,

```

SeaStack

```

        if
self.current_token and
self.current_token.type in
('{', '$', '('):
            operand =
self.id_tail(inner_tok.value,
inner_tok)
            else: operand =
IdentNode(inner_tok.value,
inner_tok)
            elif not_prod == 187:
                self.eat('(');
            operand = self.value_grp();
            self.eat(')')
            else:
                if inner_tok.type
== 'AYE':
                    self.eat('AYE');
                operand =
LiteralNode('BOOL', True,
inner_tok)
                else:
                    self.eat('NAY');
                operand =
LiteralNode('BOOL', False,
inner_tok)
            return
UnaryOpNode(op, operand,
op_tok)

def bool_exp_fold(self,
left):
    prod =
self.get_production('<bool-e
xp>')
    if prod == 227:

```

semantic_analyzer.py

```

FunctionSymbol,
StructTypeSymbol,
StructVarSymbol,
SymbolTable, )

```

```

        op_tok =
self.current_token; op =
self.log_op()
        right =
self.bool_val()
        return
self.bool_exp_fold(BinaryO
pNode(left, op, right,
op_tok))
        return left

def _logeq_op(self):
    tok =
self.current_token;
self.eat(tok.type); return
tok.type

def rel_op(self):
    tok =
self.current_token;
self.eat(tok.type); return
tok.type

def eq_op(self):
    tok =
self.current_token;
self.eat(tok.type); return
tok.type

def log_op(self):
    tok =
self.current_token;
self.eat(tok.type); return
tok.type

```

```

from semantic.ast_nodes
import NamedInitNode,
PositionalInitNode

```

```
from
backend.semantic.sem_error
_msg import
SemanticErrorHandler

class SemanticAnalyzer:
    def __init__(self, ast,
source_code):
        self.ast = ast
        self.source_code =
source_code
        self.sym =
SymbolTable()
        self.errors = []
        self.err =
SemanticErrorHandler(sourc
e_code)

        # Context tracking
        self.current_func_return
= None
        self.loop_depth = 0
        self.in_conditional = 0
        self.in_chart = False
        self.in_adrift = False
        self.known_values = {}

    def analyze(self):
        self.visit(self.ast)
        return self.errors

    def visit(self, node):
        if node is None: return
None
        method =
f'visit_{type(node).__name_
__}'
        return getattr(self,
method,
self._visit_unknown)(node)
```

```
def _visit_unknown(self,
node):
    self._e(self.err.internal_no_v
isitor(type(node).__name__
)

    # Append a pre-built error
dict from
SemanticErrorHandler.
    def _e(self, error_dict):
self.errors.append(error_dict
)

    # Assignment
compatibility: exact match
or COIN → DIME
promotion.
    def _compatible(self,
expected, actual):
        if expected == actual:
return True
        if expected == 'DIME'
and actual == 'COIN': return
True
        return False

    # Expression
compatibility: COIN ↔
DIME are compatible.
    def
_compatible_expr(self, left,
right):
        if left == right: return
True
        if left in ('COIN',
'DIME') and right in
('COIN', 'DIME'): return
True
        return False
```

```
def _is_numeric(self,
dtype): return dtype in
('COIN', 'DIME')
    def _is_bool(self, dtype):
return dtype == 'BOOL'
    def _type_name(self,
dtype): return dtype if dtype
else 'unknown'

    # Get compile-time COIN
int value, or None for
runtime values.
    def _literal_int(self,
node):
        if type(node).__name__
== 'LiteralNode':
            if getattr(node,
'dtype', None) != 'COIN':
return None
            try: return
int(getattr(node, 'value',
None))
        except (TypeError,
ValueError): return None
        if type(node).__name__
== 'IdentNode':
            name = getattr(node,
'name', None)
            if name and name in
self.known_values:
                dtype, val =
self.known_values[name]
                if dtype ==
'COIN':
                    try: return
int(val)
                except
(TypeError, ValueError):
return None
                return None
```

```
# Get compile-time
SCROLL string length, or
None for runtime values.
def
_known_scroll_length(self,
node):
    if type(node).__name__
    == 'LiteralNode' and
    getattr(node, 'dtype', None)
    == 'SCROLL':
        raw = getattr(node,
'value', None)
        if raw is not None:
            return len(str(raw))
        if type(node).__name__
    == 'IdentNode':
            name = getattr(node,
'name', None)
            if name and name in
self.known_values:
                dtype, val =
self.known_values[name]
                if dtype ==
'SCROLL' and val is not
None:
                    return
len(str(val))
                return None

    def _check_bounds(self,
arr_name, dim_label,
idx_expr, size, token):
        v =
self._literal_int(idx_expr)
        if v is not None and (v
< 0 or v >= size):

self._e(self.err.array_index_
out_of_bounds(token,
arr_name, dim_label, v,
size))
```

```
_SPEC_MAP = {'C':
'COIN', 'D': 'DIME', 'P':
'PARCH', 'S': 'SCROLL', 'B':
'BOOL'}

def _parse_specs(self,
fmt):
    return
[self._SPEC_MAP[ch] for
ch in
re.findall(r'%([CDPSB])',
fmt)]

def
visit_ProgramNode(self,
node):
    # Pass 1: pre-register
functions and struct types for
forward refs
    for d in
node.global_decls:
        cn =
type(d).__name__
        if cn ==
'FuncDefNode':
            self._pre_register_func(d)
        elif cn ==
'StructDefNode':
            self._pre_register_struct(d)
        # Pass 2: full analysis
        for d in
node.global_decls:
            self.visit(d)

self.visit(node.ahoy_body)

def
_pre_register_func(self,
node):
    if not
self.sym.lookup_current_sco
pe(node.name):
```

```
self.sym.declare(FunctionSy
mbol(node.name,
node.return_type,
node.params, node.token))

def
_pre_register_struct(self,
node):
    if not
self.sym.lookup_current_sco
pe(node.name):
        members = {m.name:
m.dtype for m in
node.members}
        order = [m.name for
m in node.members]

self.sym.declare(StructType
Symbol(node.name,
members, order,
node.token))

def visit_AhoyNode(self,
node):
    self.sym.push_scope()
    for d in
node.local_decls:
        self.visit(d)
    for s in
node.statements: self.visit(s)
    self.sym.pop_scope()

def
visit_ConstDeclNode(self,
node):
    # LOCKE can only be
declared globally
    if not
self.sym.is_global_scope():
```

```

        # Track compile-time
        known value for constant
        propagation
        if
        type(node.init_value).__name__ == 'LiteralNode':
            self.known_values[node.name] = (node.dtype,
            node.init_value.value)

        self.sym.declare(Symbol(node.name, node.dtype, 'var',
        node.token,
        is_initialized=True))

        def
        visit_VarDeclNode(self,
        node):
            if
            self.sym.lookup_current_scope(node.name):

            self._e(self.err.duplicate_variable(node.token,
            node.name))
            return
            if node.init_value is not
            None:
                init_type =
                self.visit(node.init_value)
                if init_type and not
                self._compatible(node.dtype,
                init_type):

                self._e(self.err.type_mismatch_init(node.token,
                node.name, node.dtype,
                self._type_name(init_type)))

        for ri, row in
        enumerate(node.init_values):
            if len(row) > dc:

            self._e(self.err.array_init_row_too_long(node.token,
            node.name, ri, len(row), dc))
            for ci, elem in
            enumerate(row):
                et =
                self.visit(elem)
                if et and not
                self._compatible(node.dtype,
                et):

                self._e(self.err.type_mismatch_array_element(node.token,
                node.name, f'[{ri}][{ci}]',
                node.dtype,
                self._type_name(et)))
                else:
                    ds =
                    node.dimensions[0]
                    if
                    len(node.init_values) > ds:

                    self._e(self.err.array_init_too_many_elements(node.token
                    , node.name,
                    len(node.init_values), ds))
                    for i, elem in
                    enumerate(node.init_values):
                        et =
                        self.visit(elem)
                        if et and not
                        self._compatible(node.dtype,
                        et):

                        self._e(self.err.type_mismatch_array_element(node.token,
                        node.name, f'[{i}]',

```

```
node.dtype,
self._type_name(et)))

self.sym.declare(ArraySymbol(
node.name, node.dtype,
node.dimensions,
node.is_2d, node.token))

def
visit_StructDefNode(self,
node):
    existing =
self.sym.lookup_current_scope(
node.name)
    if existing and
existing.kind == 'struct':
        # Already
pre-registered — just
validate members
        seen = set()
        for m in
node.members:
            if m.name in seen:

self._e(self.err.duplicate_struct_member(
m.token,
m.name, node.name))
        seen.add(m.name)
        return
    if existing:

self._e(self.err.duplicate_identifier(
node.token,
node.name))
        return
    members, order = {}, []
    for m in node.members:
        if m.name in
members:

self._e(self.err.duplicate_struct_member(
m.token,
m.name, node.name))
```

```
ct_member(m.token,
m.name, node.name))
        else:
            members[m.name]
= m.dtype;
order.append(m.name)

self.sym.declare(StructTypeSymbol(
node.name,
members, order,
node.token))

def
visit_MemberDeclNode(self,
node): pass

def
visit_StructVarDeclNode(self,
node):
    type_sym =
self.sym.lookup(node.struct_type)
    if type_sym is None or
type_sym.kind != 'struct':

self._e(self.err.undefine_struct_type(
node.token,
node.struct_type)); return
    members, order =
type_sym.members,
type_sym.member_order
    if
self.sym.lookup_current_scope(
node.var_name):

self._e(self.err.duplicate_variable(
node.token,
node.var_name)); return
    if node.inits:
        if len(node.inits) >
len(order):
```

```
self._e(self.err.struct_too_many_inits(
node.token,
node.struct_type, len(order),
len(node.inits)))
        else:
            pos = 0
            for init in
node.inits:
                if isinstance(
init, NamedInitNode):
                    if
init.member_name not in
members:

self._e(self.err.no_such_member(
init.token,
node.struct_type,
init.member_name))
            else:
                actual =
self.visit(init.value)
                if actual
and not
self._compatible(members[init.member_name],
actual):

self._e(self.err.type_mismatch_struct_member(
init.token,
init.member_name,
node.struct_type,
members[init.member_name],
self._type_name(actual)))
                if
init.member_name in order:
                    pos =
order.index(init.member_name) + 1
                elif
isinstance(
init, PositionalInitNode):
```

```
        if pos >=
len(order):
    self._e(self.err.struct_positio
nal_overflow(node.token,
node.struct_type)); break
        mname =
order[pos]; expected =
members[mname]
        actual =
self.visit(init.value)
        if actual and
not
self._compatible(expected,
actual):

self._e(self.err.type_mismatch_struct_member(node.token, mname, node.struct_type,
expected,
self._type_name(actual)))
        pos += 1

self.sym.declare(StructVarSymbol(node.var_name,
node.struct_type,
node.token))

    def
visit_PositionalInitNode(self
, node): return
self.visit(node.value)
    def
visit_NamedInitNode(self,
node): return
self.visit(node.value)

    def
visit_FuncDefNode(self,
node):

        existing =
self.sym.lookup_current_scope(node.name)
        if existing is None:
            if not
self.sym.declare(FunctionSymbol(node.name,
node.return_type,
node.params, node.token)):

self._e(self.err.duplicate_function(node.token,
node.name)); return
            elif existing.kind !=
'func':

self._e(self.err.function_name_conflict(node.token,
node.name, existing.kind));
return

        outer_ret =
self.current_func_return
        self.current_func_return
= node.return_type
        self.sym.push_scope()

        for p in node.params:
            if
self.sym.lookup_current_scope(p.name):

self._e(self.err.duplicate_parameter(p.token, p.name,
node.name))
            else:

self.sym.declare(Symbol(p.name, p.dtype, 'param',
p.token, is_initialized=True))

        for d in
node.local_decls:
self.visit(d)
            for s in node.body:
self.visit(s)

            if node.return_type !=
'ABYSS' and
node.return_expr is not
None:
                rt =
self.visit(node.return_expr)
                if rt and not
self._compatible(node.return_type, rt):

self._e(self.err.type_mismatch_return(node.token,
node.name,
node.return_type,
self._type_name(rt)))

                self.sym.pop_scope()
                self.current_func_return
= outer_ret

        def visit_ParamNode(self,
node): pass

        def _target_label(self,
var_name, target_kind,
index1, index2, member):
            if target_kind ==
'member' and member:
                return
            f' {var_name} ${member}'
            elif target_kind ==
'array2d' and index1 is not
None and index2 is not
None:
                i1 = getattr(index1,
'value', '?')
```

```
        i2 = getattr(index2,
'value', '?')
        return
f"{var_name}{{{i1}}}{i2
}}}"
        elif target_kind ==
'array1d' and index1 is not
None:
            i1 = getattr(index1,
'value', '?')
            return
f"{var_name}{{{i1}}}"
            return var_name

def visit_AssignNode(self,
node):
    target_dtype =
self._resolve_target(
        node.var_name,
node.target_kind,
node.index1, node.index2,
node.member, node.token)
    if target_dtype is not
None:
        val_type =
self.visit(node.value)
        if val_type and not
self._compatible(target_dtyp
e, val_type):
            label =
self._target_label(
                node.var_name,
node.target_kind,
node.index1, node.index2,
node.member)

self._e(self.err.type_mismatch,
assign(node.token, label,
target_dtype,
self._type_name(val_type)))
        # Mark initialized
```

```
self.sym.update_initialized(n
ode.var_name)
        # Track compile-time
known value for simple
variable assignments
        if node.target_kind
== 'var':
            if
type(node.value).__name__
== 'LiteralNode':

self.known_values[node.var
_name] = (target_dtype,
node.value.value)
            else:
                # Value is now
runtime-determined; remove
from known values

self.known_values.pop(node
.var_name, None)

def
visit_CompoundAssignNode
(self, node):
    target_dtype =
self._resolve_target(
        node.var_name,
node.target_kind,
node.index1, node.index2,
node.member, node.token)
    if target_dtype is not
None:
        label =
self._target_label(
            node.var_name,
node.target_kind,
node.index1, node.index2,
node.member)
```

```
        if not
self._is_numeric(target_dtyp
e):

self._e(self.err.compound_as
sign_not_numeric(node.toke
n, node.operator, label,
target_dtype))
            val_type =
self.visit(node.value)
            if val_type and not
self._is_numeric(val_type):

self._e(self.err.compound_as
sign_rhs_not_numeric(node.
token, node.operator,
self._type_name(val_type)))
            # Mark the variable
as initialized after compound
assignment

self.sym.update_initialized(n
ode.var_name)
            # Value is now
runtime-determined; remove
from known values

self.known_values.pop(node
.var_name, None)

        # Resolve assignment
target dtype. Returns None
on error.
        def _resolve_target(self,
var_name, target_kind,
index1, index2, member,
token):
            sym =
self.sym.lookup(var_name)
            if sym is None:

self._e(self.err.undeclared_v
```

```

variable(token, var_name));
return None
    if sym.kind == 'const':

self._e(self.err.locke_assign
ment(token, var_name));
return None

    if target_kind == 'var':
        return sym.dtype
    elif target_kind in
('array1d', 'array2d'):
        if sym.kind != 'array':

self._e(self.err.not_an_array(
token, var_name)); return
None

        if index1 is not None:
            it =
self.visit(index1)
            if it and it !=
'COIN':

self._e(self.err.array_index_
not_coin(token, var_name,
0, self._type_name(it)))
            lbl = 'row index' if
sym.is_2d else 'index'

self._check_bounds(var_na
me, lbl, index1,
sym.dimensions[0], token)
            if index2 is not None:
                it =
self.visit(index2)
                if it and it !=
'COIN':

self._e(self.err.array_index_
not_coin(token, var_name,
1, self._type_name(it)))

```

```

if
len(sym.dimensions) > 1:

self._check_bounds(var_na
me, 'column index', index2,
sym.dimensions[1], token)
    return sym.dtype
    elif target_kind ==
'member':
        if sym.kind !=
'struct_var':

self._e(self.err.not_a_struct_
variable(token, var_name));
return None

        ts =
self.sym.lookup(sym.struct_t
ype_name)
        if ts is None or
ts.kind != 'struct':

self._e(self.err.unresolvable_
struct_type(token,
sym.struct_type_name));
return None

        if member not in
ts.members:

self._e(self.err.no_such_me
mber(token,
sym.struct_type_name,
member)); return None

        return
ts.members[member]
        return None

    def visit_AskNode(self,
node):
        specs =
self._parse_specs(node.form
at_string)

```

```

if len(specs) !=
len(node.targets):

self._e(self.err.ask_specifier
_count_mismatch(node.toke
n, len(specs),
len(node.targets)))
        for i, tgt in
enumerate(node.targets):
            tgt_dtype =
self._resolve_addr_target(tgt
)
                if i < len(specs) and
tgt_dtype:
                    if not
self._compatible(specs[i],
tgt_dtype):
                        spec_ch = [k for
k, v in
self._SPEC_MAP.items() if
v == specs[i]]

self._e(self.err.ask_specifier
_type_mismatch(tgt.token,
spec_ch[0] if spec_ch else
'?', specs[i], tgt.var_name,
tgt_dtype))
                    # ASK reads input
into the target, so mark it as
initialized

self.sym.update_initialized(t
gt.var_name)
                    # Value is now
runtime-determined; remove
from known values

self.known_values.pop(tgt.v
ar_name, None)

```



```
def
_resolve_addr_target(self,
node):
    sym =
self.sym.lookup(node.var_name)
    if sym is None:

self._e(self.err.undeclared_variable_in_context(node.token, node.var_name, 'ASK target')); return None
        if sym.kind == 'const':

self._e(self.err.locke_ask_target(node.token, node.var_name)); return None
            if node.target_kind in ('array1d', 'array2d'):
                if sym.kind != 'array':

self._e(self.err.not_an_array(node.token, node.var_name)); return None
                    if node.index1 is not None:
                        it =
self.visit(node.index1)
                        if it and it != 'COIN':

self._e(self.err.ask_array_index_not_coin(node.token, node.var_name, 0, self._type_name(it)))
                            lbl = 'row index' if sym.is_2d else 'index'

self._check_bounds(node.var_name, lbl, node.index1,
```

```
sym.dimensions[0],
node.token)
            if getattr(node, 'index2', None) is not None:
                it =
self.visit(node.index2)
                if it and it != 'COIN':

self._e(self.err.ask_array_index_not_coin(node.token, node.var_name, 1, self._type_name(it)))
                    if
len(sym.dimensions) > 1:

self._check_bounds(node.var_name, 'column index', node.index2, sym.dimensions[1], node.token)
                        return sym.dtype
                        elif node.target_kind == 'member':
                            if sym.kind != 'struct_var':

self._e(self.err.not_a_struct_variable(node.token, node.var_name)); return None
                                ts =
self.sym.lookup(sym.struct_type_name)
                                if ts and
node.member in ts.members:
                                    return
ts.members[node.member]

self._e(self.err.no_such_member(node.token,
```

```
sym.struct_type_name,
node.member)); return None
        return sym.dtype

def
visit_AddressNode(self,
node):
self._resolve_addr_target(node)

def visit_EchoNode(self,
node):
    specs =
self._parse_specs(node.format_string)
    if len(specs) != len(node.args):

self._e(self.err.echo_specifier_count_mismatch(node.token, len(specs), len(node.args)))
        for i, arg in enumerate(node.args):
            at = self.visit(arg)
            if i < len(specs) and at:
                if not
self._compatible(specs[i], at):
                    sc = [k for k, v
in self._SPEC_MAP.items()
if v == specs[i]]

self._e(self.err.echo_specifier_type_mismatch(node.token, sc[0] if sc else '?', specs[i], self._type_name(at)))

def visit_LookNode(self,
node):
```

```
ct =
self.visit(node.condition)
    if ct and not
self._is_bool(ct):

self._e(self.err.condition_not
_bool(node.token, 'LOOK',
self._type_name(ct)))
    self.sym.push_scope();
self.in_conditional += 1
    for s in node.body:
self.visit(s)
        self.in_conditional -= 1;
self.sym.pop_scope()
        for (dc, db) in
node.droplooks:
            dt = self.visit(dc)
            if dt and not
self._is_bool(dt):

self._e(self.err.condition_not
_bool(node.token,
'DROPLOOK',
self._type_name(dt)))

self.sym.push_scope();
self.in_conditional += 1
    for s in db:
self.visit(s)
        self.in_conditional -=
1; self.sym.pop_scope()
        if node.drop_body is
not None:

self.sym.push_scope();
self.in_conditional += 1
    for s in
node.drop_body: self.visit(s)
        self.in_conditional -=
1; self.sym.pop_scope()
```

```
def visit_ChartNode(self,
node):
    expr_type =
self.visit(node.expr)
        if expr_type and
expr_type not in ('COIN',
'PARCH', 'SCROLL'):

self._e(self.err.chart_invalid
_expr_type(node.token,
self._type_name(expr_type))
)
        outer_chart =
self.in_chart; self.in_chart =
True
        # duplicate COURSE
label check using
value+dtype
        seen = {}
        for course in
node.courses:
            case_type =
self.visit(course.value)
            if expr_type and
case_type and not
self._compatible(expr_type,
case_type):

self._e(self.err.course_type_
mismatch(course.token,
self._type_name(case_type),
self._type_name(expr_type))
)

        # Build label key
from dtype+value for
reliable comparison
        label_key =
self._course_label_key(cours
e.value)
        if label_key in seen:
```

```
self._e(self.err.course_duplic
ate_label(course.token))
        else:
            seen[label_key] =
True

self.sym.push_scope();
self.in_conditional += 1
    for s in course.body:
self.visit(s)
        self.in_conditional -=
1; self.sym.pop_scope()

        if node.adrift_body is
not None:
            outer_adrift =
self.in_adrift; self.in_adrift =
True

self.sym.push_scope();
self.in_conditional += 1
    for s in
node.adrift_body:
self.visit(s)
        self.in_conditional -=
1; self.sym.pop_scope()
        self.in_adrift =
outer_adrift
        self.in_chart =
outer_chart

        # Generate a reliable key
for COURSE label duplicate
detection.
        def
_course_label_key(self,
node):
            cn =
type(node).__name__
```

```

        if cn == 'LiteralNode':
return (node.dtype,
node.value)
        if cn ==
'ScrollCharAccessNode':
            inner =
getattr(node.scroll_expr,
'value', '?')
            idx =
getattr(node.index, 'value',
'?')
            return
('SCROLL_CHAR', inner,
idx)
            return id(node) #
fallback

        def
visit_CourseNode(self,
node): pass

        def visit_HoistNode(self,
node):
            self.sym.push_scope()
            for init in node.inits:
self.visit(init)
            ct =
self.visit(node.condition)
            if ct and not
self._is_bool(ct):

self._e(self.err.condition_not
_bool(node.token, 'HOIST',
self._type_name(ct)))
            for upd in node.updates:
self.visit(upd)
            self.loop_depth += 1
            for s in node.body:
self.visit(s)
            self.loop_depth -= 1
            self.sym.pop_scope()

```

```

        def
visit_HoistInitNode(self,
node):
            # Visit the init value
expression and type-check it.
            # The value can now be
a literal, variable, element,
member, function call, or
arithmetic expression — but
must resolve to COIN.
            val_type = None
            if node.value is not
None:
                val_type =
self.visit(node.value)

                if node.declares_new:
                    # COIN id =
<coin-val> — declares a
new loop variable
                    if
self.sym.lookup_current_sco
pe(node.var_name):

self._e(self.err.loop_variable
_conflict(node.token,
node.var_name))
                    else:

self.sym.declare(Symbol(no
de.var_name, 'COIN', 'var',
node.token,
is_initialized=True))
                    # Type-check the init
value: must be COIN (or
compatible)
                    if val_type and not
self._compatible('COIN',
val_type):

self._e(self.err.type_mismatch
_hoist_init(node.token,

```

```

node.var_name,
self._type_name(val_type)))
            else:
                # id = <coin-val> —
assigns to an existing
variable
                sym =
self.sym.lookup(node.var_na
me)
                if sym is None:

self._e(self.err.undeclared_v
ariable_in_context(node.toke
n, node.var_name, 'HOIST
init'))
                elif sym.kind ==
'const':

self._e(self.err.locke_hoist_i
nit(node.token,
node.var_name))
                elif sym.dtype !=
'COIN':

self._e(self.err.type_mismatch
_hoist_var(node.token,
node.var_name, sym.dtype))
                else:
                    # Type-check the
init value: must be COIN
                    if val_type and not
self._compatible('COIN',
val_type):

self._e(self.err.type_mismatch
_hoist_init(node.token,
node.var_name,
self._type_name(val_type)))
                    # Mark the
variable as initialized

```

```

self.sym.update_initialized(n
ode.var_name)

    def
    visit_HoistUpdateNode(self,
node):
        sym =
self.sym.lookup(node.var_na
me)
        if sym is None:

self._e(self.err.undeclared_v
ariable_in_context(node.toke
n, node.var_name, 'HOIST
update')); return
        if sym.kind == 'const':

self._e(self.err.locke_hoist_u
pdate(node.token,
node.var_name)); return
        if node.update_kind ==
'unary':
            if sym.dtype !=
'COIN':

self._e(self.err.hoist_update
unary_not_coin(node.token,
node.unary_op,
node.var_name, sym.dtype))
            elif node.update_kind
== 'compound':
                if not
self._is_numeric(sym.dtype):

self._e(self.err.hoist_update
compound_not_numeric(nod
e.token, node.var_name,
sym.dtype))
                if node.value is not
None:

                    vt =
self.visit(node.value)
                    if vt and not
self._is_numeric(vt):

self._e(self.err.hoist_update
value_not_numeric(node.tok
en, self._type_name(vt)))

                    def visit_HeaveNode(self,
node):
                        ct =
self.visit(node.condition)
                        if ct and not
self._is_bool(ct):

self._e(self.err.condition_not
_bool(node.token, 'HEAVE',
self._type_name(ct)))
                        self.sym.push_scope();
self.loop_depth += 1
                        for s in node.body:
self.visit(s)
                        self.loop_depth -= 1;
self.sym.pop_scope()

                        def
                        visit_HaulHeaveNode(self,
node):
                            self.sym.push_scope();
self.loop_depth += 1
                            for s in node.body:
self.visit(s)
                            self.loop_depth -= 1;
self.sym.pop_scope()

                            ct =
self.visit(node.condition)
                            if ct and not
self._is_bool(ct):

self._e(self.err.condition_not
_bool(node.token,

'HAUL-HEAVE',
self._type_name(ct)))

                            def visit_SailNode(self,
node):
                                if self.in_adrift:

self._e(self.err.sail_inside_ad
rift(node.token)); return
                                if self.loop_depth == 0:

self._e(self.err.sail_outside_l
oop(node.token))

                                def visit_LandNode(self,
node):
                                    if self.loop_depth == 0
and not self.in_chart:

self._e(self.err.land_outside_
loop(node.token))

                                    def visit_ReturnNode(self,
node):
                                        if
self.current_func_return is
None:

self._e(self.err.back_outside
_function(node.token));
return
                                        if
self.current_func_return ==
'ABYSS':

self._e(self.err.back_value_i
n_abyss(node.token)); return
                                        rt =
self.visit(node.value)
                                        if rt and not
self._compatible(self.current
_func_return, rt):

```

```
self._e(self.err.type_mismatch_return(node.token,
'(current function)',
self.current_func_return,
self._type_name(rt)))
```

```
def visit_BackNode(self,
node):
    if
self.current_func_return is
None:
```

```
self._e(self.err.back_outside_function(node.token))
    elif
self.current_func_return !=
'ABYSS':
```

```
self._e(self.err.back_missing_value(node.token,
self.current_func_return))
```

```
def
visit_UnaryStmtNode(self,
node):
    sym =
self.sym.lookup(node.var_name)
    if sym is None:
```

```
self._e(self.err.undeclared_variable(node.token,
node.var_name)); return
    if sym.kind == 'const':
```

```
self._e(self.err.locke_operator(node.token, node.operator,
node.var_name)); return
    # Unary ++ -- operates
on COIN only (rule p.38)
    if sym.dtype != 'COIN':
```

```
self._e(self.err.invalid_operand_unary_stmt(node.token,
node.operator,
node.var_name, sym.dtype))
    # Value is now
runtime-determined; remove
from known values
```

```
self.known_values.pop(node
.var_name, None)
```

```
    # Validate the call directly
so that ABYSS-returning
functions are allowed.
```

```
def
visit_FuncCallStmtNode(self
, node):
    call = node.call_expr
    sym =
self.sym.lookup(call.name)
    if sym is None:
```

```
self._e(self.err.undeclared_function(call.token,
call.name)); return
    if sym.kind != 'func':
```

```
self._e(self.err.not_a_function(call.token, call.name));
return
    exp_cnt, act_cnt =
len(sym.params),
len(call.args)
    if exp_cnt != act_cnt:
```

```
self._e(self.err.arg_count_mismatch(call.token, call.name,
exp_cnt, act_cnt))
    else:
```

```
for idx, (p, a) in
enumerate(zip(sym.params,
call.args)):
```

```
    at = self.visit(a)
    if at and not
self._compatible(p.dtype,
at):
```

```
self._e(self.err.arg_type_mismatch(call.token, call.name,
idx+1, p.dtype,
self._type_name(at)))
```

```
    # Non-ABYSS
functions may also be called
as statements (return value is
discarded).
```

```
    # ABYSS functions are
intentionally allowed here
and nowhere else.
```

```
def visit_LiteralNode(self,
node): return node.dtype
```

```
def visit_IdentNode(self,
node):
    sym =
self.sym.lookup(node.name)
    if sym is None:
```

```
self._e(self.err.undeclared_variable(node.token,
node.name)); return None
    # A lone identifier (no
parentheses) is ONLY a
variable or constant.
```

```
    # If the name resolves
to a function, it must be
treated as undeclared
```

```
    # because function
references require
parentheses: greet() not
greet.
```

```
        if sym.kind == 'func':
            self._e(self.err.undeclared_v
                ariable(node.token,
                    node.name)); return None
            if not
            sym.is_initialized:
                self._e(self.err.uninitialized_
                    variable(node.token,
                        node.name))
                return sym.dtype

            def
            visit_ArrayAccessNode(self,
                node):
                sym =
                self.sym.lookup(node.name)
                if sym is None:
                    self._e(self.err.undeclared_v
                        ariable(node.token,
                            node.name)); return None

                    # SCROLL character
                    indexing: scroll{n} yields a
                    single PARCH character.
                    # The parser emits
                    ArrayAccessNode for all
                    {}-indexing, so we redirect
                    # here instead of raising
                    "not an array" for SCROLL
                    variables.
                    if sym.kind in ('var',
                        'param') and sym.dtype ==
                        'SCROLL':
                        if not
                        sym.is_initialized:
                            self._e(self.err.uninitialized_
                                variable(node.token,
                                    node.name))

                    if len(node.indices)
                    != 1:
                        self._e(self.err.scroll_char_re
                            quires_scroll(node.token,
                                node.name))
                        return None
                        it =
                        self.visit(node.indices[0])
                        if it and it != 'COIN':
                            self._e(self.err.scroll_char_in
                                dex_not_coin(node.token,
                                    self._type_name(it)))
                            # Compile-time
                            bounds check using known
                            SCROLL length
                            # Look up the
                            variable name directly, since
                            _known_scroll_length
                            # expects a
                            LiteralNode or IdentNode,
                            not an ArrayAccessNode.
                            str_len = None
                            if node.name in
                            self.known_values:
                                kv_dtype, kv_val
                                =
                                self.known_values[node.nam
                                    e]
                                if kv_dtype ==
                                'SCROLL' and kv_val is not
                                None:
                                    str_len =
                                    len(str(kv_val))
                                    if str_len is not None:
                                        idx_val =
                                        self._literal_int(node.indices
                                            [0])
                                        if idx_val is not
                                        None and (idx_val < 0 or
                                            idx_val >= str_len):
                                            self._e(self.err.array_index_
                                                not_coin(node.token,
                                                    node.name, i,
                                                        self._type_name(it)))
                                            if i <
                                            len(sym.dimensions):
                                                lbl = 'column
                                                    index' if (sym.is_2d and i ==
                                                        1) else 'row index' if
                                                        (sym.is_2d and i == 0) else
                                                        'index'

                                                self._check_bounds(node.na
                                                    me, lbl, idx,
                                                        sym.dimensions[i],
                                                            node.token)
                                                return sym.dtype

                                            def
                                            visit_MemberAccessNode(s
                                                elf, node):
                                                    if sym.kind != 'array':
                                                        self._e(self.err.not_an_array(
                                                            node.token, node.name));
                                                        return None
                                                        for i, idx in
                                                        enumerate(node.indices):
                                                            it = self.visit(idx)
                                                            if it and it != 'COIN':
                                                                self._e(self.err.array_index_
                                                                    not_coin(node.token,
                                                                        node.name, i,
                                                                            self._type_name(it)))
                                                                if i <
                                                                len(sym.dimensions):
                                                                    lbl = 'column
                                                                        index' if (sym.is_2d and i ==
                                                                            1) else 'row index' if
                                                                            (sym.is_2d and i == 0) else
                                                                            'index'

                                                                    self._check_bounds(node.na
                                                                        me, lbl, idx,
                                                                            sym.dimensions[i],
                                                                                node.token)
                                                                    return sym.dtype

                                                                def
                                                                visit_MemberAccessNode(s
                                                                    elf, node):
```

```

    sym =
self.sym.lookup(node.var_name)
    if sym is None:

self._e(self.err.undeclared_variable(
node.token,
node.var_name)); return
None
    if sym.kind !=
'struct_var':

self._e(self.err.not_a_struct_variable(
node.token,
node.var_name)); return
None
    ts =
self.sym.lookup(sym.struct_type_name)
    if ts is None or ts.kind
!= 'struct':

self._e(self.err.unresolvable_struct_type(
node.token,
sym.struct_type_name));
return None
    if node.member_name
not in ts.members:

self._e(self.err.no_such_member(
node.token,
sym.struct_type_name,
node.member_name)); return
None
    return
ts.members[node.member_name]

    def
visit_ScrollCharAccessNode
(self, node):

```

```

    st =
self.visit(node.scroll_expr)
    if st and st !=
'SCROLL':

self._e(self.err.scroll_char_requires_scroll(
node.token,
self._type_name(st)))
    it =
self.visit(node.index)
    if it and it != 'COIN':

self._e(self.err.scroll_char_in_dex_not_coin(
node.token,
self._type_name(it)))
    # Compile-time bounds
check (works for both
literals and known variables)
    str_len =
self._known_scroll_length(
node.scroll_expr)
    if str_len is not None:
        idx_val =
self._literal_int(node.index)
        if idx_val is not
None:
            if idx_val < 0 or
idx_val >= str_len:

self._e(self.err.scroll_char_in_dex_out_of_bounds(
node.token, idx_val, str_len))
        return 'SCROLL' #
SCROLL char access returns
a single-char SCROLL

    def
visit_StringConcatNode(self,
node):
    for op in
node.operands:
        ot = self.visit(op)

```

```

    if ot and ot !=
'SCROLL':

self._e(self.err.scroll_concat_requires_scroll(
node.token,
self._type_name(ot)))
    return 'SCROLL'

    def
visit_FuncCallNode(self,
node):
    sym =
self.sym.lookup(node.name)
    if sym is None:

self._e(self.err.undeclared_function(
node.token,
node.name)); return None
    if sym.kind != 'func':

self._e(self.err.not_a_function(
node.token, node.name));
return None
    exp_cnt, act_cnt =
len(sym.params),
len(node.args)
    if exp_cnt != act_cnt:

self._e(self.err.arg_count_mismatch(
node.token,
node.name, exp_cnt,
act_cnt))
    else:
        for i, (p, a) in
enumerate(zip(sym.params,
node.args)):
            at = self.visit(a)
            if at and not
self._compatible(p.dtype,
at):

self._e(self.err.arg_type_mis

```

```
match(node.token,
node.name, i+1, p.dtype,
self._type_name(at)))
    if sym.return_type ==
'ABYSS':

self._e(self.err.abyss_in_exp
ression(node.token,
node.name))
    return None
    return sym.return_type

def
visit_BinaryOpNode(self,
node):
    lt = self.visit(node.left);
    rt = self.visit(node.right)
    op = node.operator
    if op in ('+', '-', '*', '/',
'%', '^'):
        if lt and not
self._is_numeric(lt):

self._e(self.err.invalid_opera
nd_binary(node.token, op,
'left', self._type_name(lt)))
        if rt and not
self._is_numeric(rt):

self._e(self.err.invalid_opera
nd_binary(node.token, op,
'right', self._type_name(rt)))
```

```
    return 'DIME' if (lt
== 'DIME' or rt == 'DIME')
    else 'COIN'
        elif op in ('<', '>', '<=',
'>='):
            if lt and not
self._is_numeric(lt):

self._e(self.err.invalid_opera
nd_relational(node.token,
op, 'left',
self._type_name(lt)))
            if rt and not
self._is_numeric(rt):

self._e(self.err.invalid_opera
nd_relational(node.token,
op, 'right',
self._type_name(rt)))
            return 'BOOL'
            elif op in ('==', '!='):
                if lt and rt and not
self._compatible_expr(lt, rt):

self._e(self.err.incompatible_
comparison(node.token, op,
self._type_name(lt),
self._type_name(rt)))
                return 'BOOL'
            elif op in ('&&', '||'):
                if lt and not
self._is_bool(lt):

self._e(self.err.invalid_opera
```

```
nd_logical(node.token, op,
'left', self._type_name(lt)))
        if rt and not
self._is_bool(rt):

self._e(self.err.invalid_opera
nd_logical(node.token, op,
'right', self._type_name(rt)))
        return 'BOOL'
        return None

def
visit_UnaryOpNode(self,
node):
    ot =
self.visit(node.operand); op
= node.operator
    if op == '-':
        if ot and not
self._is_numeric(ot):

self._e(self.err.invalid_opera
nd_unary_neg(node.token,
self._type_name(ot)))
        return ot
        elif op in ('!', '#'):
            if ot and not
self._is_bool(ot):

self._e(self.err.invalid_opera
nd_not(node.token, op,
self._type_name(ot)))
            return 'BOOL'
            return None
```

```
class Symbol:
    # Base symbol entry.
```

symbol_table.py

```
def __init__(self, name,
dtype, kind, token,
```

```
is_initialized=False,
init_expr=None):
    self.name = name
```



```

    self.dtype = dtype
    self.kind = kind      #
'var'|'const'|'array'|'func'|'struct'|'struct_var'|'param'
    self.token = token
    self.is_initialized =
is_initialized
    self.init_expr =
init_expr # Store the
initialization expression
(AST node) for bounds
checking

    def __repr__(self):
        return
(f'Symbol(name={self.name!r}, dtype={self.dtype!r}, "
f'kind={self.kind!r},
init={self.is_initialized})')

# Array declaration symbol.
class ArraySymbol(Symbol):
    def __init__(self, name,
dtype, dimensions, is_2d,
token):
        super().__init__(name,
dtype, 'array', token,
is_initialized=True)
        self.dimensions =
dimensions # list[int]
        self.is_2d = is_2d

    def __repr__(self):
        dims = 'x'.join(str(d) for
d in self.dimensions)
        return
(f'ArraySymbol(name={self.name!r},
dtype={self.dtype!r}, "
f'dims={dims},
is_2d={self.is_2d})')

```

SeaStack

```

# Function definition
symbol.
class
FunctionSymbol(Symbol):
    def __init__(self, name,
return_type, params, token):
        super().__init__(name,
return_type, 'func', token,
is_initialized=True)
        self.return_type =
return_type
        self.params = params
# list[ParamNode]

    def __repr__(self):
        param_types = '
'.join(p.dtype for p in
self.params)
        return
(f'FunctionSymbol(name={self.name!r}, "
f'return={self.return_type!r}
,
params=[{param_types}])')

# Struct TYPE definition
(MAST Ship [...]).
class
StructTypeSymbol(Symbol):
    def __init__(self, name,
members, member_order,
token):
        super().__init__(name,
'struct_type', 'struct', token,
is_initialized=True)
        self.members =
members # dict[name
→ dtype]

```

```

    self.member_order =
member_order # list[str] in
declaration order

    def __repr__(self):
        mem_str = '
'.join(f'{k}:{v}' for k, v in
self.members.items())
        return
f'StructTypeSymbol(name=
{self.name!r},
members={{ {mem_str} }}')

# Struct VARIABLE
instance (MAST Ship s1).
class
StructVarSymbol(Symbol):
    def __init__(self, name,
struct_type_name, token):
        super().__init__(name,
struct_type_name,
'struct_var', token,
is_initialized=True)
        self.struct_type_name =
struct_type_name

    def __repr__(self):
        return
(f'StructVarSymbol(name=
{self.name!r}, "
f'struct_type={self.struct_ty
pe_name!r})')

# Scope-stack symbol table
class SymbolTable:

    def __init__(self):
        self._scopes: list = [{}]
# index 0 = global, index -1
= current

```

```

    return True

# --- Scope Management
---

def push_scope(self):
    self._scopes.append({})

def pop_scope(self):
    if len(self._scopes) <=
1:
        raise
RuntimeError("Cannot pop
global scope.")
    self._scopes.pop()

def scope_level(self) ->
int:
    return len(self._scopes)

def is_global_scope(self)
-> bool:
    return len(self._scopes)
== 1

# --- Declaration ---

# Insert into current scope.
Returns False if duplicate.
def declare(self, symbol:
Symbol) -> bool:
    current =
self._scopes[-1]
    if symbol.name in
current:
        return False
    current[symbol.name] =
symbol

    return True

# --- Lookup ---

# Search from innermost
to outermost scope.
def lookup(self, name:
str):
    for scope in
reversed(self._scopes):
        if name in scope:
            return
    scope[name]
    return None

# Search only the current
(innermost) scope.
def
lookup_current_scope(self,
name: str):
    return
self._scopes[-1].get(name)

# Search only the global
(outermost) scope.
def
lookup_global_scope(self,
name: str):
    return
self._scopes[0].get(name)

# --- Mutation ---

# Mark a variable as
initialized (after
assignment).

def update_initialized(self,
name: str):
    for scope in
reversed(self._scopes):
        if name in scope:
            scope[name].is_initialized =
True
            return

# --- Diagnostics ---

def dump(self) -> str:
    lines = [f"SymbolTable
--- {len(self._scopes)}
scope(s):"]
    for depth, scope in
enumerate(self._scopes):
        label = "global" if
depth == 0 else
f"scope[{depth}]"
        lines.append(f"
{label}:")
        if scope:
            for name, sym in
scope.items():
                lines.append(f"
{name!r:25s} -> {sym!r}")
            else:
                lines.append("
(empty)")
    return "\n".join(lines)

```

```

import re

code_generator.py
from codegen.ir_instructions
import (

```

```

IRProgram, Quad,

```

```
PROGRAM_START,
PROGRAM_END,
FUNC_BEGIN,
FUNC_END,
PARAM_DECL,
    AHOY_BEGIN,
    AHOY_END,
    DECL_VAR,
    DECL_CONST,
    DECL_ARR,
    DECL_STRUCT_TYPE,
    DECL_STRUCT_VAR,
    ARR_INIT_1D,
    ARR_INIT_2D,
    STRUCT_INIT,
    ASSIGN, ASSIGN_ARR,
    ASSIGN_ARR2,
    ASSIGN_MEMBER,
    ADD, SUB, MUL, DIV,
    MOD, POW,
    LT, GT, LE, GE, EQ, NE,
    LOG_AND, LOG_OR,
    LOG_NOT, LOG_DNOT,
    UNARY_NEG,
    UNARY_INC,
    UNARY_DEC,
    CONCAT,
    SCROLL_CHAR,
    LOAD_ARR,
    LOAD_ARR2,
    LOAD_MEMBER,
    LABEL, JUMP,
    JUMP_FALSE,
    JUMP_TRUE, BREAK,
    CONTINUE,
    ARG, CALL,
    CALL_VOID, RETURN,
    RETURN_VOID,
    INPUT, OUTPUT, NOP,
    LITERAL, COMMENT,
    COMPOUND_ASSIGN,
)
```

SeaStack

```
def
_convert_parch_escapes(raw
):
    # Convert SeaStack
    PARCH escape sequences to
    Python equivalents.
    if raw is None:
        return ""
    s = str(raw)
    # Remove surrounding
    single quotes if present
    if len(s) >= 2 and s[0] ==
    "" and s[-1] == "":
        s = s[1:-1]
    s = s.replace("\\'",
    '\x00BACKSLASH\x00')
    s = s.replace("\\s', '')
    s = s.replace("\\n', 'n')
    s = s.replace("\\t', 't')
    s = s.replace("\\0', '0')
    s =
    s.replace("\x00BACKSLAS
    H\x00', '\\')
    return s
```

```
def
_convert_scroll_escapes(raw
):
    """"Convert SeaStack
    SCROLL escape sequences
    to Python equivalents.""
    if raw is None:
        return ""
    s = str(raw)
    # Remove surrounding
    double quotes if present
    if len(s) >= 2 and s[0] ==
    "" and s[-1] == "":
        s = s[1:-1]
```

```
s = s.replace("\\'",
    '\x00BACKSLASH\x00')
s = s.replace("\\d', '')
s = s.replace("\\n', 'n')
s = s.replace("\\t', 't')
s = s.replace("\\0', '0')
s =
s.replace("\x00BACKSLAS
H\x00', '\\')
return s
```

```
def
_format_spec_to_python(fmt
_str):
    """"Convert SeaStack
    format specifiers in a
    SCROLL literal to Python
    format-string
    placeholders.
```

Input: 'Hello %S, you are
 %C years old\n'

Output: ('Hello {}, you are
 {} years old\n', ['SCROLL',
 'COIN'])

```
raw =
_convert_scroll_escapes(fmt
_str)
specs = []
spec_map = {'C': 'COIN',
'D': 'DIME', 'P': 'PARCH',
'S': 'SCROLL', 'B': 'BOOL'}
```

```
def replacer(m):
    ch = m.group(1)

    specs.append(spec_map.get(
    ch, 'SCROLL'))
    return '{}'
```

```

converted =
re.sub(r'%([CDPSB])',
replacer, raw)
    return converted, specs

_DEFAULT_VALUES = {
    'COIN': '0',
    'DIME': '0.0',
    'PARCH': '"\\x00"', #
SeaStack default: null char
'\\x00'
    'SCROLL': '"\\x00"', #
SeaStack default: null string
"\\x00"
    'BOOL': 'False', #
SeaStack NAY
}

_INPUT_CONVERTERS =
{
    'COIN': 'int',
    'DIME': 'float',
    'PARCH': 'str',
    'SCROLL': 'str',
    'BOOL': '_ss_bool_input',
}

class CodeGenerator:

    def __init__(self,
ir_program: IRProgram):
        self.ir = ir_program
        self._lines = []
        self._indent = 0
        self._temps = {}
        self._arg_stack = []
        self._var_types = {}
        self._const_names =
set()
        self._func_bodies = {}
        # func_name → list of Quad

        self._ahoy_body = []
        self._global_decls = []

        def generate(self):
            # Generate executable
            Python code.

            self._partition_instructions()
            self._emit_preamble()
            self._emit_globals()
            self._emit_functions()
            self._emit_ahoy()
            self._emit_entry()
            return
            '\\n'.join(self._lines)

            def
_partition_instructions(self):
                """Split IR into global
                decls, function bodies, and
                AHoy body."""
                instrs =
                self.ir.instructions
                current_func = None
                current_body = []
                in_ahoy = False

                for q in instrs:
                    if q.op ==
PROGRAM_START:
                        continue
                    elif q.op ==
PROGRAM_END:
                        continue
                    elif q.op ==
FUNC_BEGIN:
                        current_func =
q.arg1
                        current_body = [q]
                    elif q.op ==
FUNC_END:
                        self._ahoy_body.append(q)

                        self._func_bodies[current_fu
nc] = current_body
                        current_func =
None
                        current_body = []
                        elif q.op ==
AHoy_BEGIN:
                            in_ahoy = True
                            current_body = []
                        elif q.op ==
AHoy_END:
                            self._ahoy_body =
current_body
                            in_ahoy = False
                            current_body = []
                            elif current_func is
not None:
                                current_body.append(q)
                                elif in_ahoy:
                                    current_body.append(q)
                                    else:
                                        self._global_decls.append(q)

                                def _emit(self, line):
                                    self._lines.append(' '*
self._indent + line)

                                def _emit_blank(self):
                                    self._lines.append("")

                                def _indent_inc(self):
                                    self._indent += 1

                                def _indent_dec(self):
                                    self._indent = max(0,
self._indent - 1)

```

SeaStack

```
def _val(self, operand):
    if operand is None:
        return 'None'
    if isinstance(operand,
bool):
        return 'True' if
operand else 'False'
    if isinstance(operand,
int):
        return str(operand)
    if isinstance(operand,
float):
        return repr(operand)
    if isinstance(operand,
str):
        if operand in
self._temps:
            return
self._temps[operand]
        return
self._sanitize_name(operand
)
        return repr(operand)

def _sanitize_name(self,
name):
    py_reserved = {
        'False', 'None', 'True',
'and', 'as', 'assert', 'async',
'await',
        'break', 'class',
'continue', 'def', 'del', 'elif',
'else',
        'except', 'finally', 'for',
'from', 'global', 'if', 'import',
        'in', 'is', 'lambda',
'nonlocal', 'not', 'or', 'pass',
'raise',
        'return', 'try', 'while',
'with', 'yield',
    }
```

SeaStack

```
        return f'_ss_{name}' if
name in py_reserved else
name

def _python_literal(self,
dtype, value):
    if dtype == 'COIN':
        return str(int(value))
    if dtype == 'DIME':
        f = float(value)
        s = repr(f)
        # Guarantee a
decimal point so whole
numbers display as e.g. 5.0
        if '.' not in s and 'e'
not in s and 'n' not in s:
            s += '.0'
        return s
    if dtype == 'BOOL':
        return 'True' if value else
'False'
    if dtype == 'PARCH':
        return
repr(_convert_parch_escapes
(value))
    if dtype == 'SCROLL':
        return
repr(_convert_scroll_escapes
(value))
        return repr(value)

def _resolve_type(self,
operand):
    # Resolve the SeaStack
type of an operand.
    if isinstance(operand,
bool):
        return 'BOOL'
    if isinstance(operand,
int):
        return 'COIN'
```

```
    if isinstance(operand,
float):
        return 'DIME'
    if isinstance(operand,
str):
        # Check
code-generator local type
registry first
        t =
self._var_types.get(operand)
        if t:
            return t
        # Fall back to the IR
program's temp-type
metadata
        t =
self.ir.temp_types.get(operan
d)
        if t:
            return t
        return None

def _emit_preamble(self):
    self._emit("# =====
SeaStack → Python
(Generated Code) =====")
    self._emit("import sys,
math")
    self._emit_blank()
    self._emit("_ss_line =
0")
    self._emit("_ss_col =
0")
    self._emit_blank()
    self._emit("def
_ss_bool_input(s):")
    self._indent_inc()
    self._emit("s =
s.strip()")
    self._emit("if s ==
'AYE': return True")
```



```

        self._emit("if s ==
'NAY': return False")
        self._emit("raise
ValueError(f'Expected AYE
or NAY, got {s!r}'))
        self._indent_dec()
        self._emit_blank()
        self._emit("def
_ss_make_array(dims,
default=None):")
        self._indent_inc()
        self._emit("for _d in
dims:")
        self._indent_inc()
        self._emit("if not
isinstance(_d, int) or _d <=
0:")
        self._indent_inc()
        self._emit("raise
ValueError(f'Invalid array
size {_d!r}. array
dimensions must be positive
integers.').")
        self._indent_dec()
        self._indent_dec()
        self._emit("if len(dims)
== 1: return [default] *
dims[0]")
        self._emit("return
[[default] * dims[1] for _ in
range(dims[0])]")
        self._indent_dec()
        self._emit_blank()
        self._emit("def
_ss_check_div(divisor):")
        self._indent_inc()
        self._emit("if divisor ==
0:")
        self._indent_inc()
        self._emit("raise
ZeroDivisionError('Division
by zero is not allowed.').")

```

```

        self._indent_dec()
        self._indent_dec()
        self._emit_blank()
        self._emit("def
_ss_check_arr_bounds(arr,
idx, arr_name='array'):")
        self._indent_inc()
        self._emit("if not
isinstance(idx, int):")
        self._indent_inc()
        self._emit("raise
TypeError(f'Array index
must be an integer, got
{type(idx).__name__!r}.').")
        self._indent_dec()
        self._emit("if idx < 0:")
        self._indent_inc()
        self._emit("raise
IndexError(f'index {idx} is
negative for array
\\\"+arr_name+\\\"').")
        self._indent_dec()
        self._emit("if idx >=
len(arr):")
        self._indent_inc()
        self._emit("raise
IndexError(f'index {idx}
exceeds size {len(arr)} of
array \\\"+arr_name+\\\"').")
        self._indent_dec()
        self._indent_dec()
        self._emit_blank()
        self._emit("def
_ss_display(val):")
        self._indent_inc()
        self._emit("if
isinstance(val, bool): return
'AYE' if val else 'NAY'")
        self._emit("return
str(val)")
        self._indent_dec()
        self._emit_blank()

```

```

        self._emit("def
_ss_scroll_char(s, idx):")
        self._indent_inc()
        self._emit("if idx < 0 or
idx >= len(s): return '\\x00'")
        self._emit("return
s[idx]")
        self._indent_dec()
        self._emit_blank()

        # Enforces SeaStack
        numeric limits at runtime:
        # COIN : integer part
        must not exceed 16 digits
        # DIME : integer part
        ≤ 16 digits AND decimal
        part ≤ 8 digits
        self._emit("def
_ss_check_overflow(val,
dtype, context='value'):")
        self._indent_inc()
        self._emit("from
decimal import Decimal as
_D")
        self._emit("if dtype ==
'COIN':")
        self._indent_inc()
        self._emit("if
abs(int(val)) >
9_999_999_999_999_999:")
        self._indent_inc()
        self._emit("raise
OverflowError(f'COIN
{context} exceeds the
16-digit limit.').")
        self._indent_dec()
        self._indent_dec()
        self._emit("elif dtype
== 'DIME':")
        self._indent_inc()
        self._emit("import math
as _m")

```

```
self._emit("if
_m.isinf(val) or
_m.isnan(val):")
self._indent_inc()
self._emit("raise
OverflowError(f'DIME
{context} is not a finite
number.')"")
self._indent_dec()
self._emit("if
abs(int(val)) >
9_999_999_999_999_999:")
self._indent_inc()
self._emit("raise
OverflowError(f'DIME
{context} integer part
exceeds the 16-digit limit.')"")
self._indent_dec()
self._emit("_sign,
_digits, _exp =
_D(repr(abs(float(val))))).as_
tuple()")
self._emit("if _exp <
-8:")
self._indent_inc()
self._emit("val =
round(val, 8)")
self._indent_dec()
self._indent_dec()
self._emit("return val")
self._indent_dec()
self._emit_blank()
self._emit("def
_ss_parse_input(raw_line,
types):")
self._indent_inc()
self._emit("if len(types)
== 1 and types[0] ==
'SCROLL': return
[raw_line]")
self._emit("tokens =
raw_line.split()")
```

```
self._emit("results =
[]")
self._emit("for i, t in
enumerate(types):")
self._indent_inc()
self._emit("tok =
tokens[i] if i < len(tokens)
else """)
self._emit("if t ==
'COIN':")
self._indent_inc()
self._emit("if not tok:")
self._indent_inc()
self._emit("raise
ValueError('Expected an
integer (COIN) but received
empty input.')"")
self._indent_dec()
self._emit("try:")
self._indent_inc()
self._emit("_coin_val =
int(tok)")
self._indent_dec()
self._emit("except
ValueError:")
self._indent_inc()
self._emit("raise
ValueError(f'Expected a
whole number (COIN), got
{tok!r}. Do not use decimals
or letters.')"")
self._indent_dec()
self._emit("if
len(tok.lstrip('-+')) > 16:")
self._indent_inc()
self._emit("raise
RuntimeError('COIN input
exceeds 16 digits.')"")
self._indent_dec()
```

```
self._emit("results.append(_
coin_val)")
self._indent_dec()
self._emit("elif t ==
'DIME':")
self._indent_inc()
self._emit("if not tok:")
self._indent_inc()
self._emit("raise
ValueError('Expected a
decimal number (DIME) but
received empty input.')"")
self._indent_dec()
self._emit("try:")
self._indent_inc()
self._emit("_dime_val
= float(tok)")
self._indent_dec()
self._emit("except
ValueError:")
self._indent_inc()
self._emit("raise
ValueError(f'Expected a
numeric value (DIME), got
{tok!r}. Please enter a
number.')"")
self._indent_dec()
# Validate integer and
decimal parts separately
(max 16 and 8 digits
respectively)
self._emit("_dime_clean =
tok.lstrip('-+').split('.')[0]")
self._emit("if
len(_dime_clean) > 16:")
self._indent_inc()
self._emit("raise
RuntimeError('DIME input
```

```
exceeds 16 digits in integer
part.')"
    self._indent_dec()
    self._emit("if '.' in
tok:")
    self._indent_inc()
    self._emit("_dime_dec
= tok.split('.')[1]")
    self._emit("if
len(_dime_dec) > 8:")
    self._indent_inc()
    self._emit("raise
RuntimeError('DIME input
exceeds 8 digits in decimal
part.')"")
    self._indent_dec()
    self._indent_dec()

self._emit("results.append(_
dime_val)")
    self._indent_dec()

    self._emit("elif t ==
'BOOL: ")
    self._indent_inc()
    self._emit("if not tok:")
    self._indent_inc()
    self._emit("raise
ValueError('Expected AYE
or NAY (BOOL) but
received empty input.')"")
    self._indent_dec()

self._emit("results.append(_s
s_bool_input(tok)")
    self._indent_dec()

    self._emit("elif t ==
'PARCH: ")
    self._indent_inc()
    self._emit("if not tok:")
    self._indent_inc()
```

```
self._emit("raise
ValueError('Expected a
single character (PARCH)
but received empty input.')"")
    self._indent_dec()
    self._emit("if len(tok)
!= 1:")
    self._indent_inc()
    self._emit("raise
ValueError(f'Expected
exactly one character
(PARCH), got {tok!r}
({len(tok)} characters).'"")
    self._indent_dec()

self._emit("results.append(to
k)")
    self._indent_dec()

    self._emit("else:")
    self._indent_inc()

self._emit("results.append(to
k)")
    self._indent_dec()
    self._indent_dec()
    self._emit("return
results")
    self._indent_dec()
    self._emit_blank()

    def
_maybe_emit_line_tracker(s
elf, q):
    # Exact SeaStack
source line
    line = getattr(q, 'line',
None)
    col = getattr(q, 'col',
None)
    if line is not None:
```

```
self._emit(f"_ss_line
= {line}")
    self._emit(f"_ss_col
= {col if col is not None else
0}")

def _emit_globals(self):
    if self._global_decls:
        self._emit("# ---
Global Declarations ---")

self._emit_block(self._global
_decls)
    self._emit_blank()

def _emit_functions(self):
    for fname, body in
self._func_bodies.items():

self._emit_function(fname,
body)

def _emit_function(self,
fname, instrs):
    # Extract params
params = []
    body_start = 0
    ret_type = None
    for i, q in
enumerate(instrs):
        if q.op ==
FUNC_BEGIN:
            ret_type = q.arg2
        elif q.op ==
PARAM_DECL:

params.append(self._sanitize
_name(q.arg1))

self._var_types[q.arg1] =
q.arg2
```

```

        elif q.op ==
FUNC_END:
            break
        else:
            if body_start == 0:
                body_start = i
            # Find body (between
last PARAM_DECL and
FUNC_END)
            body_instrs = []
            found_first_non_param
= False
            for q in instrs:
                if q.op in
(FUNC_BEGIN,
PARAM_DECL,
FUNC_END):
                    continue

body_instrs.append(q)

            safe_name =
self._sanitize_name(fname)
            params_str = ',
'.join(params)
            self._emit(f'def
{safe_name}({params_str}):
")
            self._indent_inc()
            global_var_names = [

self._sanitize_name(q.arg1)
            for q in
self._global_decls
                if q.op in
(DECL_VAR,
DECL_CONST,
DECL_ARR,
DECL_STRUCT_VAR)
                    ]
            all_globals = ['_ss_line',
'_ss_col'] +
global_var_names
            self._emit(f'global {'
'.join(all_globals)}")
            if self._ahoy_body:

self._emit_block(self._ahoy_
body)
            else:

                self._emit("pass")
                self._indent_dec()
                self._emit_blank()

            def _emit_entry(self):
                self._emit("if
__name__ == '__main__':")
                self._indent_inc()

self._emit("_ss_ahoy()")
                self._indent_dec()

                # Emit a sequence of IR
instructions as Python
statements.
                def _emit_block(self,
instrs):
                    i = 0
                    emitted_any = False
                    while i < len(instrs):
                        q = instrs[i]
                        if q.op == LITERAL:
                            py_val =
self._python_literal(q.arg1,
q.arg2)

                            self._temps[q.result] =
py_val

                            self._emit(f'{q.result} =
{py_val}')
                            emitted_any =
True

                            i += 1
                            continue

                # --- Simple
assignment ---
                if q.op == ASSIGN:
                    if q.result:

```

```

self._maybe_emit_line_tracker(q)

self._emit(f'{self._sanitize_name(q.result)} = {self._val(q.arg1)}')
    emitted_any =
True
    i += 1
    continue

    # --- Declarations ---
    if q.op ==
DECL_VAR:

self._maybe_emit_line_tracker(q)
    name =
self._sanitize_name(q.arg1)

self._var_types[q.arg1] =
q.arg2
    if q.result is not
None:

self._emit(f'{name} = {self._val(q.result)}')
    else:

self._emit(f'{name} = {_DEFAULT_VALUES.get(q.arg2, 'None')}')
    emitted_any =
True
    i += 1
    continue

    if q.op ==
DECL_CONST:

self._maybe_emit_line_tracker(q)
    name =
self._sanitize_name(q.arg1)

self._var_types[q.arg1] =
q.arg2
    dims = q.result
    default =
_DEFAULT_VALUES.get(q.arg2, 'None')
    for _d in dims:
        if isinstance(_d,
int) and _d <= 0:

self._emit(f'raise
ValueError("Invalid array
size {_d} for {name!r}:
dimensions must be positive
integers.")')

self._maybe_emit_line_tracker(q)
    name =
self._sanitize_name(q.arg1)

self._var_types[q.arg1] =
q.arg2
    if q.op ==
DECL_ARR:

self._maybe_emit_line_tracker(q)
    name =
self._sanitize_name(q.arg1)

self._var_types[q.arg1] =
q.arg2
    if q.op ==
DECL_STRUCT_TYPE:
    i += 1
    continue

    if q.op ==
DECL_STRUCT_VAR:
    name =
self._sanitize_name(q.arg1)
    members =
self.ir.struct_types.get(q.arg2, {})
    md = ',
'.join(f'{m}':
{_DEFAULT_VALUES.get(d, 'None')} for m, d in
members.items())

self._emit(f'{name} = {{{md}}}')
    emitted_any =
True
    i += 1
    continue

    if q.op ==
STRUCT_INIT:
    name =
self._sanitize_name(q.arg1)
    if q.arg2:
        for member, vt
in q.arg2.items():

```

```

                                rows = q.arg2 or
self._emit(f'{name}['{memb        []
er}'] = {self._val(vt)}")
                                if rows:
                                n_rows =
                                emitted_any =
True
                                len(rows)
                                self._emit(
                                f'if
                                len({name}) < {n_rows}: "
                                f'{name}
                                += [[{default}]] * ({n_rows}
                                - len({name}))"
                                )
                                for ri, row in
                                enumerate(rows):
                                n_cols =
                                len(row)
                                self._emit(
                                f'if
                                len({name}[{ri}]) <
                                {n_cols}: "
                                f'{name}[{ri}] +=
                                [{default}] * "
                                f'({n_cols} -
                                len({name}[{ri}]))"
                                )
                                for ci, v in
                                enumerate(row):
                                self._emit(f'{name}[{ri}][{c
                                i}] = {self._val(v)}")
                                emitted_any =
                                True
                                i += 1
                                continue

                                # --- Binary/Unary
                                ops ---
                                if q.op in (ADD,
                                SUB, MUL, DIV, MOD,
                                POW):

self._emit(f'{name}['{memb        []
er}'] = {self._val(vt)}")
                                emitted_any =
True
                                i += 1
                                continue

                                if q.op in
                                (ARR_INIT_1D,
                                ARR_INIT_2D):
                                name =
                                self._sanitize_name(q.arg1)
                                dtype =
                                self._var_types.get(q.arg1, ")
                                default =
                                _DEFAULT_VALUES.get(d
                                type, 'None')

                                if q.op ==
                                ARR_INIT_1D:
                                vals = q.arg2 or
                                []
                                if vals:
                                val_strs =
                                [self._val(v) for v in vals]
                                n =
                                len(val_strs)
                                self._emit(
                                f'if
                                len({name}) < {n}: "
                                f'{name}
                                += [{default}] * ({n} -
                                len({name}))"
                                )
                                for idx, vs in
                                enumerate(val_strs):

                                self._emit(f'{name}[{idx}]
                                = {vs}")
                                else: #
                                ARR_INIT_2D

                                self._maybe_emit_line_track
                                er(q)
                                a, b =
                                self._val(q.arg1),
                                self._val(q.arg2)
                                py_ops = {ADD:
                                '+', SUB: '-', MUL: '*', DIV:
                                '/', MOD: '%', POW: '**'}
                                op = py_ops[q.op]
                                if q.op == DIV:
                                t1 =
                                self._resolve_type(q.arg1)
                                t2 =
                                self._resolve_type(q.arg2)

                                self._emit(f'_ss_check_div(
                                {b})")
                                if t1 == 'COIN'
                                and t2 == 'COIN':
                                expr =
                                f'int(({a}) / ({b}))"
                                else:
                                expr = f'({a}
                                {op} {b})"
                                elif q.op == MOD:

                                self._emit(f'_ss_check_div(
                                {b})")
                                expr =
                                f'int(math.fmod({a}, {b}))"
                                else:
                                expr = f'({a}
                                {op} {b})"

                                self._temps[q.result] = expr

                                self._emit(f'{q.result} =
                                {expr}")
                                rtype =
                                self._resolve_type(q.result)

```

<pre> if rtype in ('COIN', 'DIME'): self._emit(f'{q.result} = _ss_check_overflow({q.resul t}, {repr(rtype)}, 'result')") emitted_any = True i += 1 continue if q.op in (LT, GT, LE, GE, EQ, NE): self._maybe_emit_line_track er(q) a, b = self._val(q.arg1), self._val(q.arg2) py_ops = {LT: '<', GT: '>', LE: '<=', GE: '>=', EQ: '==', NE: '!='} expr = f'({a} {py_ops[q.op]} {b})" self._temps[q.result] = expr self._emit(f'{q.result} = {expr}') emitted_any = True i += 1 continue if q.op in (LOG_AND, LOG_OR): a, b = self._val(q.arg1), self._val(q.arg2) py_op = 'and' if q.op == LOG_AND else 'or' </pre>	<pre> expr = f'({a} {py_op} {b})" self._temps[q.result] = expr self._emit(f'{q.result} = {expr}') emitted_any = True i += 1 continue if q.op == LOG_NOT: expr = f'(not {self._val(q.arg1)})" self._temps[q.result] = expr self._emit(f'{q.result} = {expr}') emitted_any = True i += 1 continue if q.op == LOG_DNOT: expr = f'(not not {self._val(q.arg1)})" self._temps[q.result] = expr self._emit(f'{q.result} = {expr}') emitted_any = True i += 1 continue if q.op == UNARY_NEG: </pre>	<pre> expr = f'(-{self._val(q.arg1)})" self._temps[q.result] = expr self._emit(f'{q.result} = {expr}') rtype = self._resolve_type(q.result) if rtype in ('COIN', 'DIME'): self._emit(f'{q.result} = _ss_check_overflow({q.resul t}, {repr(rtype)}, 'result')") emitted_any = True i += 1 continue if q.op == UNARY_INC: self._maybe_emit_line_track er(q) vname = self._sanitize_name(q.arg1) self._emit(f'{vname} += 1") vtype = self._var_types.get(q.arg1) or self.ir.temp_types.get(q.arg1) if vtype in ('COIN', 'DIME'): self._emit(f'{vname} = _ss_check_overflow({vname }, {repr(vtype)}, 'result')") emitted_any = True </pre>
--	---	--

```

        i += 1
        continue

    if q.op ==
UNARY_DEC:

    self._maybe_emit_line_track
    er(q)
        vname =
    self._sanitize_name(q.arg1)

    self._emit(f'{vname} -= 1')
        vtype =
    self._var_types.get(q.arg1)
    or
    self.ir.temp_types.get(q.arg1
    )
        if vtype in
('COIN', 'DIME'):

    self._emit(f'{vname} =
    _ss_check_overflow({vname
    }, {repr(vtype)}, 'result')')
        emitted_any =
    True
        i += 1
        continue

    # --- Compound
    assignment (e.g. +=, -=, *=,
    /=, %=) ---
        if q.op ==
COMPOUND_ASSIGN:

    self._maybe_emit_line_track
    er(q)
        name =
    self._sanitize_name(q.result)
        py_ops = {'+': '+=',
    '-': '-=', '*': '*=', '/': '/=', '%':
    '%=', '**': '**='}

        op_str =
    py_ops.get(q.arg2,
    f'{q.arg2}=') if q.arg2 else
    '+='
        val =
    self._val(q.arg1)

    self._emit(f'{name}
    {op_str} {val}')
        vtype =
    self._var_types.get(q.result)
    or
    self.ir.temp_types.get(q.resul
    t)
        if vtype in
('COIN', 'DIME'):

    self._emit(f'{name} =
    _ss_check_overflow({name}
    , {repr(vtype)}, 'result')')
        emitted_any =
    True
        i += 1
        continue

    # --- String ---
    if q.op == CONCAT:
        a, b =
    self._val(q.arg1),
    self._val(q.arg2)
        expr = f'(str({a})
    + str({b}))'

    self._temps[q.result] = expr

    self._emit(f'{q.result} =
    {expr}')
        emitted_any =
    True
        i += 1
        continue

    if q.op ==
    SCROLL_CHAR:
        s, idx =
    self._val(q.arg1),
    self._val(q.arg2)
        # _ss_scroll_char
    honours the SeaStack rule:
    s[len(s)] == '\x00'
        expr =
    f'_ss_scroll_char({s},
    {idx})'

    self._temps[q.result] = expr

    self._emit(f'{q.result} =
    {expr}')
        emitted_any =
    True
        i += 1
        continue

    # --- Loads ---
    if q.op ==
    LOAD_ARR:

    self._maybe_emit_line_track
    er(q)
        name =
    self._sanitize_name(q.arg1)
        idx =
    self._val(q.arg2)

    self._emit(f'_ss_check_arr_
    bounds({name}, {idx},
    {name!r})')
        expr =
    f'{name}[{idx}]'

    self._emit(f'{q.result} =
    {expr}')
        emitted_any =
    True

```

i += 1		bounds({name}, {r1},
continue	self._emit(f'{q.result} =	{name!r}"))
	{expr}")	
if q.op ==	emitted_any =	self._emit(f'_ss_check_arr_
LOAD_ARR2:	True	bounds({name}[{r1}], {r2},
	i += 1	{name!r}+'[row]'))
self._maybe_emit_line_track	continue	
er(q)		self._emit(f'{name}[{r1}][{
name =	# --- Stores ---	r2}] = {self._val(q.result)}")
self._sanitize_name(q.arg1)	if q.op ==	emitted_any =
i1, i2 = q.arg2	ASSIGN_ARR:	True
r1, r2 =		i += 1
self._val(i1), self._val(i2)	self._maybe_emit_line_track	continue
	er(q)	
self._emit(f'_ss_check_arr_	name =	if q.op ==
bounds({name}, {r1},	self._sanitize_name(q.arg1)	ASSIGN_MEMBER:
{name!r}"))	idx =	
	self._val(q.arg2)	self._maybe_emit_line_track
self._emit(f'_ss_check_arr_		er(q)
bounds({name}[{r1}], {r2},	self._emit(f'_ss_check_arr_	name =
{name!r}+'[row]'))	bounds({name}, {idx},	self._sanitize_name(q.arg1)
expr =	{name!r}"))	
f'{name}[{r1}][{r2}]"		self._emit(f'{name}['{q.arg2
	self._emit(f'{name}[{idx}]	}] = {self._val(q.result)}")
self._emit(f'{q.result} =	= {self._val(q.result)}")	emitted_any =
{expr}")	emitted_any =	True
emitted_any =	True	i += 1
True	i += 1	continue
i += 1	continue	
continue		# --- I/O ---
	if q.op ==	if q.op == INPUT:
if q.op ==	ASSIGN_ARR2:	
LOAD_MEMBER:		self._maybe_emit_line_track
	self._maybe_emit_line_track	er(q)
self._maybe_emit_line_track	er(q)	self._gen_input(q)
er(q)	name =	emitted_any =
name =	self._sanitize_name(q.arg1)	True
self._sanitize_name(q.arg1)	i1, i2 = q.arg2	i += 1
expr =	r1, r2 =	continue
f'{name}['{q.arg2}']"	self._val(i1), self._val(i2)	
		if q.op == OUTPUT:
	self._emit(f'_ss_check_arr_	

```

                                end_label =
self._maybe_emit_line_tracker(q)      None
                                is_do_while =
                                False
self._gen_output(q)                    while j <
    emitted_any =                      len(instrs):
True                                  if instrs[j].op
    i += 1                            == JUMP and instrs[j].arg1
    continue                         == label_name:
                                    # This is
                                    the back-edge — loop ends
                                    here
    # --- Control flow ---
    if q.op == LABEL:
        # Check if this
        label is a loop start (jumped
        back to later)
        label_name =
q.arg1                                if j <
    is_loop_target =                 len(instrs) and instrs[j].op
False                               == LABEL:
    for k in range(i +
1, len(instrs)):
        if instrs[k].op
        == JUMP and instrs[k].arg1
        == label_name:
            is_loop_target
            = True
            break
            if instrs[k].op
            == JUMP_TRUE and
            instrs[k].arg2 ==
            label_name:
                is_loop_target
                = True
                break
                if instrs[k].op
                == JUMP_TRUE and
                instrs[k].arg2 ==
                label_name:
                    is_loop_target
                    = True
                    break
                    if is_loop_target:
                        # Collect the
                        entire loop body up to and
                        including the back-jump
                        loop_body = []
                        j = i + 1
                                end_label = instrs[j].arg1
                                j += 1
                                break
                                if instrs[j].op
                                == JUMP_TRUE and
                                instrs[j].arg2 == label_name:
                                    # do-while
                                    back-edge
                                    # The
                                    condition is in arg1
                                    cond =
                                    self._val(instrs[j].arg1)
                                    j += 1
                                    if j <
                                    len(instrs) and instrs[j].op
                                    == LABEL:
                                        j += 1
                                        # Emit
                                        do-while as while True +
                                        break
                                        self._emit("while True:")
self._indent_inc()                    if
if
loop_body:
self._emit_block(loop_body)
self._emit(f"if not
({cond}):")
self._indent_inc()
self._emit("break")
self._indent_dec()
self._indent_dec()
emitted_any = True
i = j
is_do_while = True
break
loop_body.append(instrs[j])
j += 1
else:
    # Reached
end without finding
back-jump
i += 1
continue
if not
is_do_while and (instrs[j -
1].op != JUMP_TRUE or
instrs[j - 1].arg2 !=
label_name):
    # while-loop
pattern: first check for
JUMP_FALSE in body

```

<pre> # Pattern: condition_calc → JUMP_FALSE end → body cond_instrs = [] body_instrs_loop = [] found_jf = False jf_end = None for bi, bq in enumerate(loop_body): if bq.op == JUMP_FALSE and not found_jf: found_jf = True jf_end = bq.arg2 cond_instrs = loop_body[:bi] body_instrs_loop = loop_body[bi + 1:] cond_val = self._val(bq.arg1) break if found_jf: # Emit condition calculations before the while if cond_instrs: self._emit_block(cond_instrs) self._emit(f"while {cond_val}:") self._indent_inc() </pre>	<pre> if body_instrs_loop: self._emit_block(body_instrs _loop) # Re-evaluate condition if cond_instrs: self._emit_block(cond_instrs) else: self._emit("pass") self._indent_dec() else: # Infinite loop self._emit("while True:") self._indent_inc() if loop_body: self._emit_block(loop_body) else: self._emit("pass") self._indent_dec() emitted_any = True i = j continue else: i += 1 continue if q.op == JUMP: </pre>	<pre> # Unconditional jump — handled by control flow structures i += 1 continue if q.op == JUMP_FALSE: # If-block: collect body up to target label cond_val = self._val(q.arg1) target_label = q.arg2 # Find the target label index body_instrs_inner = [] j = i + 1 while j < len(instrs): if instrs[j].op == LABEL and instrs[j].arg1 == target_label: break body_instrs_inner.append(in strs[j]) j += 1 # Check if the last body instruction is a JUMP (→ else branch) else_instrs = [] if body_instrs_inner and body_instrs_inner[-1].op == JUMP: else_label = body_instrs_inner[-1].arg1 body_instrs_inner = </pre>
--	--	--

```

body_instrs_inner[:-1] #
remove the JUMP
    # Skip past the
target label and collect until
the else-end label
    k = j + 1 # past
the LABEL
    while k <
len(instrs):
    if instrs[k].op
== LABEL and
instrs[k].arg1 == else_label:
        break

else_instrs.append(instrs[k])
    k += 1
    j = k + 1 # skip
past the else-end label
    else:
        j = j + 1 # skip
past the target label

    self._emit(f"if
{cond_val}:".")
    self._indent_inc()
    if
body_instrs_inner:

self._emit_block(body_instrs
_inner)
    else:

self._emit("pass")
    self._indent_dec()
    if else_instrs:

self._emit("else:")

self._indent_inc()

self._emit_block(else_instrs)

self._indent_dec()
    emitted_any =
    True
    i = j
    continue

    if q.op ==
JUMP_TRUE:
        # Used in do-while
(HAUL-HEAVE): just skip -
handled by loop pattern
        i += 1
        continue

    if q.op == BREAK:
        self._emit("break")
        emitted_any =
    True
        i += 1
        continue

    if q.op ==
CONTINUE:
        self._emit("continue")
        emitted_any =
    True
        i += 1
        continue

    # --- Functions ---
    if q.op == ARG:
        self._arg_stack.append(self._
val(q.arg1))
        i += 1
        continue

    if q.op == CALL:
        self._maybe_emit_line_track
er(q)
        fname =
        self._sanitize_name(q.arg1)
        n = q.arg2 or 0
        args =
        self._arg_stack[-n:] if n else
[]
        self._arg_stack =
        self._arg_stack[:-n] if n else
self._arg_stack
        self._emit(f"{fname}({{'',
'.join(args)}})")

        self._temps[q.result] = expr

        self._emit(f"{q.result} =
{expr}")
        emitted_any =
    True
        i += 1
        continue

        if q.op ==
CALL_VOID:
            self._maybe_emit_line_track
er(q)
            fname =
            self._sanitize_name(q.arg1)
            n = q.arg2 or 0
            args =
            self._arg_stack[-n:] if n else
[]
            self._arg_stack =
            self._arg_stack[:-n] if n else
            self._arg_stack
            self._emit(f"{fname}({{'',
'.join(args)}})")

```

<pre> emitted_any = True i += 1 continue if q.op == RETURN: self._emit(f'return {self._val(q.arg1)}") emitted_any = True i += 1 continue if q.op == RETURN_VOID: self._emit("return") emitted_any = True i += 1 continue # --- NOP / COMMENT --- if q.op == NOP: i += 1 continue if q.op == COMMENT: self._emit(f"# {q.arg1}") i += 1 continue # --- Fallthrough --- i += 1 if not emitted_any: self._emit("pass") </pre>	<pre> def _gen_input(self, q): fmt = q.arg1 targets = q.arg2 spec_map = {'C': 'COIN', 'D': 'DIME', 'P': 'PARCH', 'S': 'SCROLL', 'B': 'BOOL'} raw = _convert_scroll_escapes(fmt) if fmt else " specs = [spec_map[ch] for ch in re.findall(r'%([CDPSB])', raw)] prompt = re.sub(r'%([CDPSB])', ", raw) if prompt: self._emit(f'print({repr(pro mpt)}, end=", flush=True)") # Ask the server for one full line of input self._emit(f"_ss_raw_in = input('__ss:LINE:~)") # Let the injected parser split it according to the format specs self._emit(f"_ss_parsed = _ss_parse_input(_ss_raw_in, {repr(specs)}") for i, tgt in enumerate(targets): var = self._sanitize_name(tgt['var_ name']) val_expr = f'_ss_parsed[{i}]' </pre>	<pre> dtype = specs[i] if i < len(specs) else None if tgt['target_kind'] == 'var': self._emit(f'{var} = {val_expr}') elif tgt['target_kind'] == 'array1d': idx = self._val(tgt['index1']) self._emit(f'{var}[{idx}] = {val_expr}') elif tgt['target_kind'] == 'array2d': i1 = self._val(tgt['index1']) i2 = self._val(tgt['index2']) self._emit(f'{var}[{i1}][{i2}] = {val_expr}') elif tgt['target_kind'] == 'member': member = tgt['member'] self._emit(f'{var}['{member }'] = {val_expr}') def _gen_output(self, q): fmt_str = q.arg1 arg_temps = q.arg2 fmt_converted, specs = _format_spec_to_python(fmt_ str) if not arg_temps: </pre>
---	--	--

```
self._emit(f"print({repr(fmt_
converted)}, end=)")
    else:
        val_strs = []
        for i, at in
enumerate(arg_temps):
            v = self._val(at)
            spec = specs[i] if i
< len(specs) else None
            # Overflow guard:
reject values that exceed
display limits before output

        if spec in ('COIN',
'DIME'):
            if i < len(specs)
and specs[i] == 'BOOL':
                val_strs.append(f"_ss_displa
y({v})")
            else:
                val_strs.append(f"_ss_check
_overflow({v}, {repr(spec)},
'output')")
            elif i < len(specs)
and specs[i] == 'BOOL':
```

```
import sys
import os
import json
import copy
import threading
import traceback
import re
import io
import queue

sys.path.append(os.path.dirn
ame(os.path.dirname(os.path
.abspath(__file__))))
sys.path.append(os.path.dirn
ame(os.path.abspath(__file_
__)))

from flask import Flask,
request, jsonify, Response
from flask_cors import
CORS

try:
    from backend.lexical.lexer
import Lexer
```

SeaStack

```
server.py
from
backend.syntax.syn_parser
import Parser
from
backend.semantic.ast_parser
import ASTParser
from
backend.semantic.semantic_
analyzer import
SemanticAnalyzer
from
backend.codegen.ir_generato
r import IRGenerator
from
backend.codegen.optimizer
import IROptimizer
from
backend.codegen.code_gene
rator import CodeGenerator
except ImportError as e:
    print(f"\n[ERROR] Import
Failed! Details: {e}")
    sys.exit(1)

app = Flask(__name__)
CORS(app)
```

```
val_strs.append(f"_ss_displa
y({v})")
    else:
        val_strs.append(str(v))
        args = ',
'.join(val_strs)

self._emit(f"print({repr(fmt_
converted)}.format({args}),
end=)")

import flask.cli
flask.cli.show_server_banner
= lambda *args, **kwargs:
None

import logging
class
_SuppressDevWarning(loggi
ng.Filter):
    def filter(self, record):
        return 'Do not use it in a
production deployment' not
in record.getMessage()
logging.getLogger('werkzeu
g').addFilter(_SuppressDev
Warning())

_running_flags = {} #
session_id →
threading.Event (set = stop
requested)
_input_queues = {} #
session_id → queue.Queue
(frontend → execution
thread)
```

```

    err_val =
    getattr(err, 'value', '?')

    clean.append({
        "line":
        getattr(err, 'line', '?'),
        "col":
        getattr(err, 'col', '?'),
        "found":
        (f"Unknown Character
        '{err_val}'"
        if
        err_msg == "Unknown
        Character"
        else
        f"Invalid character
        '{err_val}'"),
        "expected":
        getattr(err, 'expected', ["Valid
        Token"]),
        "message":
        err_msg,
        "phase":
        "Lexical"
    })
    raise
    _CompileError("Lexical",
    clean)
    except _CompileError:
        raise
    except Exception as e:
        raise
    _CompileError("Lexical",
    [{"line": "-", "col": "-",
    "found": "CRASH",
    "expected": [], "message":
    f"Lexer crashed: {e}",
    "phase": "Lexical"}])

    # Syntax

@app.route('/api/run',
methods=['POST'])
def run_code():
    data = request.json
    code_string =
    data.get('code', "")
    session_id =
    data.get('session_id',
'default')

    def _sse(obj):
        """Wrap a dict as a
single SSE data frame."""
        return f"data:
        {json.dumps(obj)}\n\n"

    # Compile: Lexical →
    Syntax → Semantic → IR
    Generation → Optimisation
    → Code Generation
    def _compile():
        # Lexical
        try:
            lexer =
            Lexer(code_string)
            tokens, lex_errors =
            lexer.tokenize()
            if lex_errors:
                clean = []
                for err in
            lex_errors:
                if isinstance(err,
dict):

            clean.append(err)
            else:
                err_msg =
            getattr(err, 'error_msg',
            str(err))

            try:
                parser =
                Parser(tokens, code_string)
                syn_errors =
            parser.parse()
                if syn_errors:
                    for e in syn_errors:
                        e['phase'] =
                        'Syntax'
                        raise
                        _CompileError("Syntax",
                        syn_errors)
                    except _CompileError:
                        raise
                    except Exception as e:
                        raise
                        _CompileError("Syntax",
                        [{"line": "-", "col": "-",
                        "found": "CRASH",
                        "expected": [], "message":
                        "expected": [], "message":
                        f"Parser crashed: {e}",
                        "phase": "Syntax"}])

                # Semantic
                try:
                    ast_parser =
                    ASTParser(tokens,
                    code_string)
                    program_node =
                    ast_parser.build()
                    analyzer =
                    SemanticAnalyzer(program_
                    node, code_string)
                    sem_errors =
                    analyzer.analyze()
                    if sem_errors:
                        for e in
                    sem_errors:
                        e['phase'] =
                        'Semantic'

```

```
        raise
    _CompileError("Semantic",
sem_errors)
    except _CompileError:
        raise
    except Exception as e:
        raise
    _CompileError("Semantic",
[{"line": "-", "col": "-",
"found": "CRASH",

"expected": [], "message":
f"Semantic analyzer crashed:
{e}",

"phase": "Semantic"}])

    # IR Generation
    try:
        ir_gen =
IRGenerator(program_node)
        ir_program =
ir_gen.generate()
    except Exception as e:
        raise
    _CompileError("Code
Generation", [{"line": "-",
"col": "-", "found":
"CRASH",

"expected": [], "message":
f"IR generation failed: {e}",

"phase": "Code
Generation"}])

    # Optimisation
    try:
        ir_program =
IROptimizer(ir_program).op
timize()
    except Exception as e:
```

SeaStack

```
        raise
    _CompileError("Code
Generation", [{"line": "-",
"col": "-", "found":
"CRASH",

"expected": [], "message":
f"Optimisation failed: {e}",

"phase": "Code
Generation"}])

    # Code Generation
    try:
        return
CodeGenerator(ir_program).
generate()
    except Exception as e:
        raise
    _CompileError("Code
Generation", [{"line": "-",
"col": "-", "found":
"CRASH",

"expected": [], "message":
f"Code generation failed:
{e}",

"phase": "Code
Generation"}])

    def event_stream():
        try:
            generated_code =
_compile()
        except _CompileError
as ce:
            yield _sse({"type":
"compile_error", "errors":
ce.errors, "phase":
ce.phase})
```

```
        yield _sse({"type":
"done", "success": False})
        return

        output_q =
queue.Queue()
        input_q =
queue.Queue()
        stop_event =
threading.Event()

        _running_flags[session_id] =
stop_event

        _input_queues[session_id] =
input_q

        exec_globals =
{'__builtins__': __builtins__,
'_ss_line': 0, '_ss_col': 0}

        def
captured_print(*args,
**kwargs):
            end =
kwargs.get('end', '\n')
            sep =
kwargs.get('sep', ' ')
            text = sep.join(str(a)
for a in args) + end

        output_q.put(('output', text))

        def
captured_input(prompt=""):
            dtype = 'SCROLL'
            display_prompt =
prompt
            if isinstance(prompt,
str) and
prompt.startswith('__ss:')
```

```
parts =
prompt.split(':', 2)
if len(parts) >= 2:
    dtype = parts[1]
    display_prompt = "
if display_prompt:
```

```
output_q.put(('output',
str(display_prompt)))
```

```
output_q.put(('input_needed'
, {'dtype': dtype}))
while not
stop_event.is_set():
    try:
        return
input_q.get(timeout=0.5)
    except
queue.Empty:
        continue
    return "
```

```
exec_globals['print'] =
captured_print
exec_globals['input'] =
captured_input
```

```
modified_code =
generated_code.replace(
    "if __name__ ==
'__main__':\n    _ss_ahoy()",
    "_ss_ahoy()"
)
```

```
exec_result = {'error':
None}
done_event =
threading.Event()
```

```
def _run():
    try:
```

```
exec(modified_code,
exec_globals) # noqa: S102
except SystemExit:
    pass
except Exception as
exc:
```

```
if not
stop_event.is_set():
```

```
exec_result['error'] =
_map_runtime_error(
    exc,
code_string, generated_code,
exec_globals
)
finally:
```

```
output_q.put(('done', None))
done_event.set()
```

```
thread =
threading.Thread(target=_ru
n, daemon=True)
thread.start()
```

```
def _watchdog():
    if not
done_event.wait(timeout=12
00):
        stop_event.set()
```

```
output_q.put(('timeout',
None))
```

```
threading.Thread(target=_wa
tchdog,
daemon=True).start()
```

```
while True:
    try:
```

```
kind, payload =
output_q.get(timeout=1.0)
except queue.Empty:
    yield ":
keep-alive\n\n"
    continue
```

```
if kind == 'output':
    yield _sse({"type":
"output", "text": payload})
```

```
elif kind ==
'input_needed':
    dtype =
payload.get('dtype',
'SCROLL') if
isinstance(payload, dict) else
'SCROLL'
    yield _sse({"type":
"input_needed", "dtype":
dtype})
```

```
elif kind == 'timeout':
    yield _sse({
        "type": "error",
        "error": {
            "line": "-",
            "col": "-",
            "error_type":
"Timeout Error",
            "message":
"Program ran for over 5
minutes and was stopped.
Check for infinite loops.",
            "actual_line":
"",
            "phase":
"Runtime"
        }
    })
    yield _sse({"type":
"done", "success": False})
```

```
        break

    elif kind == 'done':
        if
            exec_result['error']:
                yield
            _sse({"type": "error",
                "error": exec_result['error']})
            yield _sse({"type":
                "done", "success":
                exec_result['error'] is None})
            break

    _running_flags.pop(session_
id, None)

    _input_queues.pop(session_i
d, None)

    return Response(
        event_stream(),

    mimetype='text/event-stream
',
        headers={
            'Cache-Control':
            'no-cache',
            'X-Accel-Buffering':
            'no',
        }
    )

@app.route('/api/tokenize',
methods=['POST'])
def tokenize_code():
    data = request.json
    code_string =
data.get('code', "")
```

```
    SKIP_TYPES = {'eof',
'EOF'}

    _WS_LABEL = {'\n':
'newline', ' ': 'whitespace', '\t':
'tab', '\r': 'carriage_return'}

    def _token_row(t):
        if t.type ==
'whitespace':
            label =
            _WS_LABEL.get(t.value,
'whitespace')
            return {"lexeme":
label, "token": label}
        return {"lexeme":
t.value, "token": t.type}

    try:
        lexer =
        Lexer(code_string)
        tokens, lex_errors =
lexer.tokenize()

        ui_tokens = [
            _token_row(t)
            for t in tokens
            if t.type not in
SKIP_TYPES
        ]

        formatted_errors = []
        for err in lex_errors:
            if isinstance(err,
dict):
                formatted_errors.append(err)
            else:
                formatted_errors.append({
                    "line":
getattr(err, 'line', "?"),
```

```
                    "col":
getattr(err, 'col', "?"),
                    "found":
getattr(err, 'value', "?"),
                    "message":
getattr(err, 'error_msg',
str(err)),
                })

            return jsonify({
                "success": True,
                "tokens":
ui_tokens,
                "errors":
formatted_errors,
                "error_count":
len(formatted_errors),
            })
        except Exception as e:
            return jsonify({
                "success": False,
                "tokens": [],
                "errors": [{"line":
"-", "col": "-", "found":
"CRASH", "message":
str(e)}],
                "error_count": 1,
            })

    def _quad_to_dict(q, idx:
int) -> dict:
        def _s(v):
            if v is None:
                return ""
            if isinstance(v, bool):
                return str(v)
            if isinstance(v, (int,
float, str)): return str(v)
            return repr(v)
        return {
            "index": idx,
```

```
"op": q.op,
"arg1": _s(q.arg1),
"arg2": _s(q.arg2),
"result": _s(q.result),
"comment": q.comment
or "",
"line": q.line,
}
```

```
@app.route('/api/tac',
methods=['POST'])
def tac_code():
    data = request.json
    code_string =
data.get('code', "")
    try:
        lexer =
        Lexer(code_string)
        tokens, lex_errors =
        lexer.tokenize()
        if lex_errors:
            return
        jsonify({"success": False,
"error": "Lexical errors in
source."})

        parser = Parser(tokens,
code_string)
        syn_errors =
        parser.parse()
        if syn_errors:
            return
        jsonify({"success": False,
"error": "Syntax errors in
source."})
```

```
ast_parser =
ASTParser(tokens,
code_string)
    program_node =
ast_parser.build()
```

SeaStack

```
analyzer =
SemanticAnalyzer(program_
node, code_string)
    sem_errors =
    analyzer.analyze()
    if sem_errors:
        return
    jsonify({"success": False,
"error": "Semantic errors in
source."})
```

```
ir_gen =
IRGenerator(program_node)
    ir_program =
    ir_gen.generate()
```

```
raw_quads =
[_quad_to_dict(q, i)
    for i, q in
    enumerate(copy.deepcopy(ir
_program.instructions))]
```

```
optimizer =
IROptimizer(ir_program)
    ir_optimized =
    optimizer.optimize()
```

```
return jsonify({
    "success": True,
    "raw_quads":
raw_quads,
    "optimized_quads":
[_quad_to_dict(q, i)
        for i, q
in
    enumerate(ir_optimized.instr
uctions)],
```

```
    "pass_snapshots":
optimizer.pass_snapshots,
    "optimizer_stats":
optimizer.stats,
```

```
"temp_types":
dict(ir_optimized.temp_type
s),
    })
    except Exception as e:
        return
    jsonify({"success": False,
"error": f"TAC generation
failed: {str(e)}"})
```

```
@app.route('/api/stop',
methods=['POST'])
def stop_code():
    data = request.json
    session_id =
data.get('session_id',
'default')
```

```
    if session_id in
_running_flags:
        _running_flags[session_id].s
et()
```

```
    if session_id in
_input_queues:
        try:
```

```
        _input_queues[session_id].p
ut_nowait()
        except queue.Full:
            pass
        return
    jsonify({"success": True,
"message": "Stop signal
sent."})
```

```
    return jsonify({"success":
False, "message": "No
running program found."})
```



```
@app.route('/api/input',
methods=['POST'])
def provide_input():
    data = request.json
    session_id =
data.get('session_id',
'default')
    user_input =
data.get('input', '')

    if session_id in
_input_queues:

_input_queues[session_id].p
ut(user_input)
    return
jsonify({"success": True})

    return jsonify({"success":
False, "message": "No
running program is waiting
for input."})

def
_map_runtime_error(exc,
source_code,
generated_code,
exec_globals=None):
    """Translate a Python
exception into a
SeaStack-context error
object."""
    tb =
traceback.extract_tb(exc.__tr
aceback__)
    py_line = None
    for frame in reversed(tb):
        if frame.filename ==
'<string>':
            py_line =
frame.lineno

SeaStack
```

```
break

exc_msg = str(exc)

if isinstance(exc,
ZeroDivisionError):
    error_type = "Division
by Zero"
    message = "Cannot
divide by zero. Check your
arithmetic expressions for a
denominator that evaluates
to 0."
    elif isinstance(exc,
TypeError):
        error_type = "Type
Error"
        message =
"Incompatible types in
operation. Check that your
variables and expressions
use matching types."
        elif isinstance(exc,
IndexError):
            error_type = "Index Out
of Bounds"
            message = "Array
index is out of bounds. The
index used exceeds the
declared size of the array."
            elif isinstance(exc,
KeyError):
                error_type =
"Undefined Member"
                message =
f'Attempted to access an
undefined struct member:
{exc_msg}'
            elif isinstance(exc,
NameError):
```

```
match =
re.search(r'name ' + '\w+') is
not defined", exc_msg)
    var_name =
match.group(1) if match else
"unknown"
    error_type =
"Undefined Variable"
    message = f'Variable
'{var_name}' is not defined
at this point in execution."
    elif isinstance(exc,
ValueError):
        error_type = "Value
Error"
        m = exc_msg.lower()
        if 'aye' in m or 'nay' in
m:
            message = "Invalid
BOOL input. Enter AYE for
true or NAY for false."
            elif 'int' in m or 'base
10' in m:
                message = ("Invalid
input: a whole number
(COIN) was expected but
what you "
                    "entered
cannot be converted. Make
sure you type only digits.")
                elif 'float' in m:
                    message = ("Invalid
input: a decimal number
(DIME) was expected but
what you "
                        "entered
cannot be converted. Make
sure you type a valid
number.")
                else:
                    message =
f'{exc_msg}'
```

```

elif isinstance(exc,
OverflowError):
    error_type = "Overflow
Error"
    message = "Numeric
result is too large. Check for
very large exponents or
multiplication values."
    elif isinstance(exc,
RecursionError):
        error_type = "Stack
Overflow"
        message = "Too many
nested function calls. Check
for runaway recursion."
        elif isinstance(exc,
SyntaxError):
            error_type = "Syntax
Error"
            m = exc_msg
            m =
re.sub(r'\s*((<[^\]]*>)\s*line\s
*\d+)', ", m).strip()
            if "'break'" in m or
'break' in m.lower():
                message = "LAND!!
can only be used inside a
loop or CHART block."
                elif "'continue'" in m or
'continue' in m.lower():
                    message = "SAIL!!
can only be used inside a
loop."
                    elif "'return'" in m or
'return' in m.lower():
                        message = "BACK
can only be used inside a
function."
                        else:
                            message =
_sanitize_py_message(m)
                        else:

```

SeaStack

```

error_type = "Runtime
Error"
    m =
re.sub(r'\s*((<[^\]]*>)\s*line\s
*\d+)', ", exc_msg).strip()
    message =
_sanitize_py_message(m)

    ss_line = "-"
    ss_col = "-"
    actual_line = ""

    if exec_globals:
        raw_line =
exec_globals.get('_ss_line',
0)
        raw_col =
exec_globals.get('_ss_col',
0)
        if raw_line and
int(raw_line) > 0:
            ss_line =
str(raw_line)
            if raw_col:
                ss_col =
str(raw_col)
            src_lines =
source_code.split("\n") if
source_code else []
            idx = int(raw_line) -
1
            if 0 <= idx <
len(src_lines):
                actual_line =
src_lines[idx]

            if ss_line == "-" and
py_line and generated_code:
                gen_lines =
generated_code.split('\n')
                if 0 < py_line <=
len(gen_lines):

```

```

py_code_line =
gen_lines[py_line - 1].strip()
    src_lines =
source_code.split("\n") if
source_code else []
    for si, sl in
enumerate(src_lines):
        stripped = sl.strip()
        if stripped and not
stripped.startswith('~):
            if any(w in
stripped for w in
_extract_identifiers(py_code
_line)):
                ss_line =
str(si + 1)
                actual_line =
sl
                break

    return {
        "line": ss_line,
        "col": ss_col,
        "found": error_type,
        "expected": [],
        "message": message,
        "actual_line":
actual_line,
        "phase": "Runtime",
        "error_type":
error_type,
    }

def
_sanitize_py_message(msg):
    """Replace
Python-specific terms with
SeaStack equivalents in error
messages."""
    replacements = [

```

```

    ("break",
"LAND!!"),
    ("continue",
"SAIL!!"),
    ("return", "BACK"),
    ("pass", "NOP"),
    ("int", "COIN"),
    ("float", "DIME"),
    ("str",
"SCROLL"),
    ("bool", "BOOL"),
    ("list", "array"),
    ("dict", "struct"),
    (" int ", " COIN "),
    (" float ", " DIME "),
    (" str ", " SCROLL
"),
    (" bool ", " BOOL "),
    (" list ", " array "),
    ("<string>", "the
program"),
    ("<module>", "the
program"),

    ]
    for old, new in
replacements:
        msg = msg.replace(old,
new)
    return msg

def
_extract_identifiers(code_line):
    """Extract SeaStack-style
identifiers from a Python
code line."""
    return [m for m in
re.findall(r'\b([a-z][a-z0-9_]*
)\b', code_line)
        if m not in {'print',
'input', 'int', 'float', 'str',
'bool',
                        'true', 'false',
'none', 'not', 'and', 'or',

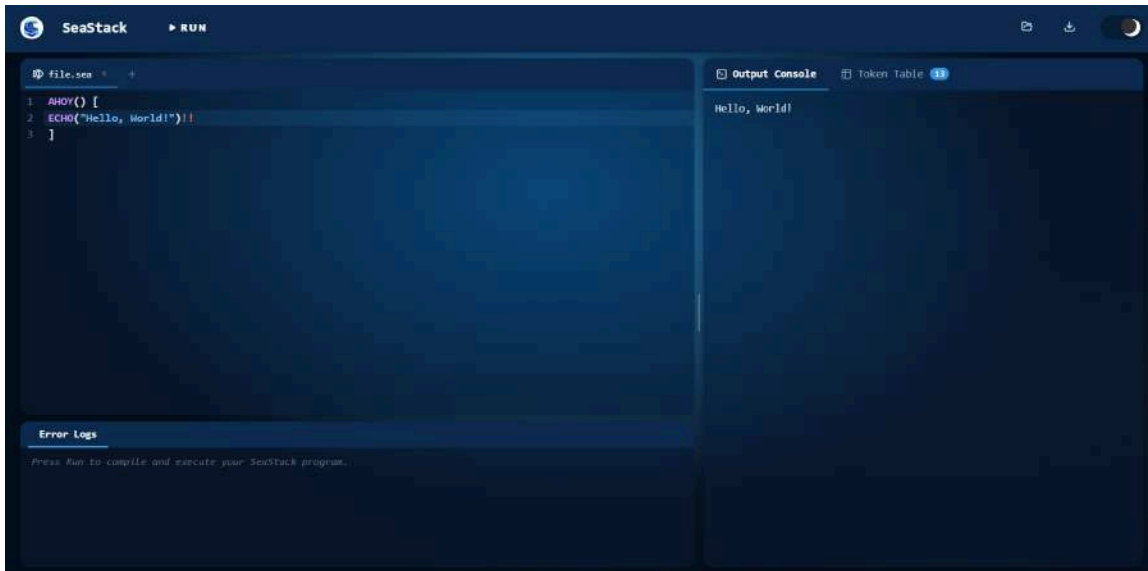
                        'if', 'else',
'while', 'for', 'return', 'def',
                        'pass', 'break',
'continue', 'math', 'sys',
                        'end',
'format', 'self', 'flush'}]

class
_CompileError(Exception):
    def __init__(self, phase,
errors):
        self.phase = phase
        self.errors = errors

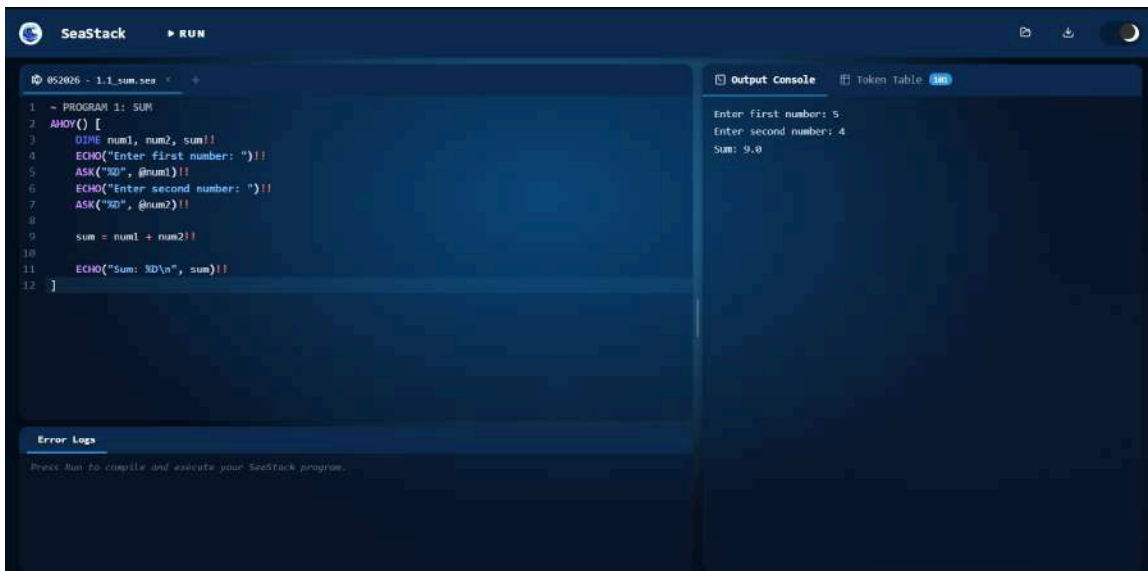
if __name__ == '__main__':
    port =
int(os.environ.get('PORT',
5000))
    app.run(debug=False,
host='0.0.0.0', port=port)

```

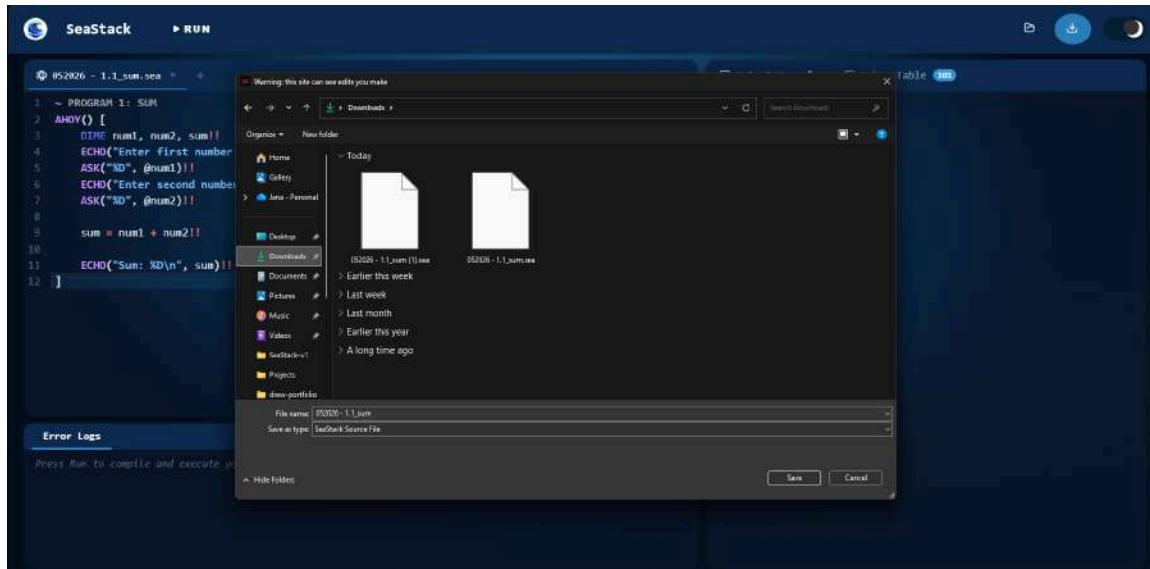
Screenshot of GUI



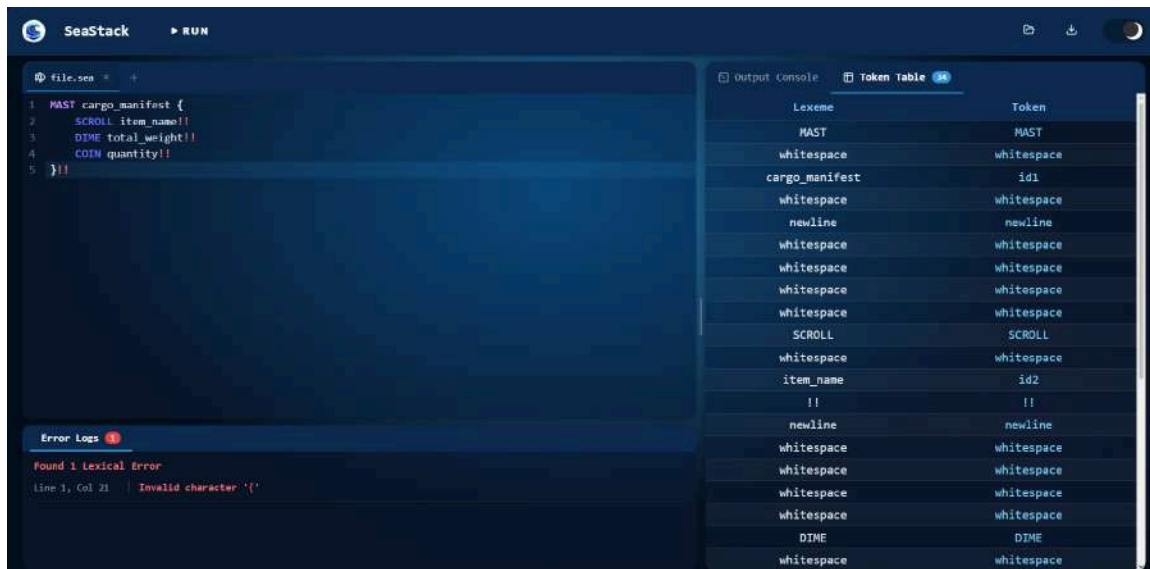
Running Program with Output



Saving Program



Program with Lexical Errors



Program with Syntax Errors



The screenshot shows the SeaStack IDE interface. The main editor displays a program with the following code:

```
1 AHoy() [
2   DIME y = 5.5!!
3   DIME wrong = (y + )!!
4   ECHO("Invalid\n")!!
5 ]
```

The Error Logs panel at the bottom left shows a syntax error:

```
Found 1 Syntax Error
Line 3, Col 23 | Unexpected token: ')'
+ DIME wrong = (y + )!!
+
Expected: DIME-lit, COIN-lit, (, -, id
```

The Token Table on the right shows the tokens for the program:

Lexeme	Token
AHOY	AHOY
(	(
)	)
whitespace	whitespace
[	[
newline	newline
whitespace	whitespace
whitespace	whitespace
whitespace	whitespace
whitespace	whitespace
DIME	DIME
whitespace	whitespace
y	id1
whitespace	whitespace
=	=
whitespace	whitespace
5.5	DIME-lit
!!	!!
newline	newline
whitespace	whitespace

Program with Semantic Errors



The screenshot shows the SeaStack IDE interface. The main editor displays a program with the following code:

```
1 ABYSS sail() [
2   ECHO("SeaStack Sail!")!!
3 ]
4
5 AHoy() [
6   sail = 10!!
7 ]
```

The Error Logs panel at the bottom left shows a semantic error:

```
Found 1 Semantic Error
Line 6, Col 3 | Type Mismatch
+ sail = 10!!
+
Cannot assign 'COIN' to 'sail' (declared as 'ABYSS').
```

The Token Table on the right shows the tokens for the program:

Lexeme	Token
ABYSS	ABYSS
whitespace	whitespace
sail	id1
(	(
)	)
whitespace	whitespace
[	[
newline	newline
whitespace	whitespace
whitespace	whitespace
ECHO	ECHO
(	(
"SeaStack Sail!"	SCROLL-lit
)	)
!!	!!
newline	newline
]	]
newline	newline
newline	newline
AHOY	AHOY

Program with Runtime Errors



The screenshot shows the SeaStack IDE interface. The main editor displays a program named "1-1_sum.sea" with the following code:

```
1 ~ PROGRAM 1: SUM
2 ANDY() [
3     DIME num1, num2, sum!!
4     ECHO("Enter first number: ")!!
5     ASK("%d", @num1)!!
6     ECHO("Enter second number: ")!!
7     ASK("%d", @num2)!!
8
9     sum = num1 + num2!!
10
11     ECHO("Sum: %d\n", sum)!!
12 ]
```

The right-hand side of the IDE is split into two panels. The top panel, "Output Console", shows the prompt "Enter first number: a". The bottom panel, "Error Logs", shows a runtime error:

```
Found 1 Runtime Error
Line 5, Col 5 | Value Error
+ ASK("%d", @num1)!!
+
Expected a numeric value (DIME), got 'a'. Please enter a number.
```

ALISWAG, K. V. J.
ALTEZA, J. R. D.
CAYACAP, F. A. S.

DEL ROSARIO, J. M. A.
MAGAAN, F. M. V.
MULLENO, J. M. A.



DOCUMENTATION